

The Design of eLearning Course Using The Power of AI

Computer Science
(Graduation Project Documentation)

Submitted by

Alaa Bahaa Eldeen Yousef Kobasiy

Jessica Ashraf Anis Azer

Hassan Tarek Mohammed Hassan

Omar Alaa Abdulaziz Mohamed

Omar Gad Saad Alhebiny

Ahmed Ali Ali Ibrahim

Amr Hazem Ismail Abdelrahman

Zeinab Nagi Mohammed Mamoon

Supervised by

Prof. DR. Mohamed Anwar Assal

T.A. Omnia Adel

T.A. Hassan Abdelshafy

T.A Mohab Mustafa

2025

Acknowledgment

All praise and thanks to God, who gave us the strength and ability to complete this work. We would like to express our deepest gratitude to the **Modern University for Technology and Information (M.T.I)** for playing a pivotal role in shaping our characters and providing us with all the necessary tools for our educational development. M.T.I. also created a wonderful environment for us to learn throughout our academic years.

This accomplishment would not have been possible without the efforts of **Prof. Dr. Olfat Kamel**, President of M.T.I University, who established such a conducive learning environment, offering the best facilities for our success.

It is a pleasure to thank **Prof. Dr. Mohamed Taher El-Mayah**, Dean of the Faculty of Computers & Artificial Intelligence, and **Prof. Dr. Mohamed Mohamed Elgazzar**, Vice Dean for Academic Affairs for their unlimited help, support, encouragement, and guidance.

We are honored to have worked under the supervision of **Prof. Dr. Mohammed Anwar Assal**, head of the AI department, who has been both a mentor and a guide. We owe him a great deal for his advice and support, as well as for providing the necessary information regarding the project and following up with us through every stage until completion. It was also a pleasure to work with our Teaching Assistants **T.A. Mohab Mustafa, T.A. Omnia, and T.A. Hassan.**

Special thanks to **Prof. Dr. Hanafy Ismail**, head of the Computer Science Department, as well as the department staff, **Prof. Dr. Alaa Abd El-Raheem** for their efforts during our university studies .

A special thank you to **Prof. Dr. Hafez Abdel-Wahab**, head of the Information System Department, and **Prof. Dr. Mahmoud El-Shishtawy**, for their efforts and guidance throughout our studies.

We would also like to extend our special thanks to **Prof. Dr. Hesham El- Deeb**, head of the Artificial Intelligence Department, and his staff for their ongoing support throughout our academic journey .

We would like to thank **Prof. Dr. Eman Taha**, the head of the Basic Science Department, and her staff, including **Prof. Dr. Sayed Bakar**, **Prof. Dr. Rasha Saeed**, and **Dr. Rania Ahmed**, for their role in providing the foundational knowledge for our computer science degree.

Special thanks go to **Prof. Dr. Mohammed Anwar Assal**, head of the IT Center, for his efforts in imparting essential knowledge for our computer science degree.

We are grateful to all **the Assistant Lecturers** and **Teaching Assistants** who helped us throughout our studies with practical demonstrations that deepened our understanding and provided valuable hands-on experience.

Special thanks and gratitude to our **families** for their unwavering support throughout our academic journey. We couldn't have done it without their help and encouragement.

Abstract

"**The Design of eLearning Course Using the Power of AI**" represents a transformative approach to digital education by developing an intelligent, conversational learning assistant that enhances student engagement and knowledge retention. We got the idea inspiration from Educational Experience in the Age of Artificial Intelligence Conference by Microsoft in June 2024. This project creates an AI-powered virtual tutoring system where learners interact with a responsive avatar teacher through natural dialogue, either via text input or voice conversation. The system is designed to function as a knowledgeable and reliable teaching assistant, strictly operating within the boundaries of a structured curriculum to ensure academic accuracy and relevance.

At the core of this platform lies an integration of cutting-edge AI technologies that work in harmony to deliver a seamless educational experience. **OpenAI's Whisper** provides robust speech recognition capabilities, converting student voice queries into text for processing. The system then employs **Retrieval-Augmented Generation (RAG) techniques** combined with **Large Language Models (LLMs)** to retrieve and generate precise, contextually appropriate responses from a curated knowledge base. These responses are delivered back to the learner through a **text-to-speech (TTS) interface**, giving the AI tutor a lifelike, interactive presence.

A critical innovation of this project is its **domain-specific response filtering mechanism**, which ensures the assistant remains focused on the designated subject matter. When faced with questions outside its knowledge scope, the system either declines to answer or redirects the learner appropriately, thereby maintaining academic integrity and preventing misinformation. This controlled interaction model makes the platform particularly valuable for structured learning environments where curriculum fidelity is essential.

The implications of this AI-enhanced eLearning system are significant for modern education. It addresses key limitations of traditional online courses—such as passive content delivery and lack of personalized instruction—by introducing an adaptive, dialogue-based learning companion. The platform's ability to process both voice and text inputs makes it accessible to diverse learning styles, while its real-time response capability mimics the immediacy of classroom interactions. Furthermore, the underlying architecture is designed for scalability, allowing for future expansion into multiple disciplines, languages, and more sophisticated adaptive learning features.

By merging advanced AI with pedagogical design principles, this project pioneers a new paradigm in digital education—one where artificial intelligence does not merely supplement human teaching but actively replicates its most valuable aspects: responsiveness, personalization, and guided knowledge exploration. The system demonstrates how AI can be responsibly implemented in education, balancing technological innovation with academic rigor to create truly effective and trustworthy learning tools.

Contents

Chapter One : Introduction.....	13
1.1 Introduction	13
1.2 Problem Definition.....	13
1.3 Proposed Solutions.....	15
1.3.1 Core Components.....	15
1.3.2 Key Functionalities.....	16
1.3.3. Innovations	17
1.3.4. Expected Outcomes.....	17
1.4 Statistics on AI in Education	18
1.5 Statistics on the Impact of AI in Education.....	19
1.6 Documentation Organization	20
Chapter Two: Project planning.....	23
2.1 Introduction	23
2.2 Project Manageable Tasks.....	23
2.2.1 Tasks Description	24
2.3 Software tool for planning tasks (project plan 365).....	27
Chapter Three: Literature Review.....	31
3.1 Introduction	31
3.2. AI in Education: A Historical Perspective.....	31
3.3. ASR models.....	31
3.3.1 What is Automatic speech recognition (ASR) models?	31
3.3.2 How does Automatic speech recognition (ASR) work?.....	32
3.3.3 Key components of automatic speech recognition (ASR).....	32
3.3.4 Challenges and limitations face the (ASR) models.....	33
3.3.5 Evaluation metrics for Speech Recognition models (ASR).....	34
3.3.6 Speech recognition Architectures	37
3.4 What is a Large Language Model?.....	43
3.5 Knowledge Retrieval & AI Tutors	44
3.6 Text To Speech (TTS).....	45
3.6.1 What is TTS?.....	45
3.6.2 Importance and Applications.....	45
3.6.2 TTS vs Speech Recognition	46

3.7 Limitations of Existing Solutions.....	46
3.8 Related Work and Similar Technologies	47
Chapter Four: System Analysis and Design.....	50
4.1 Introduction	50
4.2 System Requirements	50
4.2.1 Functional Requirements.....	50
4.2.2 System Specifications using UML Use Case Diagram	50
4.2.3 System Specifications using UML Activity Diagrams.....	52
4.2.4 Nonfunctional Requirements.....	57
4.3 System Analysis Basic Concept	58
4.3.1 Function View	58
4.3.2 Data Dictionary	59
4.3.3 System Scenario	59
4.4 System Features	60
4.5 Graphical User Interface (GUI)	60
4.5.1 System GUI.....	60
System Design	67
4.6 Introduction	67
4.6.1 Importance of System Design	68
4.6.2 Process of System Design	68
4.7 System Objectives	68
4.7.1 Personalized and Interactive Learning	68
4.7.2 Enhanced Accuracy and Engagement	69
4.7.3 Accessibility and Inclusivity	69
4.7.4 Time and Resource Efficiency	69
4.7.5 Cost-Effective and Scalable	69
4.8 System Block Diagram	69
4.8.1 Block Diagram Importance	71
4.8.2 Block Diagram Modules	71
4.9 Entity Relationship Diagram (ERD)	72
4.9.1 ERD Mapping	74

Chapter Five : Implementation.....	76
Introduction	76
5.1 ECAI Frontend Implementation	76
5.1.2 Learning Hub	77
5.2 ECAI Backend Implementation	80
5.2.2 Authentication & User Management.....	80
5.2.3 Chat Operations.....	81
5.2.4 Data Models & Schemas	81
5.2.5 Database & Infrastructure	82
5.2.6 Error Handling & Validation.....	82
5.2.7 API Endpoints	82
5.3 Whisper and how it works.....	84
5.3.2 Massive Training Dataset.....	85
5.3.3 Input Processing: From Audio to Mel Spectrogram.....	88
5.3.4 Architecture: Transformer Encoder-Decoder	91
5.3.4. Tokenization: Byte Pair Encoding (BPE).....	94
5.3.5 Multitask Learning	99
5.3.6 Zero-shot Capabilities	102
5.3.7 Output with Timestamps	104
5.4 Large Language Models and Retrieval-Augmented Generation.....	107
5.4.2 How do LLMs Work?	107
5.4.3 Components of LLMs	107
5.4.4 Qwen-14B: What is it?	109
5.4.5 Challenges and Limitations of LLMs.....	113
5.4.6 What is Retrieval-Augmented Generation (RAG)	115
5.4.7 Knowledge Base for eLearning Projects	122
5.4.8 Prompt Engineering.....	124
5.4.9 Experimentation with RAG and its Limitations.....	127
5.5 How TTS Works: The Pipeline.....	133
5.5.2 Overview	133
5.5.3 Frontend (Text Analysis / Preprocessing).....	133
5.5.4 Backend (Speech Synthesis)	134
5.5.5 TTS Synthesis Techniques	135

5.5.6 Key Neural TTS Models	135
5.5.4 Cloud Model vs. Local Model	136
5.5.5 Models Evaluated.....	138
5.5.6 Evaluation Criteria for TTS Models.....	145
5.5.7. Comparison of Evaluated TTS Models	146
5.5.8. Selected TTS Model and Justification.....	147
5.6 Avatar	148
5.6.2 Ready Player Me	148
5.6.2.1 gltfjsx.....	150
5.6.3 Animations:	153
5.6.4 Blender:	155
5.6.5 Lip sync:.....	163
5.6.6 Oculus OVR LipSync.....	168
5.6.7 Facial Expression and Blinking Logic	173
5.6.7.1 Apple ARKit.....	173
5.6.7.2 Facial Expression	176
5.6.8 Blinking Logic	177
Chapter Six : Testing and Evaluation	181
6.1 Introduction	181
6.2 System Testing	181
6.2.1 Unit Testing.....	181
6.2.2 Integration Testing	182
6.3 Usability Testing	182
Chapter Seven : Conclusion and Future Work.....	238
7.1 Introduction	238
7.2 Conclusion.....	238
7.3 Earned Capabilities.....	238
7.4 Project Achievements	239
7.5 Future Work	239
References.....	241
Appendix.....	245

List of Figures

FIGURE 2-1 SDLC.....	23
FIGURE 2-2 PROJECT PLAN.....	28
FIGURE 2-3 GANNT CHART	28
FIGURE 2-4 NETWORK DIAGRAM (NODES).....	29
FIGURE 2-5 NETWORK DIAGRAM	30
FIGURE 2-5 NETWORK DIAGRAM	30
FIGURE 3-1 EXAMPLE ON WER.....	35
FIGURE 3-2 EXAMPLE ON WER.....	36
FIGURE 3-3 RECURRENT NEURAL NETWORKS	41
FIGURE 3-4 THE RISE OF LLM MODELS	44
FIGURE 4-1 USECASE DIAGRAM	51
FIGURE 4-2 SIGN UP ACTIVITY DIAGRAM.....	52
FIGURE 4-3 LOGIN ACTIVITY DIAGRAM.....	53
FIGURE 4-4 EDIT PROFILE ACTIVITY DIAGRAM	54
FIGURE 4-5 TYPE QUESTION ACTIVITY DIAGRAM.....	55
FIGURE 4-6 RECORD VOICE ACTIVITY DIAGRAM	56
FIGURE 4-7 ACCESS HISTORY ACTIVITY DIAGRAM	57
FIGURE 4-8 SIGN UP.....	60
FIGURE 4-9 LOGIN PAGE.....	61
FIGURE 4-10 HOME PAGE LIGHT MODE	61
FIGURE 4-11 DARK MODE.....	62
FIGURE 4-12 SIDE BAR	62
FIGURE 4-13 ACCOUNT DETAILS	63
FIGURE 4-14 SECURITY	63
FIGURE 4-15 PROFILE.....	64
FIGURE 4-16 SUPPORT	64
FIGURE 4-17 FIRST PAGE IN HOME.....	65
FIGURE 4-18 SECOND PAGE IN HOME	65
FIGURE 4-19 THIRD PAGE IN HOME.....	66
FIGURE 4-20 FORTH PAGE IN HOME	66
FIGURE 4-21 FREQ ASKED QUESTION	67
FIGURE 5-1 SIGN-IN	76
FIGURE 5-2 SIGN-UP	76
FIGURE 5-3 ECAI CHAT PAGE	77
FIGURE 5-4 SIDEBAR.....	77
FIGURE 5-5 SUPPORT CENTER	78
FIGURE 5-6 CHAT PAGE	79
FIGURE 5-7 KEY CAMERA BEHAVIORS FOR AVATAR MOVEMENT	80
FIGURE 5-8 CLERK’S USER MANAGEMENT DASHBOARD.....	81
FIGURE 5-9 CLERK’S ANALYTICS DASHBOARD	81
FIGURE 5-10 MONGODB MANAGEMENT INTERFACE	82
FIGURE 5-11 WHISPER ARCHITECTURE	85
FIGURE 5-12 LOG-MEL SPECTROGRAM	89
FIGURE 5-13 MODEL FAMILY STRUCTURE OF QWEN.....	110
FIGURE 5-14 COMPONENT AND THEIR ROLES IN THE OVERALL PROCESS	115
FIGURE 5-15 THE WORKFLOW OF A RETRIEVAL-AUGMENTED GENERATION (RAG) SYSTEM.....	117
FIGURE 5-16 VECTOR DATABASE CODE	117

FIGURE 5-17 CREATE VECTOR DB	118
FIGURE 5-18 .FLOW OF QUERY EMBEDDING	120
FIGURE 5-19 FLOW OF RESPONSE GENERATION.....	121
FIGURE 5-20 TABLE EXAMPLE OF ZERO-SHOT PROMPTING	125
FIGURE 5-21 TABLE1 EXAMPLE OF FEW-SHOT PROMPTING	126
FIGURE 5-22 TABLE2 EXAMPLE OF FEW-SHOT PROMPTING	127
FIGURE 5-23 PROMPT WITH RAG FLOW	128
FIGURE 5-24 RESPONSE EXAMPLE OF RAG FLOW	128
FIGURE 5-25 RESPONSE EXAMPLE2 OF RAG FLOW	128
FIGURE 5-26 FIGURE PROMPT OF RAG FLOW WITH DEEPSEEK MODEL	129
FIGURE 5-27 RESPONSE EXAMPLE OF RAG FLOW WITH DEEPSEEK MODEL	129
FIGURE 5-28 (TEMPERATURE)HYPERPARAMETER SETTING.....	130
FIGURE 5-29 RESPONSE EXAMPLE1 WITH RAG FLOW WITH LLAMA 3.2	130
FIGURE 5-30 RESPONSE EXAMPLE2 WITH RAG FLOW WITH LLAMA 3.2.....	131
FIGURE 5-31 3D AVATAR	149
FIGURE 5-32 REACT COMPONENT (. JSX FILE) GENERATED BY GLTFJSX	153
FIGURE 5-33 SELECT ANIMATIONS FROM MIXAMO	155
FIGURE 5-34 DOWNLOAD ANIMATIONS FROM MIXAMO.....	155
FIGURE 5-35 CONVERT GLB AVATAR CHARACTER TO FBX USING BLENDER.....	157
FIGURE 5-36 IMPORT FBX ANIMATION INTO BLENDER.....	157
FIGURE 5-37 KEYFRAMED ARMATURE ANIMATION IN BLENDER.....	158
FIGURE 5-38 CHANGE NAME OF ANIMATION IN BLENDER	158
FIGURE 5-39 PUSH DOWN ACTION TO NLA STACK IN BLENDER.....	159
FIGURE 5-40 ANOTHER ANIMATION AND DELETE THE ADDED ARMATURE HIERARCHY IN BLENDER .	159
FIGURE 5-41 CHANGING NAME OF NEW ANIMATION AND PUSH DOWN TO NLA STACK IN BLENDER.	160
FIGURE 5-42 ALL ANIMATIONS INSIDE SINGLE ARMATURE IN NLA	160

List of tables

TABLE 3-1 RELATED WORK	48
TABLE 4-1 BLOCK DIAGRAM MODULE	72
TABLE 5-1 WHISPER DATASETS	87
TABLE 5-2 DATASETS ADVANTEGES	88
TABLE 5-3 WHY LOG-MEL SPECTROGRAM	90
TABLE 5-4 MULTITASKS BENEFITS	101
TABLE 5-5 REASONS FOR WHISPER'S ZERO-SHOT SUCCESS	104
TABLE 5-6 BENEFITS OF HAVING TIMESTAMPS DIRECTLY	106
TABLE 5-7 WELL-KNOWN LLMS	108
TABLE 5-8 REASONS TO PREFER QWEN-14B	109
TABLE 5-9 LLM VS RAG VS FINE-TUNING(17)(18)	115
TABLE 5-10 OTHER MODELS	120
TABLE 5-11 COMPARISON OF RAG FLOW	132
TABLE 5-12 COMPARISON OF EVALUATED TTS MODELS	147
TABLE 6-1 QUESTION SCORING TABLE FOR EVALUATION	234
TABLE 6-2 RESULTS SUMMARY FOR EVALUATION	235
TABLE 6-3 COMPARISON BETWEEN THE PROJECT AND CHAT GPT	236

Chapter 1

Introduction

Chapter One : Introduction

1.1 Introduction

The rapid advancement of artificial intelligence has opened new possibilities in education, transforming how knowledge is delivered and absorbed. *"The Design of eLearning Course Using the Power of AI"* introduces an innovative AI-powered virtual teaching assistant that brings interactive, personalized learning to students through natural conversations. This intelligent tutor, embodied as a responsive avatar, allows learners to ask questions via text or voice and receive accurate, curriculum-bound explanations in real time. By leveraging cutting-GTTS technologies like speech recognition (Whisper), text-to-speech synthesis, and retrieval-augmented generation (RAG), the system creates an engaging, tutor-like experience while strictly adhering to predefined subject matter—ensuring reliable, focused educational support. This project represents a meaningful step toward making AI-assisted learning more dynamic, accessible, and effective for students worldwide

1.2 Problem Definition

Given that:

1. Traditional educational methods often lack personalization, making it difficult to cater to the unique learning needs and preferences of individual students.
2. Many learners find traditional text-based or lecture-style learning environments less engaging, leading to reduced motivation and retention of information.
3. Accessibility remains a significant challenge in education, particularly for individuals with disabilities, limited literacy skills, or those who prefer auditory learning over text-based methods.
4. The rapid advancement of Artificial Intelligence (AI) technologies, such as speech-to-text, text-to-speech, and large language models, presents an opportunity to revolutionize the way educational content is delivered and consumed.
5. There is a growing demand for flexible, interactive, and on-demand learning solutions that can adapt to the fast-paced, technology-driven world.

It is required to:

1. Develop an AI-powered educational platform that functions as a virtual teacher assistant, providing learners with an interactive and personalized learning experience.
2. Enable learners to engage in natural, voice-based conversations with an AI-driven avatar, simulating real-time interaction with a knowledgeable tutor.
3. Ensure that the platform is accessible and user-friendly, allowing learners to simply select a subject, ask questions, and receive immediate, spoken responses.
4. Incorporate advanced AI tools, such as:

- Whisper for accurate speech-to-text transcription of the learner's questions.
 - GTTS for natural-sounding text-to-speech conversion of the avatar's responses.
 - Retrieval-Augmented Generation (RAG) systems to retrieve relevant information from a curated knowledge base and generate contextually appropriate responses.
 - Large Language Models (LLMs) to enhance the avatar's ability to understand and respond to complex queries.
5. Design the platform to operate within the boundaries of a predefined knowledge base, ensuring that all responses are accurate, relevant, and limited to the selected subject.
 6. Include a transcription feature that allows learners to view a text-based version of their conversation with the avatar, enabling them to review and revisit the information at their own pace.
 7. Create an intuitive and visually appealing interface that requires minimal technical expertise, making the platform accessible to users of all ages and backgrounds.

Such that:

1. The platform provides a highly engaging and interactive learning experience, encouraging active participation and fostering deeper understanding of the subject matter.
2. Learners receive accurate and relevant information tailored to their specific queries, ensuring a high-quality educational experience.
3. The platform is accessible to a wide range of users, including those with disabilities, limited literacy skills, or a preference for auditory learning.
4. The integration of advanced AI tools ensures a seamless and efficient learning process, with minimal delays or errors in transcription, response generation, and speech synthesis.
5. The subject-specific focus of the platform ensures that learners stay on track with their chosen subject, avoiding distractions or irrelevant information.
6. The platform empowers learners to take control of their education, providing them with the tools and resources they need to succeed in an increasingly digital world.

1.3 Proposed Solutions

The design of eLearning course using the power of AI project proposes a comprehensive, web-based platform that leverages advanced artificial intelligence (AI) technologies to create an interactive, voice-driven virtual teaching assistant. The solution is designed to address the challenges of personalized learning, accessibility, and educator workload by providing a scalable, user-friendly, and adaptable system. Below, we outline the key components and functionalities of the proposed solution.

1.3.1 Core Components

The proposed solution consists of the following core components:

1. Pre-Populated Knowledge Base:

- The platform comes with a pre-populated knowledge base containing high-quality, subject-specific learning materials (e.g., textbooks, lecture notes, and curated educational content).
- The system continuously updates and expands the knowledge base using trusted sources, ensuring that the content remains current and relevant.

2. AI-Driven Virtual Avatar:

- The platform features an AI-powered avatar that serves as a virtual teacher assistant.
- The avatar engages learners in natural, voice-based conversations, providing explanations, answering questions, and offering clarifications based on the knowledge base.

3. Voice-Based Interaction:

- Learners interact with the avatar using voice commands, making the platform accessible and intuitive.
- The system integrates **Whisper** for speech-to-text conversion and **GTTS** for text-to-speech synthesis, ensuring smooth and natural communication.

4. Retrieval-Augmented Generation (RAG) System:

- The platform uses RAG systems combined with **Large Language Models (LLMs)** to retrieve relevant information from the knowledge base and generate contextually accurate responses.

- This ensures that the avatar's answers are precise, relevant, and tailored to the learner's queries.

5. Subject-Specific Responses:

- The AI assistant is constrained by the knowledge base, ensuring that responses are limited to the subject matter.
- If a question falls outside the scope of the knowledge base, the system responds with a message like, *"I have no idea about what you are asking,"* maintaining relevance and focus.

1.3.2 Key Functionalities

The proposed solution offers the following functionalities:

1. Personalized Learning:

- The AI assistant adapts to the learner's pace and level of understanding, providing tailored explanations and responses.
- Learners can ask follow-up questions and explore topics in depth, fostering a deeper understanding of the subject matter.

2. Active Learning:

- The platform encourages learners to actively engage with the material by asking questions and thinking critically.
- The interactive nature of the system promotes higher retention rates and a more engaging learning experience.

3. Educator Support:

- By automating routine teaching tasks, such as answering frequently asked questions, the platform reduces the workload on educators.
- Educators can focus on higher-level instructional activities, such as lesson planning and student mentoring.

4. Accessibility:

- The voice-based interaction and real-time transcription features make the platform accessible to learners with diverse needs, including those with hearing impairments or language barriers.
- The system supports multiple languages, further enhancing its inclusivity.

5. Scalability and Adaptability:

- The platform is designed to be scalable, making it suitable for use in various educational settings, from individual learners to large classrooms.
- It can be adapted to different subjects, languages, and learning levels, ensuring its versatility and broad applicability.

1.3.3. Innovations

The proposed solution introduces several innovations that set it apart from existing systems:

1. **Integration of Advanced AI Tools:**

- The platform combines state-of-the-art AI tools, such as **Whisper**, **GTTS**, and **RAG systems**, to create a seamless and efficient learning experience.
- These technologies enable the system to understand and respond to learner queries with high accuracy and relevance.

2. **Focus on Subject-Specific Learning:**

- Unlike many existing solutions, the platform restricts responses to the scope of the pre-populated knowledge base, ensuring that interactions remain relevant and productive.
- This feature addresses the common issue of irrelevant or inaccurate answers in AI-driven educational tools.

3. **Voice-Driven Interaction:**

- The use of voice-based interaction makes the platform more intuitive and accessible, particularly for learners who struggle with traditional text-based interfaces.
- The integration of real-time transcription further enhances its usability and inclusivity.

4. **Scalable and User-Friendly Design:**

- The platform is designed to be scalable, user-friendly, and adaptable, making it accessible to a wide range of users, from individual learners to large institutions.
- Its web-based architecture ensures ease of use and broad accessibility.

1.3.4. Expected Outcomes

The proposed solution is expected to deliver the following outcomes:

- Enhanced personalized learning experiences for students, leading to improved engagement and comprehension.

- Reduced workload for educators, enabling them to focus on higher-level teaching activities.
- Increased accessibility and inclusivity, particularly for learners with disabilities or language barriers.
- A scalable and adaptable platform that can be used in diverse educational settings.

1.4 Statistics on AI in Education

1. Conversational AI and Chatbots: (1)

- **24/7 Access to Academic Assistance**

Unlike traditional office hours, chatbots are available around the clock, offering immediate support for homework help, concept clarification, and study guidance. This constant availability is particularly beneficial for students balancing education with other commitments.

- **Instant Feedback on Assignments**

Chatbots can evaluate student responses in real-time, providing immediate feedback that helps learners understand their mistakes and improve. This prompt feedback loop enhances the learning process and aids in better retention of information.

- **Streamlined Administrative Support**

Beyond academics, chatbots assist with administrative tasks such as course registration, deadline reminders, and answering frequently asked questions about campus services. This support reduces the administrative burden on students and allows them to focus more on their studies.

- **Enhanced Accessibility for Diverse Learners**

Conversational AI tools often include features like text-to-speech, language translation, and customizable interfaces, making education more accessible to students with disabilities or those who speak different languages.

- **Support for Educators**

By handling routine queries and administrative tasks, chatbots free up educators' time, allowing them to focus on personalized teaching and student engagement. This collaboration enhances the overall educational experience for students.

2. Personalized learning

Personalized learning is increasingly recognized for its potential to enhance educational outcomes by tailoring instruction to individual student needs. Here's an overview of recent statistics highlighting its impact:

- **Academic Achievement**

- **Improved Test Scores:** A study published in the *Journal of Educational Psychology* found that students participating in personalized learning programs scored an average of **30% higher on standardized tests** compared to peers in traditional classrooms. (2)
- **Broad Benefits Across Achievement Levels:** Research by RAND Corporation indicated that over half of students engaged in personalized learning surpassed their peers in both mathematics and reading, regardless of their initial achievement levels. (3)
- **Educator Perspectives**
 - **Positive Impact on Engagement:** According to a McKinsey & Company report, **76% of teachers** believe that personalized learning paths can significantly improve student engagement and academic performance. (4)
 - **Enhanced Teaching Efficiency:** Educators adopting personalized learning approaches have reported increased time for one-on-one instruction, allowing for more targeted support to students. (5)
- **Market Growth**
 - **Rapid Expansion:** The global personalized learning market was valued at approximately **\$6.99 billion in 2024** and is projected to reach **\$33.1 billion by 2033**, growing at a compound annual growth rate (CAGR) of **18.87%**. (6)
 - **AI Integration:** The integration of AI in personalized learning is also on the rise, with the market expected to grow from **\$6.5 billion in 2024** to **\$208.2 billion by 2034**, reflecting a CAGR of **41.4%**. (7)

These statistics underscore the growing recognition of personalized learning's benefits in enhancing student performance and engagement. As technology continues to evolve, personalized learning is poised to play an increasingly significant role in education.

If you're interested in exploring specific personalized learning tools or platforms suitable for your educational context, feel free to ask!

1.5 Statistics on the Impact of AI in Education

1. Student Adoption of AI Tools

- A 2024 global survey by the Digital Education Council found that **86% of students** use AI in their studies, with **54%** utilizing it weekly and **24%** daily. (8)
- The most common applications include:
 - Information search (69%)
 - Grammar checking (42%)

- Document summarization (33%)
 - Paraphrasing (28%)
 - Draft creation (24%) . (8)
- Despite widespread use, **58%** of students feel they lack sufficient AI knowledge, and **48%** feel unprepared for an AI-driven workforce.

2. AI in Higher Education

- AI usage in higher education has surged, with a 2025 report indicating a **2.3x increase** over the previous year. Furthermore, **93%** of institutional leaders anticipate even greater AI integration in the next two years. (9)
- A study at IU International University of Applied Sciences demonstrated that using the AI-powered assistant Syntea reduced students' study time by **27%**, highlighting AI's potential to accelerate learning through personalization.(10)

1.6 Documentation Organization

This documentation is structured into seven main chapters, each focusing on a key aspect of the project. Below is a summary of the content and purpose of each chapter:

- **Chapter Two: Project Planning**
This chapter outlines the initial planning phase of the project. It includes the project's timeline, task assignments, and scheduling tools such as the Task Sheet, Gantt Chart, and Network Diagram to ensure effective time and resource management.
- **Chapter Three: Literature Review**
This chapter investigates the foundational knowledge behind the project. It discusses the role of Artificial Intelligence in education, reviews Automatic Speech Recognition (ASR) models, explores AI tutors, and explains knowledge retrieval methods that support the intelligent system design.
- **Chapter Four: System Analysis and Design**
This chapter provides insights into the analysis and design of the system. For the analysis, it elaborates the functional and non-functional requirements presented in Unified Modeling Language (UML) diagrams. While for the design, an overview is given via system block diagrams and Graphical User Interface (GUI) designs. The system architecture illustrated using the Entity Relationship Diagram (ERD).
- **Chapter Five: System Implementation**
This chapter focuses on the implementation phase. It explains how the system components were developed and integrated, including the Whisper model for Automatic Speech Recognition (ASR), Qwen as the Large Language Model (LLM), and Google Text-to-Speech (gTTS) for speech synthesis. It also highlights the backend and frontend integration, Retrieval Augmented Generation (RAG) pipeline

development, avatar creation using Ready Player Me, and animation synchronization for seamless system interaction.

- **Chapter Six: Testing and Evaluation**

The results and performance of the system are evaluated in this chapter. It covers both system testing and usability testing. The effectiveness of the proposed system is analyzed and compared with existing models like GPT, with statistical data and performance analysis provided.

- **Chapter Seven: Conclusion and Future Work**

This chapter concludes the documentation by summarizing the capabilities and achievements gained throughout the project. It also outlines proposed enhancements and future development directions to improve the system further.

Chapter 2

Project Planning

Chapter Two: Project planning

2.1 Introduction

Project planning is presented using Microsoft Project Plan software to follow up activities of the project.

The project planning contains three outputs.

- Project task sheet.
- Gantt chart.
- Network diagram.

In this Chapter, we are going to divide the project into manageable tasks in section (2.2), Describe our project tasks in section (2.2.1), illustrate the work breakdown structure in section (2.2.2), present the task sheet in section (2.2.3), and we are going develop a primary schedule in section (2.3), present Gantt chart in section (2.3.1), present network diagram in section (2.3.2).

2.2 Project Manageable Tasks

Teamwork exists so that a group of individuals with a diverse set of skills and talents can work together to create something better than one could create on their own.



Figure 2-1 SDLC

Tasks are divided according to the Software Development Life Cycle (SDLC) which is a process used by the software industry to design, develop, and test high-quality software, The SDLC aims to produce high-quality software that meets or exceeds customer expectations, and reaches completion within times and cost estimates.

2.2.1 Tasks Description

The Work Breakdown Structure (WBS) task involves breaking down the project into smaller, manageable components and sub-tasks. This includes:

- **Defining Deliverables:** Identifying key outputs such as the platform, documentation, and demo.
- **Task Hierarchy:** Organizing tasks into a hierarchical structure (e.g., design, development, testing, deployment).
- **Resource Allocation:** Assigning team members and resources to each task.
- **Timeline Estimation:** Creating a timeline for task completion to ensure the project stays on track.

The WBS provides a clear roadmap for the project, ensuring all tasks are accounted for and executed efficiently.

2.2.1.1 Analysis Task

The analysis task involves evaluating the project's performance, impact, and areas for improvement. This includes:

- **Data Collection:** Gathering data on user interactions, system performance, and learning outcomes.
- **Performance Metrics:** Analyzing metrics such as response accuracy, user engagement, and system efficiency.
- **Feedback Review:** Reviewing feedback from educators and learners to identify strengths and weaknesses.
- **Recommendations:** Providing insights and suggestions for future enhancements or iterations.

The analysis task helps assess the project's success and guides future development efforts.

2.2.1.2 Design Task

The design task corresponds to the **Design Phase** of the SDLC. It involves planning the architecture, user experience, and visual elements of the platform. This includes:

- **System Architecture:** Designing the overall structure of the platform, including the knowledge base, AI models, and user interface.
- **User Experience (UX) Design:** Creating intuitive and user-friendly interfaces for educators and learners.

- **Visual Design:** Developing the visual elements, such as the AI avatar and platform layout, to ensure an engaging experience.
- **Prototyping:** Building mockups or prototypes to test and refine the design before implementation.
- **Technical Specifications:** Documenting the design decisions and technical requirements for developers.

The design task ensures the platform is both functional and visually appealing, meeting the needs of its users.

2.2.1.3 Implementation Task

The implementation task corresponds to the **Development Phase** of the SDLC. It involves building and integrating the core components of The design of eLearning course using the power of AI platform. This includes:

- **Coding:** Writing the code for the platform's front-end (user interface) and back-end (AI integration and knowledge base).
- **Integrating AI Tools:** Incorporating technologies like Whisper (speech-to-text), GTTS (text-to-speech), and RAG systems (retrieval-augmented generation).
- **Database Setup:** Creating and populating the knowledge base with subject-specific learning materials.
- **Voice Interaction:** Implementing voice-based communication between learners and the AI avatar.
- **API Integration:** Connecting the platform with external tools and services as needed.

The implementation task focuses on translating the design and requirements into a functional system.

2.2.1.4 Testing Task

The testing task aligns with the **Testing Phase** of the SDLC. It involves evaluating the platform's functionality, performance, and usability. This includes:

- **Unit Testing:** Testing individual components (e.g., speech-to-text, text-to-speech) to ensure they work as intended.
- **Integration Testing:** Verifying that all components (e.g., AI tools, knowledge base, user interface) work together seamlessly.
- **System Testing:** Ensuring the entire platform functions as a cohesive system.

- **User Acceptance Testing (UAT):** Conducting trials with educators and learners to gather feedback on usability and effectiveness.
- **Bug Fixing:** Identifying and resolving issues to improve the platform's reliability and performance.

The testing task ensures the platform meets quality standards and user expectations before deployment

2.2.1.5 Documentation Task

The documentation task involves creating a comprehensive and well-structured written record of the project. This includes detailing the project's objectives, design, implementation, and outcomes. Key deliverables include:

- **Project Overview:** A high-level summary of the project's purpose, features, and significance.
- **Technical Specifications:** Detailed descriptions of the technologies used (e.g., Whisper, GTTS, RAG systems).
- **User Guides:** Step-by-step instructions for educators and learners on how to use the platform.
- **Testing and Evaluation:** Documentation of testing procedures, results, and improvements made during development.
- **Future Work:** Suggestions for enhancements or additional features.

The documentation ensures that the project is well-documented for future reference, scalability, and knowledge sharing.

2.2.1.6 Presentation Task

The presentation task involves creating and delivering a compelling visual and verbal overview of the project. This includes:

- **Slideshow:** A visually engaging presentation highlighting the project's goals, features, technologies, and impact.
- **Key Points:** Clear and concise explanations of the problem being addressed, the proposed solution, and the results achieved.
- **Audience Engagement:** Tailoring the presentation to the audience (e.g., educators, developers, or stakeholders) to ensure clarity and relevance.
- **Q&A Preparation:** Anticipating and preparing for potential questions about the project's functionality, limitations, and future potential.

The presentation task aims to effectively communicate the project's value and innovation to a broad audience.

2.2.1.7. Demo Task

The demo task involves showcasing the functionality of The design of eLearning course using the power of AI platform in a live or recorded demonstration. Key aspects include:

- **Live Interaction:** Demonstrating how learners can interact with the AI avatar using voice commands and receive real-time responses.
- **Feature Highlights:** Showing key features such as voice-based interaction, real-time transcription, and subject-specific responses.
- **Use Cases:** Presenting real-world scenarios where the platform can be used, such as a student asking questions about a specific topic.
- **Technical Setup:** Briefly explaining the technologies powering the platform (e.g., Whisper, GTTS, RAG systems).
- The demo task provides a tangible and interactive way to showcase the platform's capabilities and potential impact.

2.3 Software tool for planning tasks (project plan 365)

1. Draw the action plan and represent it on both the Network Diagram and the Gantt chart.
 2. Allocate and organize resources for each activity.
 3. Follow-up progress of the project.
 4. Project budget management, and workload analysis.
 5. The possibility of programming work.
 6. As in all programs Microsoft Office package.
 7. Group of interconnected and interactive parts that performs an important job or task as a component of a larger system.
 8. A subsystem, while a system in itself, is also wholly contained within a larger system. The system would be subsystems, such as ERD, transactions, and GUI.
- **Project Plan:** According to package solution. Group of interconnected and interactive parts that performs an important job or task as a component of a larger system. A subsystem, while a system in itself, is also wholly contained within a larger system. The system would be subsystems, such as ERD, transactions, and GUI, [Figure 2.2] shows the project plan.
 - **Gantt chart:** a Gantt chart, commonly used in project management, is one of the most popular and useful ways of showing activities (tasks or events) displayed against time. On the left of the chart is a list of the activities and along the top is a suitable time scale. Each activity is represented by a bar; the position and length of the bar reflects the start date, duration and end date of the activity, [Figure 2.3] shows that the Gantt chart for system tasks.

	Task	Task Name	Duration	Start	Finish	Predecessors	Resource Names
1	✚	Project Scope and Research	4 wks	Thu 10/3/24	Wed 10/30/24		
2	✚	Understanding AI in eLearning	1 wk	Thu 10/3/24	Wed 10/9/24		All Students
3	✚	Identify Learning Objectives	3 wks	Thu 10/10/24	Wed 10/30/24	2	All Students
4	✚	Data Gathering and Analysis	6 wks	Fri 11/1/24	Thu 12/12/24		
5	✚	Audience Analysis	3 wks	Fri 11/1/24	Thu 11/21/24	3,2	Alaa
6	✚	Content Inventory	3 wks	Fri 11/22/24	Thu 12/12/24	3,2	Jessica
7	✚	AI-Powered Course Outline Generation	6 wks	Fri 12/13/24	Thu 1/23/25		
8	✚	Generative AI Tools	3 wks	Fri 12/13/24	Thu 1/2/25	6	Hassan,Omar A
9	✚	Quality Prompts	3 wks	Thu 1/2/25	Wed 1/22/25	6	Omar G
10	✚	Content Creation and Customization	6 wks	Mon 1/27/25	Fri 3/7/25		
11	✚	Module Design	3 wks	Mon 1/27/25	Fri 2/14/25	6	Ahmed,Zeinab
12	✚	Media Selection	3 wks	Thu 2/13/25	Wed 3/5/25	6	Amr
13	✚	User Experience (UX) Design	3 wks	Thu 3/6/25	Wed 3/26/25		
14	✚	Navigation Flow	1 wk	Thu 3/6/25	Wed 3/12/25	11	Jessica
15	✚	Accessibility	2 wks	Wed 3/12/25	Tue 3/25/25	12	Alaa
16	✚	Integration of AI Features	4 wks	Wed 3/26/25	Tue 4/22/25		
17	✚	Adaptive Learning:	2 wks	Wed 3/26/25	Tue 4/8/25	14	Hassan
18	✚	Recommendation Systems	2 wks	Tue 4/8/25	Mon 4/21/25	15	Omar A
19	✚	System documentation	4 wks	Wed 4/23/25	Tue 5/20/25	16	All Students
20	✚	Final project presentation	1 wk	Wed 5/21/25	Tue 5/27/25	17	All Students

Figure 2-2 project plan

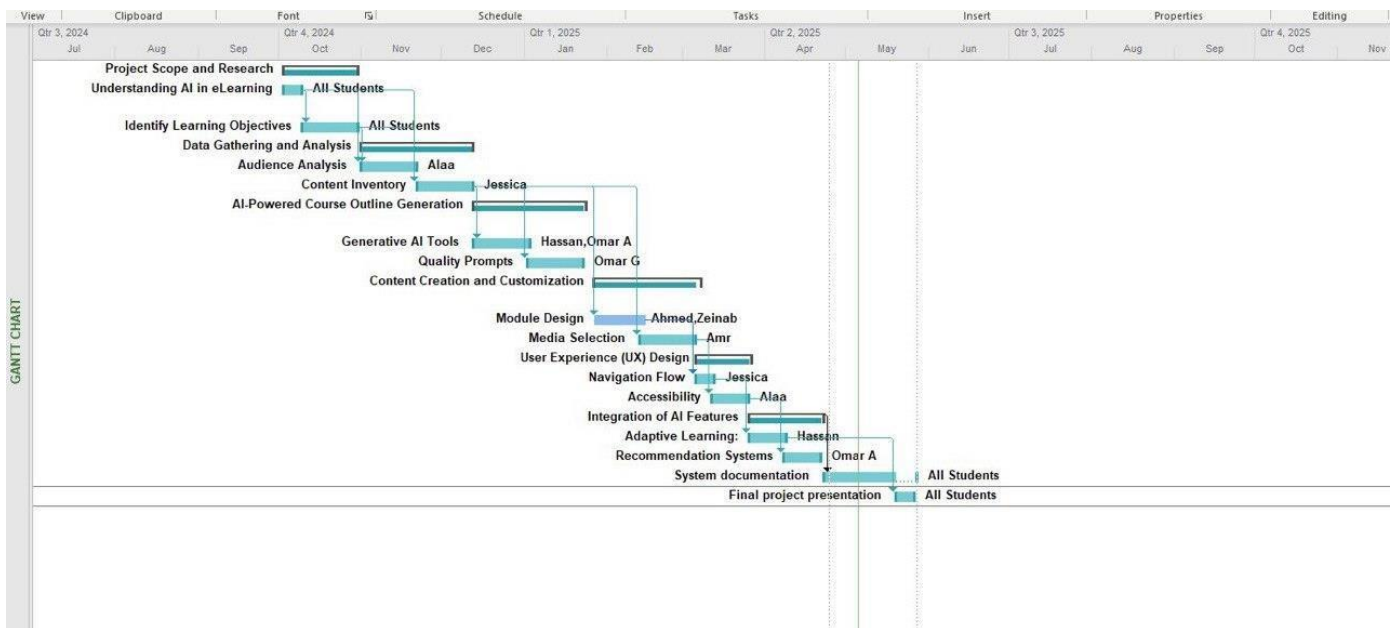


Figure 2-3 Gannt Chart

- **Network diagram** for the project showing estimated times for each activity and the earliest and latest expected completion time for each activity as shown in [Figure2-4] and [Figure2-5] .

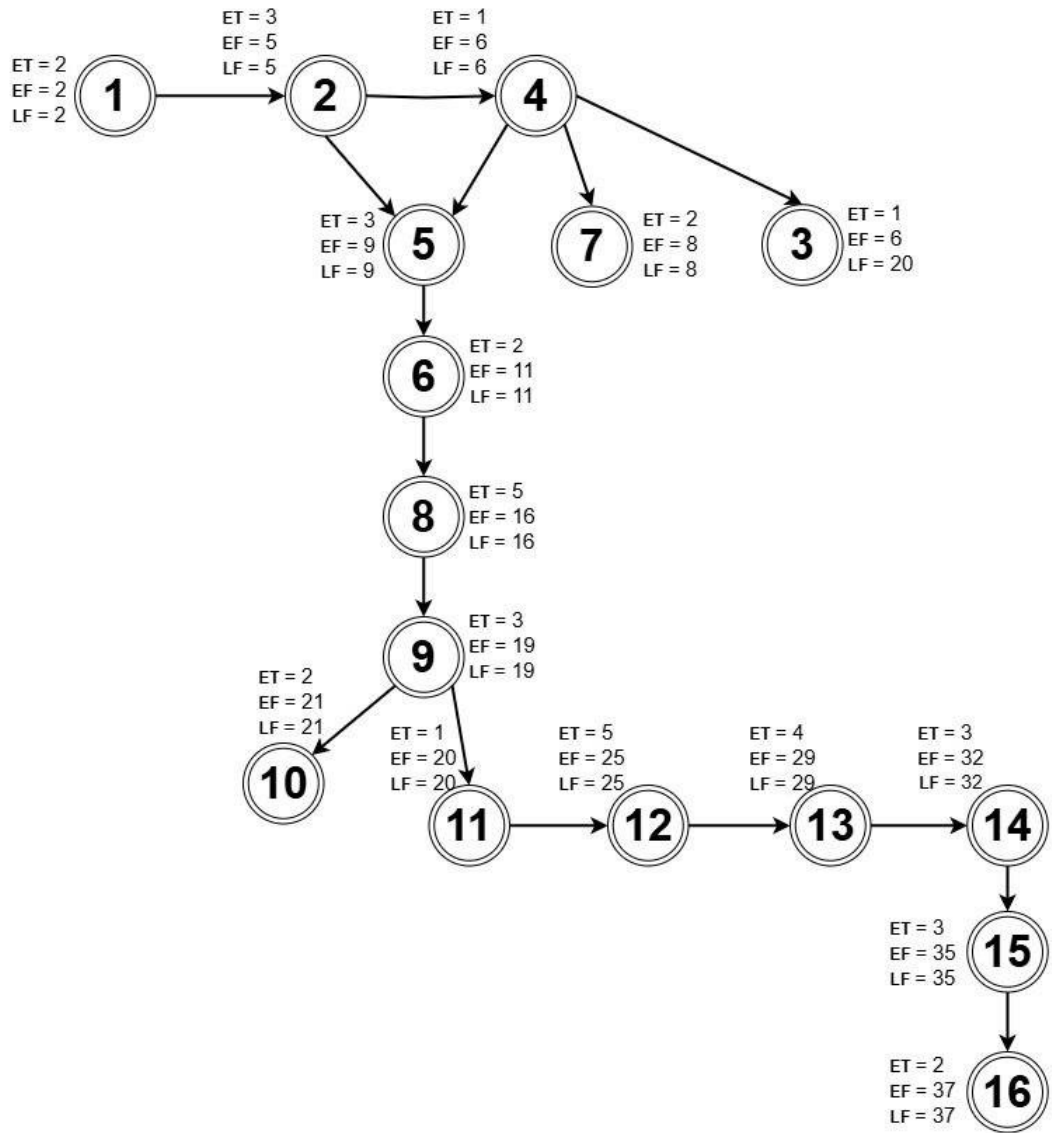


Figure 2-4 Network diagram (Nodes)

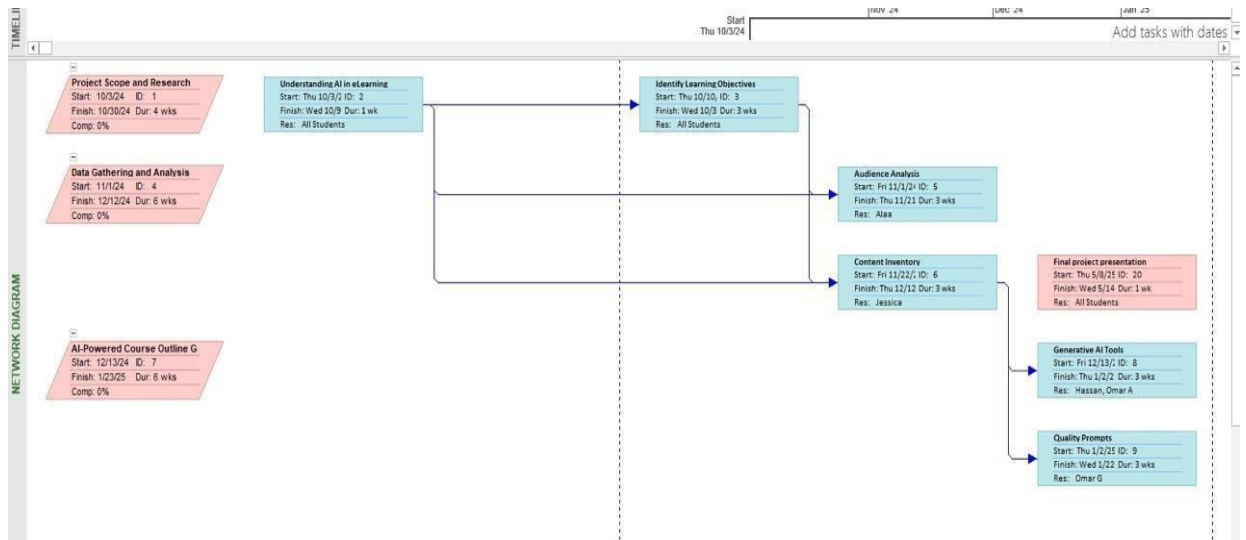


Figure 2-5 Network diagram



Figure 2-5 Network diagram

Chapter 3

Literature Review

Chapter Three: Literature Review

3.1 Introduction

The integration of **Artificial Intelligence (AI)** in education has transformed traditional learning methods, enabling personalized, interactive, and accessible learning experiences. This literature review examines key research and existing technologies related to **AI-powered virtual teacher assistants, voice-based learning systems, and knowledge retrieval methods** to contextualize the proposed solution.

3.2. AI in Education: A Historical Perspective

AI has evolved from simple rule-based tutoring systems (e.g., **ELIZA**, 1966) to advanced **Large Language Models (LLMs)** like GPT-4 (OpenAI, 2023).

Key milestones include:

- **Intelligent Tutoring Systems (ITS)** (1980s–2000s): Systems like **AutoTutor** (Graesser et al., 2001) used natural language processing (NLP) to simulate human tutors.
- **Adaptive Learning Platforms** (2010s): Tools like **Knewton** and **DreamBox** leveraged AI to personalize learning paths.
- **Conversational AI in Education** (2020s): AI chatbots (e.g., **Khanmigo**, **Duolingo's AI**) now provide real-time tutoring.

Relevance to Our Work: Modern AI tutors combine **adaptive learning** with **natural language interactions**, aligning without virtual teacher assistant.

3.3. ASR models

3.3.1 What is Automatic speech recognition (ASR) models?

Automatic speech recognition (ASR) is a technology that converts spoken language into written text. It uses complex algorithms and machine learning techniques to analyze audio signals and transcribe them into text. ASR systems are designed to recognize and interpret human speech, making it possible to convert spoken words into a format that computers can process and understand.

The development of ASR can be traced back to the mid-20th century, when researchers began exploring methods to automate speech recognition. Early systems relied on pattern matching and acoustic modeling techniques, exhibiting limited accuracy and vocabulary coverage. However, with the advent of machine learning and computational advancements, the automatic speech recognition model underwent a significant evolution.

The introduction of Hidden Markov Models (HMMs) in the 1970s marked an important moment for ASR, enabling more robust speech recognition capabilities.

Recently, speech recognition has changed a lot with the use of deep learning and neural network-based models. Not too long ago, the prevailing model for speech recognition was wave2text. This technology allowed for the conversion of spoken words into text, enabling a range of applications from virtual assistants to transcribing voice recordings. However, as technology continues to advance at a rapid pace, new models and approaches have emerged.

3.3.2 How does Automatic speech recognition (ASR) work?

ASR operates through a multi-step process that involves capturing, processing, and interpreting spoken language. The following steps outline the fundamentals of the ASR process:

- **Audio input.** The process begins with the capture of human speech through a microphone or any audio recording device. The audio input is then digitized to create a digital representation of the spoken words.
- **Preprocessing.** The digitized audio undergoes preprocessing, which involves filtering out background noise, normalizing the audio levels, and segmenting the speech into smaller units for analysis.
- **Feature extraction.** During this phase, the system extracts acoustic features from the preprocessed audio, such as Mel-frequency cepstral coefficients (MFCCs) or spectrograms. These features serve as the input for the subsequent recognition stage.
- **Speech recognition.** The extracted features are fed into a speech recognition model that employs machine learning algorithms, such as deep neural networks and Hidden Markov Models (HMMs), to match the acoustic features with linguistic units and generate the corresponding textual output.
- **Language modeling.** In this step, language models are utilized to enhance the accuracy of recognizing spoken words by considering the context and grammar of the language being spoken.
- **Output generation.** Finally, the recognized speech is transcribed into written text, which can be further processed for various applications, such as generating subtitles, transcribing meetings, or enabling voice commands for devices.

3.3.3 Key components of automatic speech recognition (ASR)

The development of ASR involves several key components that work together to accurately process and interpret spoken language by using the following components:

1) Speech signal processing

Speech signal processing is fundamental to ASR systems. It converts analog audio signals into digital form for computer analysis. Techniques like digitization, signal filtering,

and feature extraction capture speech characteristics like phonemes and intonation. This conversion into a computationally analyzable format sets the stage for further ASR steps like acoustic modeling and language processing.

2) Acoustic modeling techniques

Acoustic modeling plays an important role in ASR by capturing the relationship between speech signals and the corresponding phonetic units. Techniques such as Hidden Markov Models (HMMs) and neural networks are commonly used for acoustic modeling. HMMs are particularly effective in representing the temporal and sequential nature of speech, while neural networks, including deep learning architectures, have shown remarkable capability in learning complex patterns and variations in speech signals. These techniques enable ASR systems to recognize and differentiate between different phonemes and spoken sounds, contributing to the accuracy and robustness of speech recognition.

3) Language modeling

Language modeling is essential for understanding the structure and context of spoken language. It involves the use of statistical models, such as n-grams, as well as more advanced approaches, like neural language models, to predict the likelihood of word sequences and phrases in a given language. By incorporating linguistic knowledge and contextual information, language models enhance the accuracy of ASR systems in deciphering spoken sentences and inferring the intended meaning behind the words.

4) Decoding algorithms

Decoding algorithms are responsible for determining the most likely sequence of words that correspond to a given input speech signal. Techniques such as Viterbi decoding and beam search are commonly used to efficiently search through the space of possible word sequences and identify the most probable interpretation of the input speech. These algorithms are crucial for aligning the acoustic and language models, resolving ambiguities, and producing accurate transcriptions or commands based on the input speech.

3.3.4 Challenges and limitations face the (ASR) models

1) Background noise

A major challenge for ASR is dealing with background noise. In noisy places like crowded areas, industrial settings, or vehicles, ASR accuracy can decline. Background noise, echoes, and other acoustic interferences make it difficult for ASR to accurately recognize and transcribe speech in such environments.

2) Speaker variations and accents

Another significant challenge is the diversity of speakers and accents encountered in everyday communication. ASR systems may struggle to accurately transcribe speech from individuals with different dialects, accents, or speech impediments. The variations in

pitch, intonation, and pronunciation among speakers also pose a considerable challenge for ASR technology. It must adapt to accurately interpret a wide range of vocal characteristics.

3) Contextual ambiguity

Contextual ambiguity presents a substantial challenge for ASR, particularly in understanding and interpreting natural language. Human speech often relies on context, tone, and non-verbal cues to convey meaning, which can be challenging for ASR systems to accurately interpret. Ambiguous phrases, homophones, and colloquial expressions further complicate the task of accurate speech recognition and understanding.

4) Handling different languages

Recognizing and understanding different languages and dialects is not easy. Multilingual ASR faces difficulties because each language has its own sounds, sentence structures, and word meanings. To work well, ASR needs special models and algorithms designed for each language, making it accurate in recognizing and writing down spoken words.

5) Limitations in real-time applications

ASR encounters challenges in how quickly it processes information and responds in real-time applications like live transcription, virtual assistants, and voice-controlled devices. To be useful, ASR needs to quickly and accurately change spoken words into text. But it is difficult to keep up

with the fast pace of real-time speech. Although ASR has gotten better over the years, there is still room for improvements.

Despite these challenges, the ongoing research and development efforts aim to improve the technology and enhance its robustness and effectiveness. Nowadays, advancements in machine learning, neural networks, and data collection methodologies are contributing to overcoming some of these challenges. Several recent models have already overcome certain challenges.

3.3.5 Evaluation metrics for Speech Recognition models (ASR)

1) Word error rate in ASR models

In ASR, the most common metric for evaluating model accuracy is the word error rate (**WER**). WER is calculated using the **Levenshtein Distance concept**, which measures the differences between two strings (in this case, the predicted transcript produced by the ASR model and a human-labeled transcript). WER is defined as the number of errors divided by the total number of words in the ground truth.

There are three types of errors: substitutions, insertions, and deletions. Let's examine each one:

Substitution happens when a word gets replaced with another word (for example, “had” gets replaced with “bad”)

Insertion happens when a word that was not said is added (for example “She is playing chess” becomes “She is always playing chess”)

Deletion happens when a word is left out (for example, “you need to talk now” becomes “you need”)

Given a model prediction and a ground truth transcript, we perform an edit distance alignment to identify the positions of substitution, insertion, and deletion errors. We then sum these errors and divide the total by the number of words in the ground truth.

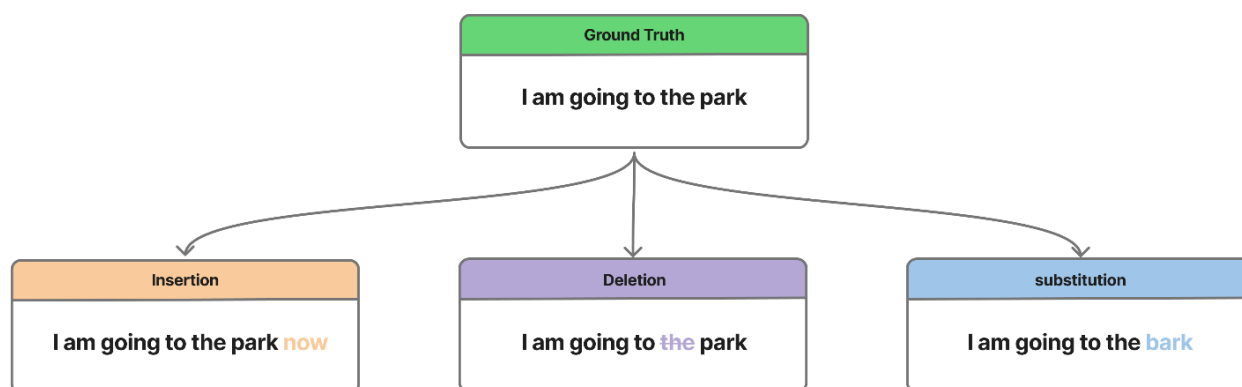


Figure 3-1 example on WER

The sum of these counts is then divided by the total number of words in the ground-truth transcript to get the **word error rate**. When represented as a percentage, WER corresponds to the number of errors an ASR model can be expected to make per 100 words spoken.

2) Character error rate in ASR models

The Character Error Rate (**CER**) metric quantifies the difference between the system's predicted text output, such as transcribed speech or extracted text from images, and the correct reference text. It calculates the minimal number of character insertions, deletions, and substitutions required to transform the predicted text into the reference text.

For instance, in a speech-to-text system, if a single letter in a complex medical term is incorrectly transcribed, the character error rate metric accurately captures this specific error more precisely than word-level metrics.

This level of precision is crucial for teams working on tonal or script-based languages, where single-character changes can dramatically alter the meaning

Calculate the Character Error Rate Metric (CER)

The basic formula for the Character Error Rate metric is:

- $CER = (I + D + S) / N$

Where:

- I = Number of character insertions
- D = Number of character deletions
- S = Number of character substitutions
- N = Total number of characters in the reference text

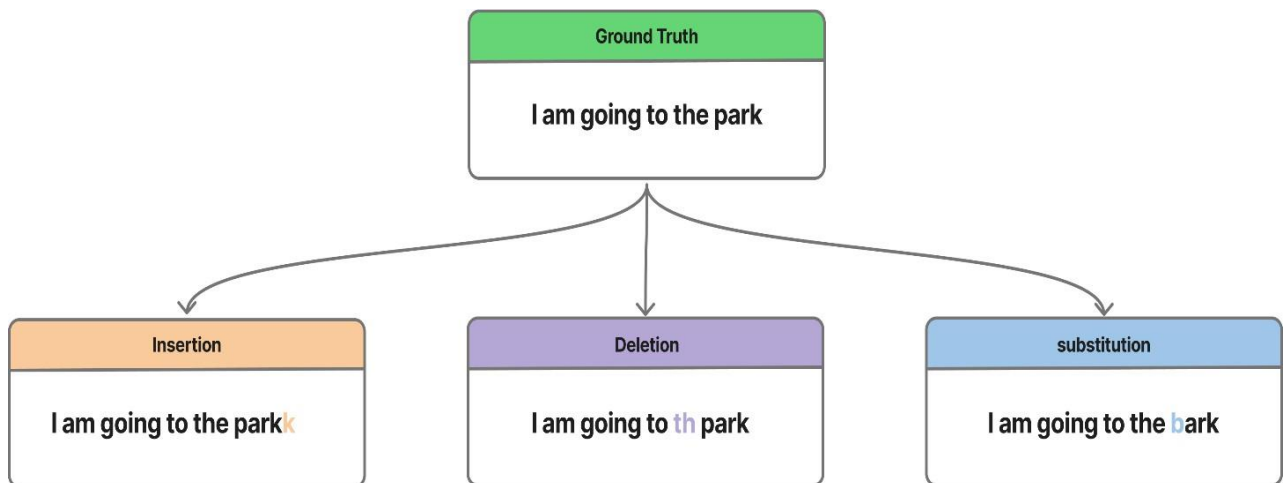


Figure 3-2 example on WER

For example, given:

- Reference text: " I am going to the park "
- Predicted text: " I am going to th barkk "

While this may seem like a minor glitch, from a character error rate metric perspective, each incorrect character contributes to the error rate. The algorithm would identify:

- 1 insertion ('k' in " park")
- 1 deletion ('e' in " the ")
- 1 substitution ('b' in " park ")
- Total characters in reference: 17

Therefore: $CER = (1 + 1 + 1) / 17 = \mathbf{0.176}$ or **17.6%**

Character Error Rate (CER) vs. Word Error Rate (WER)

The **Character Error Rate (CER)** metric evaluates text accuracy at the character level, whereas **Word Error Rate (WER)** assesses accuracy at the word level. The character error rate metric is particularly effective when small errors have significant consequences, such as in coding tasks, formal documents, or specialized jargon.

In contrast, WER is more suitable for evaluating broader semantic elements, like the coherence of sentences in a chatbot or the fluency of phrases in machine translation.

Each metric is valuable in different contexts. For example, a single missing character in a product name can cause confusion in an e-commerce listing; in this case, the character error rate metric's precision effectively captures the error.

3.3.6 Speech recognition Architectures

For a better understanding, let's explore some of the most influential models for speech recognition.

1. Hidden Markov Models (HMM)

Hidden Markov model is widely used in speech recognition due to its ability to represent the time-varying aspect of speech signal. The origin of HMM is the famous Markov chain of probability theory which can be used for sequential modelling. HMM is a finite state automaton which has a finite number of states through which the machine makes transition from one state to another state. It also has a finite set of input and output symbols. When a machine is at states S at time Instant T , depending upon the input symbol provided, it will transit to another state emitting a certain output with a certain probability at the next time instant. Four factors are associated.

2. Recurrent neural networks (RNN)

A Recurrent Neural Network (RNN) is a class of artificial neural networks where connections between nodes form a directed graph along a temporal sequence. This structure gives RNNs temporal dynamic behavior, making them suitable for tasks such as speech recognition, language modeling, and text generation.

Unlike feedforward neural networks, RNNs can use their internal state (memory) to process sequences of inputs, which makes them powerful for sequential data analysis. RNNs can connect previous information to the present task. However, they often suffer from problems like vanishing or exploding gradients, which can make training over longer sequences challenging.

Advances such as LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Units) models have addressed these issues to some extent, refining the utility and performance of RNNs in practical applications.

3.3.6.1 How Hidden Markov Model (HMMs) work?

1 - Observations (Input Layer)

In the provided review paper, what you refer to as Observations are formally called **Visible States**. They represent the tangible and measurable "output symbols" that the system generates over time. As defined in the paper and its accompanying Figure 1, the set of all possible observations is denoted by **V**, with individual observations labeled as **V1**, **V2**, and **V3**. The paper explains that while the system moves through its internal hidden states, it is "emitting a certain output with a certain probability," and this directly observable output is the visible state. This sequence of visible data is the only evidence an observer has about the system's internal workings, making it the fundamental input for analysis.

2 - Hidden States (Hidden Layer)

The concept of **Hidden States** represents the unobservable internal structure of the system, which the paper describes as the core of the Hidden Markov Model. These states, denoted as a set by **H** and individually in Figure 1 as **H1**, **H2**, and **H3**, form an underlying sequence that captures the "time-varying aspect" of a process like speech. The paper explains that the system "makes transition" from one hidden state to another over time, but an external observer cannot directly see which state the system occupies at any given moment. This unseeable sequence must be inferred from the visible states that are produced at each step, making this the "hidden" component of the model.

3. Transitions and Emissions

The process by which the system forms its output is governed by two distinct sets of probabilities: **Transitions** and **Emissions**. The paper defines **transition probability**, notated as a_{ij} , as the likelihood that the system will move from one hidden state (H_i) to another (H_j). This governs the internal dynamics, as illustrated by the example, " a_{12} is the state transition probability from hidden state H_1 to H_2 ." In parallel, the **emission probability**, notated as b_{jk} , connects the hidden states to the visible observations by defining the likelihood of emitting a specific visible state (V_k) while the system is in a particular hidden state (H_j). The paper formalizes this as $P(V_k | H_j) = b_{jk}$ and gives the example that " b_{11} is the emission probability of the hidden state H_1 if it emits visible state V_1 ." Together, these two mechanisms define the complete probabilistic behavior of the model.

3.3.6.2 The main HMMs concepts and functions

1 - Baum-Welch Algorithm

HMMs can be trained using the Baum-Welch algorithm, which is an iterative method that estimates the parameters of the HMM given a training set of observed sequences. These parameters can then be used to make predictions about future observations given a sequence of previous observations.

The Baum-Welch algorithm is based on the principle of expectation-maximization (EM), a popular technique for estimating parameters in probabilistic models.

The Baum-Welch algorithm consists of two main steps: the expectation (E) step and the maximization (M) step. In the E step, the algorithm computes the expected counts of each hidden state and transition given the observed data and the current estimates of the model parameters. In the M step, the algorithm uses these expected counts to compute new estimates of the model parameters that maximize the likelihood of the observed data given the current estimates of the model.

The Baum-Welch algorithm is an iterative process, where the E and M steps are repeated until the model parameters converge to a local maximum of the likelihood. The algorithm starts with an initial estimate of the model parameters and iteratively refines these estimates until convergence, resulting in an HMM that best explains the observed data.

2 - Forward-Backward Algorithm

The forward-backward algorithm is a widely used algorithm for solving the inference problem in Hidden Markov Models (HMMs). It provides an efficient way to calculate the likelihood of a sequence of observations given the model parameters and to estimate the hidden states that generated these observations.

The forward-backward algorithm consists of two steps: the forward step and the backward step.

The forward step starts from the first observation and calculates the forward probability, which is the probability of observing the first observation and being in a specific state at time $t=1$. It then uses this forward probability to calculate the forward probability of observing the second observation and being in a specific state at time $t=2$, and so on until the end of the sequence. The forward step can be seen as calculating the probability of observing the entire sequence given the model parameters and starting from each possible state.

The backward step starts from the last observation and calculates the backward probability, which is the probability of observing the rest of the sequence given that we are in a specific state at the end. It then uses the backward probability to calculate the probability of observing the rest of the sequence given that we are in a specific state at time $t-1$, and so on until the beginning of the sequence. The backward step can be seen as calculating the probability of observing the rest of the sequence given the model parameters and ending in each possible state.

Finally, the forward-backward algorithm combines the forward and backward probabilities to estimate the most likely hidden states that generated the sequence of observations. The probabilities of being in each state at each time step can be calculated

using the forward and backward probabilities and the transition probabilities between states.

The forward-backward algorithm is widely used in speech recognition, gene expression analysis, and other applications that involve modeling sequences of observations with hidden states. It provides a flexible and efficient solution for solving the inference problem in HMMs.

3 - Viterbi Algorithm

The Viterbi algorithm is a dynamic programming algorithm used to solve the maximum likelihood decoding problem for hidden Markov models (HMMs). The algorithm is named after its creator, Andrew Viterbi, who developed it in 1967.

The purpose of the Viterbi algorithm is to find the most likely sequence of hidden states that generated a given observation sequence (called the “decoding task”). Given an observation sequence and an HMM, the algorithm iteratively updates the probability of being in each state at each time step based on the observation and the transition probabilities between states. The final output is the sequence of hidden states with the highest overall probability.

The Viterbi algorithm starts by computing the initial probabilities of being in each state given the first observation. It then iteratively updates the probabilities of being in each state at each time step by considering all possible previous states and choosing the state with the highest probability. This process continues until the final time step is reached and the algorithm outputs the final sequence of states with the highest overall probability.

The Viterbi algorithm has many applications in natural language processing, speech recognition, and bioinformatics, among others. It is fast and efficient and has become a standard tool for solving maximum likelihood decoding problems for HMMs.

3.3.6.3 HMMs advantages and disadvantages

1 - Advantages

One of the key advantages of HMMs is that they allow for modeling the time-dependence of the data. This means that the probabilities of the hidden states are influenced by the previous hidden states. This makes HMMs well-suited for modeling sequences of data that have a temporal structure, such as speech signals or sequences of words in a sentence.

Another advantage of HMMs is that they are flexible in the sense that they can handle missing or partially observable data. This is because the hidden states are not directly observable, but instead are inferred from the observed sequence.

2 - Disadvantages

There are some disadvantages of HMMs, such as their assumption of Markovian dynamics and their sensitivity to the choice of initialization parameters. However,

despite these limitations, HMMs have proven to be powerful models for a wide range of sequential data problems.

3.3.6.4 How Recurrent Neural Networks (RNNs) work?

Recurrent Neural Networks (RNNs) are specialized neural networks designed to process sequential data by maintaining a form of memory over time. They are composed of three main layers:

1. Input Layer:

Receives the sequence data one element at a time. For example, in text processing, each word or character is fed into the network at its corresponding time step as a vector .

2. Hidden Layer:

This is the core of the RNN. It processes both the current input and the previous hidden state to compute the new hidden state. This hidden state serves as the memory of the network, allowing it to retain relevant information from earlier time steps.

3. Output Layer:

Produces an output at each time step based on the hidden state.

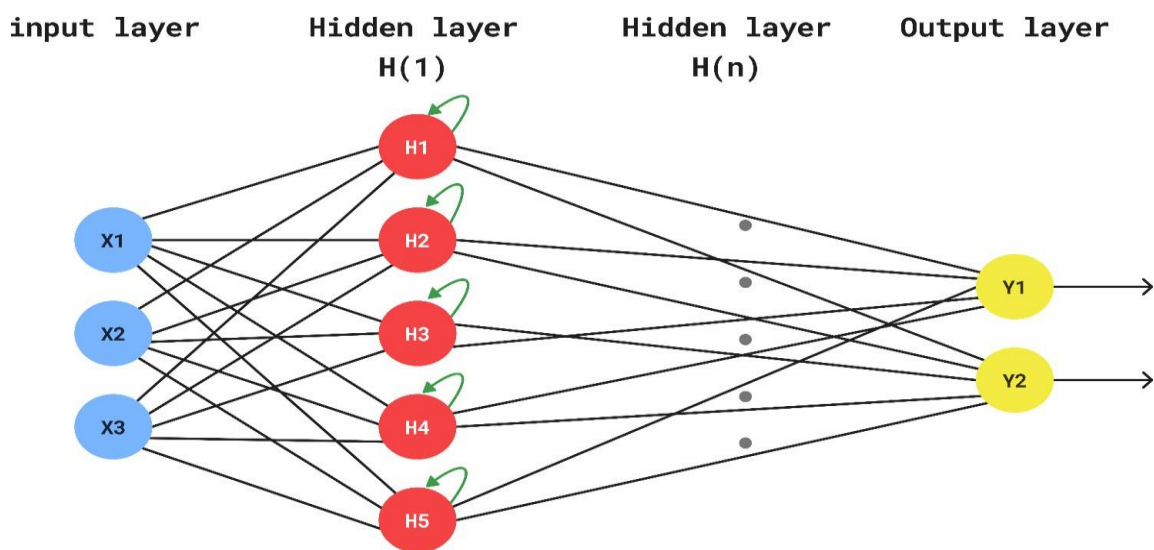


Figure 3-3 Recurrent Neural Networks

$X_1 \rightarrow X_n$ its represent the input values , $H_1 \rightarrow H_n$ its represent the Hidden layers processes , $Y_1 \rightarrow Y_n$ its represent the output values that processed via hidden layers.

3.3.6.5 The main RNNs concepts and functions

Activation Function

An activation function is a mathematical function applied to the output of a neuron. It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data. Without this non-linearity feature, a neural network would behave like a linear regression model, no matter how many layers it has.

Activation function decides whether a neuron should be activated by calculating the weighted sum of inputs and adding a bias term. This helps the model make complex decisions and predictions by introducing non-linearities to the output of each neuron.

The common types of Activation function is **RELU** , **Sigmoid** , and **Tanh**.

Gradient Descent :

Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks. It trains machine learning models by minimizing errors between predicted and actual results.

Training data helps these models learn over time, and the cost function within gradient descent specifically acts as a barometer, gauging its accuracy with each iteration of parameter updates. Until the function is close to or equal to zero, the model will continue to adjust its parameters to yield the smallest possible error. Once machine learning models are optimized for accuracy, they can be powerful tools for artificial intelligence (AI) and computer science applications.

Loss function :

A loss function, often referred to as a cost function, serves as a critical metric in evaluating the performance of our models.

It quantifies the disparity between predicted outputs and actual values. The purpose of a loss function is to measure how well a model's predictions align with the true values in the dataset.

3.3.6.6 RNNs Advantages and Disadvantages

1 - Advantages :

- RNN architecture is designed in such a way that it can process inputs of any length. Even with the input size growing larger, the model size does not increase.
- The weights of all the dependent hidden layers in between can be shared across the time steps.
- An RNN model is modeled to remember each information throughout the time which is very helpful in any time series predictor.

2- Disadvantages :

- Due to its recurrent nature, the computation becomes slow.

- Training of RNN models can be very difficult and time-consuming as compared to other Artificial Neural Networks.
- It becomes very difficult to process sequences that are very long if the activation functions used are ReLU or tanh as activation functions.
- Prone to problems such as *exploding* and *gradient* vanishing.
- RNNs cannot be stacked into very deep models
- RNNs are not able to keep track of long-term dependencies.

3.4 What is a Large Language Model?

LLMs are AI systems used to model and process human language. They are called “large” because these types of models are normally made of hundreds of millions or even billions of parameters that define the model's behavior, which are pre-trained using a massive corpus of text data.

The underlying technology of LLMs is called transformer neural network, simply referred to as a transformer . As we will explain in more detail in the next section, a transformer is an innovative neural architecture within the field of deep learning.

Presented by Google researchers in the famous paper Attention is All You Need in 2017, transformers are capable of performing natural language (NLP) tasks with unprecedented accuracy and speed. With its unique capabilities, transformers have provided a significant leap in the capabilities of LLMs. It’s fair to say that, without transformers, the current generative AI revolution wouldn’t be possible.(12)

The Rise and Rise of A.I. Large Language Models (LLMs) & their associated bots like ChatGPT

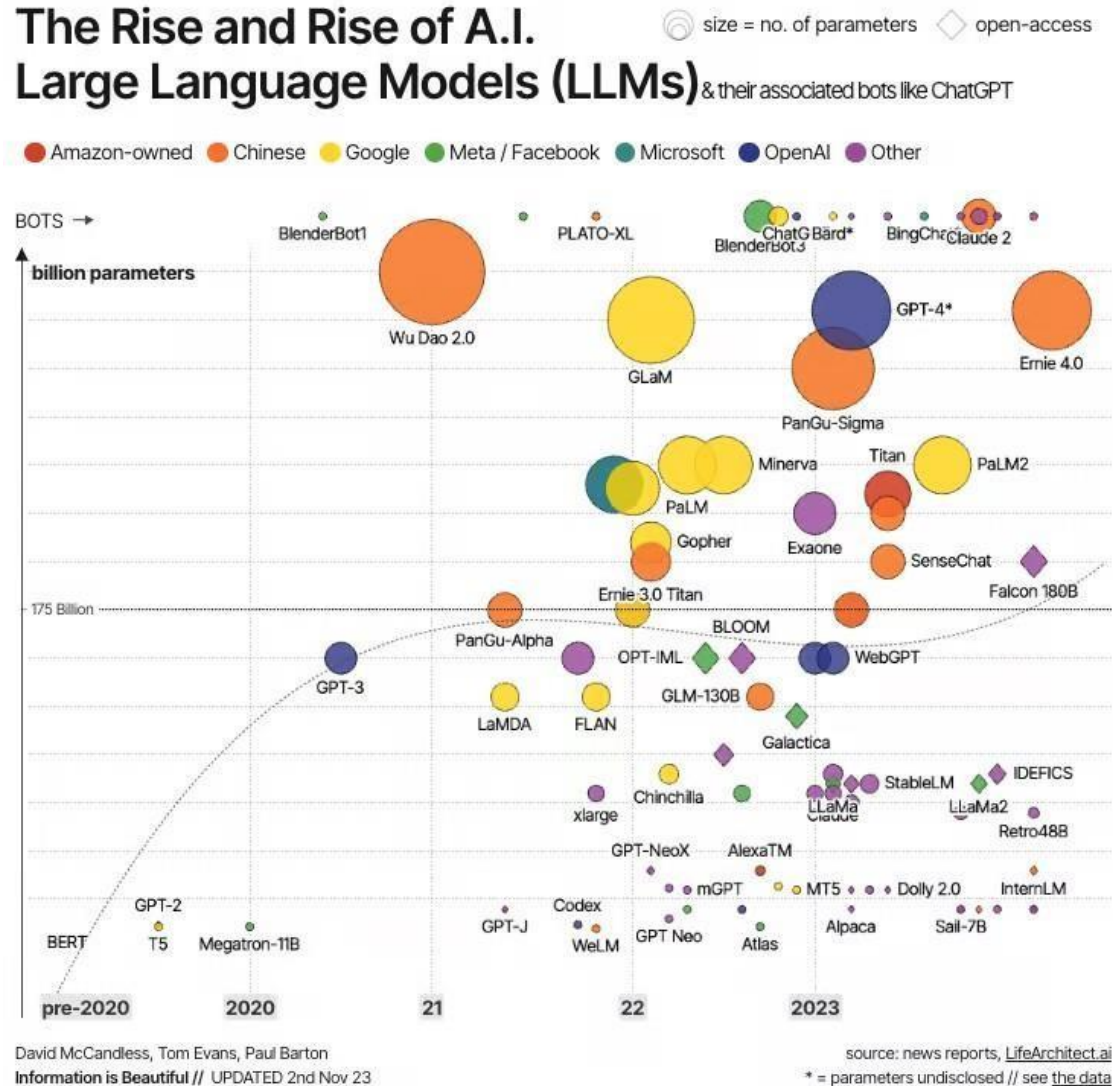


Figure 3-4 The rise of LLM Models

This evolution is illustrated in the graph above. As we can see, the first modern LLMs were created right after the development of transformers, with the most significant examples being BERT –the first LLM developed by Google to test the power of transformers–, as well as GPT-1 and GPT-2, the first two models in the GPT series created by OpenAI. But it’s only in the 2020s that LLMs become mainstream, increasingly bigger (in terms of parameters), and hence more powerful, with well-known examples like GPT-4 and LLaMa. (13)

3.5 Knowledge Retrieval & AI Tutors

Effective AI tutors rely on **accurate knowledge retrieval**:

- **Retrieval-Augmented Generation (RAG)** (Lewis et al., 2020): Combines retrieval from knowledge bases with LLM-generated responses.
- **Wolfram Alpha**: A computational knowledge engine for factual queries.

- **IBM Watson Discovery:** Retrieves structured/unstructured data for AI responses.

Our Approach: We use **RAG + curated knowledge bases** to ensure **subject-specific accuracy**.

3.6 Text To Speech (TTS)

3.6.1 What is TTS?

Text-to-Speech (TTS) technology is a system designed to convert written text into spoken audio. It enables machines to generate human-like speech by processing input text, analyzing its structure, and producing corresponding sound waves. TTS, also referred to as speech synthesis, is the process of converting written text into spoken words. **The primary goal of TTS is to make digital content accessible through auditory means**, bridging the gap between written information and users who prefer or require audio output. This technology is driven by AI algorithms and combines linguistic, mathematical, and auditory elements. TTS is a technology quietly revolutionizing how we interact with the digital world.

3.6.2 Importance and Applications

TTS technology has become an indispensable technology across various sectors. It enables machines to generate human-like speech, turning silent text into conversations, instructions, and stories. TTS bridges accessibility gaps, empowering visually impaired users or language learners.

Common applications include:

- Virtual assistants like Siri, Alexa, and Google Assistant to deliver spoken responses. These devices utilize both speech synthesis and recognition technologies to provide a conversational interface.
- Accessibility tools that help visually impaired individuals by converting text into audible speech. TTS systems are vital for creating accessible content, enabling individuals with disabilities to engage with written material.
- Content creation for audiobooks, automated news reading, and synthetic voices for podcasts and videos. An audiobook can entertain you while cooking dinner.
- Navigation systems to dictate directions. Your GPS navigates traffic with clear, turn-by-turn directions.
- Language learning apps. TTS can help language learners.
- Customer service bots and interactive voice response (IVR) systems. Automated systems can handle inquiries through voice.
- Education to enhance the learning process for students with reading difficulties, limited study time, younger pupils struggling with reading, or those with autism and emotional problems.

- Entertainment, such as storytelling apps, virtual reality, and gaming environments.
- Healthcare for articulating medication instructions or communicating alerts.
- Marketing and Branding for creating unique brand voices or powering automated customer service.
- Integration into various platforms, from mobile apps to smart home devices, enhancing user experience. Your smart speaker wakes you with a weather update. Your phone reads aloud calendar reminders.

TTS is crucial in enhancing accessibility and user interaction across various sectors. By the end, you'll understand why TTS isn't just a tool—it's the heartbeat of modern voice-driven experiences.

3.6.2 TTS vs Speech Recognition

Speech recognition and speech synthesis are two pivotal technologies in artificial intelligence, each serving distinct yet complementary roles in enhancing human-computer interaction.

3.6.2.1 Key Differences Between TTS And SR

While both technologies serve to bridge the gap between humans and machines, they operate in opposite directions. **Speech synthesis takes written text and converts it into spoken language, while speech recognition interprets spoken language and converts it into text.** Understanding the nuances between speech synthesis and speech recognition is essential for developers and users alike.

3.7 Limitations of Existing Solutions

Despite advancements, challenges persist:

1. **Overgeneralization:** Most AI tutors (e.g., ChatGPT) lack **subject boundaries**, leading to unreliable answers.
2. **Limited Voice Interaction:** Few platforms integrate **seamless voice-based learning**.
3. **Accessibility Gaps:** Many tools lack **transcription** or **multimodal support** for diverse learners.

How We Address These:

- **Strict knowledge boundaries** (prevents off-topic responses).
- **Voice-first design** (enhances accessibility).
- **Conversation transcripts** (supports review and retention).

3.8 Related Work and Similar Technologies

We explore six prominent AI-driven learning platforms, contrasting how each enhances student outcomes through unique capabilities and identifying where they fall short. By examining their key differentiators—such as personalization, interactivity, depth, and accessibility—you’ll gain insight into which tools are best suited for various educational contexts.

1. Khanmigo (Khan Academy AI)

Khanmigo is Khan Academy's AI-powered tutor designed to assist students with interactive Q&A and guided learning. It personalizes support using AI but is restricted to Khan Academy’s content and lacks voice interaction features.

2. Quizlet with AI

Quizlet’s AI enhances traditional flashcard-based studying by offering smart recommendations and adaptive learning tools. However, its use of advanced AI models like LLMs is limited, and its primary focus remains on memorization.

3. DeepSeek Chat

DeepSeek Chat integrates large language models with retrieval-augmented generation (RAG) to deliver context-aware, accurate answers in an educational setting. While promising, the platform is relatively new and currently supports a limited range of topics.

4. Socratic by Google

Socratic uses AI to help students solve homework questions by providing step-by-step explanations and relevant resources. It is helpful for quick answers but lacks deeper personalization and only handles relatively simple academic queries.

5. ChatGPT in Education

ChatGPT offers flexible, conversational AI that can support a wide range of subjects through real-time Q&A. Its adaptability makes it a powerful learning assistant, though it doesn’t provide structured lessons or detailed progress tracking by default.

6. Code.org

Code.org offers beginner-friendly, block-based programming environments enhanced by AI tools to support learning code logic. It focuses on accessibility and visual engagement but doesn’t incorporate advanced LLM features or voice interactivity.

Here's a refined and more descriptive information of each entry in the table, explaining its context, capabilities, and limitations in a more engaging and professional way:

#	Name	link	Advantages	Disadvantages
1	Khanmigo (Khan Academy AI)	khanmigo.ai	- Uses AI for tutoring. - Interactive Q&A.	- Limited to Khan Academy content. - No voice-enabled features.
2	Quizlet with AI	quizlet.com	- AI-powered study tools. - Help to recover info.	- No advanced RAG or LLM integration.
3	DeepSeek Chat	deepseek.com	- Smart AI that combines memory and search.	- Limited to specific use cases. - May generate inaccurate or misleading answers. - Take more time to be up to date.
4	Socratic by Google	socratic.org	- Instant homework help. - Uses AI for explanations.	- Limited to basic queries. - No personalized learning paths.
5	ChatGPT in Education	openai.com/chatgpt	- Versatile for various subjects. - Interactive Q&A.	- Limited to specific use cases. - May generate inaccurate or misleading answers. - Take more time to be up to date.
6	Code.org	code.org	- Block-based Coding. - Visual coding interface.	- No voice-enabled features. - No advanced RAG or LLM integration.

Table 1 related work

Chapter 4

System Analysis & System Design

Chapter Four: System Analysis and Design

4.1 Introduction

In this chapter, we illustrate the system requirements in section (4.2), functional requirements in section (4.2.1), non-functional requirements in section (4.2.2), system analysis basic concepts in section (4.3), functional view in section (4.3.1), maintain basic data in section (4.3.1.1), issue query subsystem in section (4.3.1.2), generate report in section (4.3.1.3), matrix report in section (4.3.1.4), perform transaction in section (4.3.1.5), context diagram in section (4.3.2), data flow diagram in section (4.3.3), data dictionary in section (4.3.4), data store in section (4.3.4.1), data flow in section (4.3.4.2), element description form in section (4.3.4.3), process specifications in section (4.3.4.4), system scenario in section (4.3.5), system features in section (4.4), and GUI free hand sketches in section (4.5).

4.2 System Requirements

4.2.1 Functional Requirements

The system must fulfill the following user and technical functions:

1. Accept user queries via text input or voice recordings (live or pre-recorded).
2. Validate queries to ensure they are database-related (SQL, NoSQL, schema design, etc.).
3. Reject off-topic queries with an error message: “This question is unrelated to databases. Please ask about SQL, NoSQL, or database design.”
4. Retrieve answers from a pre-loaded knowledge base (PDFs, documentation) using RAG (Retrieval-Augmented Generation).
5. Refine answers using an LLM (e.g., GPT-4) for clarity and readability.
6. Display answers as text in the chat interface.
7. Provide optional voice output via TTS synchronized with an animated avatar.
8. Log interactions (queries, responses, accuracy metrics) for system improvement.
9. Support multilingual queries (if non-English database resources are added).
10. Allow admins to update the knowledge base (add/remove documents).

4.2.2 System Specifications using UML Use Case Diagram

- In many design processes, the use case diagram is the first tool that designers utilize when starting a project.

- This diagram allows for the specification of high-level user goals that the system must fulfill.
- These goals are not necessarily tasks or actions but can represent more general required functionality of the system.
- More formally, a use case comprises a set of scenarios.
- Each scenario is a sequence of steps that describes an interaction between a user and the system.

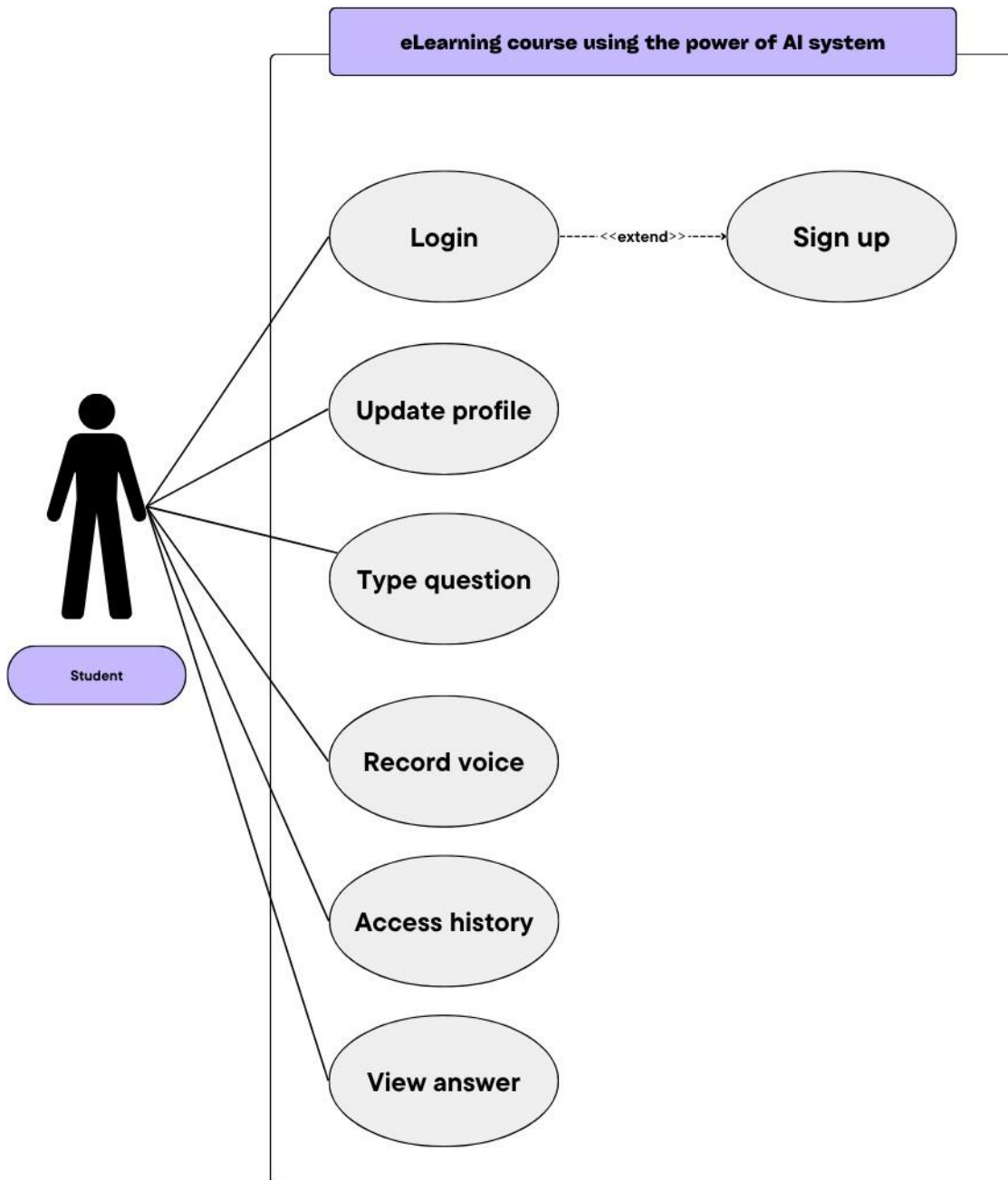


Figure 4-1 Usecase diagram

4.2.3 System Specifications using UML Activity Diagrams

- Ovals for start/end points
- Rectangles for actions
- Diamonds for decisions
- Clear, labeled transition arrows

Sign up Activity Diagram

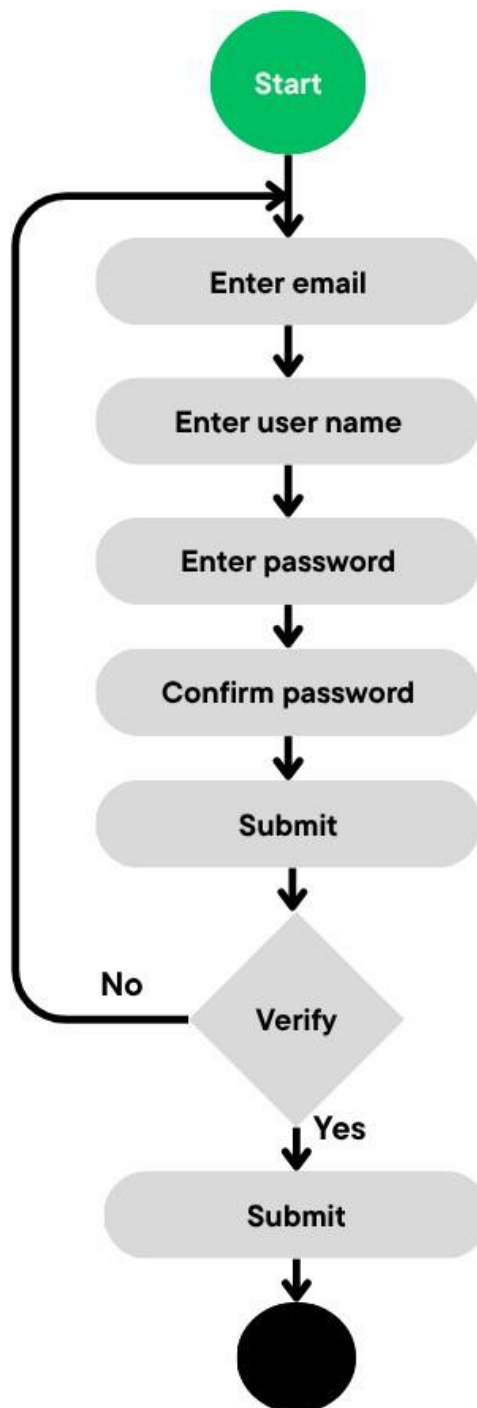


Figure 4-2 sign up activity diagram

Login Activity Diagram

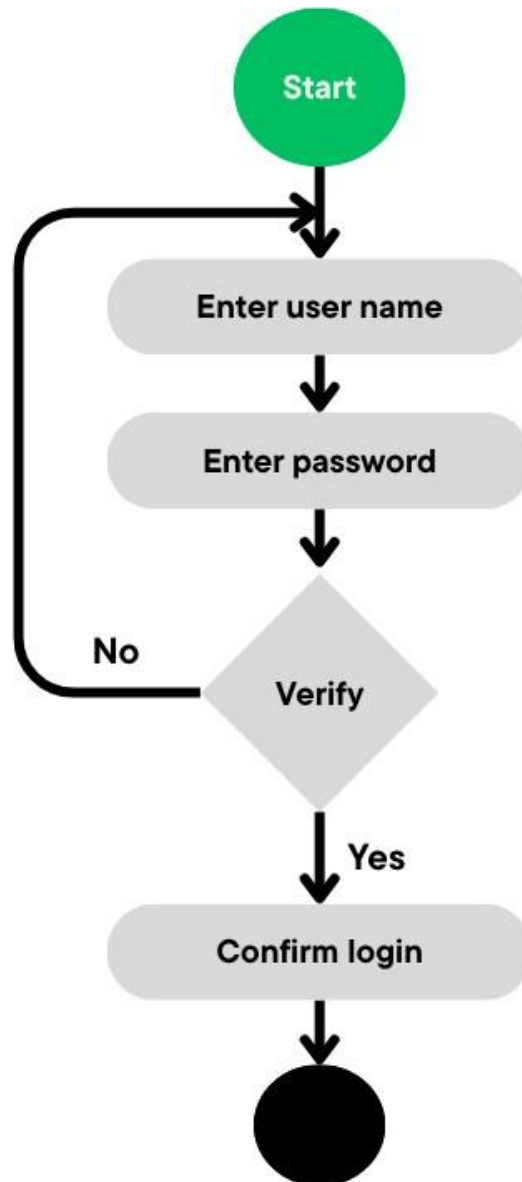


Figure 4-3 Login activity diagram

Edit profile info Activity Diagram

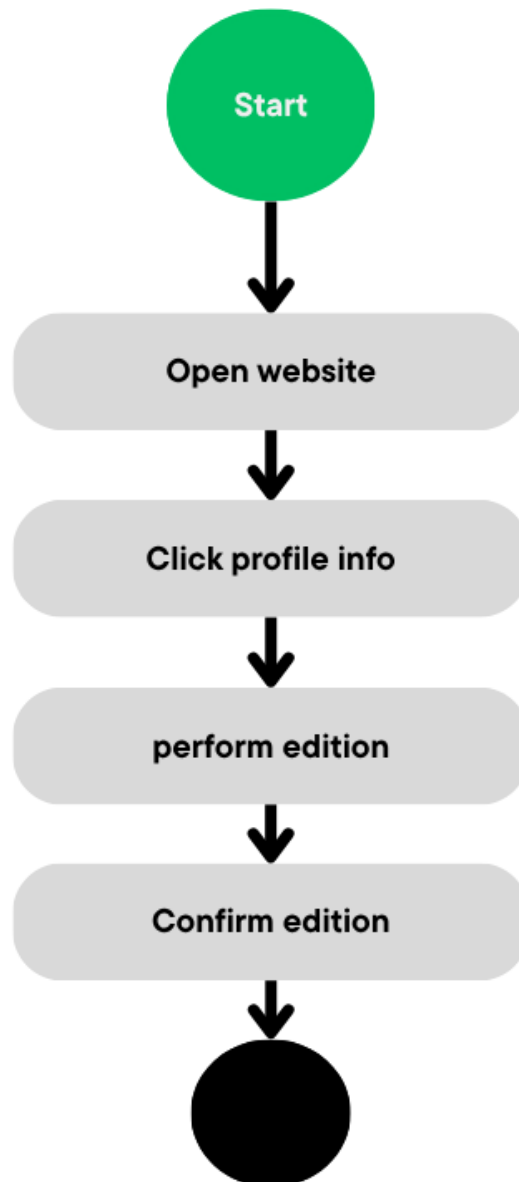


Figure 4-4 edit profile activity diagram

Type question Activity Diagram

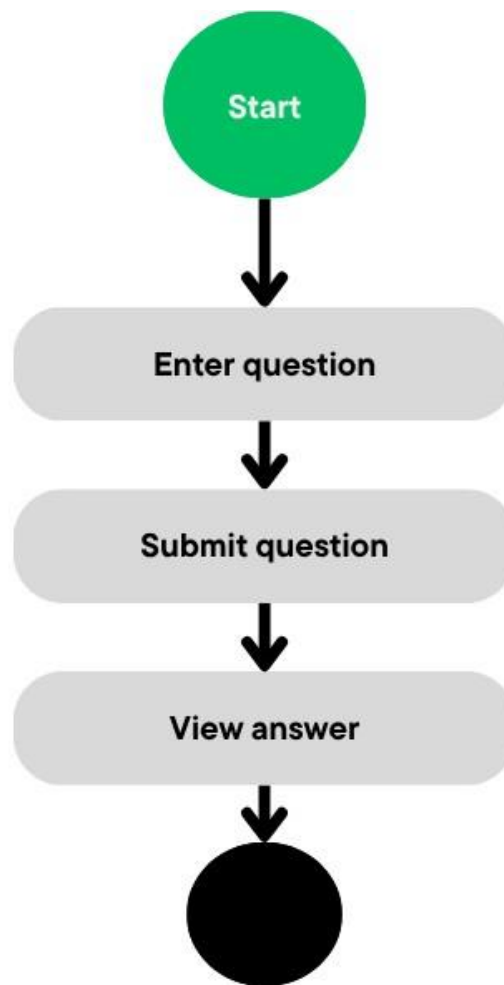


Figure 4-5 type question activity diagram

Record voice Activity Diagram

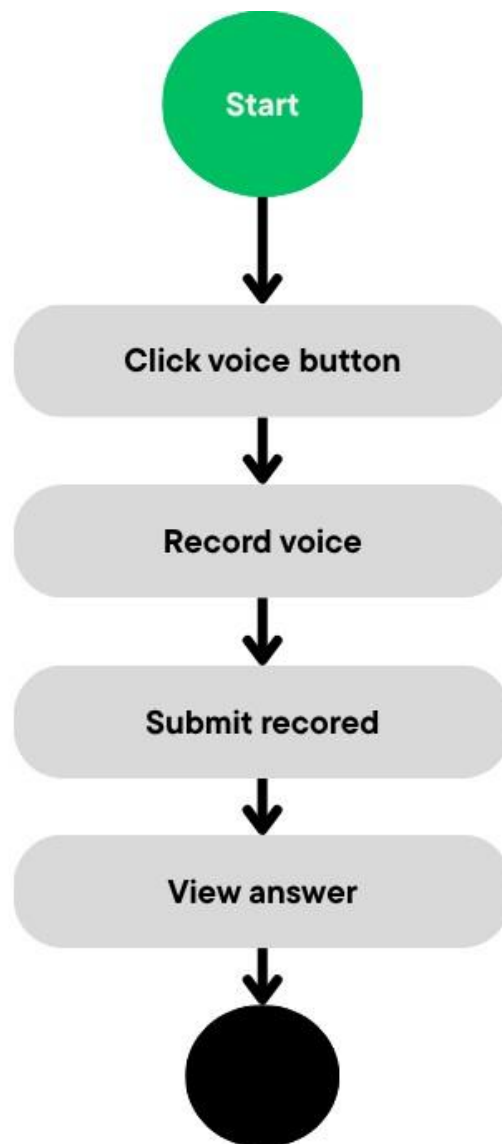


Figure 4-6 record voice activity diagram

Access history Activity Diagram

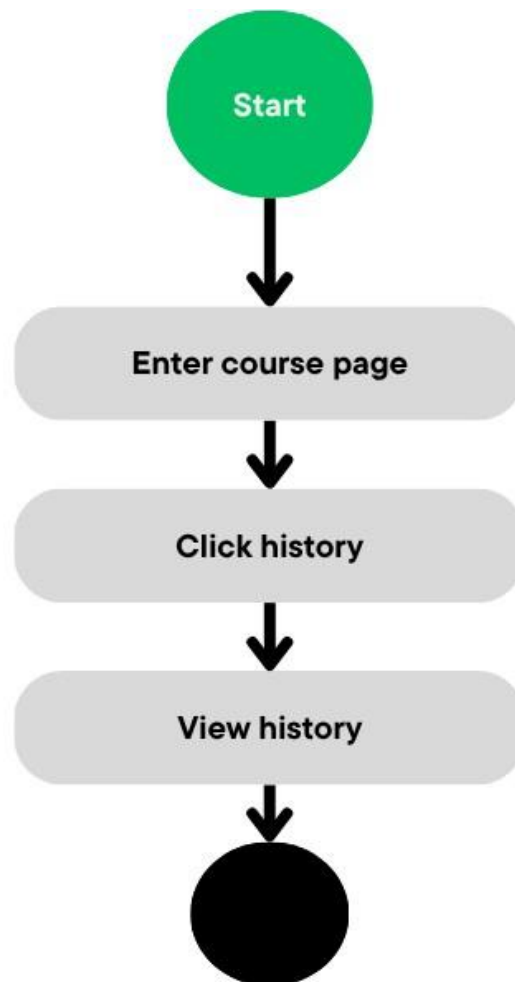


Figure 4-7 access history activity diagram

4.2.4 Nonfunctional Requirements

1. **Reliability:** System uptime $\geq 99\%$.
2. **Performance:**
 - Text query response time < 2 seconds.
 - Voice query response time < 5 seconds (including Whisper processing).
3. **Accuracy:**
 - Whisper speech-to-text accuracy $\geq 90\%$ for clear audio.
 - RAG answer relevance $\approx 94\%$

4.3 System Analysis Basic Concept

System analysis involves decomposing the chatbot into components to ensure it:

- Strictly answers **database-related questions**.
- Processes voice/text inputs efficiently.
- Delivers accurate, LLM-refined answers via multi-modal outputs.

4.3.1 Function View

4.3.1.1 Maintain Basic Data

Stores the following data:

- **Users** (User ID, Query History, Voice Preference)
- **Knowledge Base** (Document ID, Title, Content Type [PDF/Docx])
- **Logs** (Session ID, Query Text, Response Accuracy, Timestamp)

4.3.1.2 Issue Query Subsystem

1. **Input Handler:**

- Routes voice queries → Whisper → Text.
- Directly accepts text queries.

2. **Query Validator:**

- Checks if the question is **database-related** using keyword/NLP filters.

3. **RAG Processor:**

- Searches **knowledge base** for relevant answers.

4. **LLM Refiner:**

- Improves answer readability.

5. **Output Generator:**

- Displays text + optional TTS with Avatar animation.

4.3.1.3 Generate Report

Automated reports for admins:

1. **Daily Query Volume:** Total/rejected queries.

2. **Accuracy Metrics** : 100% of correct RAG retrievals.
3. **User Feedback**: Ratings (if implemented).

4.3.1.5 Perform Transaction

- **Success Path**: Query → validation → Answer → User.
- **Error Path**: Query → validation → Rejection → Error Message.

4.3.2 Data Dictionary

4.3.2.1 Data Store

Knowledge base:

- **Knowledgebase**: Stores database documentation (PDFs, SQL scripts).
- **User Sessions**: Tracks user interactions and preferences.

4.3.2.2

Data Flow

Voice Query:

- **Source**: User microphone/upload → Whisper model.

4.3.2.3 Process Specifications

Query Validation :

- **Input**: Raw text , or if voice ; converted to text then displayed on validate function that pick if related or not to database fields.
- **Output**: "Accept" (DB-related) or "Reject" (other).

4.3.3 System Scenario

1. User asks: "How to JOIN tables in PostgreSQL?"
2. System validates → Accepts (DB-related).
3. RAG extracts answer → LLM formats it:

SELECT * FROM users

JOIN orders ON users.id = orders.user_id;

4. Answer displayed as text + spoken by avatar with TTS .

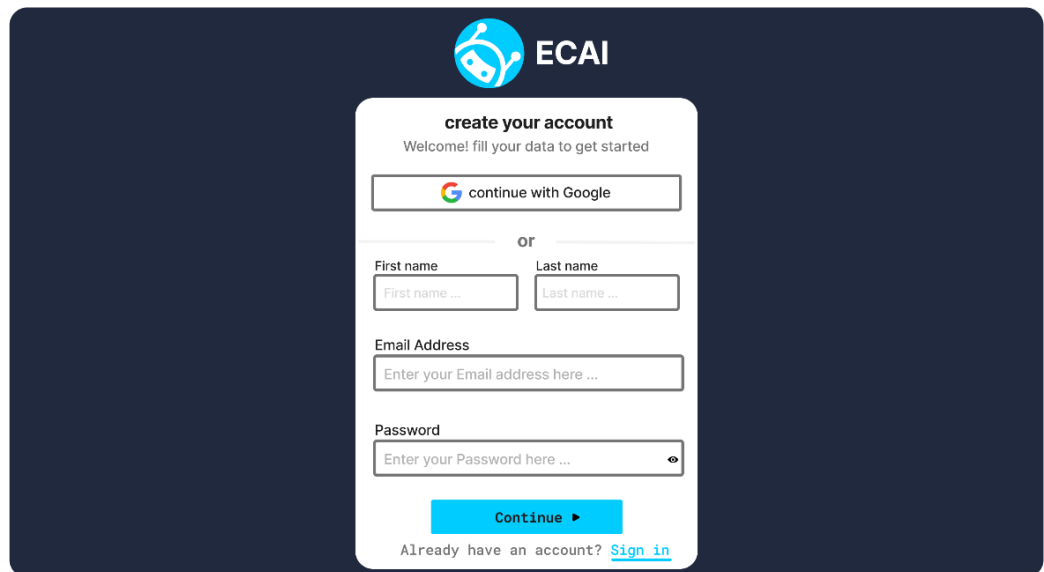
4.4 System Features

1. **Multi-Modal Interaction:** Voice/text input + animated avatar.
2. **Strict Database Focus:** Rejects off-topic questions.
3. **Admin Dashboard:** Query analytics, knowledge base management.
4. **Real-Time Lip Syncing:** Avatar mimics speech movements.

4.5 Graphical User Interface (GUI)

4.5.1 System GUI

- Textbox at the bottom for typing questions.
- Microphone button for voice input (right side of the textbox)
- Animated avatar (top-right corner)
- Left sidebar showing previous chats (user questions + bot responses)
- "+ New Chat" button (top-left)



The image shows a sign-up form for 'ECAI' (Emotionally Connected AI). The form is titled 'create your account' and includes the instruction 'Welcome! fill your data to get started'. It features a 'continue with Google' button, followed by an 'or' separator. Below this are input fields for 'First name' and 'Last name', an 'Email Address' field, and a 'Password' field. A blue 'Continue' button is at the bottom, with a link to 'Sign in' for existing users.

ECAI

create your account
Welcome! fill your data to get started

[continue with Google](#)

or

First name

Last name

Email Address

Password

[Continue](#)

Already have an account? [Sign in](#)

Figure 4-8 sign up

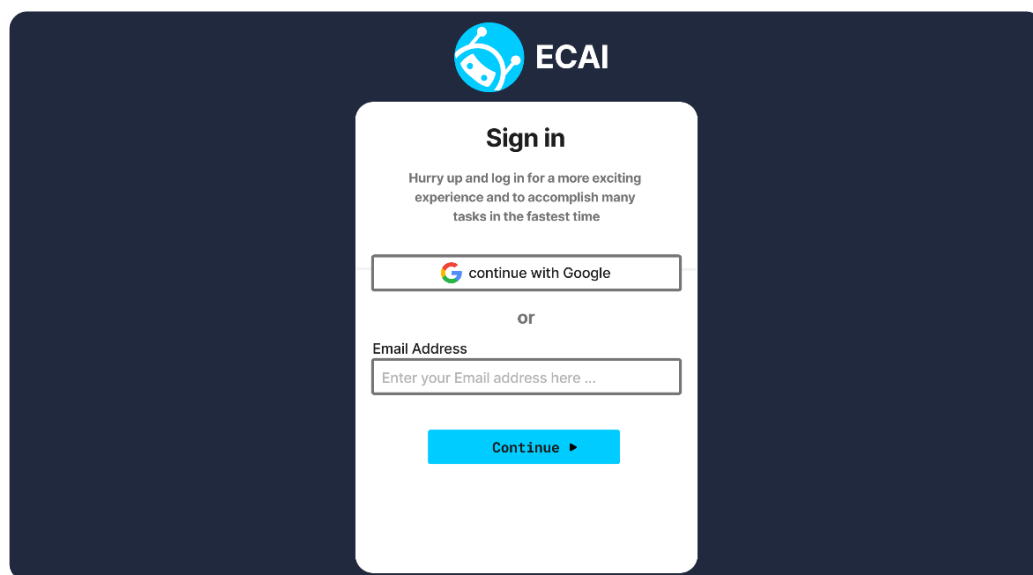


Figure 4-9 Login page

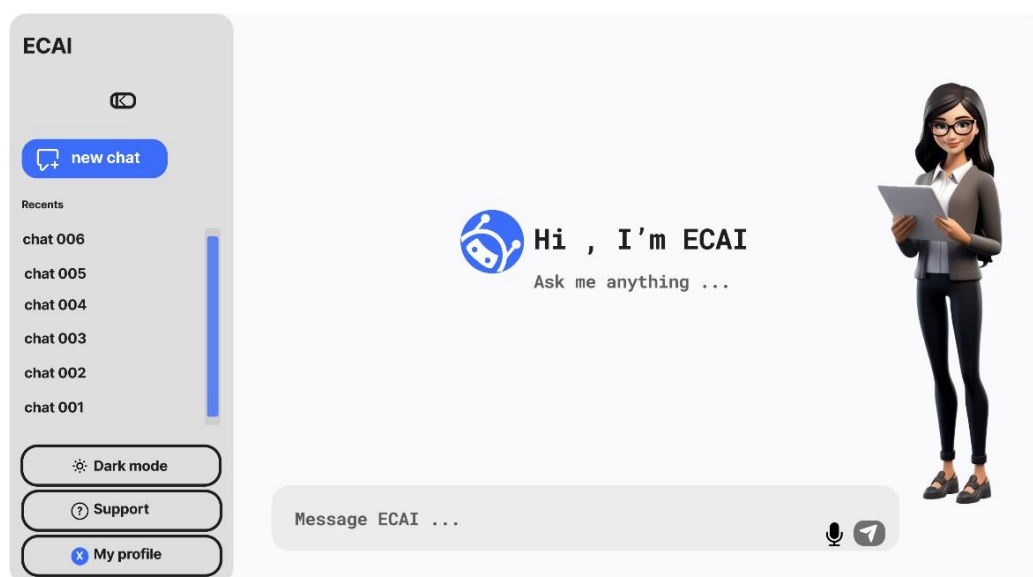


Figure 4-10 home page light mode

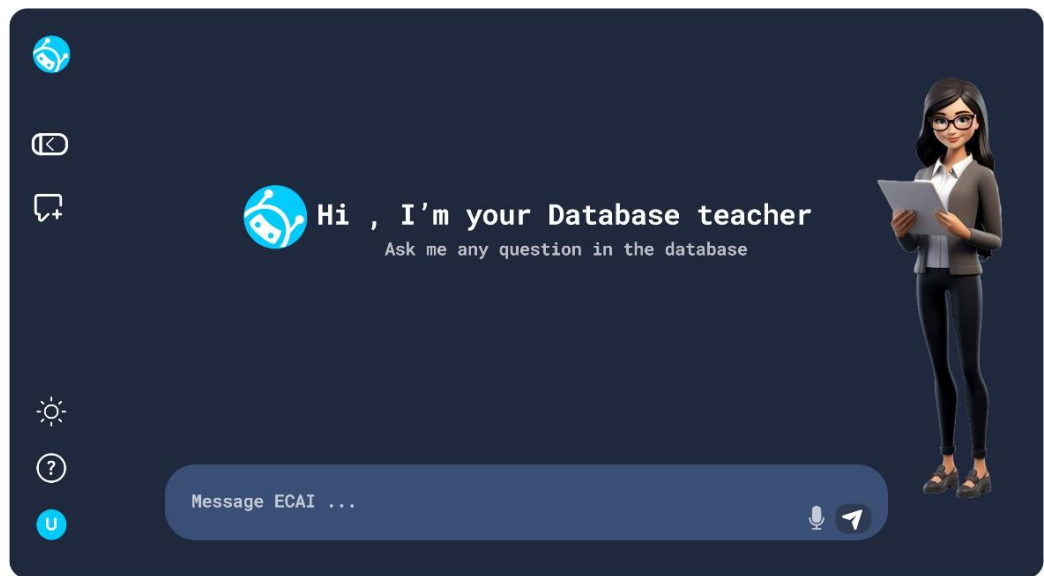


Figure 4-11 Dark mode

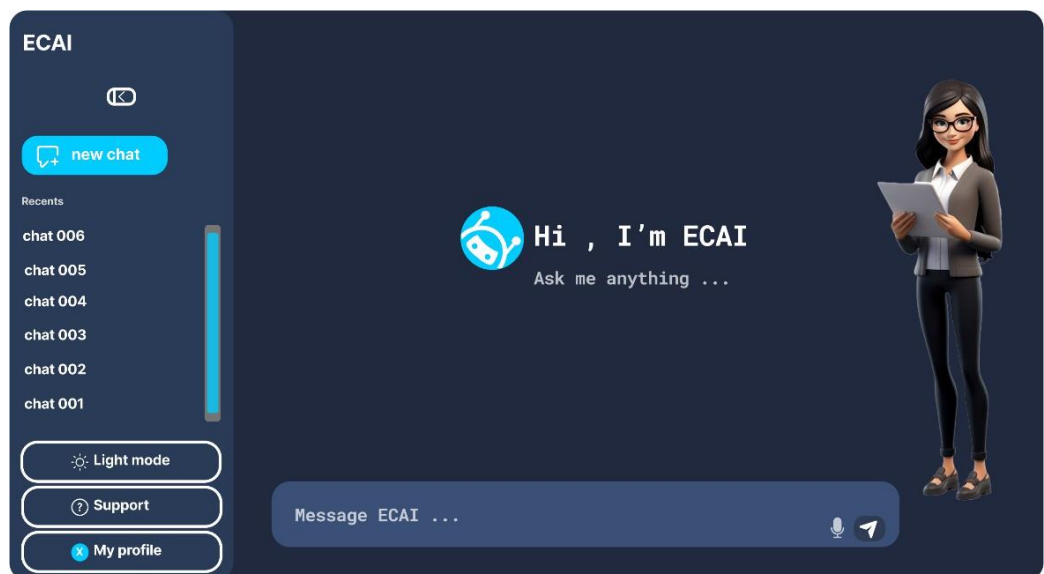


Figure 4-12 side bar

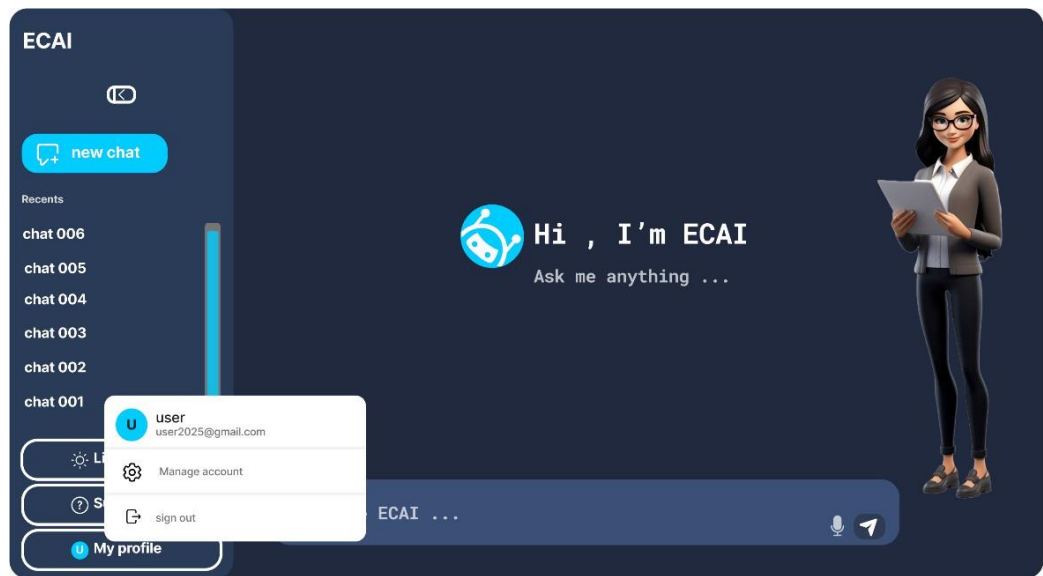


Figure 4-13 Account details

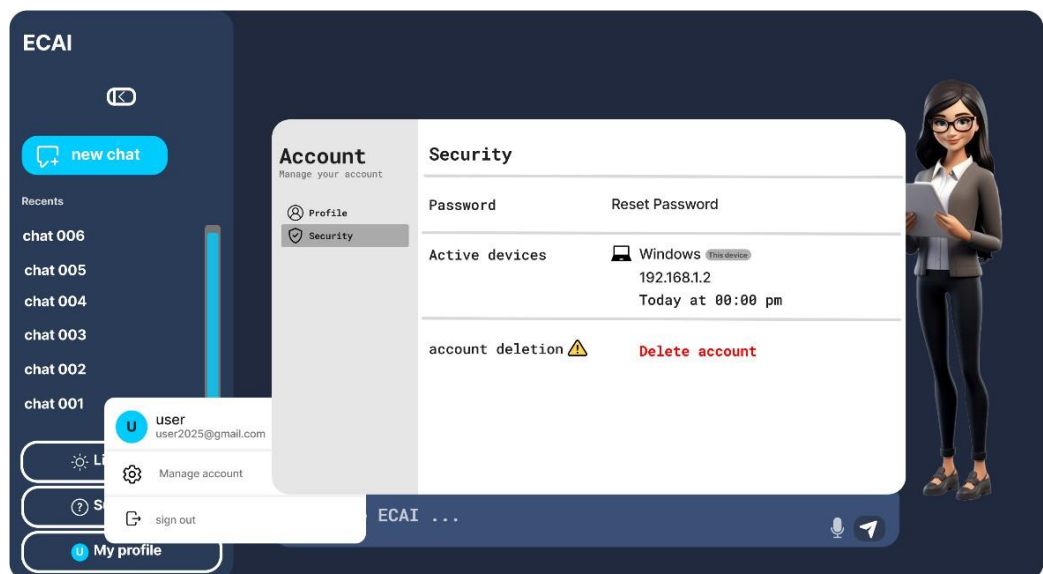


Figure 4-14 security

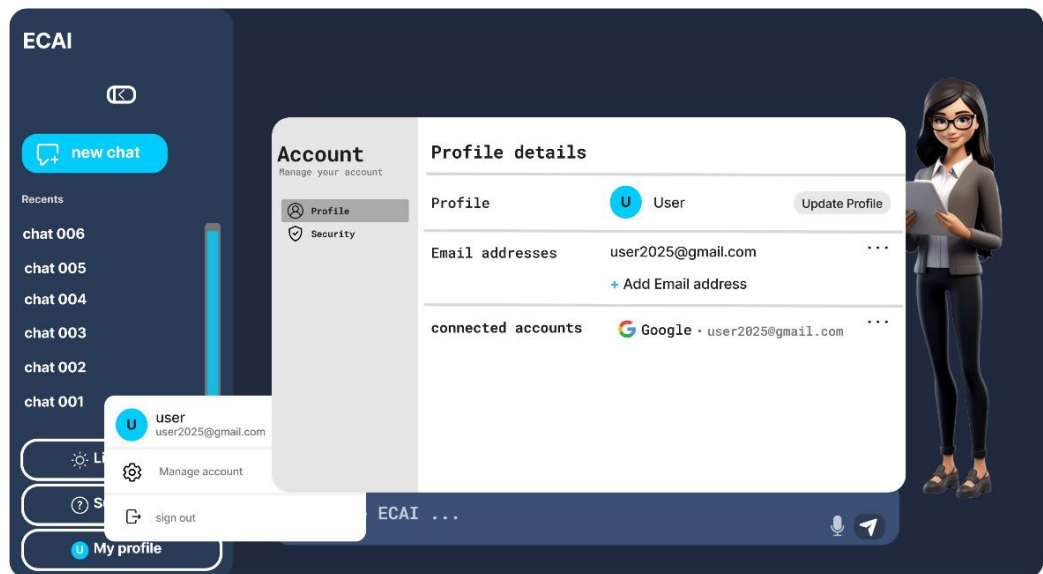


Figure 4-15 profile

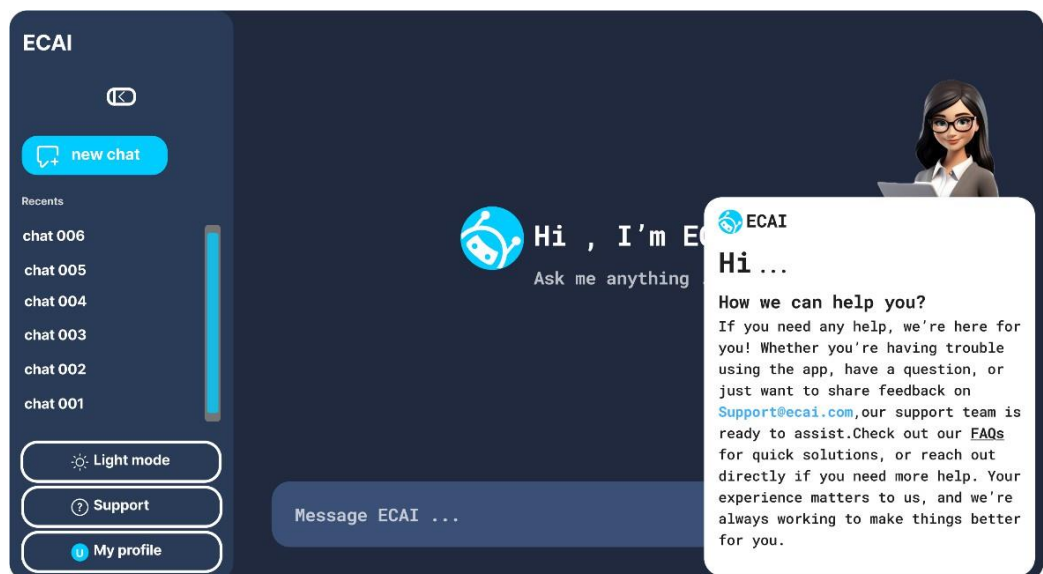


Figure 4-16 support

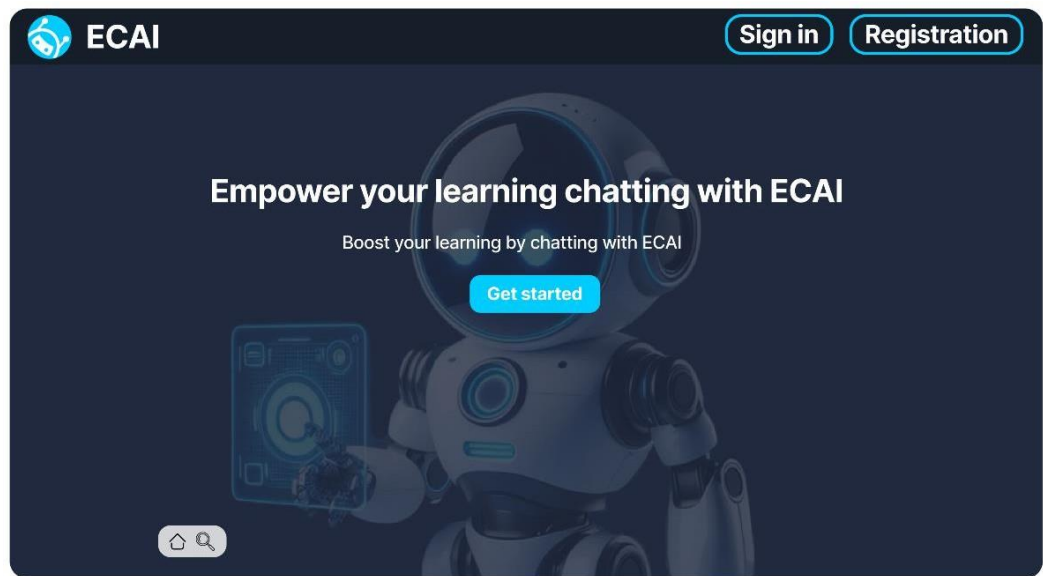


Figure 4-17 first page in home

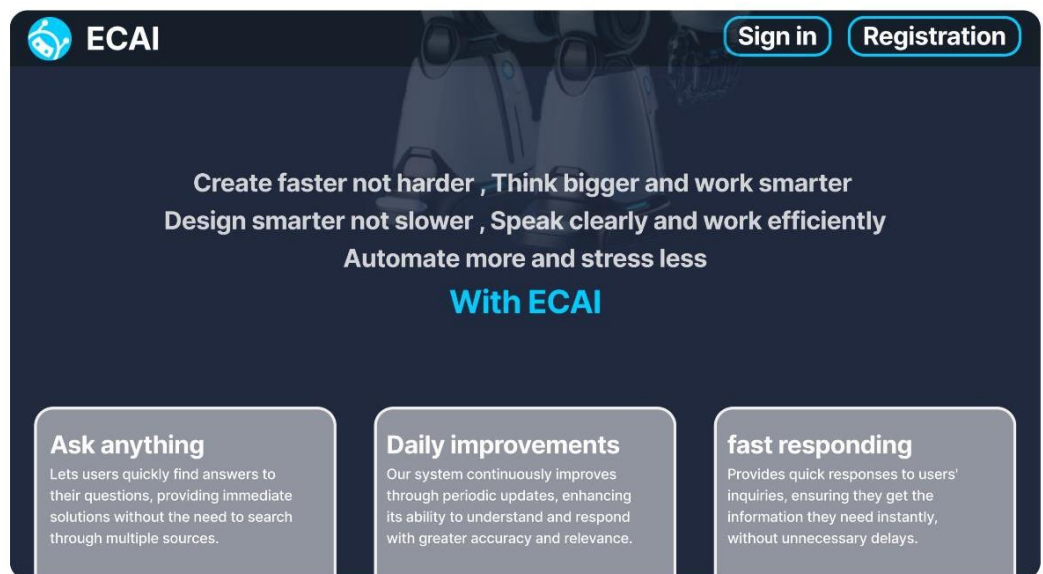


Figure 4-18 second page in home

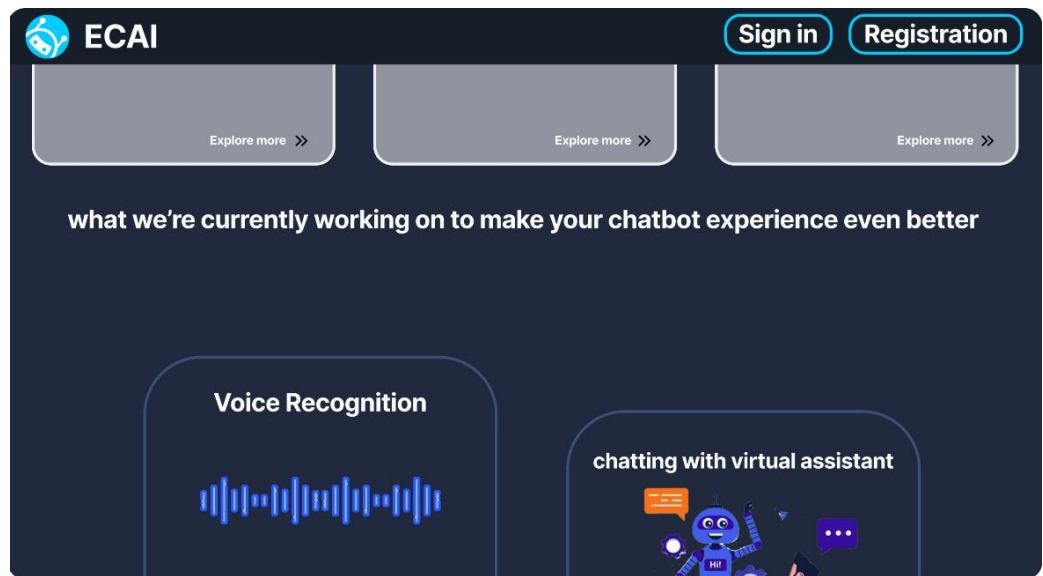


Figure 4-19 third page in home

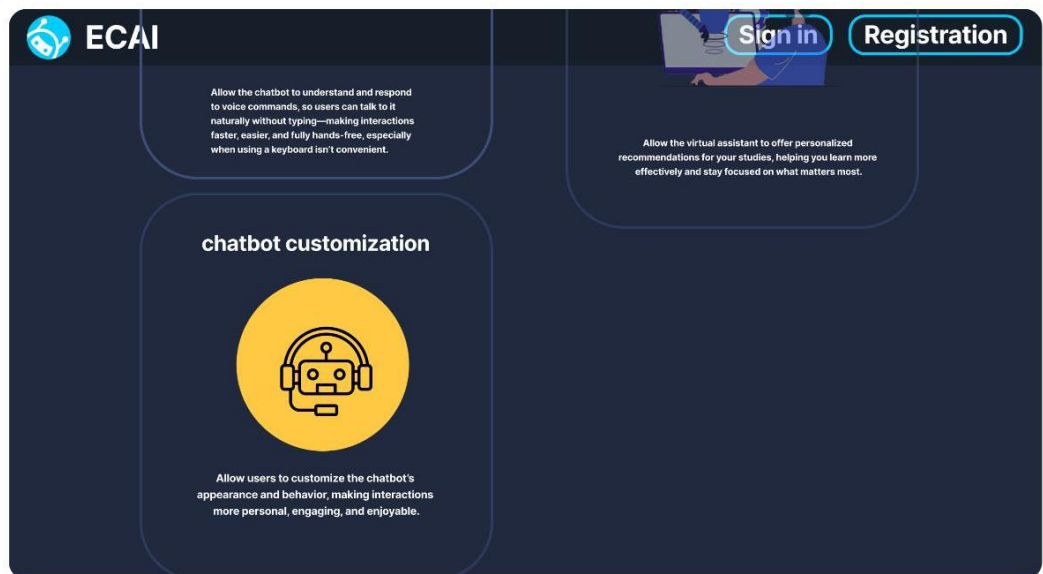


Figure 4-20 forth page in home

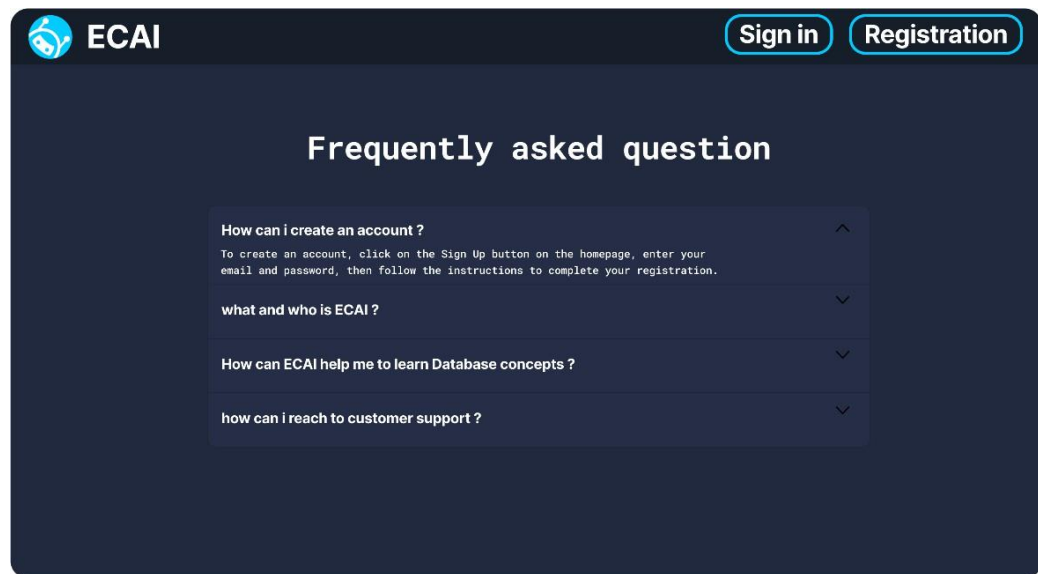


Figure 4-21 freq asked question

System Design

4.6 Introduction

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It is a critical phase in the software development lifecycle that aims to create a high-level abstract representation of a system that can be implemented in a variety of technologies and platforms. The design of a system should consider scalability, maintainability, flexibility, and reliability. The process typically involves creating a conceptual design, which describes the overall structure of the system, followed by a detailed design, which defines the specific components and interfaces needed to realize the conceptual design. Effective system design is crucial to ensuring the success of a project, as it can greatly impact factors such as performance, cost, and user experience.

System design is an essential component of software development, which aims to create a high-level abstract representation of a system that meets specified requirements. The process involves defining the architecture, components, modules, interfaces, and data for a system while considering factors such as scalability, maintainability, flexibility, and reliability. In recent years, there have been significant advancements in system design methodologies, tools, and technologies, which have led to improved software quality, faster time-to-market, and better user experiences. However, several challenges must be addressed to ensure the success of system design projects.

4.6.1 Importance of System Design

System design is a crucial process in software development because it provides a blueprint for building an efficient and effective system. The design process involves identifying the requirements and constraints of the system, analyzing the various components, and determining how they will interact to achieve the desired functionality. By going through this process, developers can identify potential problems and address them before they become issues during the development or implementation phases. This helps to save time and resources and ensures that the final product meets the requirements of the end-users.

4.6.2 Process of System Design

The process of system design involves several steps, which include

- **Requirements Gathering:** This involves identifying the requirements of the system and understanding the needs of the end-users. This can be done through interviews, surveys, or meetings with stakeholders.
- **Analysis:** This involves analyzing the requirements and identifying the various components needed to fulfill them. This may involve creating usecases, diagrams, or flowcharts.
- **Design:** This involves creating a detailed plan for the system, including the architecture, database design, and interface design.
- **Implementation:** This involves developing the system using the plan created during the design phase. This may involve coding, testing, and debugging.
- **Maintenance:** This involves maintaining the system after it has been implemented. This may include bug fixes, updates, and improvements.

4.7 System Objectives

The AI-Powered eLearning Platform is designed to enhance the learning experience for students studying database concepts by offering an interactive, accessible, and engaging method through voice-based communication with an AI avatar. It aims to revolutionize how students absorb complex technical knowledge using artificial intelligence technologies.

4.7.1 Personalized and Interactive Learning

System enables learners to engage in natural, voice-based conversations with an AI avatar, simulating real-time tutoring. The system adapts to individual learning paces and preferences, offering tailored explanations and follow-up support. By leveraging RAG, it ensures responses are accurate and contextually relevant to the selected subject.

4.7.2 Enhanced Accuracy and Engagement

Using advanced AI technologies like Whisper (speech-to-text), LLMs, and GTTS (text-to-speech), the platform minimizes misinformation. The AI avatar's interactive design boosts student engagement, making learning more dynamic and retention-focused.

4.7.3 Accessibility and Inclusivity

System prioritizes accessibility by:

- Supporting voice-based interactions for auditory learners and users with disabilities.
- Providing real-time transcripts of conversations for review and accessibility.

4.7.4 Time and Resource Efficiency

- Reduces educator workload by automating routine Q&A tasks, allowing teachers to focus on higher-level instruction.
- Instant responses to queries, cutting down learning delays.

4.7.5 Cost-Effective and Scalable

- Optimized to run on standard devices (laptops, smartphones) without expensive hardware.
- Open to institutional or individual adoption, making advanced AI education tools widely accessible.

4.8 System Block Diagram

The block diagram is a critical tool used in system design to represent the system's components and their interconnections. The diagram provides a visual representation of the system and helps stakeholders to understand how the system works. block diagram provides an in-depth analysis of system design.

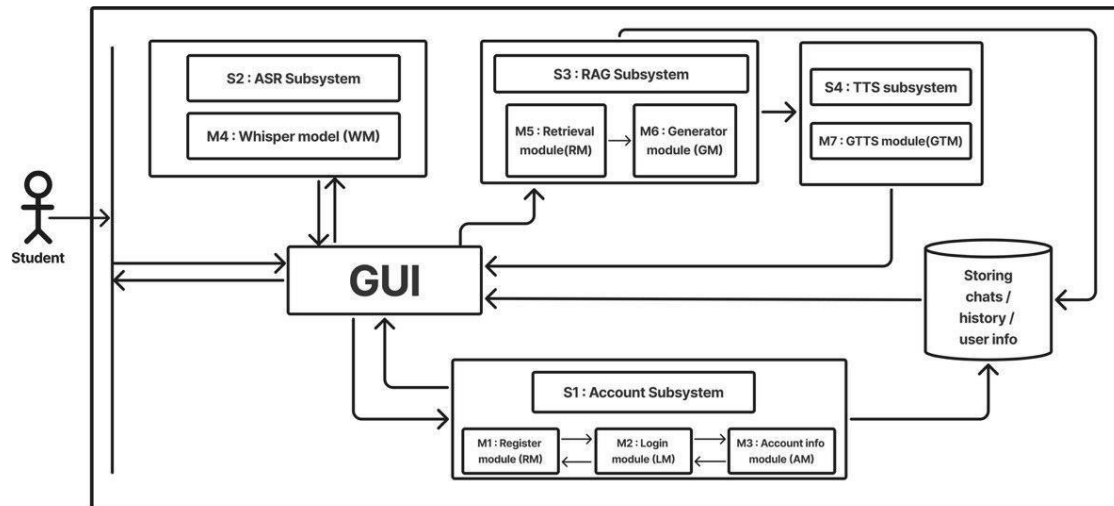


Figure 4-22 Block diagram

1. Blocks

Each block represents a major component of the platform, including both software modules and processing stages:

- Voice Input Module (Microphone)
- Speech-to-Text Engine (Whisper)
- RAG Module (LLM + Knowledge Base)
- Text-to-Speech Engine (GTTS TTS)
- Voice Output Module (Audio Playback to User)

E2. Arrows

Arrows represent the data flow between these components.

For example:

- Audio from the user → Speech-to-Text
- Transcribed text → Question Answering module
- Answer → Text-to-Speech
- Audio response → User playback

3. Inputs and Outputs

- Input: The system accepts voice-based questions from the user through a microphone.

- Output: It delivers audio answers via text-to-speech, allowing the user to hear the response.

4. Feedback Loops

Optional feedback mechanisms may be integrated for:

- User Feedback Collection: Users can rate the answer quality or ask follow-up questions, which loops back to improve the AI response.
- System Improvement: Logs and reports from user interactions may be analyzed to fine-tune the LLM responses or expand the knowledge base over time

4.8.1 Block Diagram Importance

The block diagram is a critical tool in system design, and its importance lies in its ability to provide a visual representation of the system and its components. The diagram helps stakeholders to understand the system's behavior, identify potential problems or bottlenecks, and optimize the system's performance. It also helps to identify the system's inputs and outputs, which are critical in determining the system's functionality

4.8.2 Block Diagram Modules

	Process Name	Input	Output	Functionality	Module	Subsystem
P1	Register Student Account.	Student information.	Account created Confirmation.	Register a new student in the System.	M1	S1
P2	Student Login Authentication.	Username and password.	Access to system.	Verify login credentials.	M2	S1
P3	View and Manage Account Details.	Account selection Request.	Edit account Information.	View or modify student Account details.	M3	S1
P4	Speech to Text Conversion (ASR).	Spoken query (audio).	Text transcription.	Convert student's voice query into text.	M4	S2

P5	Retrieve Relevant Documents.	Text query.	Retrieved Documents.	Search knowledge base for Related documents.	M5	S3
P6	Generate Text Answer.	Retrieved Documents.	Answer text.	Generate answer based on Retrieved documents.	M6	S3
P7	Text to Speech Conversion (TTS).	Answer Text.	Spoken response (Audio).	Convert generated text answer into speech audio.	M7	S4

Table 4-1 block diagram module

4.9 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) is a critical tool in database design used to model the relationships between entities in a database. The diagram provides a visual representation of the database's structure, including its tables, attributes, and relationships.

The Entity Relationship Diagram consists of several components.

1. Entities represent real-world objects, such as customers, products, or orders. Each entity is represented by a rectangle with its name inside it.
2. Attributes represent the characteristics of the entities, such as the customer's name, address, or phone number. Attributes are represented by ovals connected to the entities by lines.
3. Relationships represent the connections between entities, such as a customer placing an order, or a product being sold to a customer. Relationships are represented by diamond shapes with lines connecting them to the entities.
4. Cardinality defines the number of instances of one entity that can be related to another entity.

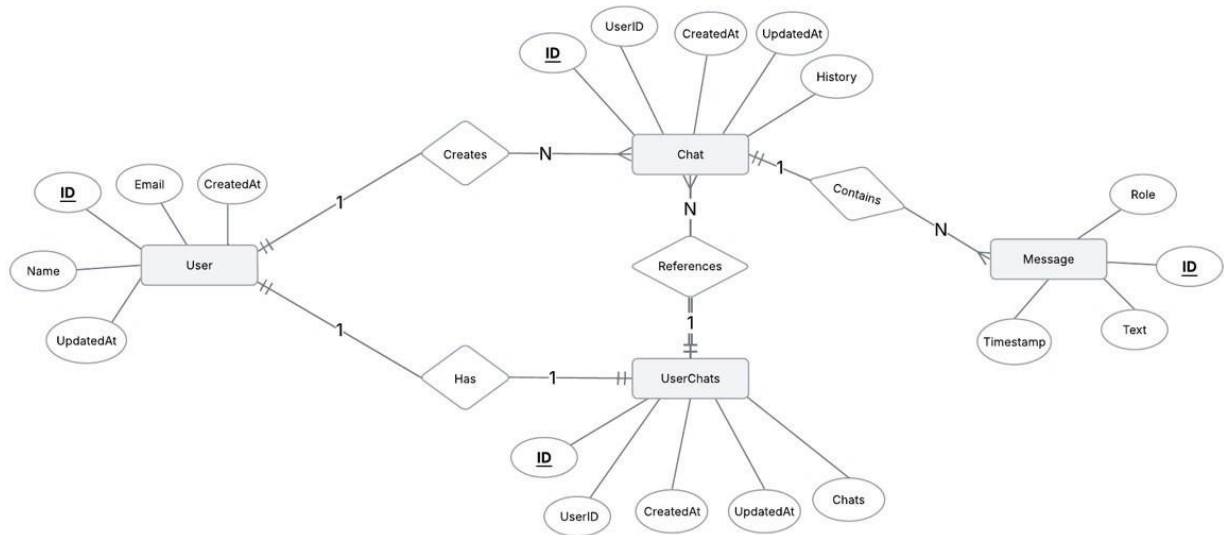


Figure 4-23 ER diagram

The Cardinality of Relationship Pair Objects

1. One-To-One (1:1)
2. One-To-Many (1: N)
3. Many-To-Many (M: N)

4.9.1 ERD Mapping

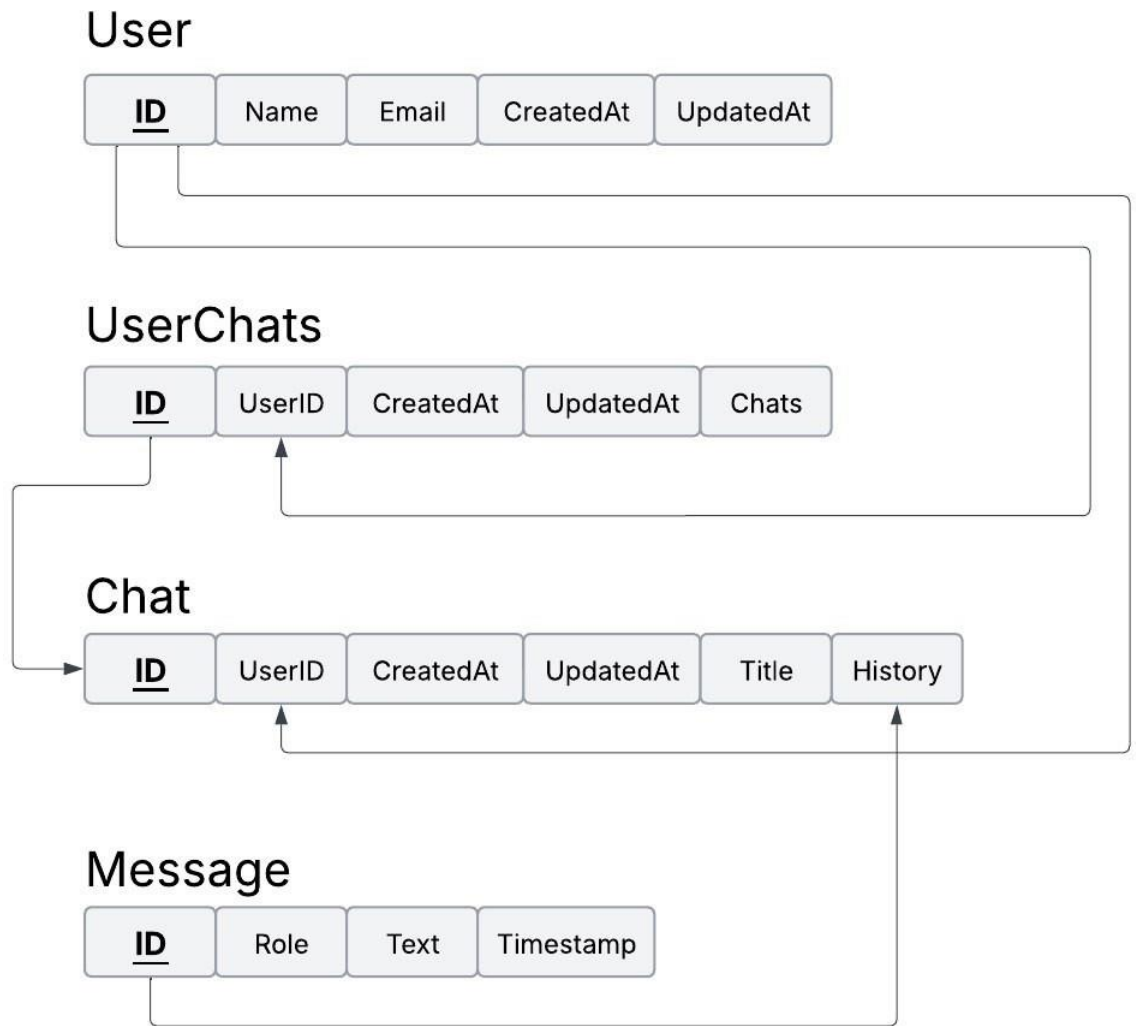


Figure 4-24 Mapping

Chapter 5

Implementation

Chapter Five : Implementation

Introduction

This chapter provides a detailed technical breakdown of the implementation process for *"The Design of eLearning Course Using the Power of AI."* The system integrates multiple AI-driven components to create an interactive virtual tutor capable of voice-based and text-based educational interactions. The implementation follows a structured pipeline, from audio processing to response generation, ensuring accurate, curriculum-aligned answers while maintaining an engaging user experience.

5.1 ECAI Frontend Implementation

The ECAI application presents a clean, student-focused interface designed to facilitate seamless interaction between learners and AI-powered educational assistance. The UI follows modern design principles with a dual-page structure that balances simplicity with functionality, creating an environment that empowers learning without overwhelming users.

5.1.1 Registration and Sign in Section

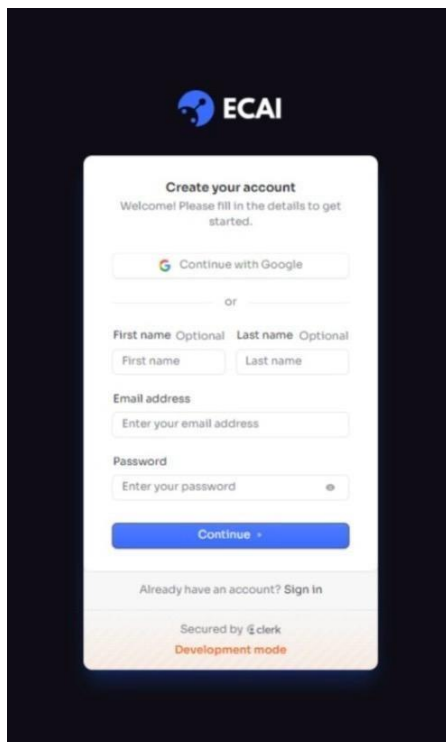
The screenshot shows the 'Create your account' page of the ECAI application. At the top, the ECAI logo is displayed. Below it, the heading 'Create your account' is followed by the instruction 'Welcome! Please fill in the details to get started.' The form includes a 'Continue with Google' button, an 'or' separator, and input fields for 'First name' (optional) and 'Last name' (optional). Below these are fields for 'Email address' and 'Password'. A blue 'Continue' button is at the bottom of the form. At the very bottom, there is a link 'Already have an account? Sign in' and a footer note 'Secured by Clerk Development mode'.

Figure 5-2 Sign-up

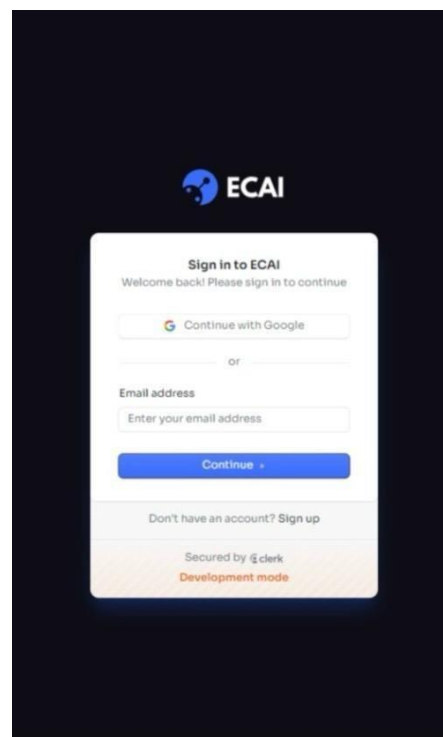
The screenshot shows the 'Sign in to ECAI' page of the application. The ECAI logo is at the top. The heading 'Sign in to ECAI' is followed by 'Welcome back! Please sign in to continue'. The form features a 'Continue with Google' button, an 'or' separator, and an 'Email address' input field. A blue 'Continue' button is positioned below the email field. At the bottom, there is a link 'Don't have an account? Sign up' and a footer note 'Secured by Clerk Development mode'.

Figure 5-1 Sign-in

5.1.2 Learning Hub

The ECAI chat page is designed as a **minimalist, conversational hub** where students interact with an AI tutor. UI features include:

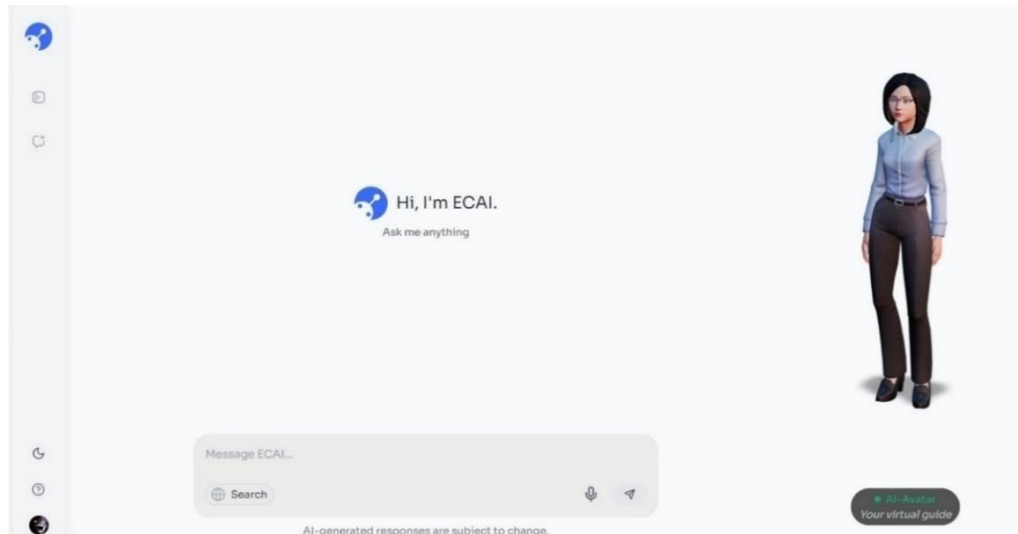


Figure 5-3 ECAI chat page

5.1.2.1 Sidebar Key Elements

Collapsible Design

- **Expanded (300px):** Shows full text labels, recent chats, and icons.
- **Collapsed (80px):** Displays only icons (space-saving)

Dynamic New Chat Button

- Resets all chat states (messages, input).

Theme Mode

- Users can switch between light and dark modes via a **toggle theme switcher**.

Chat History Feature

- Allows users to revisit past conversations, resume learning, or reference previous AI responses.



Figure 5-4 Sidebar

Support Center

Provides instant support, or guidance on using ECAI

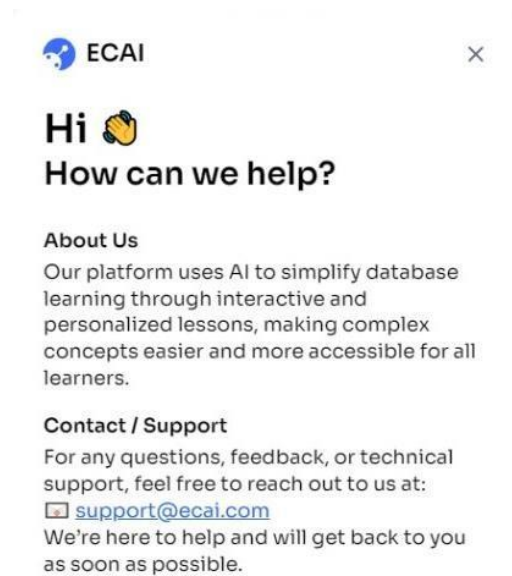


Figure 5-5 Support Center

5.1.2.2 Message Display & Search Bar Area

Shows the conversation thread between the user and ECAI.

Messages Key Features

- **Bubble Layout:**
 - **User messages:** Right aligned (e.g., "Write a Python 'Hello World' program").
 - **AI responses:** Left-aligned with distinct styling (e.g., code blocks, Markdown support).
- **Syntax Highlighting:**
 - Code snippets are formatted with monospace fonts and color coding.
- **Scroll Retention:**
 - Auto-scrolls to the latest message while preserving manual scroll positions.

Input Area & Action Components

- **Text Area:**
 - Auto-resizing (**adjustHeight()** function).
- **Action Buttons:**
 - Voice recording.
 - Shift+Enter for new lines; Enter to submit.

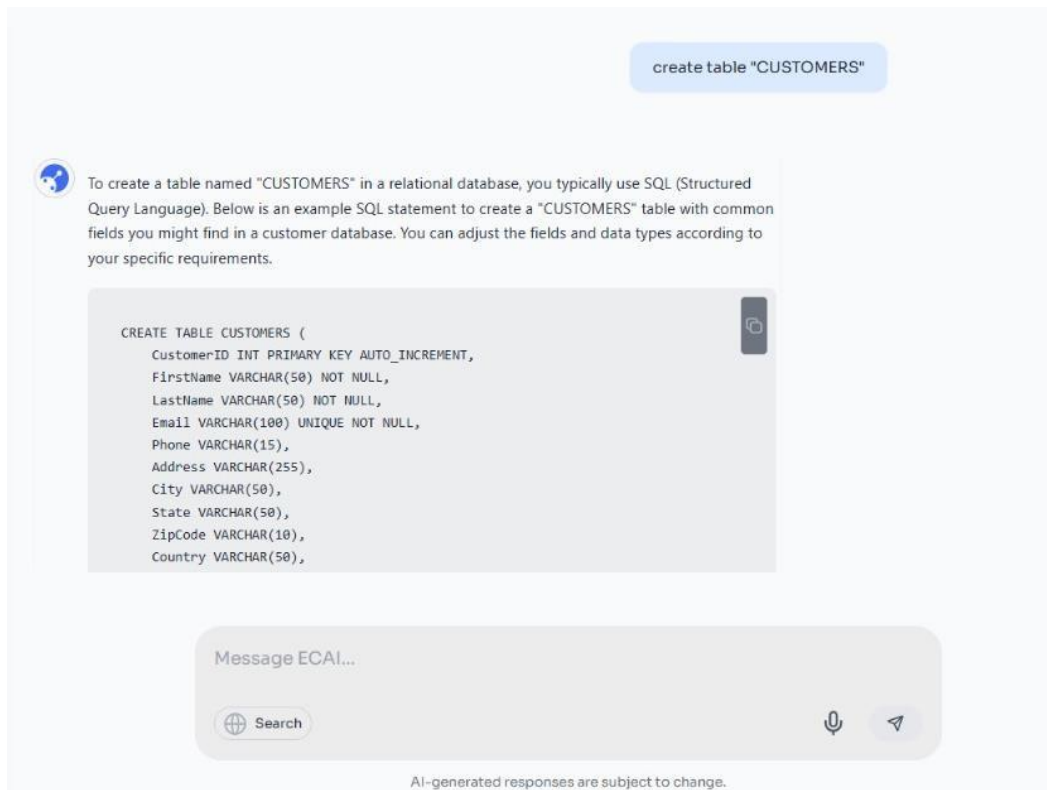


Figure 5-6 Chat page

5.1.2.3 ECAI Avatar System

The avatar system combines **3D animation, facial expressions, and real-time interactivity** to create a lifelike virtual assistant.

3D Model & Animation Pipeline

- **Model Loading:**
 - Uses **useGLTF** from **@react-three/drei** to load
- **Animation System:**
 - Maps animations (e.g., "Idle") to actions via **useAnimations** hook.
 - Smooth transitions with **fadeIn()/fadeOut()**.

- **Facial Expression System:**
 - Controls 50+ facial blend shapes (brow, mouth, cheek movements).
 - Applies presets like smile, angry, or surprised.
- **Audio-Driven Animation:**
 - Syncs mouth cues with audio playback.
- **User-Controlled Camera Adjustments:**
 - manual camera control (e.g., mouse drag), leverage CameraControls' built-in features.

Key Camera Behaviors

Action	Code Snippet	Effect
Initial Focus	<code>setLookAt(0,0,2, 0,-0.3,0)</code>	Centers on avatar's face
Zoom Adjustment	<code>controls.dolly(-0.1, true)</code>	Gradual zoom-out
User Rotation	Built into <code>CameraControls</code>	Mouse drag rotates view
Animation React	<code>useFrame</code> + distance check	Dynamic framing

Figure 5-7 Key Camera Behaviors for Avatar Movement

5.2 ECAI Backend Implementation

The ECAI backend is built with **Node.js, Express, and MongoDB**, handling **user authentication, chat storage, and real-time interactions**. It integrates with **Clerk for auth** and **MongoDB Atlas for data persistence**, ensuring secure and scalable operations.

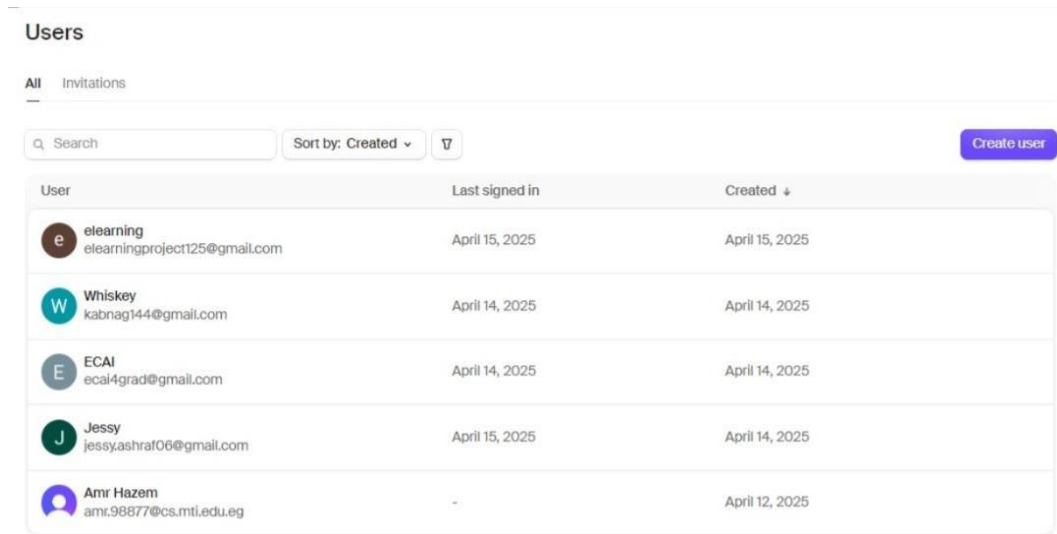
5.2.1 Authentication & User Management

Handles a secure user access and identity verification.

- Uses **Clerk** for sign-in/sign-up.
- Protects routes with middleware (`requireAuth`).

5.2.1.1 User Management Dashboard

Clerk's user management interface



The screenshot shows the Clerk User Management Dashboard. At the top, there's a 'Users' header with tabs for 'All' and 'Invitations'. Below this is a search bar and a 'Sort by: Created' dropdown. A 'Create user' button is in the top right. The main table lists five users with their avatars, names, email addresses, last signed in dates, and creation dates.

User	Last signed in	Created
 elearning elearningproject125@gmail.com	April 15, 2025	April 15, 2025
 Whiskey kabnag144@gmail.com	April 14, 2025	April 14, 2025
 ECAI ecai4grad@gmail.com	April 14, 2025	April 14, 2025
 Jessy jessy.ashraf06@gmail.com	April 15, 2025	April 14, 2025
 Amr Hazem amr.98877@cs.mtl.edu.eg	-	April 12, 2025

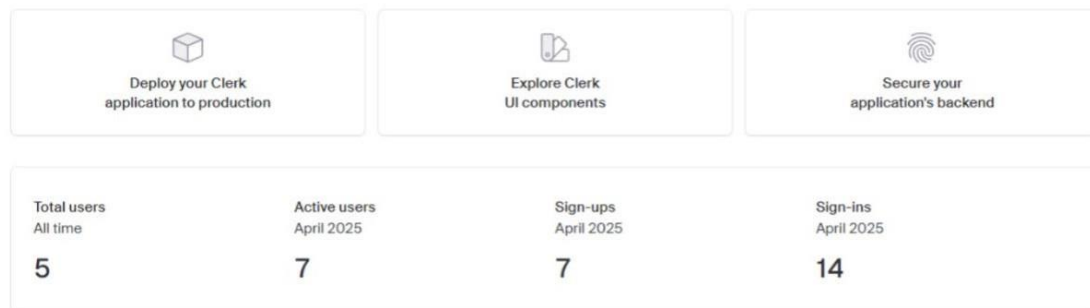
Figure 5-8 Clerk's User Management Dashboard

5.2.1.2 Application Metrics Overview

Clerk Analytics & user activity

Congratulations, your application has users!

Clerk handles authentication and user management for you. Next, learn how to access this data and use it in your application, customize the look and feel of your components, and get your Clerk app ready for production.



The screenshot shows the Clerk Analytics Dashboard. It features three cards for deployment guides: 'Deploy your Clerk application to production', 'Explore Clerk UI components', and 'Secure your application's backend'. Below these is a table with four columns: 'Total users All time', 'Active users April 2025', 'Sign-ups April 2025', and 'Sign-ins April 2025'.

Total users All time	Active users April 2025	Sign-ups April 2025	Sign-ins April 2025
5	7	7	14

Figure 5-9 Clerk's Analytics Dashboard

5.2.2 Chat Operations

Manages chat lifecycle and interactions.

- **Create Chat:** Starts new conversations (auto-titles first message).
- **Update Chat:** Appends user/AI messages to existing chats.

5.2.3 Data Models & Schemas

Defines how chat and user data are structured in MongoDB.

- **Chat Schema:** Stores message history (user/model roles).
- **UserChats Schema:** Tracks all chats per user_id (for quick lookup).

5.2.3.2 MongoDB Atlas Database Management Interface

- **MongoDB Atlas administrative dashboard** providing insights into your database cluster and collections.

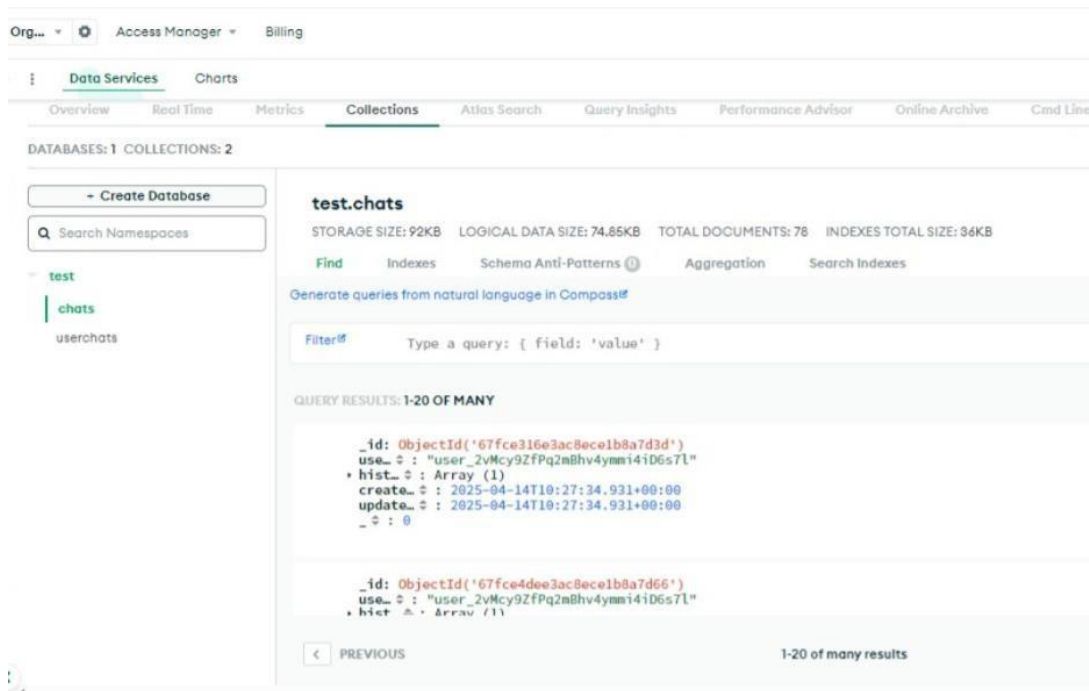


Figure 5-10 MongoDB Management Interface

5.2.4 Database & Infrastructure

Connects and configures backend services.

- **MongoDB Connection:** Links to Atlas cluster (Iowa region).
- **CORS:** Safely allows frontend requests from localhost:5173.

5.2.5 Error Handling & Validation

Catches and responds to issues gracefully.

- Rejects invalid inputs (e.g., missing text).
- Handles authentication failures (400), and server crashes (500).

5.2.6 API Endpoints

The backend provides four core RESTful endpoints for managing chat operations, each requiring authentication and interacting with a MongoDB database.

5.2.6.1 Create a New Chat

Endpoint: POST /api/chats

Functionality:

- Initializes a new chat session.
- Accepts a text input (the first message).
- Creates a document in the Chat collection with the initial message.
- Updates the UserChats collection to include the new chat's metadata (ID and truncated title).

5.2.6.2. List User Chats

Endpoint: GET /api/userchats

Functionality:

- Retrieves all chat summaries associated with the authenticated user.
- Queries the UserChats collection for the user's chat list.
- Returns an array of chat objects containing IDs and titles.

5.2.6.3 Fetch Chat History

Endpoint: GET /api/chats/:id

Functionality:

- Retrieves the full message history for a specific chat.
- Validates that the chat belongs to the authenticated user.
- Returns the history array from the Chat collection, sorted by timestamp if available.

5.2.6.4 Update Chat with New Messages

Endpoint: PUT /api/chats/:id

Functionality:

- Appends new question-and-answer pairs to an existing chat.
- Requires question and answer fields in the request body.
- Updates the chat's history array with timestamps.

5.3 Whisper and how it works

Whisper is a **speech-to-text model** developed by OpenAI. That means it takes **spoken audio** and converts it into **written text** — kind of like having a really smart assistant that can transcribe podcasts, meetings, lectures, or any spoken audio... in multiple languages.

It is designed to **transcribe** and **translate** speech across multiple languages without the need for task-specific fine-tuning. **In our project we will focus on transcription features.**

Key Highlights

- **Massive Training Dataset:** Trained on **680,000 hours** of multilingual, multitask audio data collected from the internet.
- **Zero-Shot Performance:** Capable of transcribing or translating speech it has never explicitly seen before, without additional training.
- **Multilingual and Multitask:** Supports **96 languages** for speech recognition and translation tasks.
- **Robustness:** Performs reliably even on **noisy audio**, **different accents**, and **domain shifts** (changes in topic or environment).

Technical Overview

- **Input:**
 - 16 kHz sampled audio.
 - Preprocessed into **log-Mel spectrograms**.
- **Architecture:**
 - Based on a **transformer encoder-decoder** model.
- **Tokenizer:**
 - Uses **Byte Pair Encoding (BPE)** for multilingual token generation.
- **Special Tokens:**
 - Incorporates special markers such as [startoftranscript], [translate], [nospeech], and timestamp tokens to guide tasks.

Achievements

- **Long-form Audio Handling:** Smart segmentation strategies allow Whisper to manage very long audio recordings without losing coherence.
- **Unified Model:** No need for separate models for speech recognition, translation, or voice activity detection.

Approach

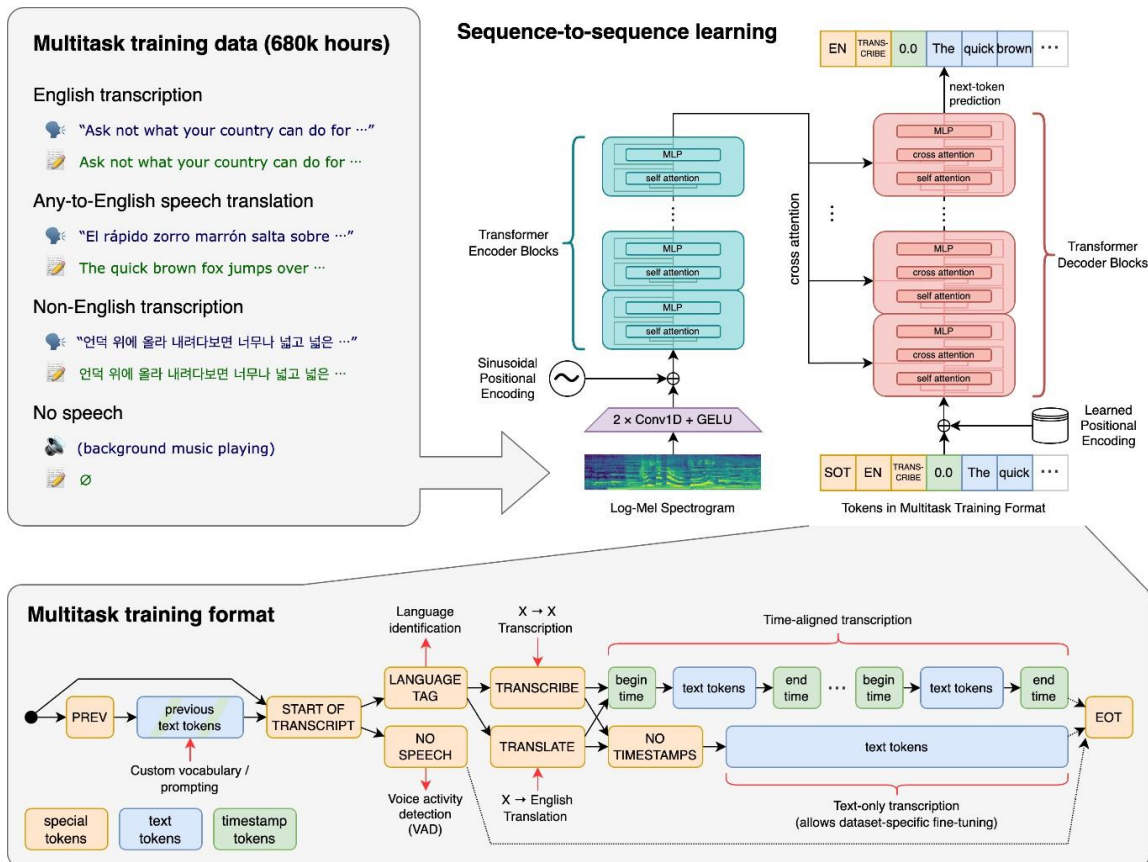


Figure 5-11 Whisper architecture

When you upload an audio file to **Whisper** for transcription, the model processes the audio through several steps to convert it into a readable text output. Here's a detailed explanation of each step from the moment the audio is uploaded until the final output is generated:

5.3.1 Massive Training Dataset

Whisper was trained on an extremely large and diverse dataset: **680,000 hours of multilingual and multitask** labeled audio collected from the **public web**.

The training audio includes:

- **Multiple languages:** About **96 languages** represented.
- **Multiple domains:**
 - **Podcasts, lectures, interviews, audiobooks, meetings, news recordings, radio shows, etc.**

- **Real-world conditions:**

- Audio recorded in **noisy environments** (e.g., street sounds, background music, overlapping conversations).

Key point: This dataset was **weakly supervised** — meaning it was **not manually annotated**. Instead, OpenAI:

- Collected raw audio and their **automatically associated transcripts** (from captions, subtitles, metadata).
- Used **automatic cleaning**, **language filtering**, and **data balancing** strategies to prepare the training set.

Why Weak Supervision?

- **Manual annotation** (human labeling) would have been impractical for hundreds of thousands of hours.
- **Weak supervision** (using imperfect but large datasets) allowed scaling the training to **very large sizes**.
- By exposing the model to **realistic, messy, noisy audio**, Whisper became **more robust** than models trained only on clean studio-quality data.

Result:

Whisper generalizes very well to:

- Noisy environments (cafes, streets).
- Accented speech.
- Domain shifts (from podcasts to formal news).
- Different languages and dialects without needing fine-tuning.

Probable Sources of Whisper's Training Data

OpenAI did **not release the exact datasets** for privacy and licensing reasons. However, based on their paper and public knowledge, the likely sources or types of datasets are:

Dataset / Source	Description
Common Voice (Mozilla)	Open-source multilingual speech corpus.
LibriSpeech	English audiobooks (clean + noisy versions).
Multilingual TEDx	TEDx talk recordings across many languages with transcripts.

Dataset / Source	Description
MLS (Multilingual LibriSpeech)	Audiobooks datasets in multiple languages.
VoxPopuli	European Parliament speeches (23 languages).
FLEURS	Google multilingual speech dataset (102 languages).
YouTube Public Captions (estimated)	Videos with autogenerated or manually uploaded subtitles.
Common Crawl Speech (hypothetical)	Scraped web speech data from various domains.

Table 5-1 whisper datasets

Note: These datasets match the **description** (multilingual, diverse, weakly supervised), but OpenAI **has not confirmed** the exact list.

Data Cleaning and Filtering Techniques

OpenAI used smart techniques to make the weakly supervised data more reliable:

- **Automatic Language Identification:** Detecting the language spoken in each audio clip.
- **Filtering Poor-Quality Matches:** Removing audio-text pairs where the audio didn't match the transcript well.
- **Removing Silence or No-speech Clips:** Clips that contained mostly silence or noise were filtered out.
- **Balancing Language Distribution:** To avoid overtraining on English and undertraining on low-resource languages.

This automatic data curation is a **critical reason** why Whisper achieves **robust performance** across diverse real-world scenarios.

5. Why Whisper's Dataset is Special

Whisper's Dataset Advantage	Meaning
Large-scale (680k hours)	More than 10x larger than typical speech datasets.
Noisy and real-world	Prepares the model for real, messy conditions, not just clean studio recordings.

Whisper's Dataset Advantage	Meaning
Multilingual and multitask	Trained jointly on multiple languages and tasks (transcription + translation).
Weakly supervised, automatically cleaned	Achieves robustness without expensive manual labor.

Table 5-2 datasets advantages

Thus, Whisper **learned to recognize**:

- Speech patterns even under strong noise.
- How different languages "sound" at the audio level.
- How to handle background conversations, reverberations, interruptions, and music.

5.3.2 Input Processing: From Audio to Mel Spectrogram

Why Whisper Doesn't Work Directly with Raw Audio

- **Raw audio** is a long, complex sequence of numbers (16,000 samples per second).
- Feeding raw audio directly into a Transformer would be **too large** and **low-level**.
- Speech recognition needs to focus on **patterns** like **syllables, rhythm, and pitch**, not the tiny fluctuations in raw sound waves.

Solution: Convert raw audio into a **log-Mel spectrogram**, a smart, compressed representation that focuses on the important parts of speech.

Step-by-Step Audio Preprocessing

a) Windowing and Short-Time Fourier Transform (STFT)

- The audio is **divided into small frames** (e.g., 25 milliseconds each).
- Apply **Fourier Transform** on each frame to measure:
 - What **frequencies** are present.
 - How **strong** each frequency is.

Result: A basic **spectrogram** (time × frequency × energy).

b) Mel Scale Transformation

- Human ears **don't hear** frequencies linearly:
 - We are **better at low frequencies** and **worse at high frequencies**.
- The **Mel scale** warps the frequency axis to match human perception.
- Frequencies are grouped into **Mel filter banks**.

Result: **Mel spectrogram**.

c) Logarithmic Compression

- Energy values vary a lot, from very quiet to very loud.
- Taking the **logarithm** compresses the dynamic range.
- Small but important sounds (like faint speech) become more noticeable.

Result: **Log-Mel spectrogram** — the final input format for Whisper.

Structure of the Log-Mel Spectrogram

- **X-axis** = Time.
- **Y-axis** = Mel-frequency bands (low to high).
- **Color/brightness** = Energy at that time and frequency.

Whisper's encoder sees this spectrogram like an **image** — a visual pattern of sound.

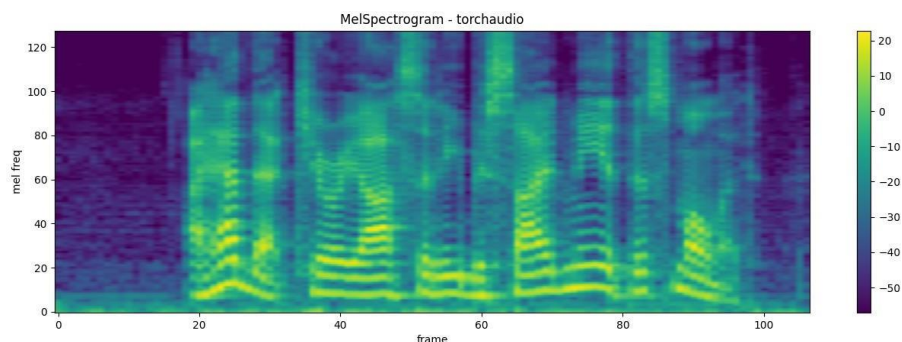


Figure 5-12 log-Mel spectrogram

How Whisper Reads the Spectrogram

What Whisper "Sees"

In a log-Mel spectrogram:

- **Vowels** (A, E, O sounds) → Broad, bright horizontal bands at mid frequencies.
- **Consonants** (T, K, S sounds) → Short, sharp bursts at high frequencies.

- **Silence** → Blank spaces (no energy).
- **Noise or Music** → Random, less structured energy patterns.

How the Transformer Encoder Understands Audio

- **Self-attention** layers in the encoder allow the model to:
 - Focus on important sounds (words, syllables).
 - Ignore irrelevant noise.
 - Capture long-term patterns (like sentence structure).
- It turns the spectrogram into a set of **hidden features** that summarize the audio's meaning.

Why Whisper Uses Log-Mel Spectrograms

- **Compresses audio** into a smaller, more manageable size.
- **Highlights the patterns** that are important for human speech.
- **Speeds up training and inference** (compared to raw audio).
- **Allows the Transformer** to focus on recognizing and understanding speech patterns efficiently.

Now the model can **efficiently read audio like a visual sequence** and **transcribe it into text** or **translate it**, depending on the task.

Summary Table

Stage	Description
Raw Audio	16,000 samples per second (continuous signal)
Spectrogram	Shows energy at different frequencies over time
Mel Spectrogram	Matches human hearing sensitivity
Log-Mel Spectrogram	Compresses energy ranges for better learning
Transformer Encoding	Understands patterns in the spectrogram
Decoder + Tokenization	Writes out text transcription/translation

Table 2-3 why Log-Mel spectrogram

5.3.3 Architecture: Transformer Encoder-Decoder

- Whisper uses a **Transformer encoder-decoder architecture**, similar to those in machine translation models.

Encoder:

- Processes the spectrogram into **hidden features**.
- Understands where the words are, when there is silence, what language is spoken, and the structure of the audio.

Decoder:

- Takes the encoder's hidden features and **generates text tokens** one-by-one.
- Predicts **transcriptions** or **translations**, using **special control tokens** (e.g., `<|startoftranscript|>`, `<|translate|>`, `<|nospeech|>`, timestamps).

The **Transformer** model, introduced in the paper *Attention is All You Need* by Vaswani et al. (2017), has revolutionized Natural Language Processing (NLP) and other domains by overcoming limitations of previous models like RNNs (Recurrent Neural Networks) and LSTMs (Long Short-Term Memory networks). It uses a mechanism called **self-attention**, which allows for better handling of long-range dependencies in data, enabling efficient parallelization during training.

Below is an in-depth exploration of the Transformer architecture, focusing on the self-attention mechanism, encoder-decoder structure, and key advantages.

1. Self-Attention Mechanism

Self-attention allows each token (or word) in the input sequence to attend to all other tokens, regardless of their position in the sequence, enabling the model to capture dependencies between distant tokens effectively.

Mathematical Formulation of Self-Attention:

For a given sequence, we denote:

- **Input sequence:** $X = [x_1, x_2, \dots, x_n]$
- $x_i \in \mathbb{R}^d$ represents the embedding vector of token i (with dimension d).

Each token embedding x_i is linearly projected into three different vectors: the **Query (Q)**, **Key (K)**, and **Value (V)**. These projections are computed using learned weight matrices:

- $Q_i = W^Q x_i$
- $K_i = W^K x_i$
- $V_i = W^V x_i$

Where $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$ are learned weight matrices, and d_k is the dimension of the query/key vectors.

Scaled Dot-Product Attention:

1. Compute raw attention scores:

The attention score between token i and token j is computed by taking the dot product between the **Query** of token i and the **Key** of token j :

$$\text{Attention}(Q_i, K_j) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

The scaling factor $\sqrt{d_k}$ helps prevent the dot product values from growing too large as the dimensions of Q and K increase.

Apply softmax:

To normalize the attention scores, we apply **softmax** across all tokens in the sequence:

$$\alpha_{ij} = \text{softmax}_j(Q_i \cdot K_j^T / \sqrt{d_k})$$

This results in a probability distribution for the attention weights α_{ij} , which indicate how much focus token i should place on token j .

Weighted sum of values:

The output of the self-attention mechanism for token i is a **weighted sum** of the **Value** vectors:

$$\text{Attention}(i) = \sum_{j=1}^n \alpha_{ij} V_j$$

Multi-Head Attention:

Instead of using a single set of query, key, and value vectors, the Transformer uses **multi-head attention**. This involves using multiple sets of Q, K, V .

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{Head}_1, \dots, \text{Head}_h) W^O$$

Where h is the number of attention heads, and W^O is the learned weight matrix for the output.

projections, allowing the model to focus on different aspects of the input sequence simultaneously. The outputs of all attention heads are concatenated and passed through a linear transformation:

2. Positional Encoding

Since the Transformer processes sequences in parallel (rather than sequentially, like RNNs), it has no inherent understanding of token order. To provide positional information, **positional encodings** are added to the input embeddings. These encodings are designed using sine and cosine functions:

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right)$$
$$\text{PE}(p, 2i + 1) = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

Where:

- p is the position of the token in the sequence.
- i is the dimension index of the encoding.
- d is the total dimension of the embeddings.

These sinusoidal functions ensure that the model can encode relative positions of tokens effectively.

3. Encoder and Decoder Architecture

The Transformer architecture consists of two main components: the **Encoder** and the **Decoder**. Each is built from identical layers that incorporate the self-attention mechanism, feed-forward networks, and residual connections.

Encoder:

The encoder is composed of a stack of N identical layers. Each layer includes:

Multi-head self-attention: Allows each token to attend to every other token.

Feed-forward network (FFN): Applies a fully connected neural network to each token's representation independently.

Layer normalization and residual connections:

Helps with the gradient flow during training and improves model convergence.

- **Decoder:** The decoder also consists of N layers, but each layer has an additional attention mechanism:
- **Masked self-attention:** Ensures that the decoder only attends to tokens generated before the current token during autoregressive generation.
- **Encoder-decoder attention:** Attends to the encoder's output to gather information about the input sequence.

- **Feed-forward network (FFN):** Same as in the encoder.
- **Layer normalization and residual connections:** For stability and efficient training.

The decoder generates the output sequence one token at a time, using its previous outputs as context.

4. Feed-Forward Networks (FFNs)

Each layer in the encoder and decoder includes a **feed-forward network (FFN)**, which is applied to each token independently after the attention mechanism. The FFN consists of two fully connected layers with a **ReLU** activation function between them:

$$\text{FFN}(z_i) = \text{ReLU}(z_i W_1 + b_1) W_2 + b_2$$

Where:

- W_1 and W_2 are the weight matrices for the FFN layers.
- b_1 and b_2 are the biases for each layer.
- z_i is the output of the attention mechanism for token i .

5. Final Output

The decoder produces output tokens one at a time. The output is passed through a **linear layer** followed by a **softmax** function to predict the probability distribution over the vocabulary for the next token.

6. Advantages of the Transformer

- **Parallelization:** Unlike RNNs, which require sequential processing of tokens, the Transformer can process the entire sequence at once, allowing for efficient parallelization.
- **Long-Range Dependencies:** The self-attention mechanism allows the model to capture dependencies between tokens regardless of their distance in the sequence.
- **Scalability:** The architecture scales well with both data and model size, leading to its widespread use in models like GPT (Generative Pretrained Transformer) and BERT (Bidirectional Encoder Representations from Transformers).

5.3.4. Tokenization: Byte Pair Encoding (BPE)

Whisper uses **BPE tokenization**:

- Breaks text into common **subwords** rather than full words.

- Handles **many languages** efficiently (different scripts and writing systems).
- Makes the model more flexible and compact.

1. What is Byte Pair Encoding (BPE)?

Byte Pair Encoding (BPE) is a subword tokenization technique used to split text into smaller units, such as characters or subwords, instead of using entire words as tokens. It was originally introduced as a data compression technique but has since become widely used in NLP tasks, particularly in neural machine translation (NMT) and ASR systems like Whisper.

The key idea behind BPE is to iteratively merge the most frequent pairs of characters or subwords in a given corpus into new subword units, producing a set of tokens that balance between efficiency (for frequent words) and flexibility (for rare or unseen words).

2. How BPE Works: A Step-by-Step Explanation

BPE builds a vocabulary based on subwords rather than entire words, breaking text into pieces that can be more easily handled by machine learning models. Here's how the process works:

Step 1: Initial Tokenization

First, the text is split into characters or byte-level tokens. For example, given the word "loved", the initial tokenization might look like this:

['l', 'o', 'v', 'e', 'd']

Each character is treated as an individual token at the beginning.

Step 2: Counting Pair Frequencies

Next, BPE counts the frequency of consecutive character pairs in the corpus. For example, in the corpus:

"loved love"

We have the following pairs:

- ('l', 'o')
- ('o', 'v')
- ('v', 'e')
- ('e', 'd')

- ('l', 'o')
- ('o', 'v')

The pair ('l', 'o') appears twice, which makes it the most frequent pair.

Step 3: Merging the Most Frequent Pair

The most frequent pair (in this case, ('l', 'o')) is merged into a single token, say "lo". The new token list looks like this:

['lo', 'v', 'e', 'd', 'lo', 'v', 'e']

After the merge, we repeat the process of counting the new pair frequencies and merging the most frequent pair again.

Step 4: Iteration

This process is repeated multiple times. After several iterations, common subword units such as "lo", "ve", or even whole words (depending on the corpus and the number of iterations) emerge as the final tokens in the vocabulary. For instance, we might end up with a vocabulary that includes subwords like "lo", "ve", "ed", "love", etc.

Step 5: Final Vocabulary

After many iterations of merging frequent pairs, BPE produces a vocabulary of subwords, where each subword is a token that can represent parts of words or full words. The final vocabulary might contain:

- Common subwords like "lo", "ve", "ed", and "love".
- Rare subwords or characters, which could represent words that are less frequent in the training data.

3. Why Whisper Uses BPE for Tokenization

Whisper, as an automatic speech recognition (ASR) system, uses BPE for tokenization because it offers several advantages, especially when dealing with diverse languages and varying writing systems. Below are the key reasons why BPE is particularly suited for this task:

a. Handling Multiple Languages

BPE allows Whisper to use a **single token set** across multiple languages. This is particularly important in multilingual models where the system needs to recognize and transcribe speech from different languages like English, French, Arabic, and Chinese, among others.

- **Efficiency:** BPE tokenizes common words and phrases into single tokens, allowing the model to handle frequent words efficiently.

- **Flexibility:** For less common or unseen words, BPE breaks them into smaller subword units that the model can still process, ensuring it can deal with a vast range of words, including out-of-vocabulary terms.

For example, in English, the word "unhappiness" might be split into the subwords "un", "happiness". In Arabic, the word might be tokenized into smaller units reflecting the script's structure, enabling Whisper to manage languages with complex morphology.

b. Efficient Representation of Common and Rare Words

BPE is particularly useful because it allows the model to efficiently represent **common words** as single tokens, while rare or out-of-vocabulary words are broken down into subwords. This is useful because:

- **Frequent words:** Common words such as "the", "is", and "apple" can be represented as a single token, which reduces the number of tokens the model needs to process.
- **Rare words:** Less frequent or novel words can be split into subwords that the model can still process. For example, "unbelievably" might be split into "un", "believ", and "ably".

This allows Whisper to be both memory- and compute-efficient while retaining the flexibility to handle unseen words.

c. Managing Different Writing Systems

BPE also helps Whisper manage **different writing systems** (e.g., Latin, Cyrillic, Arabic, Chinese) by tokenizing text at a subword level rather than word level. This is particularly important because languages with different scripts or writing systems often have different word formation rules.

- **Latin-based scripts** (like English or French) might require fewer subword tokens since the words tend to be shorter and more regular.
- **Cyrillic scripts** (like Russian or Ukrainian) may have different subword patterns, and BPE can effectively handle this.
- **Arabic and Hebrew:** BPE can split the script into meaningful subword units that represent root words and prefixes.
- **Chinese:** In Chinese, where characters often correspond to whole words or concepts, BPE will break them down into smaller chunks that still convey meaning.

For example:

- "house" in English might be a single token: ['house'].
- "خانه" (house in Persian) might be split into subwords like ['خا', 'نه'].

d. Flexibility with Rare and Unseen Words

The BPE tokenizer's ability to break rare or previously unseen words into smaller components is crucial for handling out-of-vocabulary words in a more generalized way. This is particularly beneficial for **speech recognition systems** like Whisper, which must deal with a large variety of spontaneous speech, including slang, mispronunciations, and domain-specific terminology.

For example:

- If the word "cryptocurrency" is not in the model's vocabulary, BPE might break it into subwords like "crypto" and "currency", allowing the model to recognize and transcribe it correctly even though it wasn't seen during training.

4. BPE in Practice: Advantages for Whisper

- **Language Independence:** BPE allows Whisper to handle multiple languages with a single, unified token set. Whether it's English, Arabic, or Chinese, the model can work with a consistent subword vocabulary.
- **Improved Efficiency:** By reducing the number of tokens required to represent frequent words while retaining the ability to handle rare or out-of-vocabulary words, BPE improves both computational efficiency and performance.
- **Subword Flexibility:** Whisper can process input speech data more flexibly, enabling the model to adapt to diverse speech patterns, accents, and dialects by tokenizing speech into meaningful subword units rather than rigid word boundaries.
- **Handling Complex Morphologies:** Many languages have complex inflections or compound words (e.g., in German or Finnish), and BPE can tokenize these into smaller units that maintain their semantic meaning while keeping the number of tokens manageable.

5. Example of BPE Tokenization

To illustrate how BPE tokenizes text, consider the following example with a simple vocabulary:

- **Vocabulary:** ['l', 'o', 'v', 'e', 'd', 'love', 's', 'h', 'i', 'p']

The word "loved" might first be tokenized as ['l', 'o', 'v', 'e', 'd'], but after several BPE merges, it might become ['love', 'd'], efficiently representing the word.

Conclusion: Why Whisper Uses BPE

By using **Byte Pair Encoding (BPE)**, Whisper benefits from efficient tokenization that handles multiple languages, diverse writing systems, and varying word structures while keeping the token vocabulary small and efficient. This approach is crucial for speech-to-text models because it allows them to handle a wide variety of

speech patterns, dialects, and out-of-vocabulary words, improving both accuracy and generalization.

BPE's flexibility ensures that Whisper can process and transcribe speech effectively across languages, making it a powerful tool for automatic speech recognition in multilingual and diverse linguistic settings.

5.3.5 Multitask Learning

- Whisper was trained to perform **multiple tasks simultaneously**, including:
 - **Speech recognition (ASR)**: transcribe spoken audio into the same language.
 - **Speech translation**: translate spoken audio into English.
 - **Language detection**: detect what language is spoken.
 - **Voice activity detection**: detect silence or non-speech parts.
- This multitasking helps the model generalize across different languages and noisy environments.

What is Multitask Learning (MTL)?

Multitask Learning (MTL) is a machine learning technique where a single model is trained on **several related tasks at the same time**. Instead of building separate models for each task, one model learns to **share its internal knowledge** across multiple tasks, leading to better generalization, robustness, and efficiency.

The key intuition is:

"Learning to solve multiple related tasks together improves performance on each task, because the tasks reinforce each other."

It shares representations:

- Early layers extract **common features** useful for all tasks (like speech patterns, language structure, etc.)
- Later parts specialize in task-specific outputs.

How Whisper Uses Multitask Learning

Whisper was **explicitly trained** to perform several tasks **simultaneously** using the same model architecture.

These tasks are:

1. Automatic Speech Recognition (ASR)

- **Goal**: Convert **spoken audio** into **text** in the **same language**.

- Example:
 - Audio: "Bonjour, comment ça va ?"
 - Transcription: "Bonjour, comment ça va ?"
- Importance:
 - Main task for building high-quality transcription systems across many languages.

2. Speech Translation

- **Goal:** Convert **spoken audio** into **English text**, regardless of the original language.
- Example:
 - Audio: "Buenos días" (Spanish)
 - Translation: "Good morning"
- Importance:
 - Useful for making audio content (lectures, interviews, meetings) accessible to English-speaking audiences.
 - Forces the model to **understand both** the source language and English, making it better at generalizing across languages.

3. Language Detection

- **Goal:** Detect **what language** the speaker is using.
- Example:
 - Audio: "Guten Morgen" → Detected as German.
- Importance:
 - Before transcribing or translating, the model needs to know **which language** it is dealing with.
 - Important for multilingual speech systems that must adapt on the fly.

4. Voice Activity Detection (VAD)

- **Goal:** Detect when there is **speech** vs. **silence** or **non-speech** sounds (like background noise).
- Example:
 - Silence or "uhh..." noises are **ignored**.
- Importance:

- Improves transcription quality by **ignoring pauses, noise, and irrelevant sounds**.
- Useful in real-world applications like call centers, meeting transcription, and podcasts where background noise is common.

Why Multitask Learning Matters in Whisper

Training Whisper to solve all of these tasks together gave it major advantages:

Benefit	How It Helps
Better Generalization	Since the model learns from different perspectives (ASR, translation, detection), it becomes more robust to new, unseen data.
Robustness to Noise	Tasks like VAD help the model focus only on meaningful speech, improving results in noisy environments.
Multilingual Capability	Language detection + speech recognition + translation together force the model to "understand" many languages well.
Efficiency	One model can handle many different use cases (transcription, translation, detection) without needing multiple specialized models.

Table 5-4 Multitasks benefits

Thus, **Whisper is not just a speech recognition model** — it is an advanced **multitask system** capable of **understanding and working with complex, multilingual, noisy real-world audio**.

How Multitask Learning Is Implemented in Whisper

1. Special Control Tokens:

- Whisper uses special tokens to **tell the model what task it should perform**.
- Examples:
 - `<|transcribe|>` for transcription
 - `<|translate|>` for translation
 - `<|detect-language|>` for language ID
 - `<|nospeech|>` for silence.

2. Training Process:

- During training, the model is **randomly given one of these tasks** for each audio sample.

- It learns to predict the output based on the corresponding instruction token.
- **Loss functions** are computed based on the specific task.

3. Decoder Behavior:

- The **decoder** in the Transformer generates the outputs token-by-token **according to the given task**.
- Same encoder output → multiple possible types of outputs depending on the decoder context.

5.3.6 Zero-shot Capabilities

- Because of its diverse training, Whisper can **generalize to new tasks and new languages** without fine-tuning.
- It can:
 - Recognize **new accents**.
 - Transcribe **unseen languages**.
 - Perform **robustly in noisy environments**.
- This is known as **zero-shot learning**.

What is Zero-shot Learning?

Zero-shot learning (ZSL) means the model can successfully perform a task **without having seen examples of it during training**.

In other words:

"A model is said to have zero-shot ability when it can generalize to completely **new tasks, domains, or data** without any additional training (fine-tuning)."

Key idea:

Instead of memorizing tasks, the model **learns general patterns** that help it **reason and adapt**.

Important distinction:

- **Fine-tuning** = training on new data to adapt to a new task.
- **Zero-shot** = *no* extra training. The model immediately handles new tasks with the knowledge it already has.

How Whisper Achieves Zero-shot Learning

Whisper was trained on a **huge, very diverse dataset** of audio from the web. The training included:

- Many **different languages** (nearly 100+),
- **Various accents**,
- **Noisy recordings** (real-world background sounds),
- **Different tasks** (transcription, translation, etc.).

This diversity **forces Whisper to learn very general, robust audio-linguistic representations**, not just narrow patterns.

Because of this, even if Whisper encounters a language, accent, or environment it wasn't explicitly trained on, it can still perform reasonably well — without needing fine-tuning.

What Zero-shot Abilities Whisper Demonstrates

Recognize New Accents

- Whisper can **transcribe speech** from speakers with **regional or non-standard accents** it may not have seen before.
- Example: A strong Scottish English or Nigerian English accent can still be transcribed even if Whisper had very little training data in that exact accent.

Transcribe Unseen Languages

- Whisper can **partially understand and transcribe** languages that are **underrepresented** in its training data.
- It might not be perfect but often produces **useful, readable outputs**.
- Example: Low-resource African or indigenous languages.

Handle Noisy Environments

- Since it was trained with noisy web audio, it can **filter out background noise** and **focus on the speech** better than traditional models.
- Example: Recognizing speech in a busy street recording or during a live concert.

Why Zero-shot Matters

- **Scaling to New Users:** No need to retrain the model for every new speaker, language, or environment.
- **Accessibility:** Opens up ASR and translation technology to communities without massive labeled datasets.

- **Cost-Efficient:** Saves computation and resources by avoiding fine-tuning.

Zero-shot generalization is **one of the major breakthroughs** Whisper achieves compared to earlier speech models (like Wav2Vec, DeepSpeech, or Kaldi).

Technical Reasons for Whisper's Zero-shot Success

Factor	How It Helps
Massive and diverse training data	Exposure to wide variability makes the model less sensitive to specific domains.
Multitask learning setup	Learning transcription + translation + detection jointly teaches broader language understanding.
Transformer architecture	Self-attention mechanisms capture long-range dependencies and patterns useful across languages and conditions.
Use of control tokens	Directs the model to perform specific tasks flexibly without retraining.

Table 5-5 Reasons for Whisper's Zero-shot Success

5.3.7 Output with Timestamps

- Whisper can **output timestamps** directly with transcriptions.
- Useful for generating **subtitles**, **closed captions**, or for aligning text with audio recordings.

Timestamps are predicted through special **timestamp tokens** during decoding (e.g., <|0.00|>, <|5.20|>).

What Is Timestamped Output?

In speech processing, **timestamps** refer to **marking the exact times** (in seconds) when specific pieces of speech occur in the audio.

In Whisper, timestamps are automatically generated alongside transcriptions — the model not only produces the text, but also predicts **when** each segment of text was spoken.

Why it matters:

Allows **perfect alignment** between audio and text.

Critical for building **subtitles**, **closed captions**, **audio indexing**, **video editing**, **searchable podcasts**, etc.

How Whisper Predicts Timestamps

Whisper was trained to **predict timestamps directly** during the **decoding phase**.

Key idea:

- The model's **output vocabulary** includes **special timestamp tokens**, such as:
 - `<|0.00|>` → represents time 0.00 seconds,
 - `<|5.20|>` → represents time 5.20 seconds,
 - `<|10.00|>`, and so on.

Decoding process:

- While generating the text sequence, **Whisper inserts timestamp tokens** at the right points.
- Example output tokens:

`<|0.00|> Hello everyone <|2.50|> welcome to this tutorial.`

- This means:
 - "Hello everyone" was spoken starting at 0.00 seconds,
 - "welcome to this tutorial" started at 2.50 seconds.

Prediction mechanism:

- The model "decides" when a timestamp should appear based on **audio features** and **language modeling context** during decoding.
- The timestamps are **not just heuristics** (like splitting by silence); they are **learned predictions** during training.

Granularity:

- Timestamps are generated roughly every 0.02–0.5 seconds depending on the model settings and decoder configuration.

Benefits of Having Timestamps Directly

Benefit	Why It's Important
Efficient Subtitle Generation	Directly create SRT, VTT, or subtitle files with accurate timing.

Benefit	Why It's Important
Improved Accessibility	Essential for closed captions for the hearing-impaired community.
Audio-Text Alignment	Enables searching or jumping to parts of audio based on text queries.
No Need for Forced Alignment Post-processing	Earlier speech-to-text pipelines needed separate alignment models (like Gentle, Aeneas) after transcription; Whisper does it all in one step.
Real-time Applications	Useful for live captions, meeting transcription software, and real-time translation apps.

Table 5-6 Benefits of Having Timestamps Directly

Timestamps vs. Other Systems

Older speech recognition models (like DeepSpeech, Kaldi, or Wav2Vec2) usually **did not natively predict timestamps**.

They either:

- Outputted raw text only, and
- Required **separate forced alignment tools** (which were slower and error-prone) to align text back to audio.

Whisper skips this extra step by building timestamp prediction directly into its main model.

Technical Notes: How the Tokens Work

- During training, Whisper's data included text **with time alignment labels**.
- The model learns to **insert timestamp tokens** whenever there's a **natural break** in the audio.
- Timestamps are sampled at **regular intervals** across the possible range (for example, every 0.02s).

When decoding:

- If the model predicts a timestamp token, it moves the "clock" forward in its output.
- The output text is then anchored relative to the current timestamp.

This enables:

- Phrase-level alignment (e.g., every sentence),

- Word-level alignment (with slight modifications and postprocessing),
- Potentially even real-time word-by-word subtitles with tweaks. (11)

5.4 Large Language Models and Retrieval-Augmented Generation

In recent years, the world has witnessed tremendous developments in the field of Artificial Intelligence (AI), a branch of computer science aimed at building systems capable of simulating human intelligence, such as understanding, learning, and decision-making. One of the most notable recent applications of AI is Large Language Models (LLMs).

5.4.1 How do LLMs Work?

The key to the success of modern LLMs is the transformer architecture. Before transformers were developed by Google researchers, modeling natural language was a very challenging task. Despite the rise of sophisticated neural networks —i.e., recurrent or convolutional neural networks— the results were only partly successful.

The main challenge lies in the strategy these neural networks use to predict the missing word in a sentence. Before transformers, state-of-the-art neural networks relied on the encoder-decoder architecture, a powerful yet time-and-resource-consuming mechanism that is unsuitable for parallel computing, hence limiting the possibilities for scalability.

Transformers provide an alternative to traditional neural to handle sequential data, namely text (although transformers have also been used with other data types, like images and audio, with equally successful results).(14)

5.4.2 Components of LLMs

The **main component** of large language models (LLMs) is the **Transformer**, a modern neural network architecture that differs from earlier types like recurrent neural networks (RNNs).

The goal of this architecture is to understand the relationships between words or tokens in a text by representing them using techniques called **embeddings**—which are numerical representations of words or sentences in a high-dimensional space.

What makes the Transformer unique is its ability to process these embeddings efficiently and in parallel, thanks to the **attention mechanism**. This parallel processing makes it especially suitable for training large-scale models like LLMs.

Examples of LLMs:

Model	Developer	Description
GPT (e.g., GPT-3.5, GPT-4)	OpenAI	Decoder-only model. Used in ChatGPT. Strong in conversation and text generation.
BERT	Google	Encoder-based model. Great at understanding sentence meaning and classification tasks.
Qwen	Alibaba Cloud (DAMO Academy)	Decoder-only Transformer. Optimized for instruction-following. Strong in Q&A and reasoning. Supports multilingual text. Good for RAG.
Llama	Meta	Lightweight and open-source. Good performance with fewer resources.
Mistral	Mistral AI (France)	Fast, open-source decoder-only model. Works well in multilingual environments.

Table 5-7 well-known LLMs

We used Qwen LLM as the core module to answer user questions.

Why Qwen-14B is a great choice for our project:

In our AI-based eLearning platform, we chose Qwen-14B, a powerful open-source Large Language Model developed by Alibaba, for several reasons. First, Qwen-14B supports multilingual understanding and generation, which is helpful in diverse educational settings. Second, it has 14 billion parameters, allowing it to comprehend complex database-related questions and deliver accurate, detailed answers suitable for academic use. Compared to other models like LLaMA or GPT-J, Qwen is optimized for instruction-following and reasoning tasks, which makes it highly compatible with our Retrieval-Augmented Generation (RAG) setup. This means the model doesn't just generate text — it can also retrieve accurate content from our database course materials, leading to high-quality, personalized answers. This level of accuracy is especially important in an educational system where correctness matters.

Extra reasons to prefer Qwen-14B:

- Open-source and free to use for academic purposes
- Supports RAG integration easily for course-specific content
- Well-optimized for question-answering and instructional dialogue

Model	License	Performance	Hardware Requirements	Advantages	Reference
Mistral 7B	Open Source	Outperforms LLaMA 2 13B – fast and efficient	Low – runs on consumer hardware	Lightweight, excellent for real-time apps like eLearning platforms	Mistral AI (2023) arXiv Paper
GPT-4	Closed Source – Paid	Very high top global benchmark	Very high – only via OpenAI API	Extremely intelligent, but expensive and limited control	GPT-4 Report
LLaMA 2 (13B)	Open Source (restricted use)	Great, but slower than Mistral 7B	High – due to larger size	High accuracy, good for research and academic use	Meta AI – LLaMA 2
Qwen-14B	Open Source (Apache 2.0)	Strong performance in multilingual Q&A and instruction-following tasks	High – requires GPU	Accurate, supports RAG, good for education and database teaching scenarios	Alibaba Cloud, DAMO Academy (2023) arXiv:2309.16609
T5	Open Source by Google	Strong for translation and summarization	Medium	Excellent for text-to-text tasks (not ideal for chat or RAG systems)	T5 Paper

Table 5-8 reasons to prefer Qwen-14B

5.4.3 Qwen-14B: What is it?

Qwen-14B is a Large Language Model (LLM) developed and released by Alibaba Group. At its core, an LLM is a deep learning model designed to understand and generate human-like text based on the data it's trained on. What sets Qwen-14B apart is its sheer size and the breadth of its training data. With a staggering 3T tokens under its belt, it's the longest trained model of its kind.

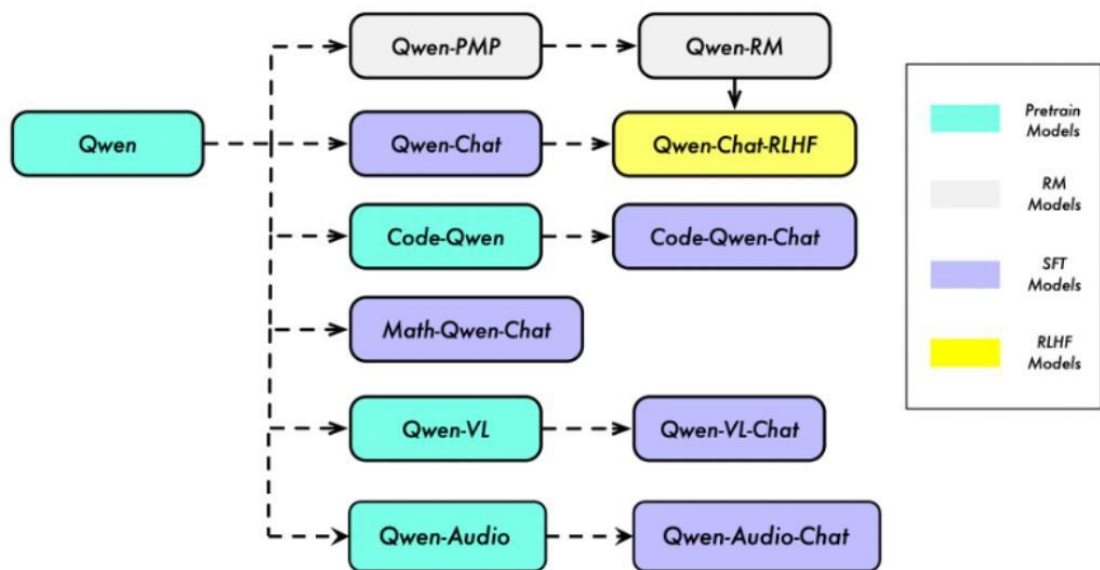


Figure 5-13 model family structure of Qwen

But size isn't the only thing that's impressive about Qwen-14B. It's available in five distinct versions, each tailored for specific tasks:

- **Base:** The foundational model upon which other versions are built.
- **Chat:** Optimized for conversational AI and chatbot applications.
- **Code:** Designed to understand and generate code across multiple programming languages.
- **Math:** Tailored for mathematical computations and problem-solving.
- **Vision:** A version that synergizes text and image processing capabilities.

Furthermore, Qwen-14B is trained for tool usage, making it a versatile asset in various tech domains

Model Specifications and Versions: A Technical Dive

When we talk about **Qwen-14B**, it's essential to understand its technical foundation. The model's architecture is a testament to the advancements in AI and deep learning. Its training on 3T tokens not only makes it the longest trained model but also equips it with a vast knowledge base, allowing it to excel in various tasks.

Model Versions and Their Significance

Qwen-14B isn't a one-size-fits-all model. Its five versions ensure that it can be applied in diverse domains with optimal results:

- **Base Version:** This is the core of Qwen-14B. It serves as the foundation upon which other specialized versions are built. It's versatile and can handle a wide range of general tasks.
- **Chat Version:** In the era of digital communication, chatbots and conversational AIs are crucial. The Chat version of Qwen-14B is optimized for this very purpose, ensuring human-like interactions.
- **Code Version:** With the tech industry booming, there's a growing need for AIs that can understand and generate code. This version of Qwen-14B does just that, making it a valuable asset for developers.
- **Math Version:** For tasks that require mathematical computations and problem-solving, the Math version is the go-to choice.
- **Vision Version:** In an age where visual content dominates, this version's ability to process both text and images makes it stand out.

Each version of Qwen-14B is a testament to Alibaba's commitment to pushing the boundaries of what AI can achieve.

Architecture

The **QWEN** model is built on a modified version of the Transformer architecture, inspired by the open-source LLaMA framework, which is known for its strong performance.

Key modifications include:

- **Embedding and Output Projection:**
We use untied embeddings (input and output are not sharing weights) to improve performance, even though it increases memory usage.
- **Positional Embedding:**
RoPE (Rotary Positional Embedding) is used to encode positional information, with FP32 precision to enhance accuracy.
- **Bias Usage:**
Most layers have no bias, but we keep bias in the attention layer (QKV) to improve generalization.
- **Normalization:**
We apply Pre-Norm and use **RMSNorm** instead of the standard LayerNorm, which gives similar results with better efficiency.
- **Activation Function:**
The activation function is **SwiGLU**, a mix of Swish and GLU, which performs better than common functions like GeLU in our tests.

- **Feed-Forward Layer Size:**

The size of the feed-forward network (FFN) is reduced from $4\times$ to $8/3\times$ the hidden size for improved efficiency.

Pretraining and Data Sources: The Foundation of Qwen-14B's Knowledge

The prowess of **Qwen-14B** is not just a result of its architecture but also the vast and diverse data it's been trained on. Pretraining is the phase where the model learns from vast amounts of data, understanding patterns, semantics, and context. This section delves into the sources and methods used to train this behemoth.

Diverse Data for Comprehensive Learning

Qwen-14B's training data is a set of various sources, ensuring a holistic learning experience:

- **Web Documents:** A treasure trove of information, web documents provide a real-world context.
- **Encyclopedias:** These offer structured and factual information, enhancing the model's knowledge base.
- **Books:** Literature, both fiction and non-fiction, helps the model understand narratives, emotions, and diverse writing styles.
- **Codes:** For its Code version, Qwen-14B was exposed to multiple programming languages, making it adept at understanding and generating code.

Ensuring Data Quality

Quality trumps quantity. While Qwen-14B had access to vast amounts of data, ensuring its quality was paramount:

- **Manual Review:** Random samples of texts from various sources were manually reviewed to ensure high standards.
- **Selective Upsampling:** Specific datasets from certain trusted sources were up-sampled to emphasize their importance in training.

Architectural Tweaks for Enhanced Performance

Qwen-14B's architecture underwent several tweaks to enhance its performance:

- **RoPE with FP32 Precision:** While RoPE (Rotary Position Embeddings) is a common feature in many models, Qwen-14B uses FP32 precision for the inverse frequency matrix, setting it apart.

- **Bias Modifications:** Biases were meticulously managed, with some being removed and others added, especially for the QKV layers, ensuring optimal performance.

How is Qwen-14B different from other LLMs?

Qwen-14B stands out due to its size, being trained on 3T tokens, making it the longest trained model of its kind. Additionally, it's available in five distinct versions: Base, Chat, Code, Math, and Vision, each optimized for specific tasks.

The general procedure of building an assistant model includes pretraining and post-training, where the latter mostly consists of SFT and RLHF. As to pretraining, similar to previous LLM, GPT-3, Llama, Qwen is a Transformer-based language model pretrained by the task of next token prediction. For simplicity and stability, we did not introduce more tasks for the language model but focus on model size scaling and data scaling. For now, we have developed 5 models of different sizes, 4 of which are open sourced. Specially, now release Qwen-1.8B, Qwen-7B, Qwen-14B, and Qwen-72B.

Alignment

We refer both techniques to the word “alignment” in post-training. Currently, it is consensus that we can obtain a chat model with a relatively small amount of finetuning data. We focus on improving the diversity and complexity of the SFT data and strictly control the quality by manual checking and automatic evaluation.

Based on a good SFT model, we can then explore the effects of RLHF. It is difficult to train RLHF, specifically PPO-based method, Besides the training instabilities of PPO, another key to the final performance is the quality of reward model. Therefore, we have spent efforts in building a reliable reward model by reward model pretraining on large-scale comparison data and finetuning on carefully labeled comparison data of high quality. In comparison with the SFT model, we find that the RLHF model is more creative and follows the instructions better, and thus its generated responses are more preferred by human annotators.(15),(16).

5.4.4 Challenges and Limitations of LLMs

Large Language Models (LLMs) like GPT-4, Llama, DeepSeek and Mistral are powerful tools for generating human-like text. However, they also come with inherent limitations that affect their reliability, especially in education or knowledge-based systems.

Here are the main challenges:

1. Wrong Answers

LLMs sometimes "make up" facts or give inaccurate answers because they rely on pre-trained data, not live or updated information.

2. **Lack of Up-to-Date Knowledge**

Most LLMs are trained on data **before a certain date** (e.g., 2023), so they can't provide **latest updates** or course-specific knowledge unless fine-tuned.

3. **Transparency and Trust**

LLMs are often “black boxes,” making it hard to trust the source of the answers. This is risky in education where **accuracy matters**.

4. **No Personalization to Specific Domains**

A general-purpose LLM might not perform well in a specific topic unless it is trained or assisted with additional tools.

LLM vs RAG vs Fine-Tuning

Feature	LLMs Alone	RAG (Retrieval-Augmented Generation)	Fine-Tuning
Accuracy of Answers	May hallucinate or guess	Uses real documents — more factual	Learns from your data — high accuracy
Up-to-Date Information	Fixed knowledge (based on training data only)	Always up-to-date (you control the document base)	Needs retraining to include new knowledge
Customization to Course	General knowledge only	Pulls information from your course materials	Model adapts to your course after training
Explainability / Sources	No reference to sources	Can return the exact source documents	Still acts like a black box
Cost and Resources	No extra cost (ready to use)	Low cost — just upload documents for context	High cost — requires expensive training on GPUs
Speed of Development	Fast to deploy and test	Fast — simply upload documents and index them	Slower — requires data collection, cleaning, and training
Scalability	Easy to scale with general use cases	Easy to scale — just add new documents	Harder to scale — needs re-training for new topics
Maintenance / Updating	Requires training new models if knowledge is outdated	Simple — just update your documents	Requires complete retraining for updates

Domain Adaptability	Limited — performance decreases on domain-specific tasks	High — retrieves domain-specific content	High — if fine-tuned correctly on domain-specific examples
Knowledge Source	Static (pre-trained dataset)	Dynamic — combines pretrained model with real-time retrieval	Static — extends the pre-trained model with specific labeled datasets
Transparency	Opaque reasoning process (black box)	Transparent — can trace answers to real sources	Still lacks transparency despite improvements
Hallucination Risk	High — model may generate incorrect or fictional information	Low — answers grounded in retrieved facts	Medium — depends on training data quality

Table 5-9 LLM vs RAG vs Fine-Tuning(17)(18)

5.4.5 What is Retrieval-Augmented Generation (RAG)

It is a sophisticated system designed to enhance the capabilities of large language models by combining them with powerful retrieval mechanisms. It's essentially a two-part process involving a retriever component and a generator component.

component and their roles in the overall process:

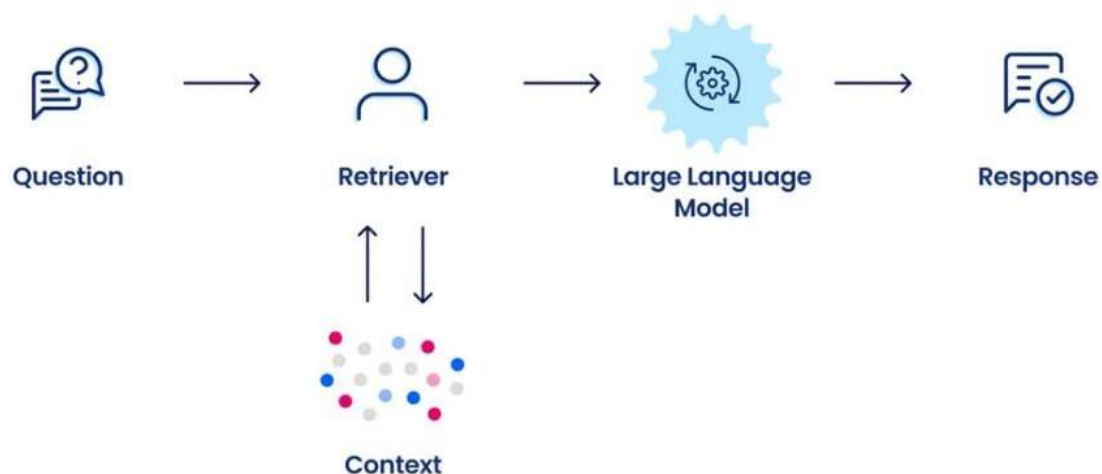


Figure 5-14 component and their roles in the overall process

5.4.5.1 Retriever:

- **Function:** The retriever's job is to find relevant documents or pieces of information that can help answer a query. It takes the input query and searches a database to retrieve information that might be useful for generating a response.

Types of Retrievers:

- **Dense Retrievers:** These use neural network-based methods to create dense vector embeddings of the text. They tend to perform better when the meaning of the text is more important than the exact wording since the embeddings capture semantic similarities.
- **Sparse Retrievers:** These rely on term-matching techniques like TF-IDF or BM25. They excel at finding documents with exact keyword matches, which can be particularly useful when the query contains unique or rare terms.(17),(19)

In the project, I used a **Dense Retriever** implemented through the **Chroma vector store** and **Ollama Embeddings**. Dense retrievers work by encoding documents and queries into high-dimensional vectors and retrieving results based on vector similarity (e.g., cosine similarity). This setup allows for **semantic search**, meaning the retriever finds relevant answers even if the query wording is different from the source text.

5.4.5.2 Generator:

- **Function:** The generator is a language model(Qwen) that produces the final text output. It takes the input query and the contexts retrieved by the retriever to generate a coherent and relevant response.
- **Interaction with Retriever:** The generator doesn't work in isolation; it uses the context provided by the retriever to inform its response, ensuring that the output is not just plausible, but also rich in detail and accuracy. (19)

```
578     session_state['llm'] = OllamaLLM(  
579         base_url="http://localhost:11434",  
580         model="qwen3:14b",  
581         verbose=True,  
582         callbacks=[StreamingStdOutCallbackHandler()],  
583     )  
584
```

Figure 5.4.5 .LLM code

5.4.5.3 The workflow of a Retrieval-Augmented Generation (RAG) system

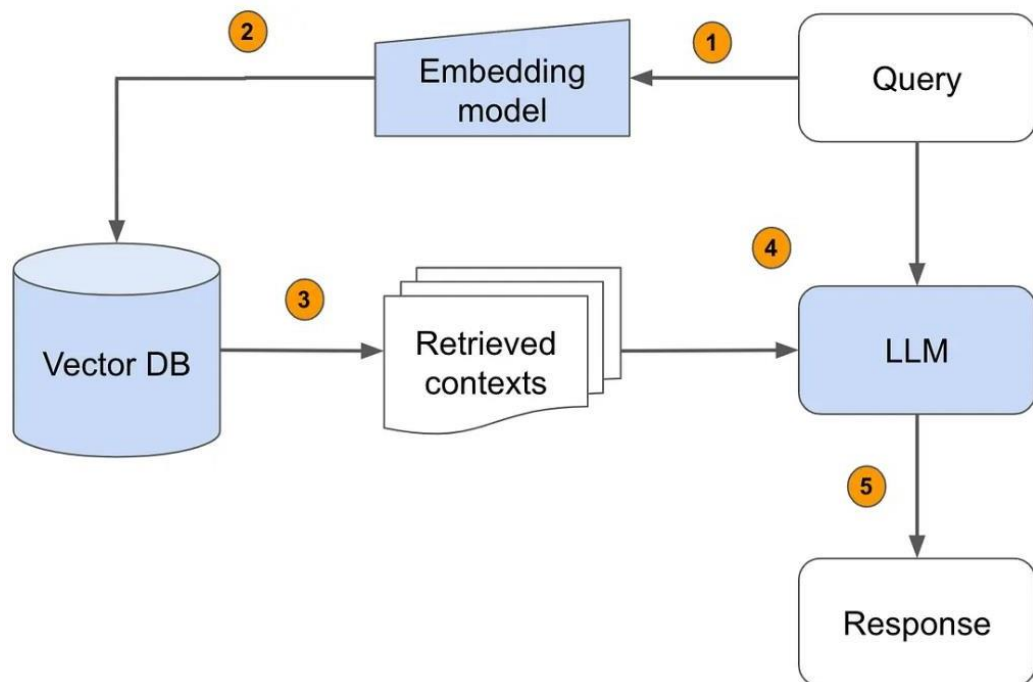


Figure 5-15 The workflow of a Retrieval-Augmented Generation (RAG) system

Vector DB creation

```
---
573     session_state['vectorstore'] = Chroma(
574         persist_directory='vector_store',
575         embedding_function=embedding_function
576     )
---
```

Figure 5-16 vector database code

Before we can start building our RAG application, we need to first create our vector DB that will contain our processed data sources.

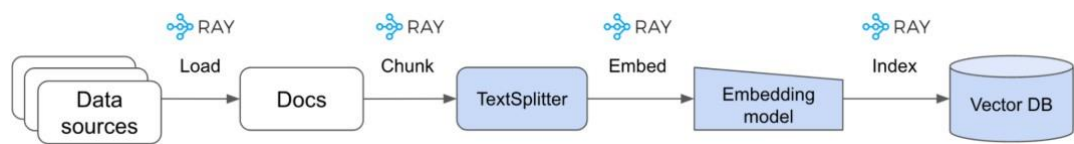


Figure 5-17 create vector DB

Load data

We're going to start by loading data from the Database Book, which is our knowledge base used, to local documents.

Chunk data

We now have a list of sections but we shouldn't directly use this as context to our RAG application just yet. The text lengths of each section are all varied and many are quite large chunks So, must chunked into smaller parts.(20)

Embed data:Converting Queries into Vectors

When interacting with a system that utilizes an embedding model, the process typically works in the following way:

User Query

The user types or submits a query, such as "CREATE TABLE student". This is the input text that the system needs to process.

Once the query is received, it is passed to an embedding model. The primary goal of this model is to convert the query into a numerical representation known as a vector.

- A vector is simply a list of numbers that represents the semantic meaning of the input text.
- The embedding model processes the query by analyzing the relationships between words and understanding the overall meaning of the sentence.

For example, in this case, the query "CREATE TABLE student" might be understood as a database-related query to create a table named "student," and the model will generate a vector based on this understanding.

Numerical Representation (Vector)

The result of the embedding model is a vector. This vector is the embedding of the query, representing it numerically.

Example vector for a query might look like:

[0.23, -0.12, 0.34, 0.56, ...] size may be 512.

This vector can be compared with other vectors to measure how similar the queries are to other data, such as database-related commands.

Why Use Vectors?

Using vectors has several advantages:

1. Efficiency: Vectors are numerical and can be processed quickly .
2. Semantic Understanding: Similar meanings (e.g., "CREATE TABLE" and "ALTER TABLE") are mapped to similar vectors, allowing the system to understand and compare their relationships efficiently.

Example: Query to Vector Conversion

Let's consider an example. Imagine you ask a system, "CREATE TABLE student".

1. The embedding model will convert this query into a vector (a numerical list).

For example, the query might become the vector:

[0.23, -0.12, 0.34, 0.56, ...]

2. This vector represents the meaning of your query, which in this case refers to an SQL command to create a table for students.
3. The system can then compare this vector to other vectors, such as those representing different SQL commands, to find the most relevant actions or results.

The Power of Embedding Models

By converting queries into vectors, the embedding model allows the system to understand the meaning of the text, even though it processes it as numbers. This enables the system to capture the nuances of language and context, which traditional keyword-based systems might miss. This approach makes it much easier for the system to compare and process text based on meaning, improving its ability to respond to user queries, provide recommendations, and perform searches.

Now, the example query has been replaced with "CREATE TABLE student", making the explanation more relevant to database operations.

We are using the **mxbai-embed-large** model through Ollama, which runs locally on my machine (localhost). This model is designed to convert user queries (like "CREATE TABLE student") into numerical vectors that can be used for semantic search, retrieval, or understanding the meaning of input data.

Why I Chose This Over Other Models

- Runs **locally** (no internet required)
- Offers **strong semantic understanding**
- Good for **privacy** and **offline** systems

Compared to:

Other Models	Limitations
OpenAI Embedding API	Needs API key and internet; may cost money for frequent use
Hugging Face Transformers	Also good but may need more setup or GPU resources
Traditional methods (Word2Vec)	Don't capture full sentence meaning (just word-level similarity)

Table 5-10 other models

- I chose **mxbai-embed-large** because it helps my system **understand full queries**, like database commands or user requests, in a deep way.
- It creates **embeddings (vectors)** that allow my app to find related queries or match input with stored knowledge.
- And because it runs **locally**, it's great for privacy and performance.

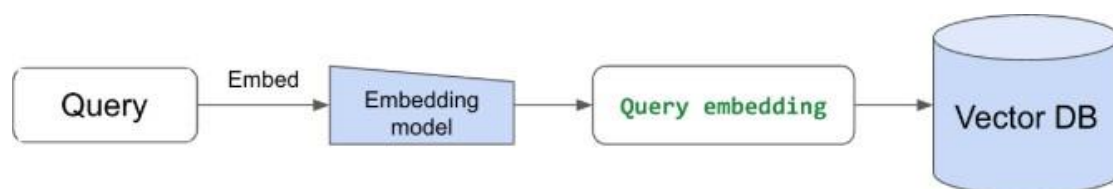


Figure 5-18 flow of Query Embedding

Query Retrieval

With our embedded chunks indexed in our vector database, we're ready to perform retrieval for a given query. We'll start by using the same embedding model we used to embed our text chunks to now embed the incoming query.

Response Generation

We can now use the context to generate a response from our LLM(Mistral). Without this relevant context that we retrieved, the LLM may not have been able to accurately answer our question. And as our data grows, we can just as easily embed and index any new data and be able to retrieve it to answer questions.

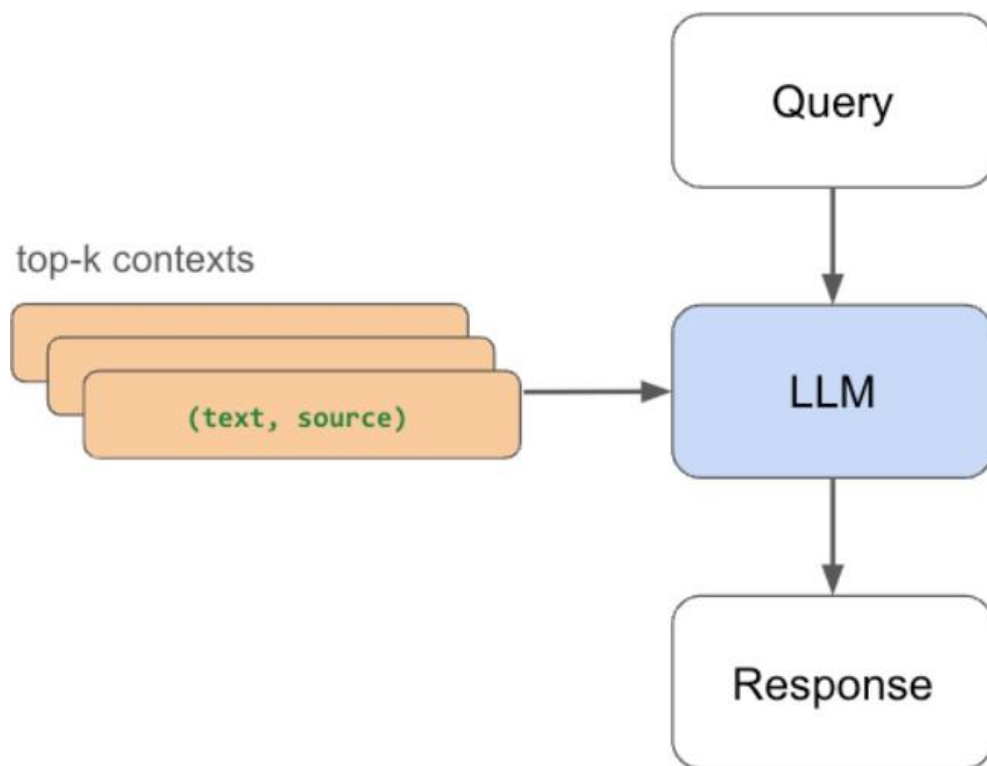


Figure 5-19 Flow of Response Generation

Agent

Let's combine the context retrieval and response generation together into a convenient query agent that we can use to easily generate our responses. This will take care of setting up our agent (embedding and LLM model), as well as the context retrieval, and pass it to our LLM for response generation.

For summary:

1. **Query Processing:** It all starts with a query. This could be a question, a prompt, or any input that you want the language model to respond to.
2. **Embedding Model:** The query is then passed to an embedding model. This model converts the query into a vector, which is a numerical representation that can be understood and processed by the system.
3. **Vector Database (DB) Retrieval:** The query vector is used to search through a vector database. This database contains precomputed vectors of potential contexts that the model can use to generate a response. The system retrieves the

most relevant contexts based on how closely their vectors match the query vector.

4. Retrieved Contexts: The contexts that have been retrieved are then passed along to the Large Language Model (LLM). These contexts contain the information that the LLM uses to generate a knowledgeable and accurate response.
5. LLM Response Generation: The LLM takes into account both the original query and the retrieved contexts to generate a comprehensive and relevant response. It synthesizes the information from the contexts to ensure that the response is not only based on its pre-existing knowledge but is also augmented with specific details from the retrieved data.
6. Final Response: Finally, the LLM outputs the response, which is now informed by the external data retrieved in the process, making it more accurate and detailed.(21)

The top frameworks/tools for building a RAG-based eLearning project

When developing an eLearning platform powered by Retrieval-Augmented Generation (RAG), choosing the right framework is essential. Below are the top three recommended tools that combine usability, performance, and educational potential:

- 1. RAGFlow** RAGFlow is a lightweight and friendly framework for quickly building a RAG pipeline. It is ideal for educational settings due to its simplicity and ease of setup. (22)
- 2. Haystack (with Open Web UI)** Haystack by deepset is a production-ready, open-source RAG framework with an integrated web UI for querying and evaluating responses. It supports modular architecture using pipelines, retrievers, generators, and document stores.(23)(24)
- 3. LangChain** LangChain is a powerful framework that allows developers to build complex LLM-based applications using “chains” of modular components. It supports memory, agents, tools, and custom workflows for RAG and more. (25)

5.4.6 Knowledge Base for eLearning Projects

A **Knowledge Base (KB)** plays a crucial role in enhancing an eLearning platform by storing structured or unstructured content that learners can easily query for answers. As part of the project, I explored different knowledge base tools to understand how they support the Retrieval-Augmented Generation (RAG) framework for delivering accurate and contextual responses. I experimented with various formats like .txt files, PDFs.

Benefits:

1. The integration of the **knowledge base** into the **RAG** pipeline is essential for ensuring that the AI-powered system can retrieve relevant information efficiently and generate accurate, contextually aware answers.
2. **Continuous Update and Maintenance** As the content in the knowledge base is updated (e.g., new textbooks, tutorials, research papers), the system should be designed to handle continuous updates. This ensures that the learning platform remains current with the latest information, ensuring learners have access to the most accurate and up-to-date knowledge.

In the project, as part of developing this eLearning project using **RAG (Retrieval-Augmented Generation)** systems, I experimented with creating several structured knowledge bases focused on **database topics**. These iterations were designed to evaluate how effectively the system could retrieve relevant information and assist learners with database-related questions.

5.4.6.1 Stages of Knowledge Base Development:

1. **Initial Stage – Create Table Only** The first version of the knowledge base included only basic CREATE TABLE statements. This served as a foundation to test how the model interpreted simple table definitions.
2. **Schema Drafting – Rough Schema** I enhanced the knowledge base by including realistic table schemas with specific field types and constraints, such as id INT PRIMARY KEY, email VARCHAR (100), etc.
3. **Table Relationships** I incorporated FOREIGN KEY constraints and relational mapping between tables to simulate real-world database relationships and test if the model could understand table interconnections.
4. **ER Modeling and Bridge Tables** To address more advanced scenarios, I added ER (Entity-Relationship) concepts and included **bridge tables** for handling many-to-many relationships. This helped test the system's comprehension of ER-to-relational mapping. (20)
5. **SQL Query Examples** In the final stage, I added a variety of **SQL query examples**, such as SELECT, JOIN, INSERT, and UPDATE statements. This helped evaluate the model's ability to assist users with practical SQL tasks and debugging.

Key chapters used:

- Relational Model Concepts
- SQL Queries and Modifications
- ER-to-Relational Mapping
- Database Design and Normalization

Purpose of These Experiments:

These iterations are aimed at:

- Explore how the structure and detail level of a knowledge base affect retrieval quality.
- Provide the RAG system with both theoretical and practical content.
- Test how well the system could assist learners with SQL writing, database design, and debugging.

5.4.7 Prompt Engineering

Prompt engineering is the process of designing and fine-tuning the input (prompt) given to a model to ensure that it generates the desired output in a consistent and efficient manner. In the context of an eLearning system that utilizes a **Retrieval-Augmented Generation (RAG)** system, prompt engineering is crucial because it influences how the model interprets the question, the context it retrieves, and the final answer it provides to the user.

The Role of Prompts in RAG Systems

In a RAG-based eLearning project, prompts are essential for ensuring that the system provides contextually accurate, relevant, and understandable answers. The primary goals of prompt engineering in such a project include:

- **Contextualization:** Ensuring the model understands the context of the question based on both the history and the knowledge base content.
- **Clarity and Specificity:** Making sure the prompt is clear and specific so that the model can generate accurate and focused responses.
- **Interaction Flow:** Ensuring the model maintains a coherent flow of interaction, especially in long or complex conversations.

5.4.7.1 Components of a Prompt

A typical prompt used in a RAG system may have several components which help direct the model's behavior:

1. **Context:** The knowledge base or documents retrieved by the RAG system.
2. **History:** The history of previous interactions or questions, which allows the model to provide responses that are consistent with past conversations.
3. **Question:** The current query from the user, for which the model needs to provide an answer.

4. **Instructions or Guidelines:** Specific instructions on how the model should behave. For example, it may be said to respond in a professional tone or to provide answers with specific citations.

5.4.7.2 Prompt Engineering Process

1. **Designing the Template:** The first step is designing the basic prompt structure that will be used across various interactions. This may involve setting up placeholders for variables like "context", "history", and "question".
2. **Fine-Tuning the Prompt:** After creating the initial prompt, we test it with various queries and responses to see how well it works. Based on these tests, we make adjustments to the prompt to improve the model's accuracy, tone, or other qualities. Fine-tuning might involve modifying the structure of the prompt or adjusting how much context/history is included.
3. **Feedback Loop:** The model's performance is evaluated continuously. Based on feedback and observed behavior, further refinements to the prompt are made. This iterative process ensures that the model's responses become more accurate and relevant over time.

5.4.7.3 Prompting techniques

- **General prompting / zero shot** A zero-shot prompt is the simplest type of prompt. It only provides a description of a task and some text for the LLM to get started with. This input could be anything: a question, a start of a story, or instructions. The name zero-shot stands for 'no examples'.

Name	1_1_movie_classification		
Goal	Classify movie reviews as positive, neutral or negative.		
Model	gemini-pro		
Temperature	0.1	Token Limit	5
Top-K	N/A	Top-P	1
Prompt	Classify movie reviews as POSITIVE, NEUTRAL or NEGATIVE. Review: "Her" is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. I wish there were more movies like this masterpiece. Sentiment:		
Output	POSITIVE		

Table 1. An example of zero-shot prompting

Figure 5-20 Table example of zero-shot prompting

- **One-shot & few-shot**

When creating prompts for AI models, it is helpful to provide examples. These examples can help the model understand what you are asking for. Examples are

especially useful when you want to steer the model to a certain output structure or pattern.

A **one-shot prompt** provides a single example, hence the name one-shot. The idea is the model has an example it can imitate to best complete the task.

A **few-shot prompt** provides multiple examples to the model. This approach shows the model a pattern that it needs to follow. The idea is similar to one-shot, but multiple examples of the desired pattern increases the chance the model follows the pattern.(22)

Goal	Parse pizza orders to JSON		
Model	gemini-pro		
Temperature	0.1	Token Limit	250
Top-K	N/A	Top-P	1
Prompt	Parse a customer's pizza order into valid JSON: EXAMPLE: I want a small pizza with cheese, tomato sauce, and pepperoni. JSON Response: ... <pre>{ "size": "small", "type": "normal", "ingredients": ["cheese", "tomato sauce", "peperoni"] }</pre> ... Continues next page...		

Figure 5-21 Table1 example of few-shot prompting

Prompt	EXAMPLE: Can I get a large pizza with tomato sauce, basil and mozzarella <pre>{ "size": "large", "type": "normal", "ingredients": [["tomato sauce", "bazel", "mozzarella"]] }</pre> Now, I would like a large pizza, with the first half cheese and mozzarella. And the other tomato sauce, ham and pineapple. JSON Response:
Output	<pre>... { "size": "large", "type": "half-half", "ingredients": [["cheese", "mozzarella"], ["tomato sauce", "ham", "pineapple"]] } ...</pre>

Table 2. An example of few-shot prompting

Figure 5-22 Table2 example of few-shot prompting

Impact of Effective Prompt Engineering

Effective prompt engineering helps ensure that:

- The **answers are relevant** to the user's question.
 - The **response quality** is maintained over long or repeated interactions.
 - The model's performance is **consistent** and accurate.
- the RAG system can provide users with contextually informed, coherent, and useful answers—enhancing the overall eLearning experience.

5.4.8 Experimentation with RAG and its Limitations

In the attempts to implement and experiment with the **RAG Flow** system, we encountered several limitations that hindered its performance and accuracy. While RAG Flow shows some promise, particularly in integrating retrieval-augmented generation, it consistently faced system breakdowns, especially when adjusting various parameters and prompts.

1. Challenges with Prompt Changes:

Despite trying multiple variations of prompts with a high number of words, the system **failed to provide more accurate answers**. Even when modifying the prompt's wording significantly, the model still struggled to generate coherent or precise responses. This limitation was evident even when I experimented with different settings, such as changing the **temperature**, **similarity**, or adjusting **top-k** parameters.

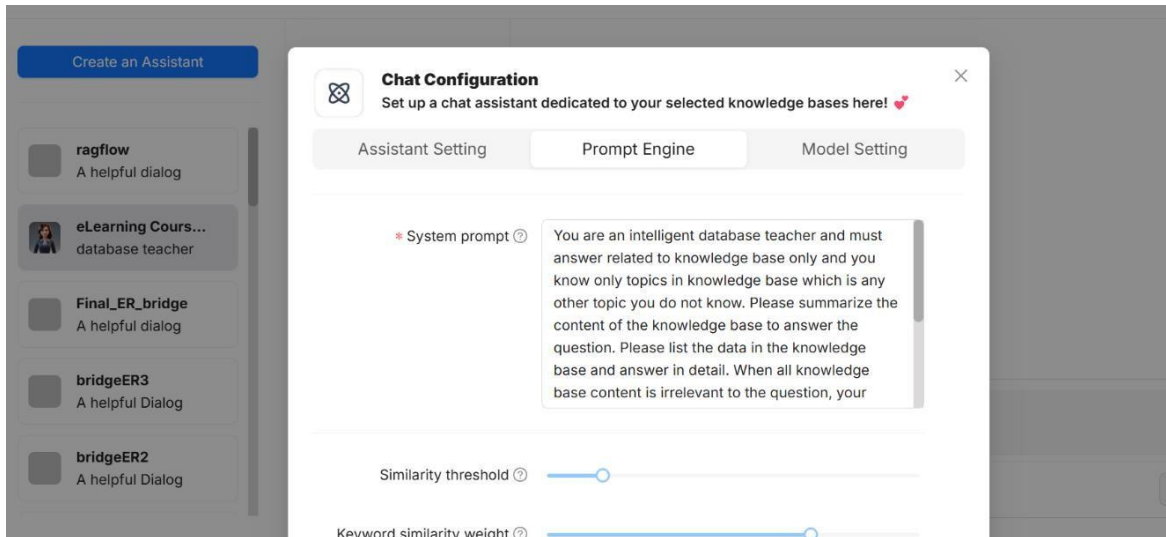


Figure 5-23 prompt with rag flow

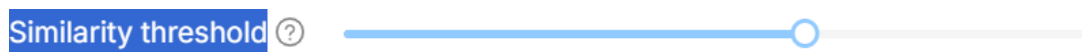


Figure 5-24 Response example of rag flow

2. Knowledge Base Limitations:

Another significant challenge I faced was the system occasionally going **out of the knowledge base**. This led to irrelevant or incomplete responses, especially when dealing with queries that required more specialized or deeper knowledge.

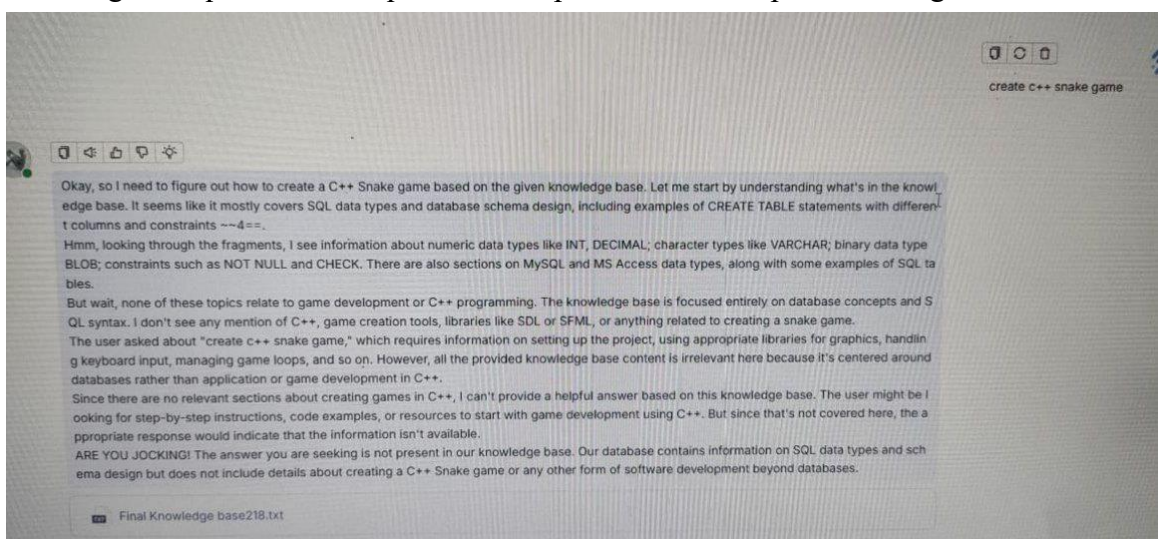


Figure 5-25 Response example2 of rag flow

3. DeepSeek Model Experiments:

Initially, I used the **DeepSeek LLM** with 128k parameters, which provided suboptimal results. Upon further experimentation, I tried **DeepSeek 14B** parameters, hoping for better performance, but the results were still unsatisfactory. The reasoning models seemed to fail, particularly when the system was engaged in general conversational tasks. I then reduced the model to **DeepSeek 1.5B**, but it still didn't meet expectations, continuing to struggle with accurate reasoning and response generation.

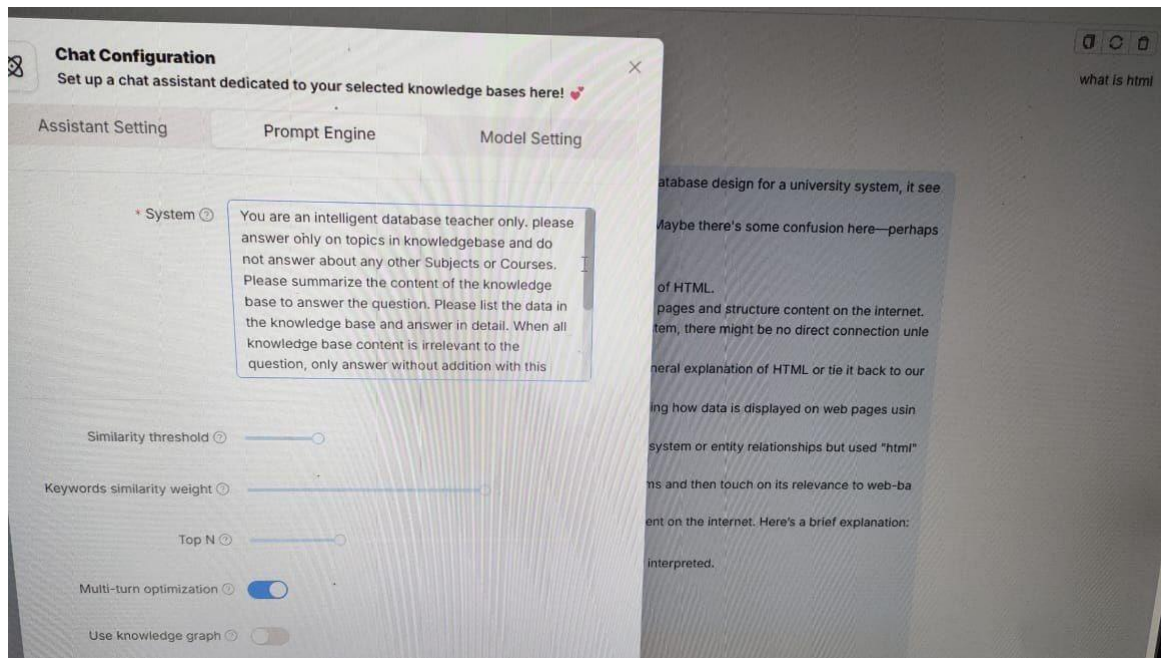


Figure 5-26 Figure prompt of rag flow with DeepSeek Model

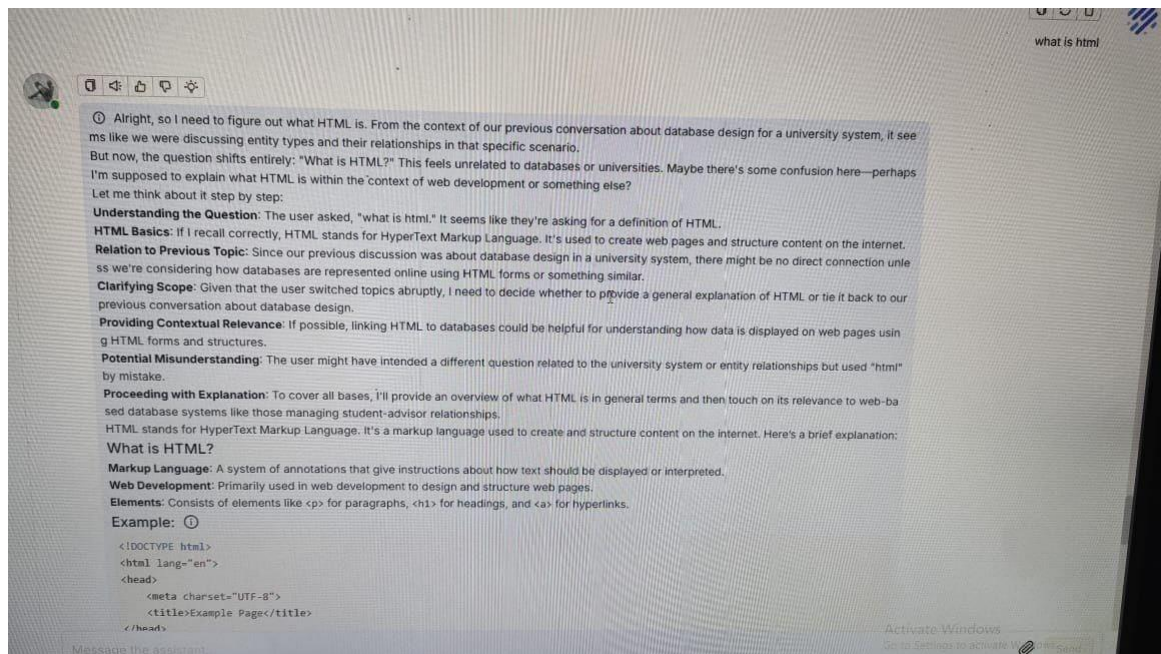


Figure 5-27 Response example of rag flow with DeepSeek Model

4. Switching to Llama 3.2:

In search of a more reliable solution, I tried **Llama 3.2**, which has 3B parameters. While it provided some improvements, it still **went out of context** in certain queries. RAG Flow worked reasonably well, but the model accuracy and contextual relevance were not ideal.

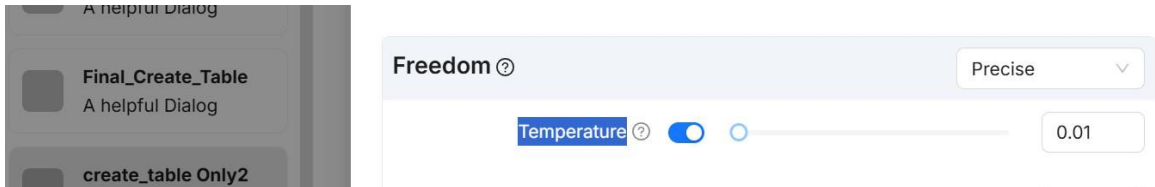


Figure 5-28 (Temperature)hyperparameter setting

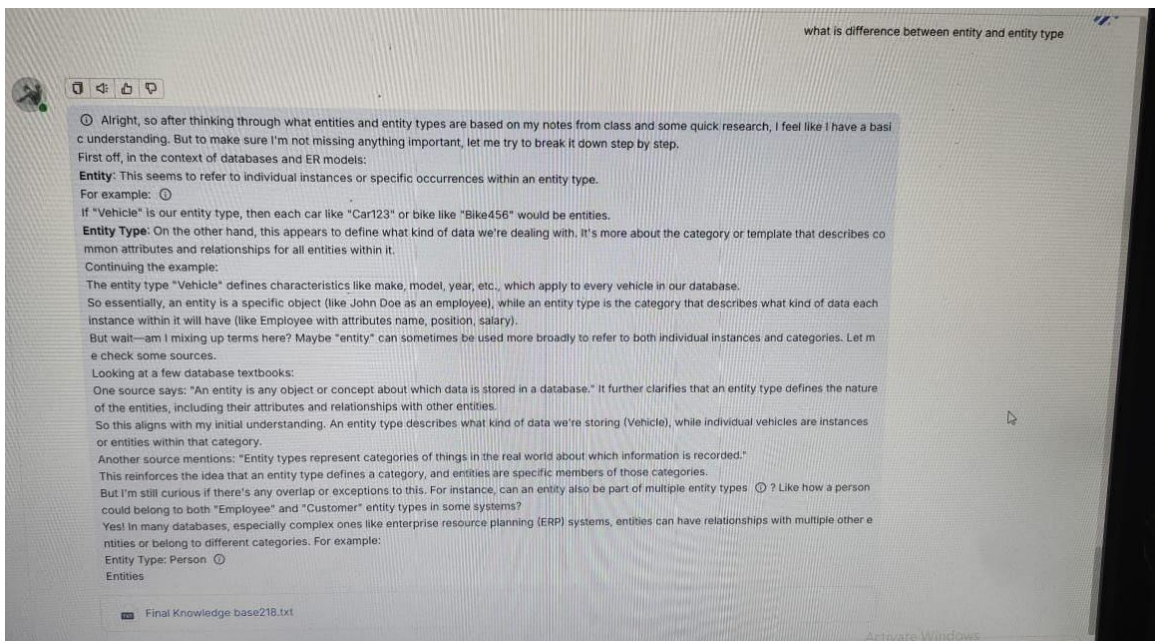


Figure 5-29 Response example1 with rag flow with Llama 3.2

Transition to Haystack in OpenWeb UI:

Since **OpenWeb UI** utilizes **Haystack** as its core retrieval-augmented generation system, I decided to experiment with it as well. Despite its capabilities, I encountered similar issues to RAG Flow, particularly with accuracy and contextual relevance. **Haystack** in OpenWeb UI also struggled with generating accurate responses, especially with complex queries, and sometimes went out of the knowledge base, providing irrelevant or incomplete answers.

Switching to LangChain with Qwen:

After facing these challenges with both **RAG Flow** and **Haystack**, I decided to switch to **LangChain**, which is better suited for handling **chatting tasks**. I experimented with the **Mistral** model, which offers better conversational abilities, and paired it with the **dense retriever** for improved retrieval accuracy. While this approach

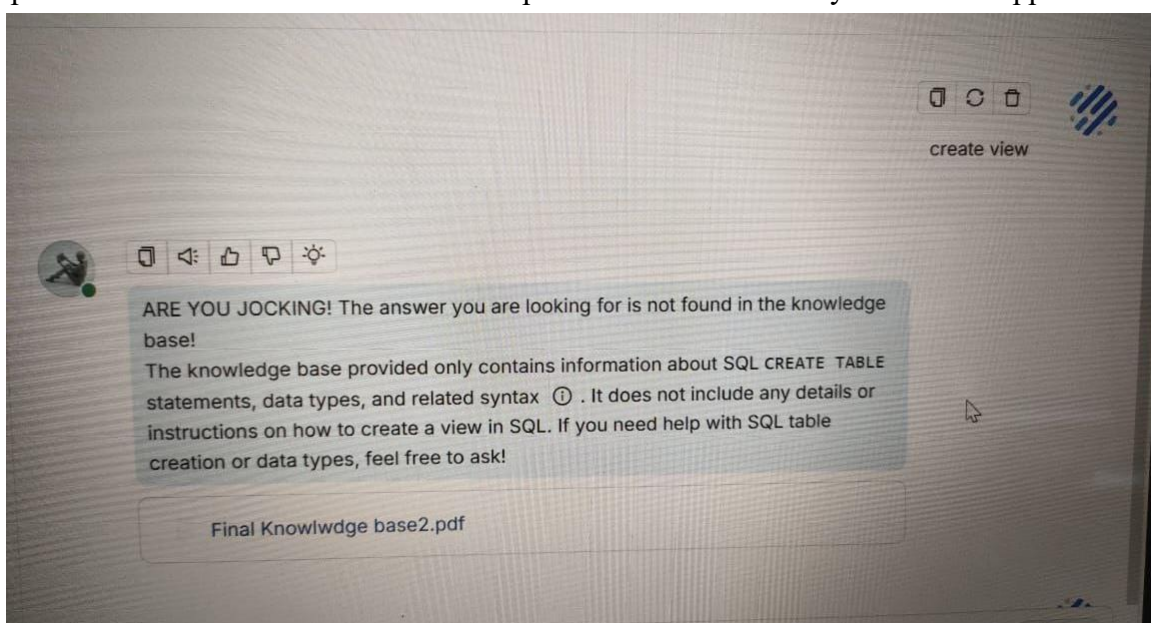


Figure 5-30 Response example2 with rag flow with Llama 3.2

worked better for some tasks, I encountered the challenge of **prompt engineering**, particularly when filtering questions related to databases versus those not related to the database. It remains a challenge to refine the prompts and ensure that the system only responds to relevant queries related to the domain.

Comparison of RAG Flow, Haystack (OpenWebUI), and LangChain for AI-based Question Filtering and Database Interaction

Feature / Tool	RAG Flow	Haystack (OpenWebUI)	LangChain
LLM Used	DeepSeek 14B, 1.5B, LLaMA 3.2 (3B) DeepSeek 128k (Smaller LLMs)	DeepSeek 14B, 1.5B, LLaMA 3.2 (3B)	Mistral (3B), suitable for chat

RAG Support	Built-in	Built-in (via Haystack)	Full support with flexible pipelines
Retriever Type	Hybrid	Sparse or hybrid (BM25 + Embeddings)	DenseRetriever (best for deep queries)
Speed / Performance	Slow, frequent timeouts, lags	Also slow, due to large models and complex retrieval	Fast, lightweight.
Accuracy (%)	80% (frequent breakdowns)	78% (similar issues)	~95% with better control or more
Out-of-domain Handling	Not well-handled	Not filtered properly	Filtered using classification prompt
Pros	Easily used as UI	Friendly UI via OpenWebUI	Highly customizable, modular, chat-friendly
Cons	LLMs not reliable in reasoning, low , poor reasoning and often off-domain	Same issues as RAGFlow , harder to customize and slower	Answers from the database still need improvement.

Table 5-11 Comparison of RAG Flow

Challenge: Filtering Questions Based on Relevance

As part of refining the system, I faced a challenge in ensuring that only relevant student queries related to the **database domain** are processed by the system. To address this, I explored an approach where the system first **filters questions** before passing them to the main RAG pipeline.

This involved thinking of a way to **classify** the input (student query) using a **simple prompt** to determine whether the question is **related to databases** or **outside the intended domain**.

- If the input is relevant, the system continues normally and provides an answer.
- If it is not, the system responds accordingly, stating that the question is not within the database scope.

This helped maintain **accuracy and contextual relevance**, especially in avoiding incorrect or misleading answers for unrelated queries.

5.5 How TTS Works: The Pipeline

5.5.1 Overview

A typical TTS system follows a multi-step pipeline to generate speech. While there are variations, modern systems generally have two main stages: a frontend and a backend. Other sources describe this as Text Analysis, Acoustic Model, and Vocoder.

5.5.2 Frontend (Text Analysis / Preprocessing)

This initial stage prepares the input text for synthesis and converts it into linguistic and prosodic features.

- **Text Processing/Normalization:** The system cleans and normalizes the raw input text into a format suitable for spoken output. This includes expanding abbreviations (e.g., "Dr." to "doctor"), handling special characters (e.g., converting "\$" to "dollar"), and converting numbers into their spoken form (e.g., "50.00" to "fifty") [45].
- **Phonetic Transcription (Grapheme-to-Phoneme):** The system converts the processed text into phonemes, which are the distinct sounds that make up words. This step is crucial for accurately capturing how the text should sound when spoken.

- **Linguistic Processing/Analysis:** This involves breaking down the text into phonetic components and understanding the text's structure and meaning. Techniques include tokenization, part-of-speech tagging, and syntactic parsing. Word segmentation may also be necessary for some languages.
- **Prosody Modeling/Prediction:** To make the speech sound more natural, this step adds rhythm, stress, and intonation, mimicking the flow of human speech. Without prosody, the speech would sound monotonous and robotic. Elements like pitch, stress, and intonation are essential for conveying emotions, emphasis, and conversational flow. In traditional systems, prosody was added separately, but modern neural models learn to handle it internally [43].

5.5.3 Backend (Speech Synthesis)

This stage generates the actual audio waveforms from the processed linguistic features [42] .

- **Acoustic Model:** This component converts the linguistic features from the text analysis phase into acoustic features. Earlier models used statistical methods like HMMs, DNNs, and RNNs, often generating features like mel-cepstral coefficients or fundamental frequency. Modern end-to-end neural models take characters or phonemes as input and generate high-dimensional acoustic features like mel-spectrograms or linear-spectrograms.
- **Waveform Generation (Vocoder):** This is the final step where the system converts the acoustic features into an actual audio waveform that can be played or saved. The vocoder takes the acoustic features as input and produces the final speech output. Different types of vocoders exist, including autoregressive, flow-based, GAN-based, VAE-based, and diffusion-based.

Together, the frontend and backend components create speech that is not only intelligible but also sounds natural and realistic[43].

5.5.4 TTS Synthesis Techniques

Several techniques are used for generating audio waveforms.

Concatenative Synthesis

Involves stitching together pre-recorded speech segments. While it can produce high-quality output, it may sound robotic if segments are not seamlessly joined. This method relies on a database of recorded speech segments.

Formant Synthesis / Parametric TTS

Uses mathematical models to simulate the human vocal tract. It offers more control but typically sounds more artificial.

Neural Synthesis / Neural TTS

The most recent and advanced method, using deep learning models to generate highly natural-sounding speech. **Modern neural TTS often follows a two-stage approach** where a neural network (like Tacotron 2) generates mel-spectrograms, and a neural vocoder (like WaveNet or HiFi-GAN) converts these spectrograms into audio waveforms [41] .

5.5.5 Key Neural TTS Models

5.5.5.1 Overview

Key models include WaveNet, Tacotron Series. These models use end-to-end deep learning processes.

5.5.5.2 Autoregressive vs Non-Autoregressive Models

Understanding the distinction between autoregressive and non-autoregressive models is crucial for selecting the appropriate approach for various tasks, particularly in machine learning and natural language processing. These are two popular models.

What Are Autoregressive Models?

Autoregressive means "self" (auto) using "past data to make predictions" (regressive). They generate outputs sequentially, where each output is conditioned on the previous outputs. The model predicts the next element in a sequence based on elements already generated. A classic example is the Recurrent Neural Network (RNN) or the Transformer model when used left-to-right. Imagine writing a sentence one word at a time, choosing the next word based on what you've already written. Autoregressive models focus on generating or predicting one element at a time based on earlier elements. Applications include language modeling, text generation, speech synthesis, and time series forecasting.

What Are Non-Autoregressive Models?

Non-autoregressive models do not rely on previous outputs to generate the next part. Instead, they try to predict or generate all parts simultaneously. They generate outputs in parallel, predicting all elements of the output sequence simultaneously. This allows for faster generation times as the model doesn't wait for the previous output. Examples include certain sequence-to-sequence models using masked attention. Imagine filling a form where you write name, age, and address all at once. Non-autoregressive models work this way – faster, but sometimes less accurate. Applications include machine translation, image captioning, and speech recognition.

Differences and Importance

Here is a quick comparison:

- **Architecture:** Autoregressive models often use RNN/Transformer (sequential); Non-autoregressive models often use Feed-forward/Normalizing Flow (parallel).
- **Alignment:** Autoregressive models handle alignment inherently; Some non-autoregressive models require external aligners, while others are self-aligning (like Glow-TTS with MAS).
- **Latent controllability:** Limited in some autoregressive models; Easy via latent manipulation in Flow-based non-autoregressive models like Glow-TTS.
- **Inference speed:** Grows linearly with output length (slow for long sequences) in autoregressive models; Fast (almost constant time) in non-autoregressive models.
- **Robustness:** Can lack robustness and produce attention errors like skips/repeats in some autoregressive models; Can be more robust by enforcing monotonic alignment in non-autoregressive models.
- **Naturalness of speech:** State-of-the-art in autoregressive models like Tacotron 2; Can be comparable or better (more expressive/precise) in advanced non-autoregressive models like Glow-TTS.

These models are crucial in real-world applications. Autoregressive models are used where **accuracy matters**, like generating realistic text or translating a novel. Non-autoregressive models are used where **speed is important**, like real-time translations during a live conversation. The choice depends on your needs. Non-autoregressive models aren't always less accurate; they perform well on simpler tasks. Sometimes, a combination of both can give the best results.

5.5.4 Cloud Model vs. Local Model

In the project, one of the first decisions I had to make was whether to use a **cloud-based TTS system** or a **local (offline) TTS model**. This choice was important

because each approach comes with its own trade-offs in terms of **Performance, Cost, Privacy, and Ease Of Deployment**. I took some time to evaluate both options based on the needs of the project

5.5.4.1 What Are Cloud and Local Models?

- **Cloud Model:** This type of model runs on remote servers (e.g., Amazon Web Services or Google Cloud). The TTS processing happens online, and the synthesized audio is streamed back to your application.
- **Local Model:** This model runs directly on your own device (e.g., your computer), processing the text and generating speech offline.

5.5.4.2 Why Might Choose a Cloud Model?

Advantages:

- **Scalability:** Cloud services can easily handle more users or complex tasks by dynamically adding resources (e.g., more servers).
- **Updates:** The provider can improve the model (e.g., fixing bugs or adding new voices) without needing to do anything.
- **No Hardware Hassle:** we don't need a powerful computer or specialized hardware; a phone or a basic device can access the service.

Drawbacks:

- **Internet Dependency:** It requires a stable internet connection, which might fail in remote areas or during network outages.
- **Privacy Concerns:** text data is sent to the cloud, so sensitive information might raise privacy considerations.
- **Cost:** Over time, cloud services typically incur fees based on usage.

5.5.4.3 Why Might Choose a Local Model?

Advantages:

- **Offline Use:** Works without an internet connection, perfect for areas with poor connectivity or secure environments.
- **Privacy:** Data stays entirely on your device, making it ideal for confidential projects (e.g., medical records).
- **Control:** You have more control over the model and can potentially tweak it or the hardware to fit your specific needs.

Drawbacks:

- **Hardware Needs:** Often requires a strong computer or dedicated hardware (like a GPU), which can be expensive or bulky.

- **Updates:** You must manually update or retrain the model, which can be a significant effort.
- **Limited Scalability:** It's harder to handle many users simultaneously on a single device.

5.5.5 Models Evaluated

To select the best TTS model for our project, we conducted a thorough evaluation of several systems, considering their architectures, features, and limitations. The models we investigated included a mix of cutting-edge neural models and practical cloud-based or local solutions.

5.5.5.1 Tacotron 2 (An Autoregressive Model)

Tacotron 2, developed by Google Research, is a well-known end-to-end autoregressive TTS system that set a high benchmark for natural-sounding speech.

Architecture and How it Works: Tacotron 2 is a **sequence-to-sequence model**. This means it takes a sequence (your input text) and transforms it into another sequence (the speech)[46].

- **An Encoder** first processes the input text, converting it into a numerical representation that the rest of the model can understand.
- **An Attention mechanism** then helps the model focus on the relevant parts of the text as it generates speech, much like how a human reader focuses on words as they speak them aloud.
- **A Decoder** (specifically, an autoregressive Recurrent Neural Network or RNN) then generates **mel-spectrogram frames** one by one. A mel-spectrogram is a visual representation of how the sound's frequencies change over time, essentially a detailed blueprint for the sound.
- **Post-Net** refines these mel-spectrograms, making them even more accurate.
- Finally, a **WaveNet vocoder** (or a similar neural vocoder) converts these mel-spectrograms into the raw audio waveform—the actual sound you hear.
- **Training Process:** Tacotron 2 requires large datasets of text and corresponding high-quality audio (like the LJSpeech dataset) for training. This training process is computationally intensive, demanding powerful hardware such as Graphics Processing Units (GPUs) or Tensor Processing Units (TPUs).
- **Features and Performance:** Tacotron 2 achieved impressive Mean Opinion Scores (MOS), a measure of perceived audio quality, very close to that of natural human speech (4.53). It primarily works with English but can be adapted to other languages with sufficient data.

Limitations:

- **High Latency:** Because it's autoregressive, it generates speech sequentially, which can lead to noticeable delays (1-2 seconds or more for longer sentences). This makes it less suitable for real-time interactive applications where immediate responses are crucial.
- **Computational Cost:** Both training and inference (generating speech from new text) are computationally demanding, requiring significant processing power.

Despite these limitations, Tacotron 2 was a landmark system that heavily influenced subsequent TTS research, especially motivating the development of faster, non-autoregressive alternatives.

5.5.5.2 Glow-TTS

Glow-TTS is a **flow-based generative model designed for parallel Text-to-Speech (TTS) synthesis**. It is a new type of parallel TTS model [47].

What is Glow-TTS and Problems Solved

Proposed to overcome limitations in previous TTS models:

- **Limitations of Autoregressive TTS:** Models like Tacotron 2 and Transformer TTS achieve state-of-the-art quality but face difficulties in real-time deployment. Inference time grows linearly with output length (undesirable delays). They can also lack robustness in some cases, producing attention errors like repeating words.
- **Limitations of Parallel TTS:** Models like FastSpeech addressed speed by generating mel-spectrograms in parallel, speeding up synthesis. They also reduce failure cases by constraining alignment to be monotonic. However, **previous parallel TTS models require guidance from autoregressive models as external aligners to train**, meaning their performance depends on the quality of these external aligners.

Glow-TTS eliminates the necessity of any external aligner and simplifies the training procedure of parallel TTS models. It is a standalone parallel TTS model that internally learns to align text and speech.

How It Works (Flow-based, MAS)

Glow-TTS generates a mel-spectrogram conditioned on a monotonic and non-skipping alignment between text and speech representations. It combines the properties of generative flows and dynamic programming.

- **Flow-based Generative Model:** Utilizes flow-based generative models, trained to maximize log-likelihood using invertible transformations. These transformations allow efficient density estimation and guarantee fast/efficient sampling. The flow-based decoder is the core part. It efficiently transforms a mel-spectrogram to a latent representation during training and transforms the prior distribution to a mel-

spectrogram during inference, enabling parallel decoding. Composed of stacked flow blocks. Models the conditional distribution of mel-spectrograms by transforming a conditional prior distribution (an isotropic multivariate Gaussian from the text encoder). It maps a complex distribution (speech features) to a simple distribution (Gaussian) using a series of invertible transformations. It supports exact likelihood optimization and parallel generation. Flow models also support controllable synthesis (pitch, speed) via latent space manipulation.

- **Monotonic Alignment Search (MAS):** Glow-TTS internally learns its own alignment using dynamic programming. MAS searches for the most probable monotonic alignment between the latent representation of speech and the statistics of the prior distribution. This search is efficiently done using dynamic programming (time complexity $O(T_{\text{text}} \times T_{\text{mel}})$). It finds the best alignment path within the model during training, enforcing monotonicity. This results in a simpler training pipeline (no external aligner) and potentially better generalization.
- **Training and Inference:**
 - Training Procedure: Directly trained to maximize the log-likelihood of speech conditioned on text and alignment. At each step, it searches for the most probable monotonic alignment (A^*) using MAS and updates model parameters to maximize log-likelihood w.r.t. A^* . This is Viterbi training.
 - Inference Procedure: Statistics (mean μ and std dev σ) of the prior are predicted by the text encoder. Alignment (duration of each text token) is predicted by a duration predictor. A latent variable is sampled from the predicted prior using the predicted alignment. A mel-spectrogram is synthesized in parallel by transforming the latent variable through the flow-based decoder. **MAS is not needed during inference.**

Model Architecture

Composed of a text encoder, a duration predictor, and a flow-based decoder. Figure 1 illustrates procedures.

- **Text Encoder:** Maps input text sequence into the statistics (μ and σ) of the prior distribution for each text element. Follows Transformer TTS encoder structure with modifications: removes positional encoding, adds relative position representations, adds a residual connection. Appends a linear projection layer to estimate statistics. Predicts statistics during inference.
- **Duration Predictor:** Appended on top of the text encoder. Trained to predict the duration of each text token to match duration labels calculated from the learned alignment (A^*). Architecture consists of two convolutional layers (ReLU), layer normalization, dropout, and a projection layer. Trained with MSE loss in the

logarithmic domain to match duration labels. A stop gradient is applied to its input. Estimates the alignment during inference, removing the need for MAS. Predicts speaker-dependent durations in the multi-speaker setting.

- **Flow-based Decoder:** Described as the core part of Glow-TTS. Functions as an invertible transformation mapping between mel-spectrogram space and a latent space with a prior distribution.
 - **Functionality:** Transforms mel-spectrogram to latent representation for training/alignment search. Transforms prior distribution to mel-spectrogram distribution during inference, enabling parallel decoding.
 - **Structure:** A stack of multiple flow blocks.
 - **Flow Block Components:** Each block has Activation Normalization, Invertible 1x1 Convolution, and Affine Coupling layers. Follows WaveGlow architecture without local conditioning.
 - **Computational Efficiency:** Splits 80-channel mel-spectrogram frames into two halves along the time dimension for flow operations. Uses grouped 1x1 convolutions (splitting feature map into 40 groups) to reduce Jacobian determinant calculation cost.
 - **Multi-Speaker:** Speaker embedding added as global conditioning in all decoder affine coupling layers.

Advantages

Glow-TTS demonstrates several advantages:

- **Fast Synthesis:** Generates mel-spectrograms in parallel. Achieved 15.7 times faster synthesis speed than Tacotron 2 in experiments. Inference time is almost constant regardless of utterance length. Speed is Fast.
- **Robustness:** Enforcing hard monotonic alignments enables robust TTS, generalizing to long utterances. Maintained robustness on texts longer than training data, unlike Tacotron 2 whose character error rate grew significantly. Robustness is High.
- **Diversity and Controllability:** As a flow-based model, can synthesize diverse samples by sampling different latent representations. Controllability is Easy via latent manipulation.
- **Pitch Control:** Varying the temperature (standard deviation T) of the prior distribution allows controlling speech pitch while maintaining intonation.
- **Intonation/Stress Diversity:** Variations in Gaussian noise sample lead to diverse stress or intonation patterns.
- **Speaking Rate Control:** Can be controlled by scaling the predicted duration from the duration predictor.
- **Simplified Training:** Does not require an external aligner. Training pipeline is simpler.

- **Extensibility:** Can be easily extended to a multi-speaker setting by adding speaker embeddings. Can predict speaker-dependent durations and perform voice conversion by disentangling latent representation and speaker identities.
- **Comparable Quality:** Achieves comparable speech quality to Tacotron 2 based on MOS evaluations. Side-by-side evaluations showed a preference for Glow-TTS over Tacotron 2. Intelligibility is High.

Evaluation and Results

Experiments were conducted on LJSpeech (single-speaker) and LibriTTS train-clean-100 (multi-speaker). Evaluations measured Audio Quality (MOS), Sampling Speed (inference time), Robustness (character error rate on long text), Diversity/Controllability (visualization), and Voice Conversion.

- Glow-TTS achieved 15.7x speed-up over Tacotron 2.
- Showed robustness to long inputs, outperforming Tacotron 2.
- MOS scores were comparable to Tacotron 2.
- Demonstrated ability to control pitch, intonation, and speaking rate.
- Successfully extended to multi-speaker synthesis, showing speaker-dependent duration and voice conversion.

5.5.5.3 Amazon Polly

Amazon Polly is a cloud-based Text-to-Speech (TTS) service provided by Amazon Web Services (AWS) that converts text into speech. Leveraging advanced deep learning technologies, Polly generates high-quality, natural-sounding speech that closely mimics human voices. It is designed for developers to integrate realistic voice output into applications such as virtual assistants, chatbots, audiobooks, e-learning platforms, and accessibility tools. Polly supports a wide array of languages and voices, covering over 40 languages (e.g., English, Spanish, French, Mandarin, Arabic) and offering multiple voice variants per language, including male, female, and regional accents[48].

Key Components:

- **Text Analysis:** Converts input text into linguistic features through normalization (e.g., expanding abbreviations) and tokenization (breaking text into words).
- **Spectrogram Generation:** For Neural voices, neural models predict acoustic features like mel-spectrograms.
- **Vocoding:** Converts the spectrograms into audible audio waveforms[49].

Training Process: Amazon Polly's models are trained on large multilingual datasets, often comprising millions of hours of speech and text. As a managed service, AWS handles all the training and infrastructure, meaning users don't need to train models themselves.

Features and Performance:

- **Speech Quality :** Neural voices sound very clear and natural, almost like a real human speaking .
- **Languages/Voices:** Offers over 100 voices across more than 40 languages, supporting various accents[50].
- **Streaming:** Supports real-time streaming of audio [50].

Limitations:

- **Cost:** While it offers a free tier (limited to 5 million characters/month for Standard voices and 1 million for Neural voices for the first 12 months), usage beyond this tier incurs costs (\$4.00 per 1 million Standard characters, \$16.00 per 1 million Neural characters)[51].
- **Setup Complexity:** Requires setting up an AWS account and integrating with the API via SDKs, which can be challenging for developers new to AWS cloud services.
- **Internet Dependency:** As a cloud service, it requires a stable internet connection.

5.5.5.4 IBM Watson Text-to-Speech

IBM Watson Text-to-Speech is another cloud-based TTS service within IBM's Watson AI, offering natural-sounding speech synthesis. It uses advanced neural network models for high-quality speech generation and is provided as a managed service, abstracting away the underlying infrastructure for the user[52]..

Key Components:

- **Text Analysis:**
Performs linguistic analysis such as normalization, tokenization, sentence boundary detection, part-of-speech tagging, and phoneme generation
- **Synthesis:** Utilizes deep neural networks to convert linguistic features into acoustic features and generate the speech waveform.
- **Output:** Delivers synthesized audio in formats like MP3 via API response, with support for SSML (Speech Synthesis Markup Language) for fine-grained control over speech characteristics.
- **Training Process:** IBM trains its models on multilingual datasets. The training is performed internally on IBM Cloud, and users access the pre-trained model via the API without needing to train models themselves.

Features and Performance:

- **Speech Quality:** Neural voices sound very clear and natural, almost like a real human speaking.
- **Languages/Voices:** Offers over 50 voices in more than 20 languages, with neural voices available for a smaller selection of languages.

Limitations:

- **Free Tier:** The free plan offers a very limited allowance of only 10,000 characters per month, which is often insufficient for significant TTS testing or deployment[53].
- **Cost:** The Standard plan costs \$20 per million characters, which is higher than Amazon Polly's (\$16.00 per million characters)[53].
- **Internet Dependency:** Like other cloud services, it requires an internet connection.

5.5.5.5 Google Text-to-Speech (gTTS)

Google Text-to-Speech (gTTS) is a popular Python library that acts as a convenient interface to Google's cloud-based TTS engine. It simplifies the process of sending text to Google's servers, where the text is synthesized into speech and returned as MP3 audio. gTTS is notably easy to use, requiring no complex setup or API keys, and supports over 40 languages. It's well-suited for basic applications like chatbots, scripts, and educational tools, though it lacks some of the advanced features found in paid, premium cloud TTS services.

Key Components:

- **Client (gTTS Library):** The Python library on your local machine acts as a client, sending the text you provide to Google's TTS endpoint.
- **Server (Google TTS Engine):** Google's powerful servers synthesize the speech using their advanced neural network models .
- **Data Flow:** The processed text is sent to the server, speech is generated, and the resulting MP3 audio is streamed back to the client for playback or saving.
- **Training Process:** Google trains its underlying TTS engine on vast multilingual datasets using advanced neural models. As a user of the gTTS library, you simply access this pre-trained service, with no involvement in the complex training process or management of the models.

Features and Performance:

- **Speech Quality :** Neural voices are known for sounding natural and high-quality, they are very close to real human speech
- **Languages:** Supports 70 languages, making it versatile for multilingual projects [54].

Limitations:

- **Limited Customization:** Lacks advanced control over speech output; for instance, it does not support SSML (Speech Synthesis Markup Language) for fine-grained adjustments to pronunciation, pitch, or emphasis, nor does it offer multiple distinct voices per language.

- **Internet Dependency:** Critically, it requires a stable internet connection to function, as all speech synthesis occurs on Google's cloud servers. This makes it unreliable for offline applications or in environments with poor network connectivity.

5.5.6 Evaluation Criteria for TTS Models

To make an informed decision, we assessed each TTS model based on the following crucial criteria:

5.5.6.1 Evaluation Metrics

- **Mean Opinion Score (MOS):** This is the main metric for evaluating speech quality. MOS is a subjective measure where human listeners rate the perceived quality of synthesized speech on a scale, typically from 1 (poor) to 5 (excellent). Scores above 4 are generally considered excellent, very close to natural human speech. For example, Tacotron 2 achieved a MOS of 4.53 .

5.5.6.2 Other Evaluation Criteria

Beyond MOS, we also considered:

- **Ease of Integration:** How simple or complex is it to incorporate the TTS system into our existing project architecture, This includes setup requirements, API complexity, and necessary dependencies.
- **Cost-Effectiveness:** Given our project's budget constraints, we carefully considered the financial implications, including free tiers, pay-as-you-go pricing, and any hidden costs like hardware requirements for local models.
- **Latency:** For an interactive application like ours, the delay between sending text and receiving the audio output (latency) is critical. Lower latency means a more responsive user experience.
- **Scalability:** We considered how well the system could handle increasing loads or a growing number of users if our project were to expand in the future.
- **Resource Demands:** For local models, this involved evaluating the computational power (e.g., CPU, GPU) required. For cloud models, it was about understanding the reliance on external infrastructure.
- **Offline Capability:** Whether the model could function without an internet connection was also a consideration, especially for potential future enhancements.

5.5.7. Comparison of Evaluated TTS Models

Here's a summary of how the models stacked up against our evaluation criteria:

Model	Tacotron 2	Amazon Polly	IBM Watson TTS	Glow-TTS	Google Text-to-Speech (gTTS)
Speech Quality	Natural and expressive	Clear and intelligible	Clear speech and high quality	Near Human Speech	Understandable and quick to generate
Ease of integration	Moderate (requires DL frameworks and training setup)	Moderate (AWS account, API keys, SDK)	Moderate (IBM Cloud account, API credentials)	Moderate	Very Simple (pip install, no API keys/accounts)
Cost	Free	Pay-as-you-go (after free tier)	Pay-as-you-go (after free tier)	Free	Free
Latency	High	Low (fast API responses with low delay)	Low	Low	Low
Scalability	Limited (hardware-dependent, complex)	High (leveraging AWS infrastructure)	High (leveraging IBM Cloud infrastructure)	Limited (local machine resources)	High (leveraging Google's infrastructure)

Resource Demands	High	low (cloud-based)	low (cloud-based)	High	low (cloud-based)
Offline Capability	Yes	No	No	Yes	No

Table 5-12 Comparison of Evaluated TTS Models

5.5.8. Selected TTS Model and Justification

After a comprehensive evaluation against our project’s specific requirements and constraints, we selected **Google Text-to-Speech (gTTS)** as the most suitable option for our initial implementation. Our decision was driven by several key factors that aligned perfectly with our priorities for simplicity, cost-effectiveness, and minimal resource demands during the prototyping and initial deployment phases.

Simplicity and Rapid Integration

The most compelling advantage of gTTS is its remarkable simplicity and the speed with which we could integrate it. To use gTTS, we simply needed to install the Python library . There was no requirement to sign up for cloud accounts, obtain API keys, or configure complex SDKs. This made integrating gTTS a straightforward task. The core functionality of converting text to speech can be achieved with just a few lines of code. This ease of use and minimal setup directly and significantly impacted our project’s development timeline, allowing us to quickly add the text-to-speech feature without getting in complex configurations.

Cost-Effectiveness

Cost was a critical factor for our project, and gTTS stood out as the most cost-effective solution among all evaluated options because it is effectively free to use. It doesn’t require any payment, subscription fees, or usage-based charges. Not like other cloud services like Amazon Polly (which charges after its free tier) and IBM Watson. The zero direct cost of gTTS perfectly aligned with our project’s budget constraints, allowing us to focus our resources on other essential aspects of development if we wanted.

Minimal Resource Demands

gTTS has minimal resource demands on our end, as it relies entirely on Google’s powerful cloud servers for the actual speech synthesis. Unlike computationally intensive models like Tacotron 2, which require specialized hardware such as GPUs or

TPUs, gTTS requires no such infrastructure on our local machine or server. While other cloud services like Polly and Watson also abstract away hardware, gTTS offers this benefit without the associated costs or setup complexity. This low resource requirement made gTTS ideal for our project, simplifying deployment and reducing operational overhead.

5.6 Avatar

The **Avatar** plays a key role in making the eLearning experience more interactive and engaging

5.6.1 Ready Player Me

Ready Player Me is a **cross-platform avatar creator** that lets you generate a personalized 3D character (full-body or half-body) that can be used in apps, games, and virtual worlds.

- You can customize the face, hair, clothes, and more.
- It provides a **GLB (GLTF)** file format that works easily in 3D engines like **Three.js**, **Unity**, or **Unreal Engine**.
- It also supports **ARKit blend shapes**, which means the avatar can be animated with facial expressions and lip sync. (26)

We used **Ready Player Me** to create a 3D avatar because it's free, easy to integrate, and provides a customizable avatar link.

By default, the avatar generated by Ready Player Me includes limited facial morph targets — for example, only a simple **mouth smile and mouth open**. However, for realistic facial animation and **lip syncing**, we needed more detailed control over facial expressions

To achieve this, we modified the GLTF URL and used the **morph target standard group** from the Ready Player Me documentation (27). These include a wide set of facial controls such as eye movements, jaw motions, and various mouth shapes

These morph targets are compatible with **Apple ARKit** and **Oculus Viseme standards**, allowing us to:

- Simulate **facial emotions** (e.g., smiling, frowning, surprise)

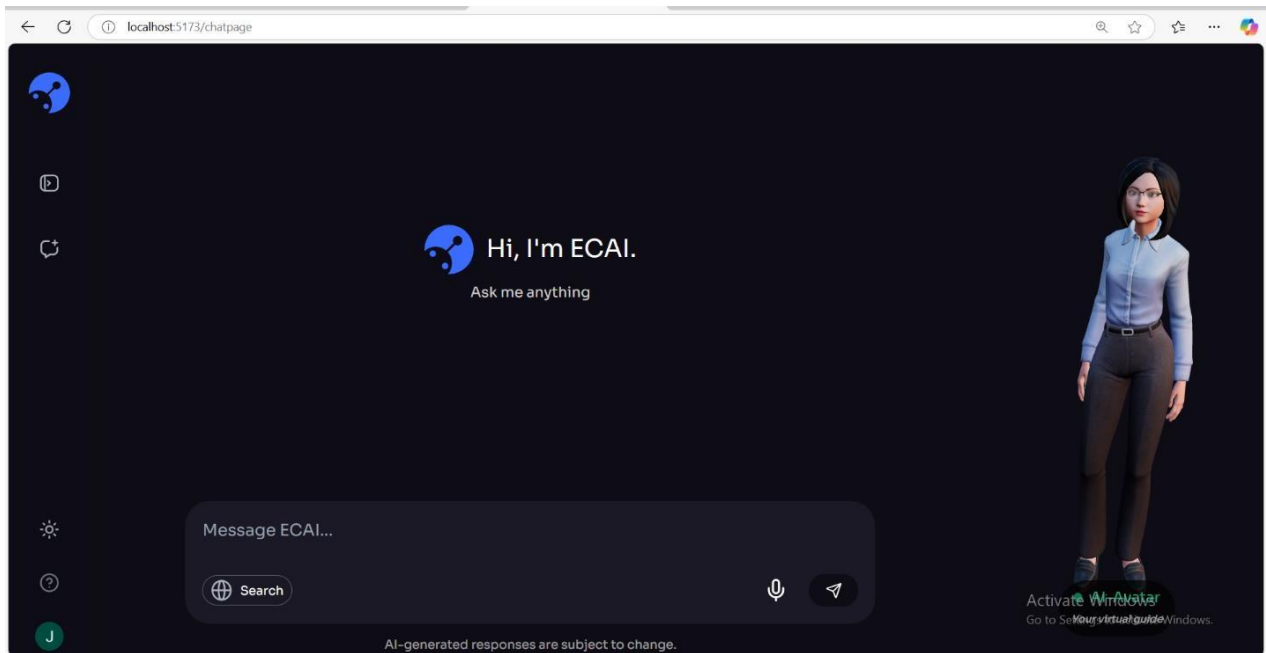


Figure 5-31 3D Avatar

- Apply **visemes** for lip syncing with audio (to make the avatar speak realistically)

5.6.2 GLTF

GLTF = Graphics Library Transmission Format or GL Transmission Format and formerly known as WebGL Transmissions Format or WebGL TF) is a standard file format for three-dimensional scenes and models. A glTF file uses one of two possible file extensions: .gltf (JSON/ASCII) or .glb (binary). Both .gltf and .glb files may reference external binary and texture resources. Alternatively, both formats may be self-contained by directly embedding binary data buffers (as base64-encoded strings in .gltf files or as raw byte arrays in .glb files). An open standard developed and maintained by the Khronos Group, it supports 3D model geometry, appearance, scene graph hierarchy, and animation. The glTF format stores data primarily in JSON. The JSON may also contain blobs of binary data known as buffers, and refer to external files, for storing mesh data, images, etc. The binary .glb format also contains JSON text, but serialized with binary chunk headers to allow blobs to be directly appended to the file.

The fundamental building blocks of a glTF scene is nodes. Nodes are organized into a hierarchy, such that a node may have other nodes defined as children. Nodes may have transforms relative to their parent. Nodes may refer to resources, such as meshes, skins, and cameras. Meshes may refer to materials, which refer to textures, which refer to images. Scenes are defined using an array of root nodes.

Most of the top-level glTF properties use flat hierarchy for storage. Nodes are saved in an array and are referred to by index, including by other nodes. A glTF scene refers to its root nodes by index. Furthermore, nodes refer to meshes by index, which

refer to materials by index, which refer to textures by index, which refer to images by index.

All glTF data structures support being extended using a JSON property, allowing arbitrary JSON data to be added. (28)

JSX = JavaScript XML A syntax extension used in React to write HTML-like code inside JavaScript.

5.6.2.1 gltfjsx

A small command-line tool that turns GLTF assets into declarative and reusable react-three-fiber JSX components.

The GLTF workflow on the web is not ideal ...

- GLTF is thrown whole into the scene which prevents re-use, in threejs objects can only be mounted once
- Contents can only be found by traversal which is cumbersome and slow
- Changes to queried nodes are made by mutation, which alters the source data and prevents re-use
- Re-structuring content, making nodes conditional or adding/removing is cumbersome
- Model compression is complex and not easily achieved
- Models often have unnecessary nodes that cause extra work and matrix updates

GLTFJSX fixes that

- It creates a virtual graph of all objects and materials. Now you can easily alter contents and re-use.
- The graph gets pruned (empty groups, unnecessary transforms, ...) and will perform better.
- It will optionally compress your model with up to 70%-90% size reduction. (29)

How used in project:

You can run command line “npx gltfjsx model.glb -o Avatar.jsx” or use online version of it like I used you can drag and drop your avatar file here (30)

This auto-generated a Avatar.jsx file.

Inside, it imports your model using useGLTF, and sets up all meshes, materials, skeletons, and animations.

5.6.2.2 Role of Avatar.jsx

- **All skinned meshes in one place:** Instead of manually importing or coding body parts separately, Avatar.jsx contains everything: body, clothes, face, hair, eyes, etc.
- **Animation-ready:** It uses useAnimations to access animation clips from the .glb file and lets you play
- **Dynamic interaction:** You can control animations or expressions by passing props (like mouthMovement, animation, etc.) to the Avatar component.

```
<group {...props} dispose={null} ref={group}>
```

- A container that groups all parts of the avatar together.
- `ref={group}`: Allows you to control the whole group in JavaScript.
- `dispose={null}`: Disables automatic memory cleanup (important when dynamically updating 3D models).

```
<primitive object={nodes.Hips} />
```

- Adds the root bone of the avatar skeleton.
- Without it, none of the **skinnedMesh** animations would move correctly.

Eye Meshes:

```
jsx
```

```
<skinnedMesh name="EyeLeft" ... />
```

```
<skinnedMesh name="EyeRight" ... />
```

- These represent the avatar's **eyeballs**.
- They use morph targets to allow **blinking or eye movement**.

Head, Teeth:

```
jsx
```

```
<skinnedMesh name="Wolf3D_Head" ... />
```

```
<skinnedMesh name="Wolf3D_Teeth" ... />
```

- `Wolf3D_Head`: The most important part! It supports **facial expressions** and **lip sync** via morph targets.
- `Wolf3D_Teeth`: Usually used with open mouth visemes.

Hair, Glasses:

```
jsx
```

```
CopyEdit
```

```
<skinnedMesh geometry={nodes.Wolf3D_Hair.geometry} ... />
```

```
<skinnedMesh geometry={nodes.Wolf3D_Glasses.geometry} ... />
```

- Accessories of the avatar. You can hide/show them dynamically if needed.

Body and Outfit:

```
jsx
```

```
<skinnedMesh geometry={nodes.Wolf3D_Body.geometry} ... />
```

```
<skinnedMesh geometry={nodes.Wolf3D_Outfit_Bottom.geometry} ... />
```

```
<skinnedMesh geometry={nodes.Wolf3D_Outfit_Footwear.geometry} ... />
```

```
<skinnedMesh geometry={nodes.Wolf3D_Outfit_Top.geometry} ... />
```

- These define the **clothing** and **body structure** of the avatar.
- They're animated along with the skeleton, but they don't usually use morph targets.

Notes:

- The parts like morphTargetDictionary and morphTargetInfluences are what make **lip sync**, **facial expressions**, and **blinking** possible.
- Each mesh is **skinned**, meaning it's attached to a skeleton (bones) for animations.
- You can animate blinking via eyeBlinkLeft and eyeBlinkRight morph targets.

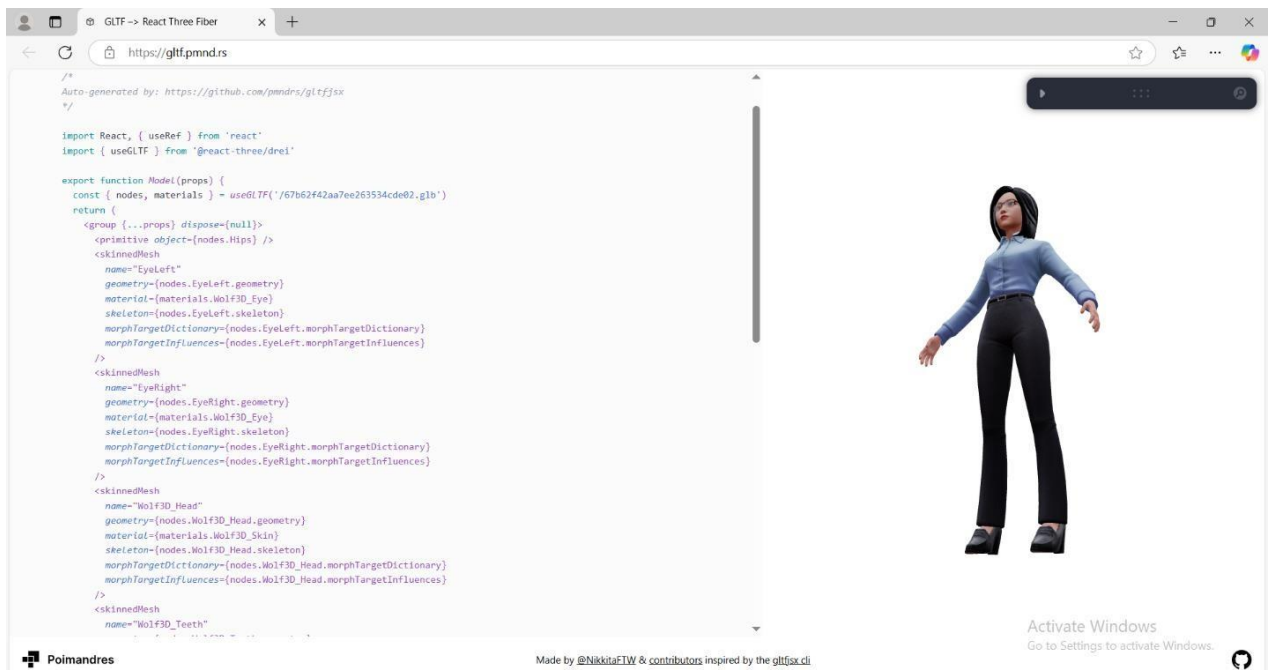


Figure 5-32 React component (.jsx file) generated by gltfjsx

5.6.3 Animations:

5.6.3.1 Mixamo

Mixamo is a 3D computer graphics technology company. Based in San Francisco, the company develops and sells web-based services for 3D character animation. Mixamo's technologies use machine learning methods to automate the steps of the character animation process, including 3D modeling to rigging and 3D animation.

Mixamo is spun off of Stanford University and has raised over \$11 million from investors Granite Ventures, Keynote Ventures, Stanford University and AMD Ventures. Mixamo was acquired by Adobe Systems in 2015, and it is free

Rigging service

This service has been discontinued to keep custom 3D models and models uploaded via fuse CC on Adobe's servers. You can still upload custom 3D models to the online auto rigging service and rig them, but they won't be kept. Mixamo also provides an online, automatic model rigging service known as the Auto-Rigger, which was the industry's first online rigging service. The AutoRigger applies machine learning to understand where the limbs of a 3D model are and to insert a "skeleton", or rig, into the 3D model as well as calculating the skinning weights. The service can take up to 2 minutes (31)

How used in project:

1. Upload the Character File:

- Upload the .fbx character file that was converted using Blender. It will appear that character is animated.
- The character should be auto-rigged with an armature already present from Ready Player Me.
- The character will appear **without texture** when uploaded.

2. Select Animations:

- Choose a default animation, such as **"Idle"**.
- select additional animations
 - **"Standing Greeting"** (for greeting)
 - **"Thinking"** (for thinking)
 - **Two animations for Talking**
- Additionally, select an **angry animation**.

3. Download Animations:

- Download the selected animations in .fbx format, **without skin**, to only capture the animation data (no need to redownload the mesh).
- Set the animation frame rate to **30 frames per second** for all animations.

Import Animations into a Single File:

- Import all the downloaded animations into **one single .glb file** using Blender. This will merge all the animations into a single file.

But for Ai teacher it was more suitable to use a default one ("Idle") and for talking one animation Talking_0 (32)

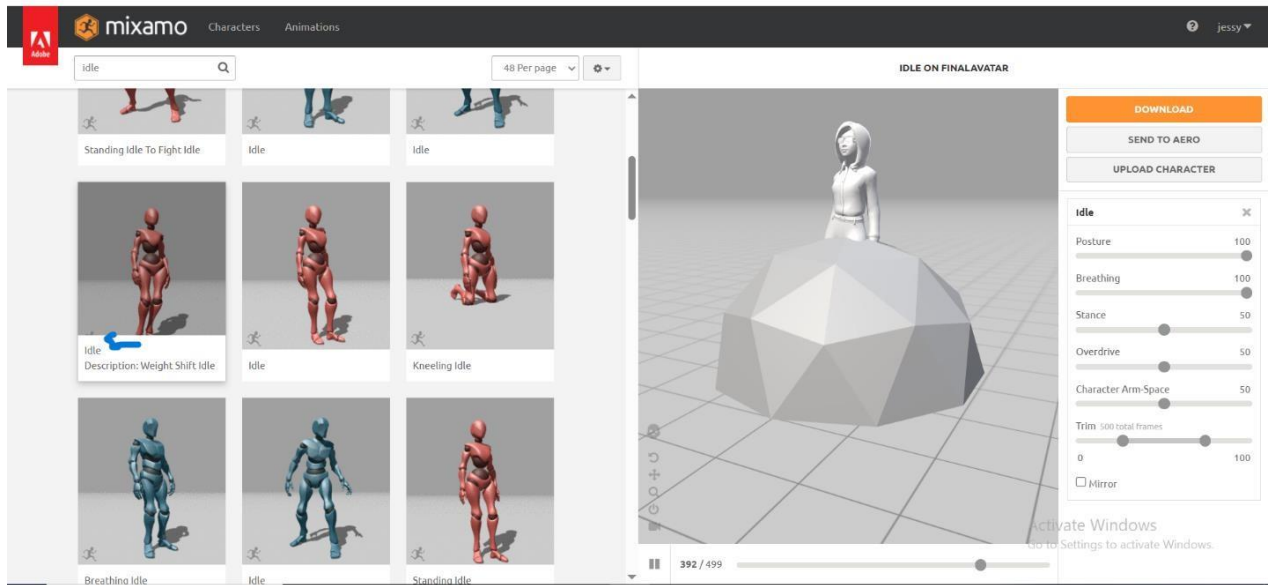


Figure 5-33 Select Animations From Mixamo

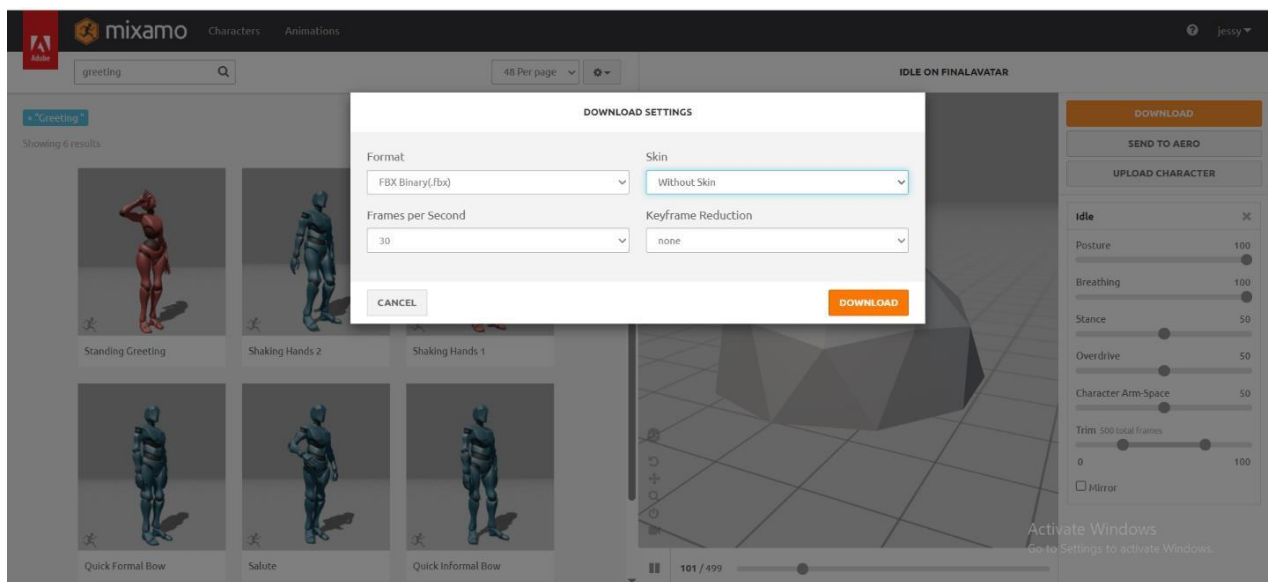


Figure 5-34 Download Animations From Mixamo

5.6.4 Blender:

Blender is a free and open-source 3D computer graphics software tool set . Blender provides a consistent experience across Linux, macOS, and Windows operating systems via OpenGL.

Blender has a wide variety of tools making it suitable for almost any sort of media production

It is used for creating animated films, visual effects, art, 3D-printed models, motion graphics, interactive 3D applications, and virtual reality. It is also used in creating video games. Blender is like a **Swiss army knife** for 3D work.

It supports:

- **3D modeling** (creating 3D shapes and characters)
- **Rigging** (adding a skeleton to a model for animation)
- **Animation** (moving characters, facial expressions, lip sync)
- **Sculpting** (fine detailing of models, like skin texture)
- **Texturing & Shading** (adding colors, materials, and textures)
- **Rendering** (producing the final image or animation)
- **Video editing** (yes, it even has a built-in video editor!)
- **Simulation** (smoke, fire, hair, physics, etc.) (33)

An armature in Blender can be thought of as similar to the armature of a real skeleton, and just like a real skeleton an armature can consist of many bones. These bones can be moved around and anything that they are attached to or associated with will move and deform in a similar way.

An “armature” is a type of object used for rigging. A rig is the controls and strings that move a marionette (puppet). Armature object borrows many ideas from real-world skeletons. (34)

Bone: The building block of an Armature. Made up of a Head, Tail and Roll Angle which define a set of local axes and a point of rotation at the Head. Also see Pose Bone.

Child: An Object that is affected by its Parent.

Mesh: Type of object consisting of Vertices, Edges and Faces. (35)

How used in project:

1. Used to convert our glb avatar character to FBX because mixamo only supports FBX, OBJ , ZIB

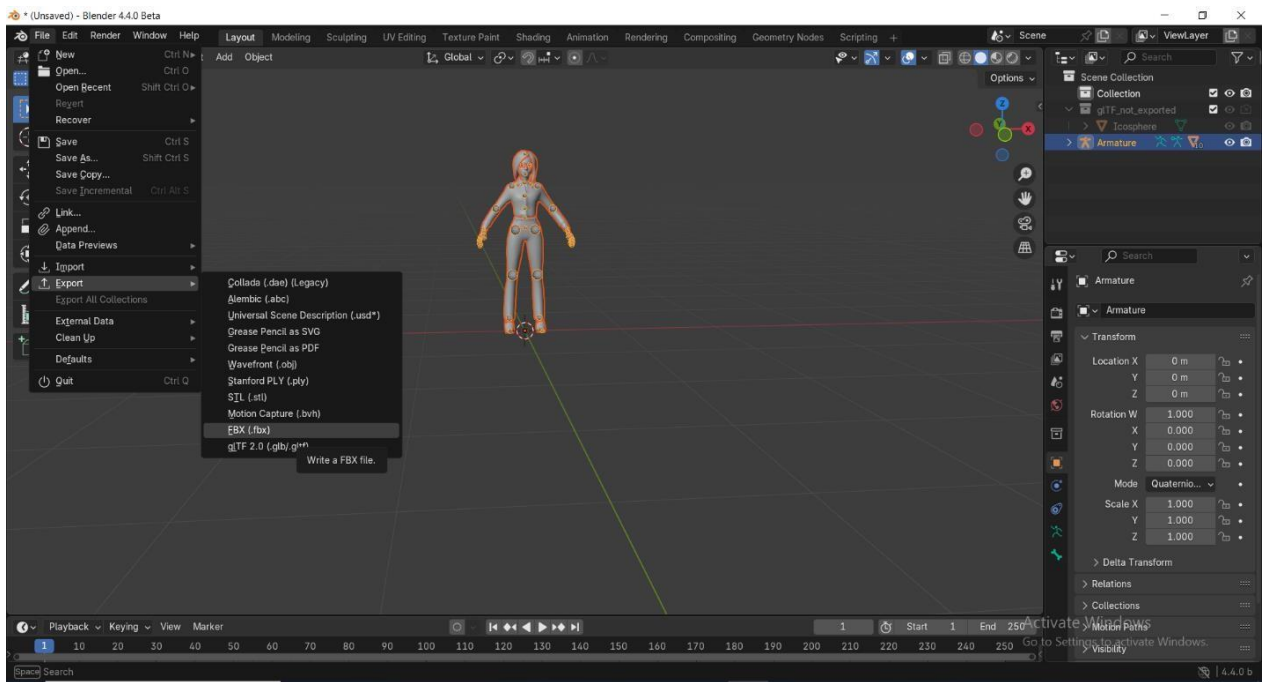


Figure 5-35 Convert GLB Avatar Caraceter To FBX Using Blender

2. Used to import all animations into one single glb file
 - a) Import FBX animation into blender

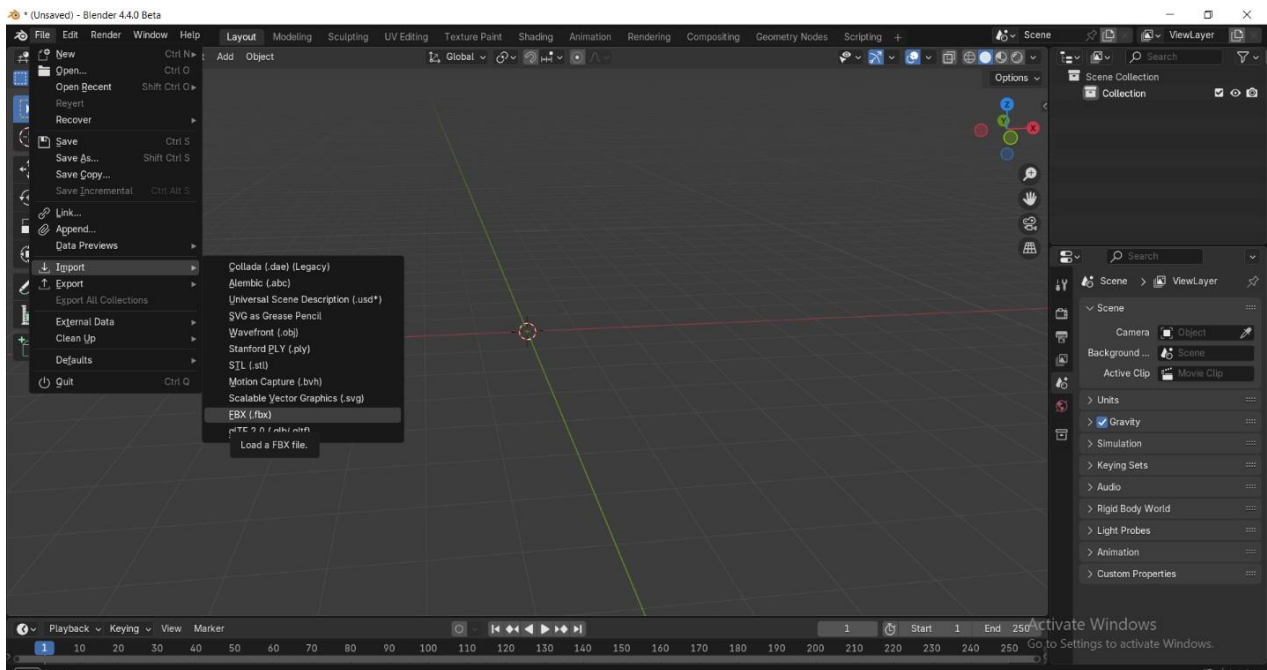


Figure 5-36 Import FBX Animation Into Blender

- a) Then armature will appear

and if we play animation, animation will play only with the bones

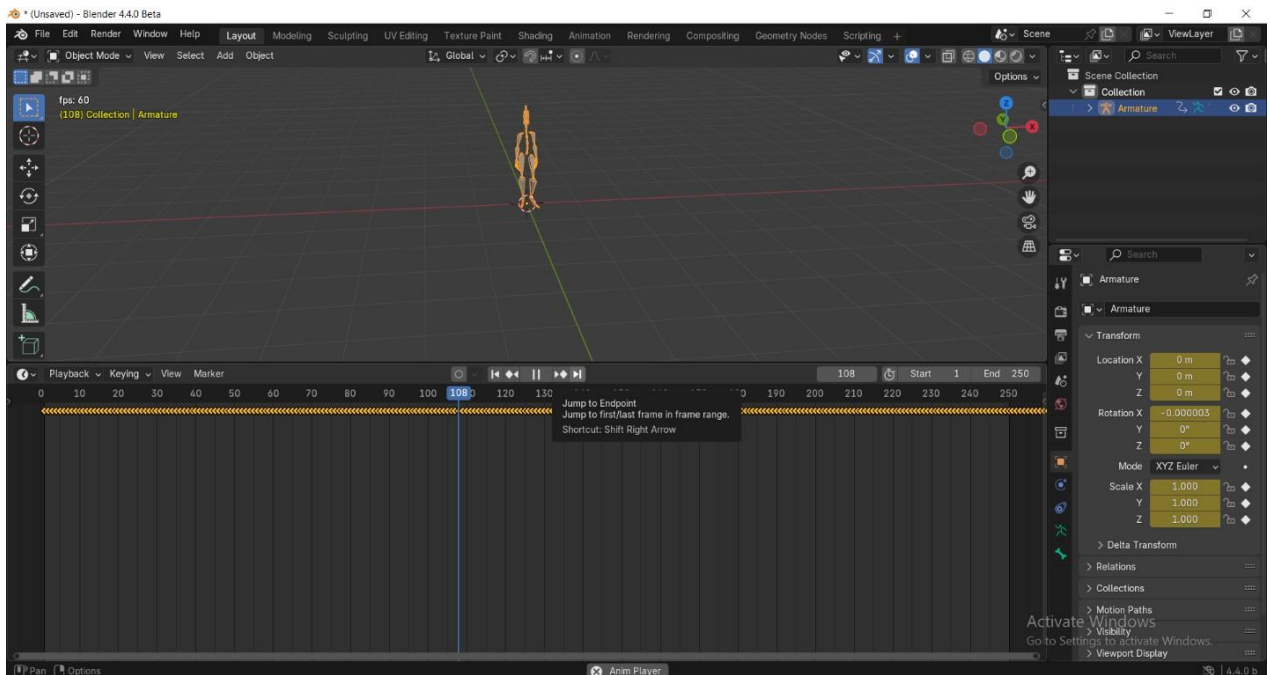


Figure 5-37 Keyframed Armature animation in Blender

- b) Then change name from Armature|mixamo.com|Layer0.001 to name that I want it in project from Dope sheet and action editor

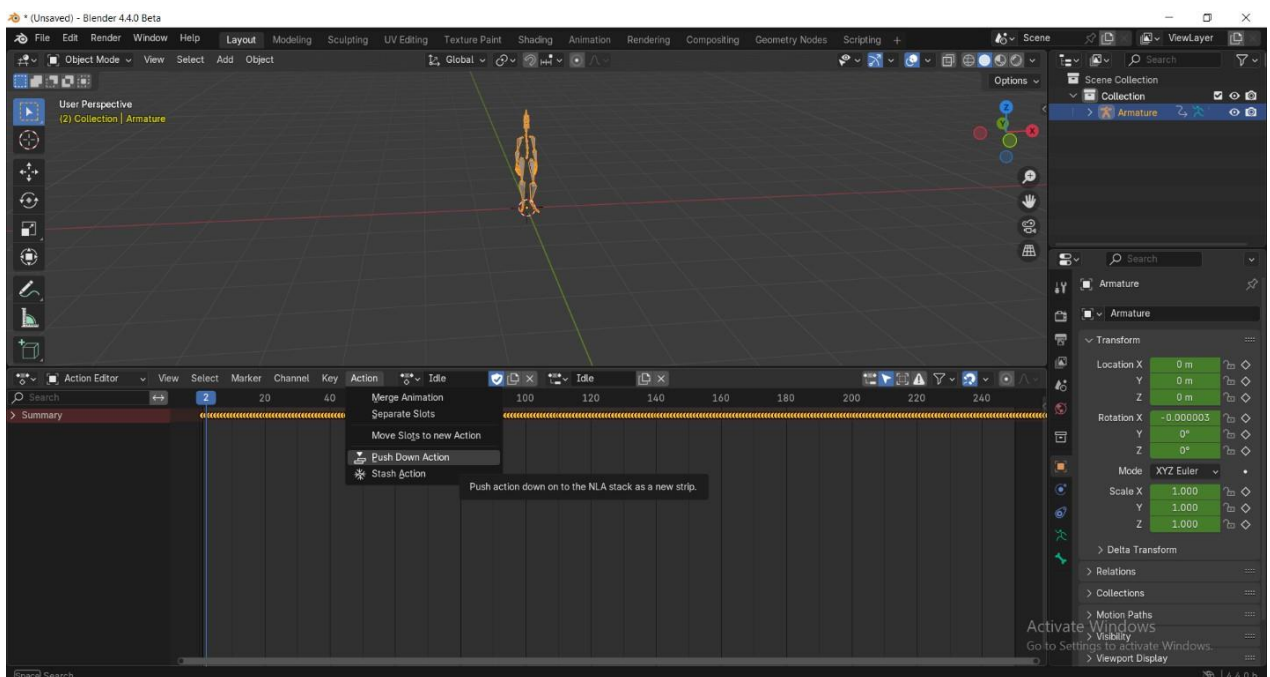


Figure 5-38 Change Name Of Animation in Blender

- c) then push down action to NLA stack (NonLinear Animation)

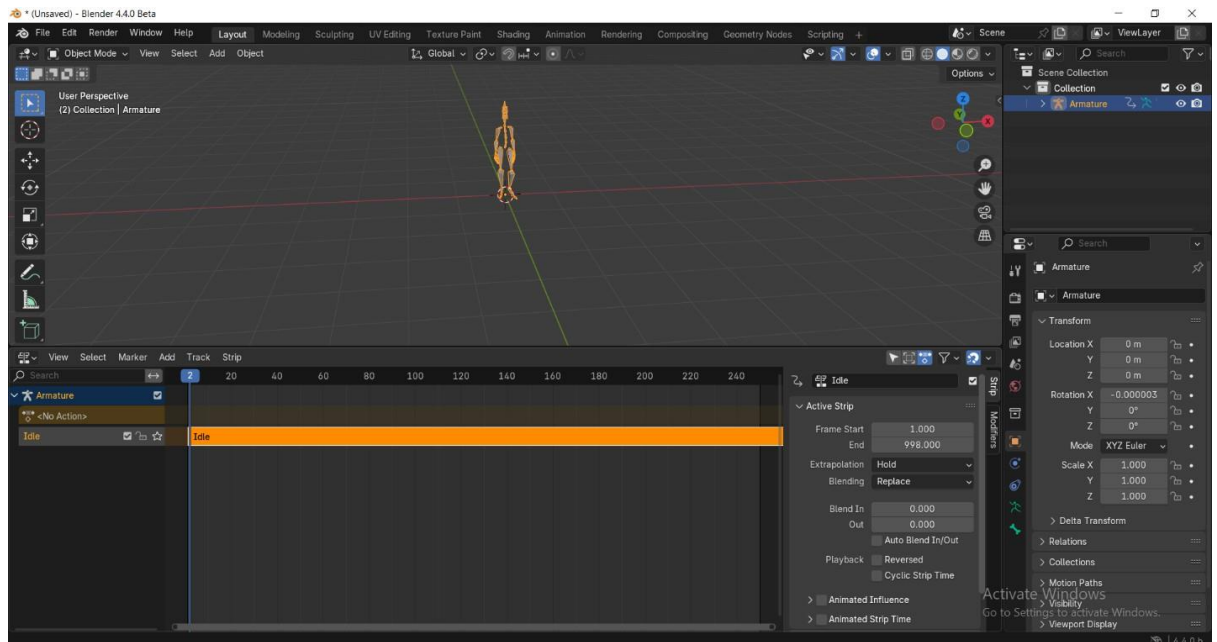


Figure 5-39 Push Down Action To NLA Stack In Blender

e) When you need to import another animation need to delete another armature hierarchy

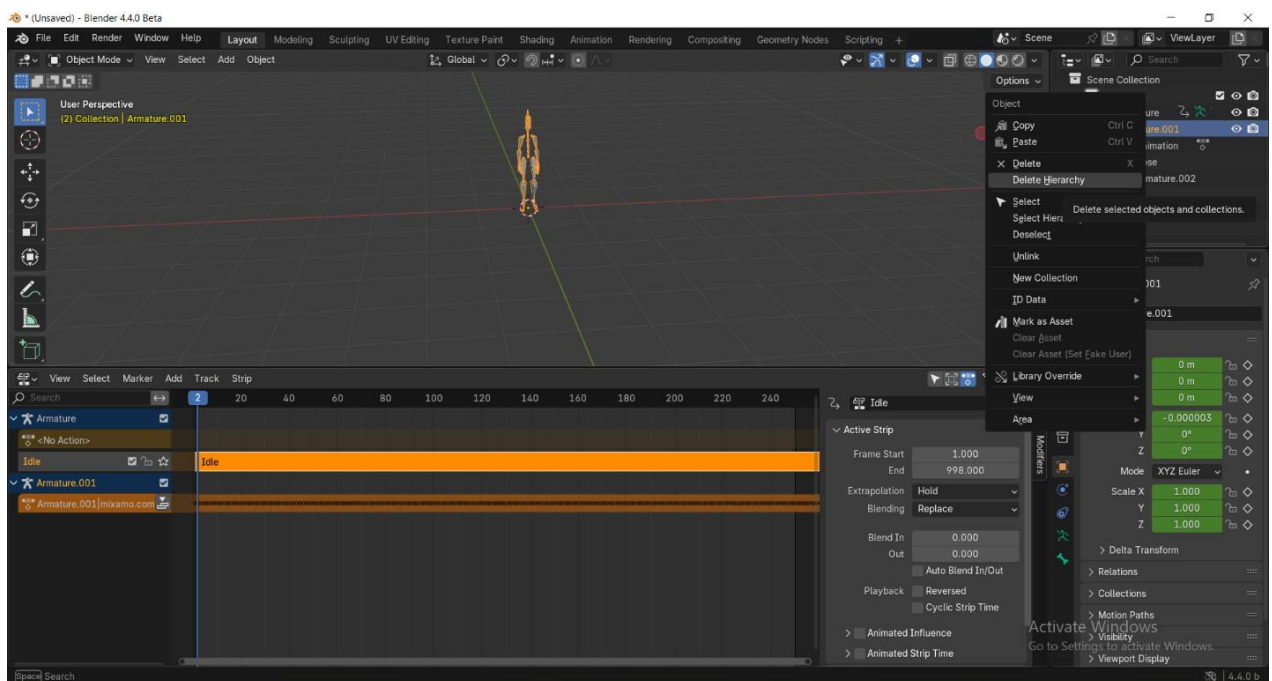


Figure 5-40 Another Animation And Delete The Added Armature Hierarchy In Blender

f) Then click on first armature and go for Dope sheet and action editor and select new animation and change name then push down action to the NLA stack

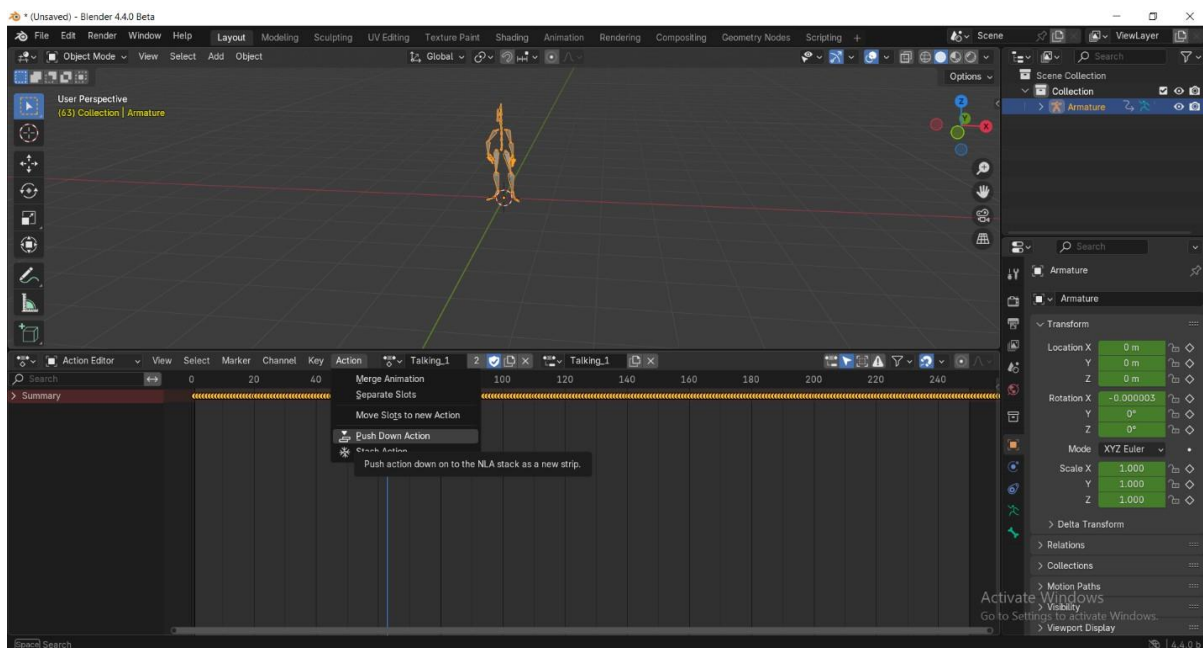


Figure 5-41 Changing Name of New Animation And Push Down To the NLA Stack In Blender

So we have 6 animations inside single armature ["Talking_0", "Talking_1", "Standing Greeting", "Angry", "Thinking", "Idle"] then exported as animations.glb file

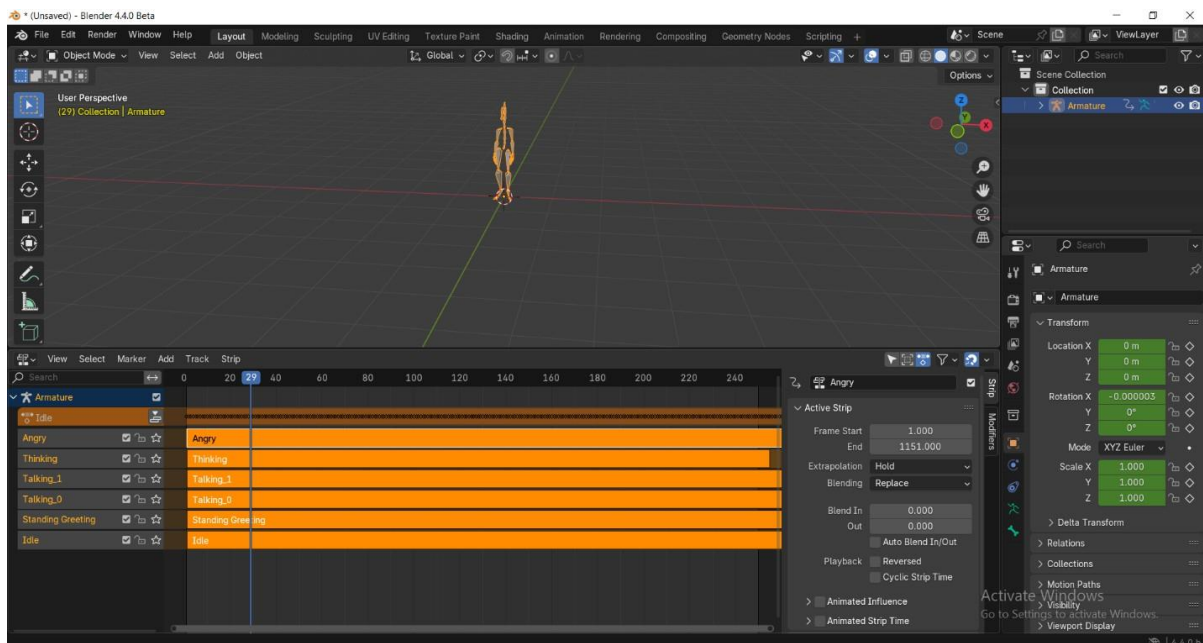


Figure 5-42 All Animations Inside Single Armature In NLA

But for Ai teacher we found out that we don't need them all and the suitable for this case a default one ("Idle") and for talking Talking_0

Apply those animations into our avatar

uses useGLTF from @react-three/drei to load the list of animation clips included in the animations.glb file. These were likely exported from Blender and contain clips like "Idle", "Talking_0", etc.

checks if there's an animation named "Idle". If it exists, it's set as the default. Otherwise, it picks the first animation in the list. This is important so the avatar has a fallback animation even if "Idle" isn't found.

Creates a state variable called animation to track the currently active animation. It starts with the default animation determined .

group refers to the root node of the avatar model.

useAnimations binds the animation clips to the avatar's skeleton so they can be played.

actions holds individual animation clips (like actions["Talking_0"]).

mixer manages transitions and blending between animations.

This effect watches for changes in the incoming message (from the chat system). When a new message arrives:

- It sets the animation (setAnimation(...)), facial expression, and lipsync data.
- If no message is present, it falls back to "
- It also loads and plays the audio if available,(unless muted)

This effect plays the selected animation when animation changes:

This decides **how long the transition to the new animation should take**.

If no animation is currently active (inUse === 0),

then no fade needed → fadeTime = 0.

If an animation is already playing, do a smooth transition → fadeTime = 0.5 seconds.

- **.reset()**: Restarts the animation from the beginning (time = 0).
- **.fadeIn(fadeTime)**: Smoothly increases the animation's influence from 0 → 1 over fadeTime.
- **play()**: Actually starts the animation.

This is a **cleanup function** that React runs **before** the next effect is applied (or before the component unmounts). It:

- Fades out the current animation smoothly over 0.5 seconds.
- Ensures that multiple animations don't overlap or "stack." (36)

5.6.5 Lip sync:

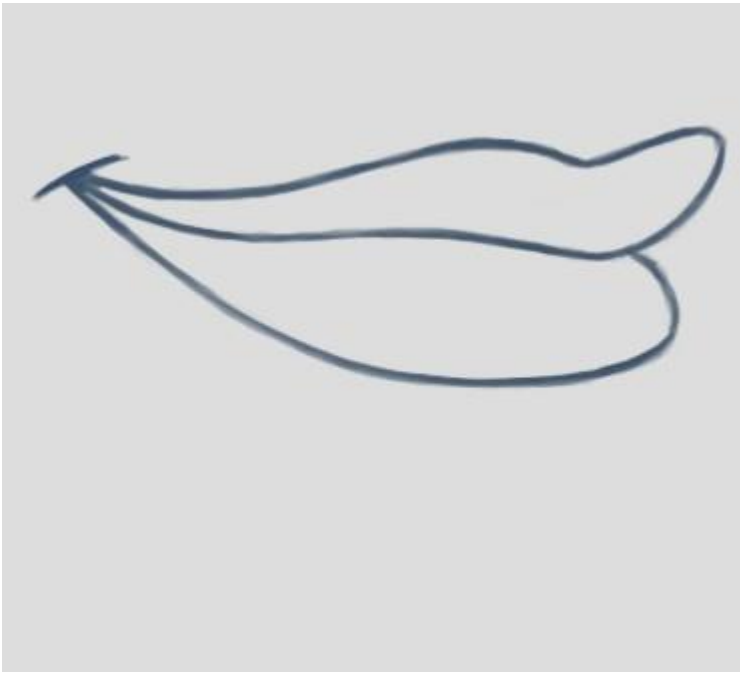
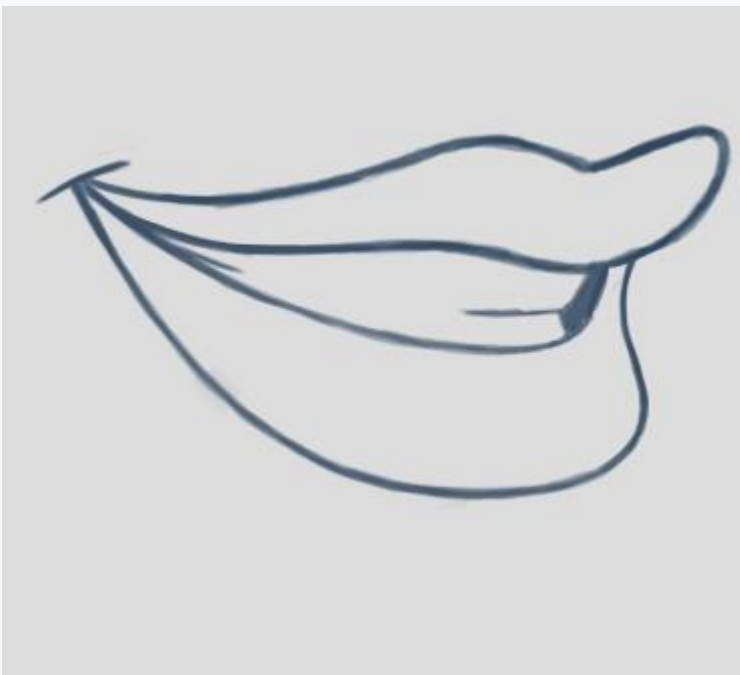
5.6.5.1 Rhubarb Lip Sync

allows you to quickly create 2D mouth animation from voice recordings. It analyzes your audio files, recognizes what is being said, then automatically generates lip sync information. You can use it for animating speech in computer games, animated cartoons, or any similar project.

Mouth shapes

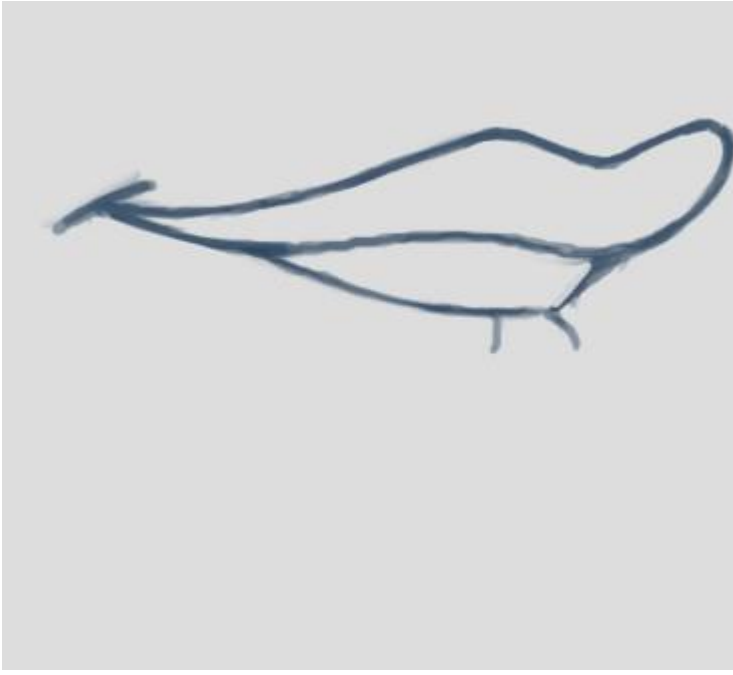
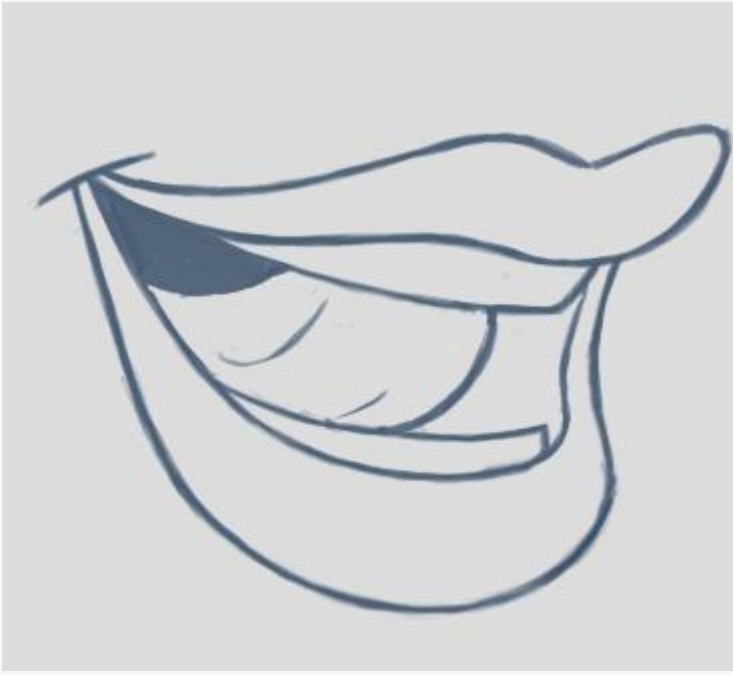
Rhubarb Lip Sync can use between six and nine different mouth positions. The first six mouth shapes (Ⓐ-Ⓕ) are the *basic mouth shapes* and the absolute minimum you have to draw for your character. These six mouth shapes were invented at the Hanna-Barbera studios for shows such as Scooby-Doo and The Flintstones. Since then, they have evolved into a *de-facto* standard for 2D animation, and have been widely used by studios like Disney and Warner Bros.

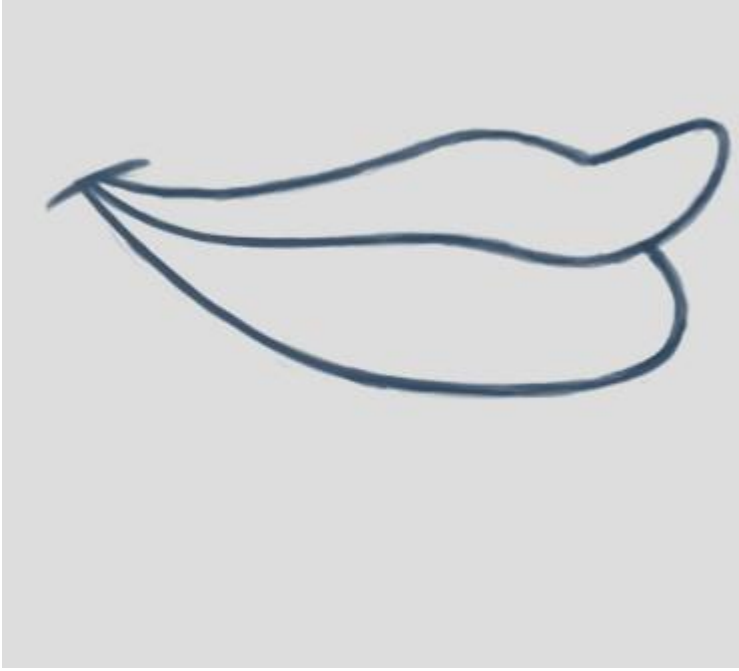
In addition to the six basic mouth shapes, there are three *extended mouth shapes*: Ⓖ, ⒼⓈ, and ⒼⓈⓈ. These are optional. You may choose to draw all three of them, pick just one or two, or leave them out entirely.(37)

Ⓐ		Closed mouth for the “P”, “B”, and “M” sounds. This is almost identical to the ⊗ shape, but there is ever-so-slight pressure between the lips.
Ⓑ		Slightly open mouth with clenched teeth. This mouth shape is used for most consonants (“K”, “S”, “T”, etc.). It’s also used for some vowels such as the “EE” sound in bee.

©		Open mouth. This mouth shape is used for vowels like “EH” as in men and “AE” as in bat. It’s also used for some consonants, depending on context. This shape is also used as an in-between when animating from Ⓐ or Ⓑ to ©. So make sure the animations Ⓐ©© and Ⓑ©© look smooth!
Ⓓ		Wide open mouth. This mouth shapes is used for vowels like “AA” as in father.

Ⓔ		Slightly rounded mouth. This mouth shape is used for vowels like “AO” as in off and “ER” as in bird. This shape is also used as an in-between when animating from Ⓒ or Ⓓ to Ⓔ. Make sure the mouth isn’t wider open than for Ⓒ. Both ⒸⒺⒻ and ⒹⒺⒼ should result in smooth animation.
Ⓕ		Puckered lips. This mouth shape is used for “UW” as in you, “OW” as in show, and “W” as in way.

©		Upper teeth touching the lower lip for “F” as in for and “V” as in very. This extended mouth shape is optional. If your art style is detailed enough, it greatly improves the overall look of the animation. If you decide not to use it, you can specify so using the extendedShapes option.
Ⓜ		This shape is used for long “L” sounds, with the tongue raised behind the upper teeth. The mouth should be at least far open as in ©, but not quite as far as in Ⓜ. This extended mouth shape is optional. Depending on your art style and the angle of the head, the tongue may not be visible at all. In this case, there is no point in drawing this extra shape. If you decide not to use it, you can specify so using the extendedShapes option.

⊗		Idle position. This mouth shape is used for pauses in speech. This should be the same mouth drawing you use when your character is walking around without talking. It is almost identical to Ⓐ, but with slightly less pressure between the lips: For ⊗, the lips should be closed but relaxed. This extended mouth shape is optional. Whether there should be any visible difference between the rest position ⊗ and the closed talking mouth Ⓐ depends on your art style and personal taste. If you decide not to use it, you can specify so using the extendedShapes option.
---	---	--

5.6.6 Oculus OVR LipSync

How used in project:

Oculus Lipsync offers a Unity plugin for use on Windows or macOS that can be used to sync avatar lip movements to speech sounds and laughter. Lipsync analyzes the audio input stream from microphone input or an audio file and predicts a set of values called [visemes](#), which are gestures or expressions of the lips and face that correspond to a particular speech sound. The term viseme is used when discussing lip reading and is a basic visual unit of intelligibility. In computer animation, visemes may be used to animate avatars so that they look like they are speaking.

Lipsync uses a repertoire of visemes to modify avatars based on a specified audio input stream. Each viseme targets a specified geometry morph target in an avatar to influence the amount that target will be expressed on the model. With Lipsync we can generate realistic lip movement in sync with what is being spoken or heard. This enhances the visual cues that one can use when populating an application with avatars, whether the character is controlled by the user or is a non-playable character (NPC).

The Lipsync system maps to 15 separate viseme targets: sil, PP, FF, TH, DD, kk, CH, SS, nn, RR, aa, E, ih, oh, and ou. The visemes describe the face expression produced when uttering the corresponding speech sound. For example the viseme **sil** corresponds to a silent/neutral expression, **PP** corresponds to pronouncing the first syllable in “popcorn” and **FF** the first syllable of “fish” (38)

Ready Player Me avatars come with blend shapes to support the Oculus LipSync API.

Both half-body (VR) and full-body Ready Player Me avatars come with a blend shape based facial rig that supports the [Oculus Lipsync API](#). All avatars have the *viseme* blend shapes required for real-time audio-based facial animation.

This makes it possible to build applications that use real-time facial animation based on audio input with Ready Player Me avatars.

Blend shapes (also called shape keys or morph targets) are predefined deformations of a 3D model. They allow you to change parts of the face—like the mouth or eyes—by blending between different facial expressions or movements.

Visemes represent **mouth shapes** corresponding to specific sounds (phonemes). For example:

- The "th" sound corresponds to viseme_TH
- The "oo" sound corresponds to viseme_U

Oculus LipSync uses these visemes to animate the mouth during speech.

List of supported blend shapes

// Visemes compatible with Oculus LipSync SDK

viseme_sil

viseme_PP

viseme_FF

viseme_TH

viseme_DD

viseme_kk

viseme_CH

viseme_SS

viseme_nn

viseme_RR

viseme_aa

viseme_E

viseme_I

viseme_O

viseme_U (39)

5.6.6.1 Ffmpeg:

FFmpeg is a powerful, open-source command-line tool that:

- Converts between many audio/video formats.
- Extracts audio/video streams.
- Compresses, trims, or resizes media files.
- Works with input/output from files, microphones, cameras, URLs, and more.

Used for audio preprocessing:

- **Used FFmpeg** to convert .mp3 audio generated by gTTS into .wav format so it can be processed by (required format) as The audio file to be analyzed. Supported file formats are WAVE (.wav) and Ogg Vorbis (.ogg).. (40)

5.6.7 Facial Expression and Blinking Logic:

5.6.7.1 Apple ARKit:

Apple ARKit is Apple's framework for building Augmented Reality (AR) applications on iOS (iPhone & iPad). One of its powerful features is **real-time facial tracking** using the TrueDepth camera (the same one used for Face ID).

ARKit can **detect and track 52 facial expressions** (called **blend shapes**) in real time – like blinking, smiling, raising eyebrows, moving the jaw, etc.

The blendShapes dictionary provided by an ARFaceAnchor object describes the facial expression of a detected face in terms of the movements of specific facial features. For each key in the dictionary, the corresponding value is a floating point number indicating the current position of that feature relative to its neutral configuration, ranging from 0.0 (neutral) to 1.0 (maximum movement).

ARKit provides many blend shape coefficients, resulting in a detailed model of a facial expression; however, you can use as many or as few of the coefficients as you desire to create a visual effect. For example, you might animate a simple cartoon character using only the jawOpen, eyeBlinkLeft, and eyeBlinkRight coefficients. A professional 3D artist could create a detailed character model rigged for realistic animation using a larger set, or the entire set, of coefficients. In the naming of blend shape coefficients, the left and right directions are relative to the face. That is, the eyeBlinkRight coefficient refers to the face's right eye. ARKit views running a face-tracking session mirror the camera image, so the face's right eye appears on the right side in the view.(55)

Ready Player Me avatars come with ARKit-compatible blend shapes.

Both half-body (VR) and full-body Ready Player Me avatars come with a blend shape based facial rig supporting Apple's [ARFaceAnchor.BlendShapeLocation](#) API.

This makes it possible to build applications that use real-time facial tracking with Ready Player Me avatars.

List of supported blend shapes

// Blend shapes compatible with Apple ARKit (52)

eyeBlinkLeft

eyeLookDownLeft

eyeLookInLeft

eyeLookOutLeft

eyeLookUpLeft

eyeSquintLeft

eyeWideLeft

eyeBlinkRight

eyeLookDownRight
eyeLookInRight
eyeLookOutRight
eyeLookUpRight
eyeSquintRight
eyeWideRight
jawForward
jawLeft
jawRight
jawOpen
mouthClose
mouthFunnel
mouthPucker
mouthLeft
mouthRight
mouthSmileLeft
mouthSmileRight
mouthFrownLeft
mouthFrownRight
mouthDimpleLeft
mouthDimpleRight
mouthStretchLeft
mouthStretchRight
mouthRollLower
mouthRollUpper
mouthShrugLower
mouthShrugUpper
mouthPressLeft
mouthPressRight

mouthLowerDownLeft
mouthLowerDownRight
mouthUpperUpLeft
mouthUpperUpRight
browDownLeft
browDownRight
browInnerUp
browOuterUpLeft
browOuterUpRight
cheekPuff
cheekSquintLeft
cheekSquintRight
noseSneerLeft
noseSneerRight
tongueOut
// Additional blend shapes
mouthOpen
mouthSmile
eyesClosed
eyesLookUp
eyesLookDown (56)

5.6.7.2 Facial Expression:

□[HYPERLINK "https://docs.readyplayer.me/ready-player-me/api-reference/avatars/morph-targets"](https://docs.readyplayer.me/ready-player-me/api-reference/avatars/morph-targets)**Morph Targets** (also called **Blend Shapes**) are a way to animate specific parts of a 3D model — like eyes, mouth, or eyebrows — by adjusting numeric values from 0 (neutral) to 1 (fully active).

For example:

- eyeBlinkLeft = 1 means the **left eye is fully closed**.
- mouthSmileLeft = 0.5 means the **left side of the mouth is half smiling**.

define different facial expressions in a JavaScript object called facialExpressions with names like:

- smile
- sad
- surprised
- angry
- crazy
- default

Each expression contains a set of **morph target keys** (e.g., eyeSquintLeft, jawOpen) with values from 0 to 1.

But suitable for database teacher is default expression when talking

In every animation frame (useFrame()), loops through all the possible morph targets. Check if the current facial expression includes it. If yes, apply its value **smoothly** using the lerpMorphTarget() function.

5.6.8 Blinking Logic:

You simulate blinking by modifying the blend shapes:

If blink is true, the avatar's eyelids close (value = 1).

After a short time (200ms), you set blink = false and the eyes open again (value = 0).

A new blink is triggered **every 1–5 seconds randomly**.

Eyes close (blink = true) for 200 milliseconds.

Then reopen and start the cycle again.

Libraries Used:

1. React, useState, useEffect, useRef

React is a popular JavaScript library for building user interfaces. It uses reusable components and built-in tools called “hooks” for managing state and side effects.

- **Hooks Used:**
 - **useState:** Allows your component to remember and update dynamic values like the current facial expression, animation state, or blinking status.
 - **useEffect:** Runs logic at specific points in the component's lifecycle (e.g., when it loads, when a new message arrives).
 - **useRef:** Creates a reference to a DOM element or object in memory, such as the avatar group or audio player.

- **used:** To manage the avatar's behavior — including blinking, speaking, and reacting to messages — all inside React components.
- **Example:**
 useEffect runs code every time a new chat message is received, triggering the avatar to animate and speak.
 useRef holds the reference to the avatar's 3D model and plays audio in sync with lip movement.(57)

2. @react-three/fiber

A React-based renderer for [Three.js](#) — lets you write 3D scenes using JSX (React's syntax).

- **used:**
 It powers the 3D rendering of the avatar, allowing you to write 3D logic inside React components instead of traditional Three.js boilerplate.
- **Example:**
 useFrame() is used to update the avatar's expressions on every animation frame, making it possible to smoothly change visemes or apply blinking.(58)

3. @react-three/drei

A helper library built on top of @react-three/fiber. It provides useful components and hooks to simplify common tasks in 3D rendering.

- **used:**
 - useGLTF: Loads your 3D avatar and animation files from .glb format.
 - useAnimations: Automatically detects and handles animations from the .glb file (like Idle or Talking).
- **Example:**
 You use useGLTF to load 67b62f42aa7ee263534cde02.glb (the avatar), and animations.glb (movement clips), then play animations with useAnimations.(59)

4. THREE (Three.js)

A low-level 3D graphics library in JavaScript used to render and animate 3D scenes in the browser.

- **used:**
 - To perform mathematical operations like linear interpolation (MathUtils.lerp) to animate facial expressions smoothly.
 - To generate random numbers for timed behaviors like blinking delays (MathUtils.randInt).

- **Example:**
When the avatar needs to transition from neutral to smiling, Three.js handles the blend between morph targets gradually for a realistic effect.(60)

5. leva, button, useControls

- **What it is:**
leva is a lightweight GUI panel for developers to interact with variables in real time, similar to debugging tools like dat.GUI.
- **used:**
 - Allows you to manually test avatar features like facial expressions or animations without changing code.
 - Useful during development for quick adjustments or testing reactions.
- **Example:**
You used useControls to create buttons like “winkLeft” or to select animations like "Idle" or "Talking", and sliders to adjust morph target values.(61)

Chapter 6

Testing

& Evaluation

Chapter Six : Testing and Evaluation

6.1 Introduction

This chapter explains how the AI-based eLearning course system was tested and evaluated after the completion of its development.

6.2 System Testing

System testing verifies whether all parts of the system function correctly after development. This includes testing individual parts (unit testing) as well as evaluating how the components interact together (integration testing).

6.2.1 Unit Testing

Unit testing involves testing the smallest parts of the system individually to ensure correct functionality.

6.2.1.1 Backend APIs Testing

The backend APIs responsible for the following tasks were tested:

- Receiving audio input from the user (sent to Whisper ASR).
- Sending the transcribed text to the LLM and RAG for processing.
- Returning the AI-generated response.
- Sending the AI response to the Text-to-Speech (TTS) model for voice generation.

Each API was tested separately to verify that it provides correct output for valid input and handles errors properly for invalid input.

6.2.1.2 Frontend Unit Testing

The frontend components were tested, including:

- Audio recording and uploading.
- Displaying the AI avatar.
- Showing the text-based response.
- Playing the TTS-generated voice answer.

Each component was verified to ensure it performs as expected and responds accurately to user interactions.

6.2.2 Integration Testing

After individual testing, the complete system was tested to verify how all components work together. For instance, the entire workflow was tested—from receiving a voice question from the user, processing it via Whisper, passing it to the LLM and RAG, generating the AI response, converting it to speech using TTS, and presenting the answer through the avatar.

Any errors or delays encountered during the integration were identified and resolved.

6.3 Usability Testing

Usability testing was conducted to assess the system’s ease of use and understandability, particularly for students engaging with the eLearning course.

A simulated usability evaluation was performed to determine whether the system would be user-friendly in an educational context.

Evaluation Approach

A typical student interaction was simulated by performing tasks such as:

- Recording a voice-based question related to the Database course.
- Receiving and listening to the AI avatar’s response.
- Observing how the interface reacts to user input.

The analysis focused on:

- The clarity and simplicity of the user interface.
- The quality and naturalness of the TTS-generated voice.
- The accuracy and relevance of the AI-provided answers.

Comparison with ChatGPT

To evaluate the effectiveness of the developed AI system, a set of sample questions from the Database course was prepared and submitted to both:

- The developed AI-powered system, and
- ChatGPT, as a reference benchmark.

The comparison was based on:

- Accuracy of the answer.
- Relevance to the original question.
- Clarity and ease of understanding for students.

This comparison helped identify the strengths of the developed system as well as areas for future enhancement.

The following examples illustrate side-by-side responses from the developed system and ChatGPT to the same set of questions.

1. What is Primary Key

The Project Response:

Primary Key

A **primary key** is a set of one or more attributes in a relation (table) that **uniquely identifies each tuple (row)** in that relation. It is selected from the candidate keys (which are attributes or combinations of attributes that can serve as a primary key) based on specific criteria.

Key Properties of a Primary Key:

1. Uniqueness

- No two tuples in a relation can have the same value for the primary key.
- **Example:** In the Employee relation, if SSN is the primary key, no two employees can have the same SSN.

2. Minimality

- The primary key must be the **smallest possible set of attributes** that ensures uniqueness.
- **Example:** If a relation has attributes {A, B, C} and {A, B} uniquely identifies all tuples, then {A, B} is the minimal set and qualifies as the primary key.

3. Non-nullability

- The primary key attributes **cannot have NULL values**.

ChatGPT Response:

Primary Key is a concept in databases.

🔑 **Definition (in simple terms):**

A **Primary Key** is a column (or a set of columns) in a table that:

- **Uniquely identifies each row** in that table.
- **Cannot be empty** (i.e., it must always have a value — **NOT NULL**).
- **Cannot be duplicated** (i.e., every value must be **unique**).

2. Create table Student

The Project Response:

```
CREATE TABLE STUDENT
```

```
CREATE TABLE STUDENT (
```

```
    Name VARCHAR(50) NOT NULL,
```

```
    SSN CHAR(9) NOT NULL,
```

```
    Address VARCHAR(100),
```

```
    Sex CHAR,
```

```
    Birth DATE,
```

```
    Class INT, -- Class: 1 (freshman), 2 (sophomore), ... , 5 (graduate)
```

```
    PRIMARY KEY (SSN)
```

```
);
```

Explanation of Attributes:

Name: Student's full name (e.g., "John Doe").

SSN: Social Security Number (unique identifier for the student).

Address: Student's residential address.

Sex: Gender (e.g., 'M' for male, 'F' for female).

Birth: Student's birthdate (stored as a DATE type).

Class: Academic classification — 1 = freshman, 2 = sophomore, ..., 5 = graduate student.

Notes:

The SSN field is defined as the primary key, ensuring uniqueness for each student.

The Class attribute is derived from the STUDENT entity in the syllabus, which distinguishes between undergraduate and graduate students.

The example follows the structure of SQL syntax shown in the syllabus (e.g., CREATE TABLE...).

ChatGPT Response:

```
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT  
);
```

3. what is the semantic data modeling concepts into the conceptual ER Model in EER?

The Project Response:

1. Object-Oriented Concepts

- **Superclass & Subclass Relationship:**

- A **superclass** represents a general entity type, while a **subclass** represents a specialized version of the superclass.
- **Example:** Vehicle (superclass) can have subclasses like *Car* and *Truck*.
 - **Attribute & Relationship Inheritance:**
- Subclasses inherit attributes and relationships from their superclasses.
- **Example:** All subclasses of *Vehicle* (e.g., *Car*) automatically inherit attributes like *LicensePlate* and relationships like *HasOwner*.

2. Specialization

- **Definition:** The process of creating **subclasses** from a superclass based on specific **real-world distinctions**.
- **Purpose:** To model hierarchical or **categorization** relationships (e.g., categorizing *Employee* into *Manager* and *Developer*).

3. Categories

- **Definition:** A **category** represents a union of **entities** from multiple classes, allowing an entity to belong to multiple **subclasses** simultaneously.
- **Example:** A *Person* entity might belong to both the *Student* and *Employee* categories.

ChatGPT Response:

Semantic Data Modeling Concepts in EER (Enhanced Entity-Relationship):

- **Entity:**
A real-world object.
Example: Student, Course
- **Attributes:**
These are the properties or details of an entity.
Example: A Student has attributes like name and age.
- **Relationship:**
Describes how entities are connected or interact with each other.
Example: A Student *enrolls in* a Course.
- **Specialization:**
Refers to creating subgroups of an entity that have more specific roles or meanings.
Example: Student can be specialized into Undergraduate and Graduate.
- **Generalization:**
Combines multiple entities into one generalized entity.
Example: Doctor and Nurse can be generalized as Staff.

4. What is the recursive relationship in database?

The Project Response:

Recursive Relationship in Database Systems

A recursive relationship occurs when the same entity type participates in a relationship more than once, playing different roles in that relationship. This concept is discussed in the syllabus (pages 16–17) and is illustrated using examples such as supervision and self-referencing attributes.

Key Characteristics of Recursive Relationships

1. **Same Entity Type Involved**

The same entity type appears multiple times in the relationship.

Example: In the supervision relationship, the EMPLOYEE entity is involved twice—once as the supervisor and once as the supervisee.

2. **Role Differentiation**

Each occurrence of the entity in the relationship takes on a different role.

Example: In the Works_For relationship, the EMPLOYEE plays the role of the worker, while the DEPARTMENT plays the role of the employer.

3. **Self-Referencing Attributes**

An attribute in the entity may refer to another instance of the same entity type.

Example: The Supervisor attribute in the EMPLOYEE entity refers to another EMPLOYEE.

Example from the Syllabus

- **Supervision Relationship:**

An EMPLOYEE can be both a supervisor (managing others) and a supervisee (being managed by someone else). This is explained in the syllabus on page 17.

- **Self-Referencing Attribute:**

The Supervisor attribute in the EMPLOYEE entity refers to another EMPLOYEE, creating a recursive relationship (page 16).

Importance in Database Design

Recursive relationships are important for modeling situations where entities have hierarchical or self-referential connections, such as:

- Organizational hierarchies (e.g., managers and employees)
- Family trees or genealogical structures

ChatGPT Response:

A **recursive relationship** in a database occurs when **an entity is related to itself**. This means the **same entity type participates in a relationship more than once**, but in **different roles**.

Example:

Consider an entity called **EMPLOYEE**.
Some employees manage other employees. In this case:

- One **EMPLOYEE** is the **supervisor**
- Another **EMPLOYEE** is the **supervisee**

This creates a recursive relationship called "**supervision**", where:

- The **EMPLOYEE** entity appears **twice** in the same relationship
- Once as the supervisor
- Once as the supervisee

In Simple Words:

A recursive relationship is when something is connected to **another of the same kind** — like a person managing another person, or a folder inside another folder

5. What is foreign key?

The Project Response:

Foreign Key

A foreign key is an attribute (or a group of attributes) in one table that references the primary key of another table. It creates a link between two tables and helps maintain referential integrity, which means keeping data consistent between related tables.

Example from the syllabus:

In the COMPANY DATABASE:

- The EMPLOYEE table has a column called DEPT_NUM, which is a foreign key.
- This DEPT_NUM refers to the DEPT_NUM in the DEPARTMENT table (which is the primary key there).
- This ensures that every employee belongs to a valid department.

Key properties of a foreign key:

1. **References a Primary Key**
A foreign key in one table must refer to a primary key in another table.
2. **Enforces Consistency**
It avoids incorrect data by preventing entries that refer to non-existent values. For example, you cannot assign an employee to a department that doesn't exist.
3. **Optional or Mandatory**
A foreign key can allow NULL values or be required, depending on business rules.

Example from a real scenario:

If a Representative is assigned to a Sales Area, then the AREA_NUM in the Representative table would be a foreign key referring to the AREA_NUM in the Sales Area table.

ChatGPT Response:

Foreign Key (in simple terms):

A **foreign key** is a field (or a group of fields) in one table that **connects** to the **primary key** in another table.

Why do we use it?

To **link two tables** together and make sure that the data stays **correct and consistent**.

Example:

Imagine you have two tables:

1. **Department Table**
 - Dept_ID (Primary Key)
 - Dept_Name
2. **Employee Table**
 - Emp_ID
 - Emp_Name
 - Dept_ID → This is a **foreign key** that refers to Dept_ID in the Department Table.

What does this do?

It makes sure that every employee belongs to a **real department**.

You cannot add an employee with a department that doesn't exist in the Department Table.

Summary:

- **Primary key** = unique ID inside its own table.
- **Foreign key** = points to a primary key in another table.

6. how many entity type in this scenario:

A database is decided for a large construction enterprise to support its resources

management. Such an enterprise has a set of Departments and Projects. Each department has a set of Equipment. Equipment's are assigned to projects for a specific period of time.

- No equipment is assigned to more than one project at a time.
- An equipment could be idle for a given period of time.
- The database is to contain the following information:
 - Department data together, with data concerning the equipment of each department.
 - Project's data together with necessary data of all equipment assigned to each project (start and finish dates of assigning an equipment to a project).
 - For each department: department number (unique), name (unique), and budget.
 - For each equipment: equipment number (unique), name, store location.
 - For each project: project number (unique), name, location, project start date and project finish date.

The Project Response:

Entity Types Breakdown:

1. Main Entity Types

1. Department

- Attributes: Department Number (unique), Name (unique), Budget.

- Key Attribute: Department Number.

2. Project

- Attributes: Project Number (unique), Name, Location, Start Date, Finish Date.
- Key Attribute: Project Number.

3. Equipment

- Attributes: Equipment Number (unique), Name, Start Location.
- Key Attribute: Equipment Number.

2. Associative Entity (Due to Relationship with Attributes)

4. Assignment (or Equipment/Assignment)

- Purpose: To store the start and finish dates of equipment assigned to a project (as specified in the scenario).
- Attributes:
 - Start Date (from the project's date).
 - Finish Date (from the project's date).

Why the Associative Entity is Needed

The syllabus explicitly states that relationships with attributes (e.g., attributes dated) require an associative entity to store these attributes. This is a standard practice in modeling to avoid redundancy and ensure data integrity. For example, in the ECOTMS scenario, the syllabus would model a similar relationship between Flight and Angular with attributes like "Date of Assignment" as an associative entity.

Summary

- Main Entity Types: 3 (Department, Project, Equipment).
- Associative Entity: 1 (Assignment).
- Total Entity Types: 4 (3 main + 1 associative).

ChatGPT Response:

 **Final Count:**

- **Entity Types:**

1. Department
2. Equipment
3. Project

7.how many entity type in this scenario:

An airline company has passengers, each identified with a unique passenger ID, along with their

name and passport number. Flights are identified by a unique flight number and have a departure

and arrival time. Each flight is operated by an airplane, which is identified by a unique airplane

ID, make, and model. Passengers book tickets for these flights, and each ticket has a unique ticket

number.

The project Response:

Key Attributes: Select the User's Scenario Settings for Online Families with their own Attributes, the answer is four.

Based on the provided scenario, there are four entity types:

Passenger

Attributes: Passenger ID (unique), Name, Passport Number.

Key Attribute: Passenger ID.

Flight

Attributes: Flight Number (unique), Departure Time, Arrival Time.

Key Attribute: Flight Number.

Airplane

Attributes: Airplane ID (unique), Make, Model.

Key Attribute: Airplane ID.

Ticket

Attributes: Ticket Number (unique).

Key Attribute: Ticket Number.

Explanation:

Each entity type is defined by its unique key attribute(s) and associated properties. The syllabus emphasizes that an entity type is a group of entities sharing the same attributes, with a key attribute ensuring uniqueness. In this scenario, the four entities (Passenger, Flight, Airplane, Ticket) meet this definition.

ChatGPT Response:

✅ **Final Answer:**

There are 4 entity types:

1. Passenger
2. Flight
3. Airplane
4. Ticket

8.List all relationships between these entity types

The Project Response:

1. Passenger – Ticket (One-to-Many)

- Description: A passenger can book multiple tickets, and each ticket belongs to exactly one passenger.
- Cardinality:
 - One Passenger can have many Tickets.
 - One Ticket is associated with one Passenger.
- Syllabus Reference: This aligns with the concept of a binary relationship (degree = 2), where one entity (Passenger) participates in multiple instances of another entity (Tickets).

2. Ticket – Flight (One-to-Many)

- Description: A ticket is associated with a single flight, and a flight can have multiple tickets.
- Cardinality:
 - One Flight can have many Tickets.
 - One Ticket is linked to one Flight.
- Syllabus Reference: This is another binary relationship. The syllabus explicitly mentions such relationships (e.g., "works for", or the example).

3. Flight – Airplane (One-to-Many)

- Description: A flight is operated by one airplane, and an airplane can operate multiple flights.
- Cardinality:
 - One Airplane can be used for many Flights.
 - One Flight is operated by one Airplane.

ChatGPT Response:

✅ Final List of Relationships:

1. **Books** – between **Passenger** and **Ticket**
2. **Issued_For** – between **Ticket** and **Flight**
3. **Operated_By** – between **Flight** and **Airplane**

9.What is Constraint and Characteristics of specialization concepts in database?

The Project Response:

Constraint and Characteristics of Specialization

Specialization in the EER model is governed by specific constraints and has distinct characteristics. These are explicitly outlined in the syllabus and include:

1. Definition Constraints

These determine how entities are assigned to subclasses within a specialization.

- a. Predicate-Defined Specialization
 - A condition (predicate) is applied to an attribute of the superclass to determine subclass membership.
 - Example: In the **EMPLOYEE** entity, subclasses like **SECRETARY**, **ENGINEER**, and **TECHNICIAN** are defined based on the **jobtype** attribute.
- b. User-Defined Specialization
 - Subclass membership is manually assigned by the user, not determined by a logical condition.
 - Example: A **Manager** subclass might be assigned to specific employees based on organizational rules rather than an attribute.

2. Disjointness Constraint

- Definition: Specifies whether an entity in the superclass can belong to multiple subclasses simultaneously.
- Disjoint Specialization: An entity can belong to only one subclass.
 - Example: An **Employee** can be either a **Manager** or a **Developer**, but not both.
- Overlapping Specialization: An entity can belong to multiple subclasses.
 - Example: An **Employee** can be both a **Manager** and a **Developer**.

3. Participation Constraints

These define whether entities in the superclass must belong to a subclass.

- a. Total Participation
 - Every entity in the superclass must belong to at least one subclass.
 - Example: In a **Customer** specialization into **VIP** and **Regular**, every customer must be classified as either **VIP** or **Regular**.
- b. Partial Participation
 - Entities in the superclass may or may not belong to any subclass.

- Example: In an **Employee** specialization into **Manager**, some employees might not be managers.

ChatGPT Response:

Characteristics of Specialization:

1. Top-Down Approach : Start from a general entity (Employee) and break it into more specific ones (Engineer , Manager).
2. Subclasses Inherit Attributes and Relationships : Subclasses automatically inherit the attributes and relationships of the superclass.
3. Each Subclass May Have Additional Attributes : E.g., Pilot might have licenseNumber , while Technician might have skillSet .

10.How many Entity Types in this Scenario:

A hospital has many registered physicians. Attributes of physicians include an id number and specialty. Patients are admitted to the hospital by physicians. Attributes of patients include a patient's id and name. Any patient who is admitted must have exactly one admitting physician. A physician may optionally admit any number of patients. Once admitted, a given patient must be treated by at least one physician. A particular physician may treat any number of patients or may not treat any patients. Whenever a patient is treated by a physician, the hospital wishes to record the details of the treatment which include the date, time, and results.

The Project Response:

Entity Types:

1. Physician
2. Patient
3. Admission (capturing admission information)
4. Treatment (recording treatment details)

Relationships Between Entity Types:

1. Physician - Patient

- There is a one-to-many relationship between Physicians and Patients for the admission process.

- A physician can admit multiple patients, but each patient must be admitted by exactly one physician.

2. Admission

- The entity type "Admission" captures the details of how a physician admits a patient.
 - One Physician to many Admissions: A single physician may admit multiple patients.
 - Many Patients to many Admissions: Each patient has exactly one admission entry.

ChatGPT Response:

Based on the question in the image, here's the breakdown:

✓ Number of Entity Types = 2

1. Physician
 - Attributes: Physician_ID (PK), Specialty
2. Patient
 - Attributes: Patient_ID (PK), Name

11. Write a query that retrieves the employee names who work on the project 'ProjectX' using the created view.

The Project Response:

Querying Data Through a View

You can query data using views similarly to how you would use base tables. For instance:

```
SELECT FNAME, LNAME
FROM WORKS_ON_VIEW
WHERE PNAME='ProjectX';
```

This query retrieves the names of employees who work on a specific project named **ProjectX**.

Dropping a View

If you no longer need a view, it can be dropped from the database using:

```
DROP VIEW WORKS_ON_VIEW;
```

Updating Views

Updating views that are based on multiple tables or involve aggregate functions is generally complex and may lead to ambiguous situations. For example, updating a single attribute in a view might require changes across several base tables, leading to potential inconsistencies.

In summary, views provide an abstraction layer over your database schema.

ChatGPT Response:

```
SELECT IE.Name
FROM IT_Employees IE
JOIN WorksOn W ON IE.EmpID = W.EmpID
JOIN Projects P ON W.ProjectID = P.ProjectID
WHERE P.ProjectName = 'ProjectX';
```

12. make a query that for each employee, retrieve the employee's first and last name and first and last name of his or her immediate supervisor.

The Project Response:

```
SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM Employee E, Employee S
WHERE E.SUPERSSN = S.SSN;
```

Explanation:

- E and S are aliases for the Employee table.
- SUPERSSN is the foreign key in the Employee table that references the SSN (primary key) of the supervisor in the same table.
- The query joins the Employee table with itself to match each employee (E) with their immediate supervisor (S).

This matches the example provided in the syllabus (see the employee relation and the self-join query).

ChatGPT Response:

```
SELECT
    E.FName AS Employee_FirstName,
    E.LName AS Employee_LastName,
    S.FName AS Supervisor_FirstName,
    S.LName AS Supervisor_LastName
FROM
    EMPLOYEE E
LEFT JOIN
    EMPLOYEE S
ON
    E.SupervisorSSN = S.SSN;
```

13.make a query that: Give me the names of all employees, and if an employee is a manager of a department, also show the name of that department.

The Project Response:

```
SELECT E.FNAME, E.LNAME, D.DNAME
FROM EMPLOYEE E
LEFT JOIN DEPARTMENT D ON E.SSN = D.MGRSSN;
```

Explanation:

1. EMPLOYEE E: Represents the employee table, with FNAME and LNAME storing the employee's first and last names.
2. DEPARTMENT D: Represents the department table, with DNAME storing the department name.
3. LEFT JOIN: Ensures all employees are listed, even if they are not managers (in which case D.DNAME will be NULL).

4. ON E.SSN = D.MGRSSN: Joins employees to departments where the employee's SSN matches the department's manager SSN (MGRSSN).

ChatGPT Response:

```
SELECT
    E.FName AS Employee_FirstName,
    E.LName AS Employee_LastName,
    D.DName AS Department_Name
FROM
    EMPLOYEE E
LEFT JOIN
    DEPARTMENT D
ON
    E.SSN = D.MgrSSN;
```

Explanation:

- EMPLOYEE table contains employee data.
- DEPARTMENT table contains department data.
- MgrSSN in DEPARTMENT table stores the SSN of the manager.
- We use LEFT JOIN to include **all employees**, even if they are **not** managers.
- If an employee is a manager, the department name (DName) will appear.
- If not, the Department_Name will be NULL

14.make this query : Retrieve the names and locations of departments that are associated with more than one location. The department information is stored in one table, and the locations are stored in another table using a foreign key relationship.

The Project Response:

```
SELECT D.DEPT_NAME, L. LOCATION_NAME
FROM DEPARTMENT D
JOIN LOCATION L ON D.DEPT_ID = L.DEPT_ID
WHERE D.DEPT_ID IN (
    SELECT DEPT_ID
    FROM LOCATION
    GROUP BY DEPT_ID
    HAVING COUNT(*) > 1
);
```

ChatGPT Response:

```
SELECT D.DEPT_NAME, L.LOCATION_NAME
FROM DEPARTMENT D
JOIN LOCATION L ON D.DEPT_ID = L.DEPT_ID
WHERE D.DEPT_ID IN (
    SELECT DEPT_ID
    FROM LOCATION
    GROUP BY DEPT_ID
    HAVING COUNT(*) > 1
);
```

15.make a query that: only employees who have a supervisor are included in the result; an EMPLOYEE tuple whose value for Super_ssn is NULL is excluded.

The Project Response:

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E INNER JOIN EMPLOYEE AS S
ON E.Super_ssn = S.Ssn);
```

Explanation:

- **INNER JOIN** ensures that only employees with a valid Super_ssn (i.e., non-NULL) are included in the result.
- Tuples where Super_ssn is NULL are excluded explicitly by this join type.

This aligns with the example provided in the syllabus document.

ChatGPT Response:

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM EMPLOYEE AS E
INNER JOIN EMPLOYEE AS S ON E.Super_ssn = S.Ssn;
```

Explanation:

- E is the alias for the employee.
- S is the alias for the supervisor (also from the EMPLOYEE table).
- INNER JOIN ensures that only rows where E.Super_ssn matches an actual S.Ssn are included.
- Any employee whose Super_ssn is NULL (i.e. no supervisor) is **automatically excluded** by the INNER JOIN.

16. make this query: Select the Essns of all employees who work the same (project, hours) combination on some project that employee ‘John Smith’ (whose Ssn = ‘123456789’) works on.

The Project Response:

To find the **ESSNs** of all employees who work on the same **(project, hours)** combination as **John Smith** (SSN = '123456789'), use the following SQL query:

```
SELECT ESSN
FROM WORKS_ON
WHERE (PNO, HOURS) IN (
    SELECT PNO, HOURS
    FROM WORKS_ON
```

```
WHERE ESSN = '123456789'
```

```
);
```

Explanation:

- The **subquery** retrieves all **(PNO, HOURS)** pairs from the WORKS_ON table where the employee is **John Smith**.
- The **main query** selects all employees whose **(PNO, HOURS)** combinations match any of these pairs.
- The **IN** operator checks for matches between the **(PNO, HOURS)** tuples in the main query and the subquery.

ChatGPT Response:

```
SELECT ESSN
FROM WORKS_ON
WHERE (PNO, HOURS) IN (
    SELECT PNO, HOURS
    FROM WORKS_ON
    WHERE ESSN = '123456789'
);
```

17.how many entity types in this :

A database is to contain information concerning sales representatives, sales areas, and products. Each representative is responsible for sales in only one area. Each area may have one or more responsible representatives. Each representative is responsible for sales of one or more products, and each product has one or more responsible representatives. A sales commission is assigned to each representative. The commission rate is determined according to the sales area, in other words, for each area there is a certain fixed commission rate for all responsible representatives of that area. The database is to contain the following information:

- For each representative: representative number (unique), representative name (unique), and sales commission.
- For each sales area: area number (unique), and location.
- For each product: product number (unique), product name and quantity sold by each

representative in each area.

The Project Response:

There are 3 entity types in the given scenario:

1. Representative

- **Attributes:**
 - RepresentativeNumber (unique)
 - Name (unique)
 - SalesCommission
- **Relationships:**
 - **One-to-many** with **SalesArea** (a representative is responsible for only one area).
 - **Many-to-many** with **Product** (a representative is responsible for multiple products, and each product has multiple representatives).

2. SalesArea

- **Attributes:**
 - AreaNumber (unique)
 - Location
 - **Relationships:**
 - **One-to-many** with **Representative** (an area may have multiple representatives).
 - **Many-to-many** with **Product** (via the quantity sold by representatives in that area).
-
- AreaNumber (unique)
 - Location
 - **Relationships:**

- **One-to-many** with **Representative** (an area may have multiple representatives).
- **Many-to-many** with **Product** (via the quantity sold by representatives in that area).

3. Product

- **Attributes:**
 - ProductNumber (unique)
 - ProductName
- **Relationships:**
 - **Many-to-many** with **Representative** (via the quantity sold in each area).
 - **Many-to-many** with **SalesArea** (via the quantity sold by representatives in that area).

Key Relationships

- **Representative -- SalesArea: One-to-many** (each representative is responsible for one area; each area has multiple representatives).
- **Representative -- Product: Many-to-many** (each representative sells multiple products, and each product is sold by multiple representatives).
- **Product -- SalesArea: Many-to-many** (via the quantity sold by representatives in that area).

ChatGPT Response:

Based on the provided database scenario, there are **3 entity types**:

1. Representative

- Attributes: RepresentativeNumber (unique), Name (unique), SalesCommission
- Relationships:
 - One-to-many with SalesArea (each rep is assigned to one area; one area can have multiple reps)

- Many-to-many with Product (each rep sells multiple products; each product is sold by multiple reps)

2. SalesArea

- Attributes: AreaNumber (unique), Location
- Relationships:
 - One-to-many with Representative
 - Many-to-many with Product (via quantity sold by reps in that area)

3. Product

- Attributes: ProductNumber (unique), ProductName
- Relationships:
 - Many-to-many with Representative
 - Many-to-many with SalesArea

Explanation:

- WORKS_ON is assumed to be a table with at least these columns:
 - ESSN (employee SSN),
 - PNO (project number),
 - HOURS (hours worked).
- The **subquery** fetches all (PNO, HOURS) combinations for John Smith.
- The **main query** selects all ESSNs of employees who match any of those (PNO, HOURS) pairs.

18.what is relationship in :

In a mini world of university, each semester, each student must be assigned an advisor who advises students about degree requirements and helps students register for classes. Each student must register for classes with the help of an advisor, but if the student's assigned

advisor is not available, then the student may register with any advisor. We want to keep track of students, the assigned advisor for each, and the name of the advisor with whom the student registered for the current term.

The Project Response:

1. Student - Assigned Advisor

- **Description:** Each student is **assigned one advisor** who is responsible for advising them about degree requirements and registration.
- **Cardinality:**
 - **Student (Many) → Advisor (One):** One advisor can be assigned to multiple students.
 - **Student (One) → Advisor (One):** Each student has exactly one assigned advisor.
- **Example:**
 - Advisor "Dr. Smith" is assigned to Student A, Student B, and Student C.
 - Student D is assigned to Advisor "Dr. Lee".

1. Student - Assigned Advisor

- **Description:** Each student is **assigned one advisor** who is responsible for advising them about degree requirements and registration.
- **Cardinality:**
 - **Student (Many) → Advisor (One):** One advisor can be assigned to multiple students.
 - **Student (One) → Advisor (One):** Each student has exactly one assigned advisor.
- **Example:**
 - Advisor "Dr. Smith" is assigned to Student A, Student B, and Student C.

- Student D is assigned to Advisor "Dr. Lee".

2. Student - Registered Advisor

- **Description:** Each student **registers for classes with an advisor** (could be their assigned advisor or another advisor if the assigned one is unavailable).
- **Cardinality:**
 - **Student (Many) → Advisor (One):** One advisor can assist multiple students with registration.
 - **Student (One) → Advisor (One):** Each student registers with exactly one advisor for the current term.
- **Example:**
 - Student A registers with their assigned advisor "Dr. Smith".
 - Student B registers with a different advisor "Dr. Lee" because their assigned advisor is unavailable.

Key Notes:

- **Advisor Entity:** The Advisor entity has attributes like **Name** and possibly **Availability** (to determine if they are available for registration).
- **Registered Advisor:** The Registered Advisor relationship is separate from the Assigned Advisor relationship, even if they are the same person.
- **Flexibility:** The system allows a student to register with any advisor (not just their assigned one), but the assigned advisor is still tracked separately.

ChatGPT Response:

Assigned to: One-to-One. Each student has one assigned advisor (permanently or long-term).

Registers with: Many-to-One. Students register with an advisor each term (might not be assigned one).

19.Retrieve all the attributes of all employees who work for the ‘IS’ department.

The Project Response:

To retrieve all attributes of employees who work in the 'IS' department we assume the following:

- There is an employee table that includes a department column (or a similar identifier like department_id linked to a department table).
- The department column holds values like IS (short for Information Systems).

SQL Query

```
SELECT *  
FROM employee  
WHERE department = 'IS';
```

Explanation

- SELECT * retrieves all attributes (columns) of the matching rows.
- FROM employee specifies the table.
- WHERE department = 'IS' filters for employees in the 'IS' department.

Notes

- If the department is stored in a separate table (e.g., department), you'd need to JOIN the tables:

```
SELECT e.*  
FROM employee e  
JOIN department d ON e.department_id = d.department_id  
WHERE d.department_name = 'IS';
```

Replace department or department_id with your actual schema's column names if they differ.

ChatGPT Response:

```
SELECT *  
FROM Employee  
WHERE Department = 'IS';
```

Explanation:

- SELECT *: selects all columns (attributes).
- FROM Employee: from the table named Employee.
- WHERE Department = 'IS': filters only those who are in the IS department.

If departments are in a separate table:

You may need a JOIN. For example:

```
SELECT E.*  
  
FROM Employee E  
  
JOIN Department D ON E.DeptID = D.DeptID  
  
WHERE D.DeptName = 'IS';
```

20.make this query : Retrive the name of each employee works on all the projects controlled by department number 5 can be written using EXISTS and NOT EXISTS in SQL systems.

The Project Response:

To retrieve the names of employees who work on **all projects controlled by department number 5**, use the NOT EXISTS approach as follows:

```
SELECT E.Lname  
FROM EMPLOYEE E  
WHERE NOT EXISTS (
```

```

SELECT P.PNO
FROM PROJECT P
WHERE P.DNO = 5
AND NOT EXISTS (
    SELECT 1
    FROM WORKS_ON W
    WHERE W.ESSN = E.Ssn AND W.PNO = P.PNO
)
);

```

Using EXISTS (Alternative Approach): While NOT EXISTS is the standard method for "all" conditions, you can achieve a similar result with EXISTS by rephrasing the logic:

```

SELECT E.Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (
    SELECT P.PNO
    FROM PROJECT P
    WHERE P.DNO = 5
    AND NOT EXISTS (
        SELECT 1
        FROM WORKS_ON W
        WHERE W.ESSN = E.Ssn AND W.PNO = P.PNO
    )
);

```

ChatGPT Response:

```

SELECT E.Lname, E.Fname
FROM EMPLOYEE E
WHERE NOT EXISTS (
    SELECT P.Pnumber
    FROM PROJECT P
    WHERE P.Dnum = 5
    AND NOT EXISTS (

```

```
SELECT W.ESSN
FROM WORKS_ON W
WHERE W.ESSN = E.SSN
AND W.PNO = P.Pnumber
)
);
```

Explanation:

- **Outer query:** Goes through each employee.
- **First NOT EXISTS:** Makes sure **there is no project** (controlled by department 5) that the employee **does not** work on.
- **Inner NOT EXISTS:** Checks if a particular project (from department 5) is **not found** in the list of projects the employee works on.

If even one such project exists (that the employee does **not** work on), the employee is **excluded**.

21.create table customers

The Project Response:

Here's a basic SQL CREATE TABLE statement for a customers table with common fields:

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone_number CHAR(15),
  address VARCHAR(255),
  city VARCHAR(100),
  state VARCHAR(50),
  postal_code VARCHAR(20),
  country VARCHAR(50),
```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

ChatGPT Response:

```
CREATE TABLE customers (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  phone VARCHAR(20)  
);
```

22. how subquery can be used in select

The Project Response:

Subqueries in the SELECT Clause

A subquery in the SELECT clause is used to compute a value for each row in the outer query. It can return a single value (scalar) or a set of values (but must be handled carefully). Below are key use cases and examples:

1. Scalar Subquery (Single Value)

A subquery that returns **one row and one column** can be used directly in the SELECT clause. This is useful for retrieving aggregated values or fixed data.

Example:

Retrieve each employee's name, salary, and the average salary in their department:

```
SELECT  
  E.Ename,  
  E.Salary,  
  (SELECT AVG(Salary) FROM EMPLOYEE WHERE DeptNo = E.DeptNo) AS  
  AvgDeptSalary  
FROM EMPLOYEE E;
```

- **Explanation:**

- The subquery calculates the **average salary** for each employee's department.

- This is a **correlated subquery** because it references E.DeptNo from the outer query.

2. Unrelated Subquery (Same Value for All Rows)

If the subquery does **not depend on the outer query**, it returns the same value for all rows.

Example:

Add a column showing the company-wide average salary:

```
SELECT
    E.Ename,
    E.Salary,
    (SELECT AVG(Salary) FROM EMPLOYEE) AS AvgCompanySalary
FROM EMPLOYEE E;
```

- **Explanation:**

- The subquery computes the **average salary** for all employees.
- This is an **uncorrelated subquery** and returns the same value for every row.

3. Complex Calculations with Subqueries

Subqueries can compute derived values based on other tables or nested logic.

Example:

List employees and their department name:

```
SELECT
    E.Ename,
    (SELECT Dname FROM DEPARTMENT WHERE DeptNo = E.DeptNo) AS DeptName
FROM EMPLOYEE E;
```

- **Explanation:**

- The subquery retrieves the **department name** for each employee.
- This is a **correlated subquery** that matches DeptNo between EMPLOYEE and DEPARTMENT.

4. Conditional Aggregations

Use subqueries to compute conditional values (e.g., counts, sums) based on specific criteria.

Example:

Count the number of projects each employee works on:

```
SELECT
    E.Ename,
    (SELECT COUNT(*) FROM WORKS_ON WHERE ESSN = E.Ssn) AS ProjectCount
FROM EMPLOYEE E;
```

- **Explanation:**

- The subquery counts the **number of projects** for each employee.
- This is a **correlated subquery** that matches ESSN between EMPLOYEE and WORKS_ON.

Important Notes

1. **Single Value Requirement:** The subquery must return **exactly one row** (or use aggregate functions like MAX, MIN, AVG, etc.) to avoid errors.
2. **Performance Considerations:** Correlated subqueries (which reference the outer query) can be slower than joins, depending on the database.
3. **Use Cases:** Ideal for **derived columns**, **aggregated data**, or **lookup values** from related tables.

ChatGPT Response:

Count of projects each employee works on

```
SELECT E.Fname, E.Lname,
    (SELECT COUNT(*)
     FROM WORKS_ON W
     WHERE W.ESSN = E.SSN) AS Project_Count
FROM EMPLOYEE E;
```

Show each employee's department name

Assume we have:

- EMPLOYEE(SSN, Fname, Dno)
- DEPARTMENT(Dnumber, Dname)

```
SELECT E.Fname, E.Lname,  
      (SELECT D.Dname  
       FROM DEPARTMENT D  
       WHERE D.Dnumber = E.Dno) AS Dept_Name  
FROM EMPLOYEE E;
```

23.SQL statement creates a stored procedure that selects Customers from a particular city with a particular postalcode from the "customers" table.

The Project Response:

Here's an SQL statement to create a stored procedure that selects customers from a specific City and PostalCode in the Customers table:

SQL Stored Procedure Example (for SQL Server/MySQL/PostgreSQL):

Create a stored procedure to retrieve customers by City and PostalCode

```
CREATE PROCEDURE GetCustomersByCityAndPostalCode
```

```
    @City NVARCHAR(50),
```

```
    @PostalCode NVARCHAR(20)
```

```
AS
```

```
BEGIN
```

```
    SELECT *
```

```
    FROM Customers
```

```
    WHERE City = @City
```

```
    AND PostalCode = @PostalCode;
```

```
END;
```

Explanation:

1. @City and @PostalCode: Parameters passed to the stored procedure.
 - NVARCHAR(50) is used for City (adjust based on your schema).
 - NVARCHAR(20) is used for PostalCode (adjust if needed).
2. SELECT *: Retrieves all columns from the Customers table.

3. WHERE Clause: Filters rows where City and PostalCode match the input parameters.

How to Call the Stored Procedure:

Example: Retrieve customers from "Berlin" with postal code "1010"

```
EXEC GetCustomersByCityAndPostalCode 'Berlin', '1010';
```

Notes:

- **Adjust Data Types:** If your City or PostalCode columns use different data types (e.g., VARCHAR or CHAR), update the parameter definitions accordingly.
- **Database Compatibility:** This syntax works for **SQL Server**, **MySQL**, and **PostgreSQL** (with minor adjustments if needed).
- **Security:** In production, consider adding input validation or using PARAMETERS with SET for more control.

ChatGPT Response:

```
DELIMITER //
```

```
CREATE PROCEDURE GetCustomersByCityAndPostalCode(
```

```
    IN input_city VARCHAR(50),
```

```
    IN input_postalcode VARCHAR(20)
```

```
)
```

```
BEGIN
```

```
    SELECT *
```

```
    FROM customers
```

```
    WHERE city = input_city
```

```
        AND postalcode = input_postalcode;
```

```
END //
```

```
DELIMITER ;
```

Explanation:

- CREATE PROCEDURE defines the stored procedure named GetCustomersByCityAndPostalCode.
- It takes **two input parameters**: input_city and input_postalcode.

- Inside the BEGIN ... END block, it runs a SELECT query filtering rows from the customers table based on those parameters.
- DELIMITER // is used to tell MySQL to interpret the procedure body correctly (you change it back to ; after the procedure).

24.why use trigger in database?

The Project Response:

Triggers are **database objects** that automatically execute **predefined SQL statements** in response to specific **events** (e.g., INSERT, UPDATE, DELETE) on a table. They are used to enforce **business rules**, maintain **data integrity**, and automate **database operations**. Below are the key reasons to use triggers:

1. Enforce Business Rules

Triggers ensure that business logic is applied consistently, even if users or applications bypass application-level validation.

- **Example:** Prevent a customer from being deleted if they have active orders.

```
CREATE TRIGGER PreventDeleteCustomerWithOrders
ON Customers
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Orders WHERE CustomerID IN (SELECT CustomerID
FROM DELETED))
        RAISERROR('Cannot delete customer with active orders.', 16, 1);
END;
```

2. Maintain Data Integrity

Triggers can validate data before it is inserted or updated, ensuring that only **valid data** enters the database.

- **Example:** Ensure a product's price is always above cost.

```
CREATE TRIGGER ValidateProductPrice
ON Products
```

```
FOR INSERT, UPDATE
AS
BEGIN
    IF EXISTS (SELECT * FROM inserted WHERE Price < Cost)
        RAISERROR('Price cannot be less than cost.', 16, 1);
END;
```

3. Automate Operations

Triggers can automate repetitive tasks, such as **logging changes, updating related tables, or generating reports.**

- **Example:** Log changes to a table in an audit table.

```
CREATE TRIGGER LogCustomerChanges
ON Customers
AFTER UPDATE
AS
BEGIN
    INSERT INTO AuditLog (TableName, Action, ChangedAt)
    VALUES ('Customers', 'UPDATE', GETDATE());
END;
```

4. Enforce Referential Integrity

Triggers can handle **cascading updates or deletes** when foreign key constraints are not sufficient.

- **Example:** Automatically update related records when a primary key changes.

```
CREATE TRIGGER UpdateOrderCustomerID
ON Customers
AFTER UPDATE
AS
BEGIN
    UPDATE Orders
    SET CustomerID = (SELECT CustomerID FROM inserted)
    WHERE CustomerID IN (SELECT CustomerID FROM deleted);
END;
```

5. Audit and Track Changes

Triggers can track **who made changes, when, and what was changed**, which is critical for

auditing and compliance.

- **Example:** Log who updated a record and what the old and new values were.

```
CREATE TRIGGER TrackEmployeeChanges
ON Employees
FOR UPDATE
AS
BEGIN
    -- [Additional logic to log old and new values can be added here]
END;
```

ChatGPT Response:

1. Data Integrity Enforcement

Triggers can ensure that certain rules are always followed, even if application logic fails to do so.

Example: Prevent inserting a negative salary in an EMPLOYEE table.

2. Automatic Auditing

You can automatically track changes to data by recording them in an audit/log table.

Example: Log every deletion from the CUSTOMERS table into a CUSTOMER_LOG table.

3. Cascading Actions

They can enforce cascading updates or deletions without explicitly coding them each time.

Example: When a department is deleted, automatically set employees' Dno to NULL.

4. Validation

Triggers can validate data before it's inserted or updated.

Example: Ensure that an ORDER total doesn't exceed a customer's credit limit.

25.What is ER Model Concepts ?

The Project Response:

The **ER Model Concepts** are foundational to designing conceptual database schemas using the Entity-Relationship (ER) model. According to the syllabus, the ER model is built using three core tools:

1. Entity/Entity Type

- An **entity** represents a distinct object or concept in the real world (e.g., "Student," "Course").
- An **entity type** is a collection of entities with the same attributes (e.g., all "Students" in a university).

2. Attribute

- Describes properties or characteristics of an entity (e.g., "StudentID," "Name," "BirthDate").
- Attributes can be **simple** (single value) or **composite** (composed of multiple simple attributes, e.g., "Address" with "Street," "City," "ZIP Code").

3. Relationships

- Represents associations between entities (e.g., "Enrolls" between "Student" and "Course").
- Relationships can be **binary** (involving two entities) or **n-ary** (involving more than two entities).

These concepts are used to create an **ER diagram**, which visually represents the structure of a database system.

ChatGPT Response:

The **ER Model** (Entity-Relationship Model) is a **high-level conceptual data model** used in database design. It helps visualize the **structure of a database** by identifying:

- **Entities** (things or objects),
- **Attributes** (properties of those things), and
- **Relationships** (how those things are related).

It is commonly used during the **design phase** of a database to ensure clarity before translating into a physical schema.

26. Make Rough Schema to:

The COMPANY DATABASE keeps track of a company's employees, departments, and projects. Suppose that after the requirements collection and analysis phase, the database designers stated the following description of the "mini-world": the part of the company to be represented in the database:

- The company is organized into departments. Each department has a name, a number, and an employee who manages the department. We keep track of the start date when that employee started managing the department. A department may have several locations.
- A department controls several projects, each of which has a name, a number, and a single location.
- We store each employee's name, social security number, address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's name, sex, birth date, and relationship to the employee.

The Project Response:

Entity Types and Attributes

1. DEPARTMENT

- Name (string)
- Number (integer)
- Locations (multi-valued attribute: list of strings, e.g., "New York," "Chicago")
- Manager (foreign key to EMPLOYEE)
- ManagerStartDate (date)

2. PROJECT

- Name (string)
- Number (integer)

- Location (string)
- ControllingDepartment (foreign key to DEPARTMENT)

3. EMPLOYEE

- Name (composite attribute: Fname, Mname, Lname)
- SSN (string, primary key)
- Address (string)
- Salary (numeric)
- Sex (string)
- BirthDate (date)
- Department (foreign key to DEPARTMENT)
- Supervisor (foreign key to EMPLOYEE, self-referential relationship)
- WorksOn (multi-valued composite attribute: Project (foreign key to PROJECT) and Hour (numeric))

4. DEPENDENT

- Employee (foreign key to EMPLOYEE)
- DependentName (string)
- Sex (string)
- BirthDate (date)
- Relationship (string, e.g., "Spouse," "Child")

Key Relationships

- **DEPARTMENT** manages **PROJECT** (one-to-many: a department controls multiple projects).
- **EMPLOYEE** belongs to **DEPARTMENT** (one-to-many: an employee is assigned to one department).
- **EMPLOYEE** supervises **EMPLOYEE** (self-referential one-to-many: a supervisor can manage multiple employees).

- **EMPLOYEE** works on **PROJECT** (many-to-many: an employee can work on multiple projects, and a project can involve multiple employees).
- **EMPLOYEE** has **DEPENDENT** (one-to-many: an employee can have multiple dependents).

Notes

- **Multi-valued attributes** (e.g., Locations in DEPARTMENT, WorksOn in EMPLOYEE) are represented as lists or separate tables in a relational model.
- **Composite attributes** (e.g., Name in EMPLOYEE) are broken into sub-attributes.
- **Foreign keys** enforce relationships between entities.

ChatGPT Response:

Entity Types and Attributes

1. DEPARTMENT

- Attributes: Dname, Dnumber (PK), Mgr_ssn (FK), Mgr_start_date

2. DEPT_LOCATIONS

- Attributes: Dnumber (FK), Dlocation

3. EMPLOYEE

- Attributes: Fname, Minit, Lname, Ssn (PK), Bdate, Address, Sex, Salary, Super_ssn (FK), Dno (FK)

4. PROJECT

- Attributes: Pname, Pnumber (PK), Plocation, Dnum (FK)

5. WORKS_ON

- Attributes: Essn (FK), Pno (FK), Hours
- Composite Key: (Essn, Pno)

6. DEPENDENT

- Attributes: Essn (FK), Dependent_name, Sex, Bdate, Relationship

- Composite Key: (Essn, Dependent_name)

27. how many entity type and list all relationship in this scenario:

A database is to contain information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at several different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:

- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings.
- For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students.
- For each teacher of a given offering; employee number (unique), and name.
- For each student of a given offering; employee number (unique), name, and grade.

The following constraints apply:

- For each course, all offerings of that course are taught by only one teacher.
- A teacher may teach more than one course type.

The Project Response:

1. Course
2. Offering
3. Teacher
4. Student

Relationships:

1. Course – Prerequisite → Course

- **Description:** A course may have one or more prerequisite courses (e.g., "Course A" requires "Course B").
- **Cardinality:** One-to-many (a course can be a prerequisite for multiple courses).

2. Course – Has → Offering

- **Description:** Each course has multiple offerings (e.g., "Course A" is offered in different semesters).
- **Cardinality:** One-to-many (a course has many offerings).

3. Offering – Taught by → Teacher

- **Description:** Each offering is taught by exactly one teacher.
- **Cardinality:** Many-to-one (a teacher can teach multiple offerings).

4. Offering – Has → Student

- **Description:** Each offering has multiple students enrolled.
- **Cardinality:** Many-to-many (a student can enroll in multiple offerings, and an offering has multiple students).

Summary:

- **Entity Types:** Course, Offering, Teacher, Student.
- **Relationships:**
 - Prerequisite (Course to Course).
 - Has (Course to Offering).
 - Taught by (Offering to Teacher).
 - Enrolled in (Offering to Student, many-to-many).

This design ensures flexibility for prerequisites, multiple offerings per course, and enrollment of students in multiple offerings.

ChatGPT Response:

Entity Types

1. Course

- Attributes: CourseNumber (PK), Title, PrerequisiteCourse

2. Offering

- Attributes: OfferingNumber (PK), Date, Location

- FK: CourseNumber (relates to Course), TeacherEmpNo

3. Teacher

- Attributes: EmpNo (PK), Name

4. Student

- Attributes: EmpNo (PK), Name, Grade

Relationships

1. Course–Offering

- One-to-Many
- A course **has many offerings**
- (CourseNumber in Offering is FK referencing Course)

2. Offering–Teacher

- Many offerings are **taught by one teacher**
- (TeacherEmpNo in Offering is FK referencing Teacher)

3. Offering–Student

- Many students **attend each offering**, and each student may **attend many offerings**
- Many-to-Many relationship
- Requires an associative table:
Enrollment(OfferingNumber, StudentEmpNo, Grade)

4. Course–Course (Prerequisite)

- One-to-One or Optional Self-Relationship
- A course may have **one prerequisite course**
- Represented via: PrerequisiteCourseNumber (FK referencing Course.CourseNumber)

28.how many entity type and list all relationship in:

A database is decided for scheduling the coming football tournament of World Cup in France. The participating teams are divided into groups each of 8 teams. The designed database must contain data about:

- Teams described by country, coach, and the score of these team in each match.
- Groups described by group number, teams in that groups and the group supervisor name.
- Matches described by the teams, referee name, court number, court city and date.
- Players described by teams, player number, player name, player position in the team, number of goals for each player in each match.

The Project Response:

Here's the combined content from the images provided, focusing on the football tournament database design:

how many entity types in A database is decided for scheduling the coming football tournament of World Cup in France. The participating teams are divided into groups each of 8 teams. The designed database must contain data about: Exercise • Teams described by country, coach, and the score of these team in each match. • Groups described by group number, teams in that groups and the group supervisor name. • Matches described by the teams, referee name, court number, court city and date. • Players described by teams, player number, player name, player position in the team, number of goals for each player in each match. and list all relationships

Teacher Thinking

Entity Types (4):

1. Team
2. Group
3. Match
4. Player

Relationships:

1. Group – Contains → Team

- Description: Each group contains multiple teams.

- Cardinality: One-to-many (a group has multiple teams, but each team belongs to one group).

2. Team – Plays → Match

- Description: Each match involves two teams, and each team plays multiple matches.
- Cardinality: Many-to-many (a match has two teams, and a team plays in multiple matches).
- Attributes: Score of the team in the match (stored as part of the relationship).

3. Team – Has → Player

- Description: Each team has multiple players, and each player belongs to one team.
- Cardinality: One-to-many (a team has many players, but each player is in one team).

4. Player – Scores in → Match

- Description: Each player can score in multiple matches, and each match has multiple players.
- Cardinality: Many-to-many (a player participates in multiple matches, and a match has multiple players).
- Attributes: Number of goals scored by the player in the match.

Notes on Additional Attributes:

- Match entity includes attributes:
 - Referee name
 - Court number
 - Court city
 - Match date
- Team entity includes attributes:
 - Country
 - Coach
- Player entity includes attributes:
 - Player number

- Name
- Position

ChatGPT Response:

Entity Types

1. Team

- Attributes: TeamID (PK), Country, Coach

2. Group

- Attributes: GroupNumber (PK), SupervisorName

3. Match

- Attributes: MatchID (PK), RefereeName, CourtNumber, CourtCity, Date

4. Player

- Attributes: PlayerID (PK), PlayerNumber, Name, Position, TeamID (FK)

Relationships

1. Group – Team

- One-to-Many
- A group contains **multiple teams**, each team belongs to **one group**
- (Team has FK: GroupNumber)

2. Team – Match

- Many-to-Many
- A match involves **two teams**, and each team plays in **many matches**
- Implemented via an associative table:
TeamMatch(MatchID, TeamID, Score)

3. Team – Player

- One-to-Many

- A team has **many players**, each player belongs to **one team**

4. **Player – Match (Goals)**

- Many-to-Many with attribute
- Tracks **number of goals scored** by a player in a match
- Requires an associative table:

PlayerMatchStats(PlayerID, MatchID, Goals)

Evaluation Methodology

To assess the performance of both “The Work” (our custom AI system) and ChatGPT, we asked each of them the same 28 questions. These questions were carefully selected from:

- The official university database textbook.
- Previous years’ university exam papers.
- ER diagram scenarios and SQL query tasks.
- Theoretical questions directly related to database concepts. Scoring

Categories

To evaluate accuracy in a structured way, we created three scoring categories:

- Category A: Worth 3 points.
- Category B: Worth 2 points.
- Category C: Worth 1 point.

After receiving the answers from both systems, a subject matter expert evaluated them and assigned scores accordingly.

Question Scoring Table

Question	The Work (Points)	ChatGPT (Points)
1	3	2
2	3	2
3	3	2
4	1	2
5	3	3
6	3	2
7	3	3
8	3	3
9	3	2
10	3	2
11	3	3
12	3	3
13	3	3
14	3	3
15	3	3
16	3	3
17	3	3
18	1	1
19	3	3
20	3	3
21	3	2
22	3	3
23	3	3
24	3	2
25	3	3
26	3	3
27	2	3
28	3	3

Table 6-1 Question Scoring Table for Evaluation

After receiving the answers from both systems, a subject matter expert evaluated them and assigned scores accordingly.

Total Score:

- The Work: 79/84
- ChatGPT: 73/84

Results Summary

System	Total Points	Out of	Accuracy (%)
The Work	79	84	94.0%
ChatGPT	73	84	86.9%

Table 6-2 Results Summary for Evaluation

This indicates that The Work achieved higher overall accuracy in providing correct and relevant responses aligned with course materials.

This evaluation clearly shows that The Work provides more accurate, focused, and reliable support for students studying database topics. Its performance, especially on textbook-based and exam-style questions, proves it to be a valuable academic assistant.

Comparison between the project and Chat GPT

Comparison	Our Project (eLearning Course using AI)	ChatGPT / General Generative Models
Scope of Knowledge	Focused on one academic subject (Database Course)	General-purpose across all fields
Answer Accuracy	Very accurate (answers come from a fixed academic book)	Can be inaccurate due to generalized knowledge
Hallucination Risk	Very low – answers are grounded in a trusted reference	High – model may generate invented information
Update Capability	Easily updated by editing the document or source	Requires model update from OpenAI
Voice Interaction	Supports voice input/output (Whisper + TTS)	No native voice, external tools needed
Avatar Response	Avatar delivers answers in both voice and animation	Not available by default
Specialization	Deep focus on a university-level subject	Broad, shallow coverage

Customization Ease	Easily customizable by changing or adding content	Hard to fine-tune, needs advanced technical work
Hardware Requirements	Can run on simple or local hardware	Requires heavy hardware for training/running
Data Control	Full control over local data so privacy done	Data is held by OpenAI
Usage Limits	Unlimited usage, 24/7 availability	Limited based on OpenAI API quotas or usage plans
Deep Search Speed	Fast – searches in a fixed, well-structured academic source	Slower – must reason over large, unstructured knowledge

Table 6-3 Comparison between the project and Chat GPT

Chapter 7

Conclusion and Future Work

Chapter Seven : Conclusion and Future Work

7.1 Introduction

In this chapter, we present a summary of our graduation project in Section 8.2. We then explain the skills we gained in Section 8.3, highlight the project achievements in Section 8.4, and finally discuss future work and improvements in Section 8.5.

7.2 Conclusion

Our project, AI-powered eLearning Course, aims to help students learn the Database course more interactively and efficiently using artificial intelligence. The system allows students to ask questions using their voice or text, and the AI avatar answers back with a clear voice response and also response generated in front. It combines technologies like Whisper (ASR), LLM + RAG (for question answering), and TTS (Text-to-Speech) to build a smart educational experience.

We also built a web interface that is simple, easy to use, and suitable for students from different technical levels.

Main Concluded Points:

- **Strong Planning:** We created a clear plan from the start, helping us organize the work step by step.
- **System Analysis and Design:** We studied the user needs (students) and designed the system to meet them.
- **Good Implementation:** We successfully connected all the AI models (Whisper, LLM, RAG, TTS) and made them work together smoothly.
- **Accurate Testing:** We tested the system with a database expert, and results showed high accuracy and performance.
- **Team Collaboration:** We worked together as a team and used our different skills to finish the project.
- **Power of AI in Education:** We proved that AI can play an important role in improving education by giving instant and helpful answers to students.

7.3 Earned Capabilities

During this project, we learned and developed the following skills:

- Skills in System Analysis, Design, and Implementation
- Skills in Testing, Evaluation, and Debugging
- Experience with AI models like Whisper, LLM, RAG, and TTS

- Frontend and Backend web development
- Integration of multiple AI components into one working system
- Writing professional documentation and presenting technical work
- Teamwork, time management, and problem-solving

7.4 Project Achievements

Our project achieved the main goals and functions we planned, including:

- Allowing users to speak their question using a microphone
- The system understands the question using Whisper
- The system generates an accurate answer using LLM + RAG based on the Database course
- The avatar responds with a clear voice using TTS
- The project succeed compared to ChatGPT, and a subject expert confirmed that the project gave more accurate answers, especially for real database exams and book-related content

7.5 Future Work

In the future, we plan to enhance and expand the project with the following:

- Add more university courses, starting with all the main courses at MTI University
- Connect our the project with the university's official eLearning platform, so students can access it easily alongside their study materials
- Support Arabic language, including Arabic TTS and Arabic questions/answers
- Make the TTS model work locally, without needing internet, to improve speed and privacy
- Add feedback system so students can rate the avatar's answers
- Improve avatar design to make it more interactive and engaging

References

- [1] **Conversational AI in Education: A New Age of Learning** By Anand Malto [JustCall](#) May 23, 2025
- [2] **Assessing the Impact of Personalized Learning Paths on Student Engagement and Outcomes** [Vorecol](#) June , 2024
- [3] **How Does Personalized Learning Affect Student Achievement?**
John F. Pane, Elizabeth D. Steiner, Matthew D. Baird, Laura S. Hamilton, Joseph D. Pane
Research Summary Published Dec 7, 2017 [RAND Corporation](#)
- [4] **Assessing the Impact of Personalized Learning Paths on Student Engagement and Outcomes** [Vorecol](#) 2024
- [5] The Benefits and Challenges of Personalized Learning **Academic Leadership**//**March 19, 2018** [Home](#)
- [6] Personalized Learning Market Size, Share, Growth, and Industry Analysis, by Type (Online Courses, Software & APP, Offline Guide, Other), by Application (Personal, Commercial), and Regional Insights and Forecast to 2034 **26 May 2025**
[marketreportsworld.com](#)
- [7] AI in Personalized Learning and Education Technology Market Size, Share & Trends Analysis Report By Type (Machine Learning Algorithms, Natural Language Processing (NLP) Systems, Expert Systems, Computer Vision Applications, Intelligent Tutoring Systems (ITS), Virtual Learning Environments (VLEs), Adaptive Learning Platforms), By Application, By Educational Level, By End-User, By Region, And By Segment Forecasts, 2025-2034 [InsightAce Analytic](#)
- [8] Survey: 86% of Students Already Use AI in Their Studies By Rhea Kelly , CAMPUS TECHNOLOGY [campustechnology.com/articles](#) 2023
- [9] Opinion: Internet access transformed society. Now Connecticut needs to embrace AI
By Kevin J. Conlan [ctinsider.com/opinion/article](#) 2023
- [10] **Why AI usage is now soaring among higher ed employees** by Alcino Donadel , November 5, 2024, [universitybusiness.com](#)
- [11] whisper paper [\[2212.04356\]](#) Robust Speech Recognition via Large-Scale Weak Supervision
- [12] [What is an LLM? A Guide on Large Language Models and How They Work](#) | [DataCamp](#) Dec,2024
- [13] [Information is Beautiful](#) Jan, 2024
- [14] Attention is all you need paper [arxiv.org/pdf/1706.03762](#) Aug, 2023
- [15] [\[2309.16609\]](#) Qwen Technical Report Sep, 2023
- [16] [Qwen-14B: Alibaba's Powerhouse Open-Source LLM](#) | by Lynn Mikami | [Medium](#) Oct, 2023
- [17] [\[2005.11401\]](#) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks Apr,2021

- [18] [What is an LLM? A Guide on Large Language Models and How They Work | DataCamp](#) 2022
- [19] [What is Retrieval-Augmented Generation\(RAG\) in LLM and How it works? | by Sahin Ahmed, Data Scientist | Medium](#) 2023
- [20] [Which is better, retrieval augmentation \(RAG\) or fine-tuning? Both. | Snorkel AI](#) 2024
- [21] [Building RAG-based LLM Applications for Production](#) 2021
- [22] [RAGFlow | RAGFlow](#) 2025
- [23] [Home | Open WebUI](#) 2025
- [24] [Open WebUI](#) 2025
- [25] [Architecture | LangChain](#) 2025
- [26] Ready player me documentation <https://docs.readyplayer.me/ready-player-me/what-is-ready-player-me> May 2025
- [27] **ReadyPlayerMe morphTarget documentation :** <https://docs.readyplayer.me/ready-pla...> 2024
- [28] **GLTF** <https://en.wikipedia.org/wiki/GLTF> May 2025
- [29] **GLTFJSX GitHub:** <https://github.com/pmndrs/gltfjsx> Nov.2024
- [30] **online version of gltfjsx :** <https://gltf.pmnd.rs/> May2024
- [31] **Mixamo** <https://en.wikipedia.org/wiki/Mixamo> June 2025
- [32] **Mixamo site:** <https://www.mixamo.com/#/> 2025
- [33] Blenderdocumentationhttps://docs.blender.org/manual/en/latest/getting_started/about/index.html May 2025
- [34] <https://docs.blender.org/manual/en/latest/animation/armatures/introduction.html> May 2025
- [35] **Glossary :**<https://docs.blender.org/manual/en/latest/glossary/index.html> May 2025
- [36] **Blender:**<https://www.blender.org/> 2024
- [37] **Rhubarb LipSync:** <https://github.com/DanielSWolf/rhubar...> April 2025
- [38] **Oculus Lipsync for Unity Development:**<https://developers.meta.com/horizon/documentation/unity/audiiovrlipsync-unity/> 2025
- [39] **Oculus OVR LipSync:** <https://docs.readyplayer.me/ready-player-me/api-reference/avatars/morph-targets/oculus-ovr-lipsync> 2024
- [40] **FFmpeg documentation:** <https://ffmpeg.org/ffmpeg.html> June 2025
- [41] [From Text to Speech: A Deep Dive into TTS Technologies - Zilliz Learn](#) 2025
- [42] <https://atlasaidev.com/how-does-text-to-speech-work/> 2025

- [43] <https://charansinghthakur9.medium.com/text-to-speech-tts-simplified-for-beginners-43218e5032d5> 2021
- [44] <https://wavel.ai/blog/what-is-text-to-speech-and-how-does-it-work> 2025
- [45] <https://milvus.io/ai-quick-reference/what-is-texttospeech-tts-technology> 2025
- [46] <https://arxiv.org/abs/1712.05884> 2017
- [47] <https://arxiv.org/abs/2005.11129> 2020
- [48] <https://www.datacamp.com/tutorial/amazon-polly> 2025
- [49] <https://docs.aws.amazon.com/polly/latest/dg/neural-voices.html> 2025
- [50] <https://aws.amazon.com/polly/> 2025
- [51] <https://aws.amazon.com/polly/pricing/?nc=sn&loc=4> 2025
- [52] https://www.ibm.com/docs/en/rpa/23.0.x?topic=SSTHBP_23.0.x/using_ibm_rpa/digital_assistant/develop/supported_languages/watson_engine.htm 2025
- [53] <https://www.ibm.com/products/text-to-speech> 2024
- [54] <https://deepwiki.com/pndurette/gTTS/3.2-language-support> 2025
- [55] <https://developer.apple.com/documentation/arkit/arfaceanchor/blendshapelocation> 2025
- [56] **Apple ARKit:** <https://docs.readyplayer.me/ready-player-me/api-reference/avatars/morph-targets/apple-arkit> 2024
- [57] **React:** <https://react.dev/learn> March 2025
- [58] **@react-three/fiber:** <https://r3f.docs.pmnd.rs/getting-started/introduction> May 2025
- [59] **@react-three/drei:** <https://drei.docs.pmnd.rs/getting-started/introduction> June 2025
- [60] **Three.js:** <https://threejs.org/docs/> March 2025
- [61] **Leva (GUI Debug Panel):** <https://leva.pmnd.rs/> June 2025

Appendix

This appendix provides supporting materials that complement the development and evaluation of **The Design of eLearning Course Using The Power of AI**. It includes code examples, tool references, and technical notes related to the implementation of the Large Language Model (LLM) system, prompt engineering, and real-time avatar animation with lip synchronization.

Large Language Models and Retrieval-Augmented Generation:

The system leverages **Large Language Models (LLMs)** with **Retrieval-Augmented Generation (RAG)** to produce accurate and context-aware responses to user questions.

function for create_qa_chain:

```

637 def create_qa_chain():
638     chain = (
639         {
640             "context": session_state['retriever'],
641             "history": lambda x: format_history(get_memory(x['session_id'])),
642             "question": RunnablePassthrough()
643         }
644         | session_state['prompt']
645         | session_state['llm']
646         | StrOutputParser()
647     )
648     return chain
649
650 session_state['qa_chain'] = create_qa_chain()

```

With prompt:

```

session_state = {}

session_state['template'] = """IMPORTANT: You are strictly forbidden from introducing any knowledge, concept, or example that is not explicitly ment
You are an expert Database teacher, possessing in-depth knowledge of database concepts, principles, and technologies. Your tone should be profession
Your primary goal is to provide accurate, concise, and easy-to-understand explanations. When answering questions:
1. Focus Exclusively on the Syllabus Content: Your knowledge is strictly limited to the information contained within the provided syllabus doc
2. Assume the User is a Student: Tailor your explanations to a student's level of understanding. Define technical terms clearly and avoid over
3. Be Comprehensive Yet Concise: Provide thorough answers that address all aspects of the user's question as it relates to the syllabus. Howeve
4. Provide Examples (If Applicable and From Syllabus): If the syllabus (referenced in the context) includes specific examples or case studies,
5. Structure Your Answers Logically: Organize your responses with clear headings, bullet points, or numbered lists when appropriate to improve
6. Encourage Further Inquiry (Within Syllabus Scope): At the end of your response, you can gently encourage the user to ask follow-up questions
7. Acknowledge Limitations: If a user asks a question that cannot be answered based on the information in the syllabus (referenced in the conte
8. Maintain a Helpful and Encouraging Demeanor: While professional, your tone should also be supportive and encouraging to facilitate a positiv

Context: {context}
History: {history}

User: {question}
Chatbot: ""

session_state['prompt'] = PromptTemplate(
    input_variables=["history", "context", "question"],
    template=session_state['template'],
)

session_state['memories'] = {}

```

Activate Windows
Go to Settings to activate Windows.

Avatar :

The **Avatar** plays a key role in making the eLearning experience more interactive and engaging .The avatar responds using realistic facial expressions, lip-syncing, and animations to create a friendly and motivating learning environment.

Tools:

- Ready Player Me
- GLTFJSX
- Mixamo
- Blender
- FFmpeg
- Rhubarb Lip Sync

Apply Animations Into Avatar 1 :

```
574  export function Avatar(props) {  
575      const audioRef = useRef(null);  
576      const { nodes, materials } = useGLTF("models/67b62f42aa7ee263534cde02.glb");  
577      const { animations: rawAnimations } = useGLTF("models/animations.glb");  
578      const { message, onMessagePlayed, registerAudioRef } = useChat();  
579      const { isMuted } = useChat();
```

Apply Animations Into Avatar 2:

```
const defaultAnimation = rawAnimations.find(a => a.name === "Idle") ? "Idle" : rawAnimations[0]?.name;  
const [animation, setAnimation] = useState(defaultAnimation);  
  
// References  
const group = useRef(null);  
const { actions, mixer } = useAnimations(rawAnimations, group);
```

Apply Animations Into Avatar 3:

```
// Handle incoming messages
useEffect(() => {
  if (!message) {
    setAnimation("Idle");
    setLipsync(null);
    if (audioRef.current) {
      audioRef.current.pause();
      audioRef.current = null;
    }
    return;
  }

  if (audioRef.current) {
    audioRef.current.pause();
    audioRef.current.onended = null;
    audioRef.current = null;
  }

  setAnimation(isMuted ? "Idle" : message.animation || "Talking_0");
  setFacialExpression(message.facialExpression);
  setLipsync(isMuted ? null : message.lipsync);
  if (message.audio) {
    const audio = new Audio(`data:audio/mp3;base64,${message.audio}`);
    audioRef.current = audio;
    audio.onended = onMessagePlayed;
    if (!isMuted) {
      audio.play().catch((error) => {
        console.error(`[ERROR] Error playing audio:`, error);
      });
    }
  } else {
    setTimeout(onMessagePlayed, 1000);
  }
}, [message, isMuted, onMessagePlayed]);
```

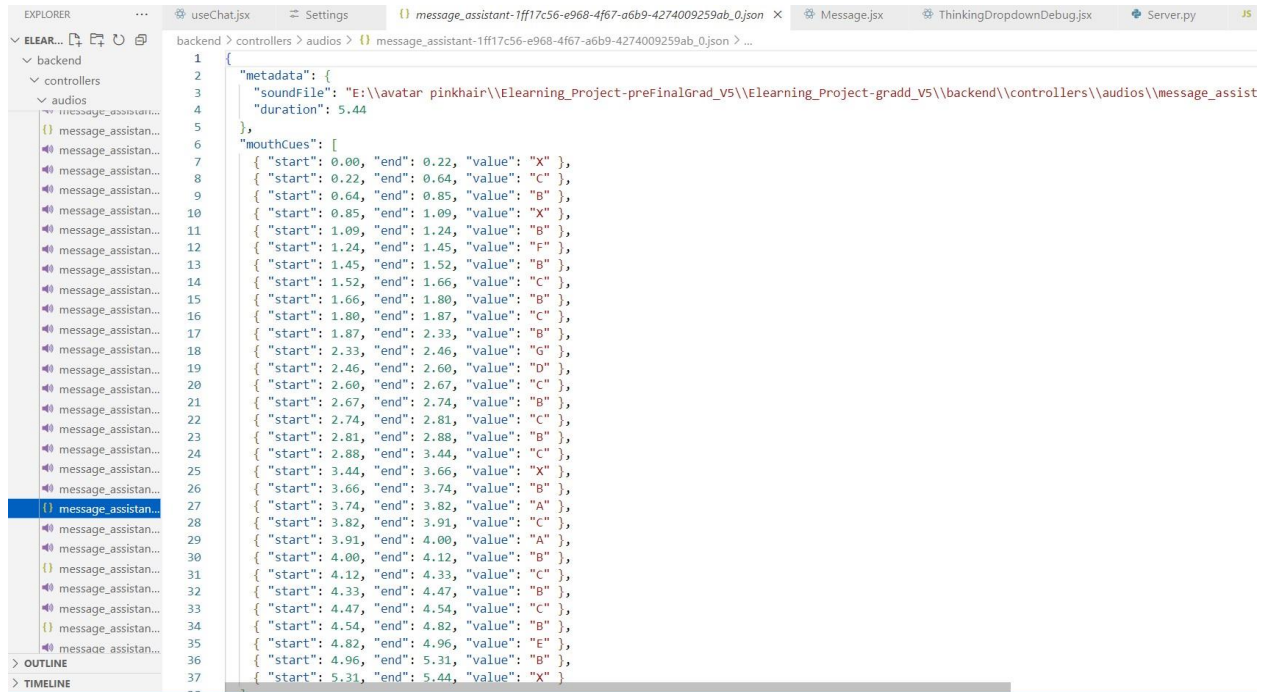
Generate A Json File By Rhubarb Library:

```
667 // Handle animation changes
668 useEffect(() => {
669   if (!actions || !animation || !actions[animation]) return;
670
671   const fadeTime = mixer?.stats?.actions?.inUse === 0 ? 0 : 0.5;
672
673   actions[animation]
674     .reset()
675     .fadeIn(fadeTime)
676     .play();
677
678   return () => {
679     if (actions[animation]) {
680       actions[animation].fadeOut(0.5);
681     }
682   };
683 }, [animation, actions, mixer]);
```

Generate A Json File By Rhubarb Library:

```
117 // Function to generate lip sync data
118 ✓ const lipSyncMessage = async (messageId) => {
119   console.log(`Processing lipsync for message: ${messageId}`);
120
121   const wavPath = `audios/message_${messageId}.wav`;
122   const jsonPath = `audios/message_${messageId}.json`;
123
124   try {
125     // Convert MP3 to WAV with specific format (44.1kHz, 16-bit, mono)
126     await execCommand(`ffmpeg -y -i audios/message_${messageId}.mp3 -ar 44100 -ac 1 -ab 16 ${wavPath}`);
127
128     // Verify WAV file exists
129     await fs.access(wavPath);
130     console.log(`✅ WAV file verified: ${wavPath}`);
131
132     // Generate lip sync data with Rhubarb (using phonetic recognizer)
133     await execCommand(`bin\\rhubarb.exe -f json -o "${jsonPath}" "${wavPath}" -r phonetic`);
134
135     // Verify JSON file exists
136     await fs.access(jsonPath);
137     console.log(`✅ Lip sync JSON verified: ${jsonPath}`);
138   } catch (error) {
139     console.error(`❌ Error generating lip sync for message ${messageId}:`, error);
140     throw error;
141   }
142 }
143 };
```


Json File Generated By Rhubarb Library:



```
1 {
2   "metadata": {
3     "soundFile": "E:\\avatar pinkhair\\Elearning_Project-preFinalGrad_V5\\Elearning_Project-gradd_V5\\backend\\controllers\\audios\\message_assist
4     "duration": 5.44
5   },
6   "mouthCues": [
7     { "start": 0.00, "end": 0.22, "value": "X" },
8     { "start": 0.22, "end": 0.64, "value": "C" },
9     { "start": 0.64, "end": 0.85, "value": "B" },
10    { "start": 0.85, "end": 1.09, "value": "X" },
11    { "start": 1.09, "end": 1.24, "value": "B" },
12    { "start": 1.24, "end": 1.45, "value": "F" },
13    { "start": 1.45, "end": 1.52, "value": "B" },
14    { "start": 1.52, "end": 1.66, "value": "C" },
15    { "start": 1.66, "end": 1.80, "value": "B" },
16    { "start": 1.80, "end": 1.87, "value": "C" },
17    { "start": 1.87, "end": 2.33, "value": "B" },
18    { "start": 2.33, "end": 2.46, "value": "G" },
19    { "start": 2.46, "end": 2.60, "value": "D" },
20    { "start": 2.60, "end": 2.67, "value": "C" },
21    { "start": 2.67, "end": 2.74, "value": "B" },
22    { "start": 2.74, "end": 2.81, "value": "C" },
23    { "start": 2.81, "end": 2.88, "value": "B" },
24    { "start": 2.88, "end": 3.44, "value": "C" },
25    { "start": 3.44, "end": 3.66, "value": "X" },
26    { "start": 3.66, "end": 3.74, "value": "B" },
27    { "start": 3.74, "end": 3.82, "value": "A" },
28    { "start": 3.82, "end": 3.91, "value": "C" },
29    { "start": 3.91, "end": 4.00, "value": "A" },
30    { "start": 4.00, "end": 4.12, "value": "B" },
31    { "start": 4.12, "end": 4.33, "value": "C" },
32    { "start": 4.33, "end": 4.47, "value": "B" },
33    { "start": 4.47, "end": 4.54, "value": "C" },
34    { "start": 4.54, "end": 4.82, "value": "B" },
35    { "start": 4.82, "end": 4.96, "value": "E" },
36    { "start": 4.96, "end": 5.31, "value": "B" },
37    { "start": 5.31, "end": 5.44, "value": "X" }
```

Real-Time Lip Sync System with Viseme Mapping for 3D Avatar 1:

```
144
145 // Function to generate audio & lip sync, returning base64-encoded audio and JSON transcript
146 ✓ const generateAudioAndLipSync = async (text, messageId) => {
147   const audioPath = `${audioDir}/message_${messageId}.mp3`;
148   const jsonPath = `${audioDir}/message_${messageId}.json`;
149
150   try {
151     // Generate audio
152     await textToSpeech(text, audioPath);
153
154     // Generate lip sync
155     await lipSyncMessage(messageId);
156
157     // Convert audio to base64
158     const audioData = await fs.readFile(audioPath);
159     const lipsyncData = await fs.readFile(jsonPath, "utf8");
160     let lipsync = JSON.parse(lipsyncData);
161
```

Real-Time Lip Sync System with Viseme Mapping for 3D Avatar 2:

```
162 // Map Rhubarb visemes to Ready Player Me visemes (match Avatar.jsx)
163 const visemeMap = {
164   A: "viseme_PP", // Closed mouth (e.g., "m")
165   B: "viseme_kk", // Small mouth (e.g., "g")
166   C: "viseme_I",  // Wide mouth (e.g., "ee")
167   D: "viseme_AA", // Open mouth (e.g., "ah")
168   E: "viseme_O",  // Smile-like (e.g., "oh")
169   F: "viseme_U",  // Pursed lips (e.g., "u")
170   G: "viseme_FF", // Lips forward (e.g., "f")
171   H: "viseme_TH", // Tongue out (e.g., "th")
172   X: "viseme_PP"  // Rest/silence
173 };
174 lipsync.mouthCues = lipsync.mouthCues.map(cue => ({
175   ...cue,
176   value: visemeMap[cue.value] || "viseme_PP" // Fallback to viseme_PP
177 }));
178
179 // Log visemes for debugging
180 console.log(`[DEBUG] Lip sync visemes for message ${messageId}:`);
181 lipsync.mouthCues.forEach(cue => {
182   console.log(`  Viseme: ${cue.value}, Start: ${cue.start}s, End: ${cue.end}s`);
183 });
184
185 return {
186   audio: audioData.toString("base64"),
187   lipsync: lipsync,
188 };
189 } catch (error) {
190   console.error(`❌ Error generating audio/lipsync for message ${messageId}:`, error);
191   throw error;
192 }
193 };
```

Real-Time Lip Sync System with Viseme Mapping for 3D Avatar 3:

```
741     lerpMorphTarget("eyeBlinkLeft", blink || winkLeft ? 1 : 0, 0.5, false);
742     lerpMorphTarget("eyeBlinkRight", blink || winkRight ? 1 : 0, 0.5, false);
743
744     if (setupMode || !message || !lipsync || !audioRef.current) return;
745
746     const currentAudioTime = audioRef.current.currentTime;
747     const appliedMorphTargets = [];
748
749     for (let i = 0; i < lipsync.mouthCues.length; i++) {
750         const mouthCue = lipsync.mouthCues[i];
751         if (currentAudioTime >= mouthCue.start && currentAudioTime <= mouthCue.end) {
752             let viseme = mouthCue.value;
753             if (viseme === "viseme_AA") viseme = "viseme_aa";
754             appliedMorphTargets.push(viseme);
755             lerpMorphTarget(viseme, 1, 0.2, true);
756             console.log(
757                 `[DEBUG] Applying viseme ${viseme} at ${currentAudioTime.toFixed(2)}s (cue: ${mouthCue.start}-${mouthCue.end}s)`
758             );
759             break;
760         }
761     }
762 }
```

Real-Time Lip Sync System with Viseme Mapping for 3D Avatar4:

```
763     const expectedVisemes = [
764         "viseme_sil",
765         "viseme_PP",
766         "viseme_FF",
767         "viseme_TH",
768         "viseme_DD",
769         "viseme_kk",
770         "viseme_CH",
771         "viseme_SS",
772         "viseme_nn",
773         "viseme_RR",
774         "viseme_aa",
775         "viseme_E",
776         "viseme_I",
777         "viseme_O",
778         "viseme_U",
779     ];
780     expectedVisemes.forEach((viseme) => {
781         if (!appliedMorphTargets.includes(viseme)) {
782             lerpMorphTarget(viseme, 0, 0.1, true);
783         }
784     });
785 });
786 }
```

ملخص

يُمثل مشروع "تصميم دورة تعليم إلكتروني باستخدام قوة الذكاء الاصطناعي" نهجًا ثوريًا في التعليم الرقمي، من خلال تطوير مساعد تعلّم ذكي حوارِي يعزز تفاعل الطلاب واحتفاظهم بالمعلومات. استلهمت الفكرة من مؤتمر "التجربة التعليمية في عصر الذكاء الاصطناعي" الذي نظّمته شركة مايكروسوفت عام ٢٠٢٤. يهدف المشروع إلى إنشاء نظام توجيه افتراضي مدعوم بالذكاء الاصطناعي، حيث يتفاعل المتعلمون مع معلم رمزي متجاوب عبر الحوار الطبيعي، سواء من خلال إدخال نصي أو محادثة صوتية. تم تصميم النظام ليعمل كمساعد تعليمي موثوق وماهر، ملتزم تمامًا بمنهج تعليمي محدد لضمان الدقة العلمية والملاءمة الأكاديمية.

يتأسس هذا المنصة على دمج تقنيات ذكاء اصطناعي متطورة تعمل بتناغم لتقديم تجربة تعليمية سلسة. يوفّر نظام Whisper من OpenAI قدرات قوية للتعرف على الكلام، حيث يحوّل استفسارات الطلاب الصوتية إلى نص قابل للمعالجة. ثم يطبق النظام تقنيات "التوليد المستند للاسترجاع" (RAG) "إلى جانب نماذج اللغة الكبيرة (LLMs) لاسترداد وإنشاء إجابات دقيقة وسياقية من قاعدة معرفية منظمة. تُعاد هذه الإجابات إلى المتعلّم عبر واجهة تحويل النص إلى كلام (TTS)، مما يمنح المعلم الآلي حضورًا تفاعليًا أشبه بالحياة.

من الابتكارات المهمة في هذا المشروع آلية تصفية الردود بحسب المجال المعرفي، التي تضمن تركيز المساعد على الموضوع المحدد. عند مواجهة أسئلة خارج نطاق معرفته، يرفض النظام الإجابة أو يوجّه المتعلّم نحو الإطار المناسب، مما يحافظ على النزاهة الأكاديمية ويمنع انتشار المعلومات الخاطئة. يجعل هذا النموذج التفاعلي المنضبط المنصة ذات قيمة خاصة في بيئات التعلم المنظم، حيث يكون الالتزام بالمنهج أمرًا جوهريًا.

تتطوي تبعات هذا النظام التعليمي المعزّز بالذكاء الاصطناعي على أهمية كبيرة للتعليم الحديث. فهو يعالج القيود الرئيسية للدورات الإلكترونية التقليدية—مثل طريقة عرض المحتوى بصورة سلبية وغياب التعليم الشخصي—بإدخال رفيق تعلم حوارِي متكيف. يتيح النظام معالجة كل من المدخلات الصوتية والنصية، مما يجعله مناسبًا لأنماط تعلم متنوعة، بينما تُحاكي قدرته على الاستجابة في الوقت الفعلي تفاعلات الفصول الدراسية. علاوة على ذلك، مصمم البنية الأساسية للنظام لتكون قابلة للتوسع، بما يتيح التوسع مستقبلًا نحو تخصصات ولغات متعددة وميزات تعليمية تكيفية أكثر تطورًا.

بدمج الذكاء الاصطناعي المتقدّم مع مبادئ التصميم البيداغوجي، يمهد هذا المشروع لانطلاق نموذج جديد في التعليم الرقمي—نموذج لا يكتفي بتعزيز التعليم البشري بل يحاكي أهم مكوناته: الاستجابة الفورية، والتخصيص، واستكشاف المعرفة الموجه. يبرهن النظام على إمكانية تطبيق الذكاء الاصطناعي في التعليم بمسؤولية، من خلال موازنة الابتكار التكنولوجي والصرامة الأكاديمية لإنشاء أدوات تعلّم فعّالة وجديرة بالثقة.

تصميم دورة التعلم الإلكتروني باستخدام الذكاء الاصطناعي

علوم الحاسوب
(توثيق مشروع التخرج)

تقديم

آلاء بهاء الدين يوسف قبيصي
جيسكا اشرف انيس عازر
حسن طارق محمد حسن
عمر علاء عبدالعزيز محمد
عمر جاد سعد الحبيبي
أحمد علي علي ابراهيم
عمرو حازم اسماعيل عبدالرحمن
زينب ناجي محمد مأمون

تحت اشراف

أ.د محمد أنور عسل
أ. امنية محمد
أ. حسن عبد الشافي
أ. مهاب مصطفى

٢٠٢٥